

Babel

Code

Version 26.12.135948
2026/09/20

Javier Bezos
Current maintainer

Johannes L. Braams
Original author

Localization and
internationalization

Unicode

T_EX

LuaT_EX

pdfT_EX

XeT_EX

Contents

1	Identification and loading of required files	3
2	locale directory	3
3	Tools	3
3.1	A few core definitions	8
3.2	$\LaTeX$: babel.sty (start)	8
3.3	base	10
3.4	key=value options and other general option	10
3.5	Post-process some options	12
3.6	Plain: babel.def (start)	13
4	babel.sty and babel.def (common)	13
4.1	Selecting the language	16
4.2	Errors	24
4.3	More on selection	25
4.4	Short tags	26
4.5	Compatibility with language.def	26
4.6	Hooks	27
4.7	Setting up language files	28
4.8	Shorthands	30
4.9	Language attributes	39
4.10	Support for saving and redefining macros	40
4.11	French spacing	42
4.12	Hyphens	42
4.13	Multiencoding strings	44
4.14	Tailor captions	49
4.15	Making glyphs available	50
4.15.1	Quotation marks	50
4.15.2	Letters	52
4.15.3	Shorthands for quotation marks	52
4.15.4	Umlauts and tremas	53
4.16	Layout	55
4.17	Load engine specific macros	55
4.18	Creating and modifying languages	55
4.19	Main loop in ‘provide’	63
4.20	Processing keys in ini	68
4.21	French spacing (again)	74
4.22	Handle language system	75
4.23	Numerals	76
4.24	Casing	77
4.25	Getting info	78
4.26	BCP 47 related commands	79
5	Adjusting the Babel behavior	80
5.1	Cross referencing macros	83
5.2	Layout	85
5.3	Marks	87
5.4	Other packages	88
5.4.1	ifthen	88
5.4.2	varioref	88
5.4.3	hhline	89
5.5	Encoding and fonts	89
5.6	Basic bidi support	91
5.7	Local Language Configuration	94
5.8	Language options	95

6	The kernel of Babel	99
7	Error messages	99
8	Loading hyphenation patterns	103
9	luatex + xetex: common stuff	107
10	Hooks for XeTeX and LuaTeX	110
10.1	XeTeX	110
10.2	Support for interchar	112
10.3	Layout	114
10.4	8-bit TeX	116
10.5	LuaTeX	116
10.6	Southeast Asian scripts	123
10.7	CJK line breaking	124
10.8	Arabic justification	126
10.9	Common stuff	131
10.10	Automatic fonts and ids switching	131
10.11	Bidi	138
10.12	Layout	140
10.13	Lua: transforms	149
10.14	Lua: Auto bidi with basic and basic-r	159
11	Data for CJK	172
12	The ‘nil’ language	173
13	Calendars	174
13.1	Islamic	174
13.2	Hebrew	175
13.3	Persian	180
13.4	Coptic and Ethiopic	180
13.5	Julian	181
13.6	Buddhist	181
14	Support for Plain TeX (plain.def)	183
14.1	Not renaming hyphen.tex	183
14.2	Emulating some L ^A T _E X features	183
14.3	General tools	184
14.4	Encoding related macros	188
15	Acknowledgements	190

The babel package is being developed incrementally, which means parts of the code are under development and therefore incomplete. Only documented features are considered complete. In other words, use babel in real documents only as documented (except, of course, if you want to explore and test them).

1. Identification and loading of required files

The babel package after unpacking consists of the following files:

babel.sty is the $\LaTeX$ package, which set options and load language styles.

babel.def is loaded by Plain.

switch.def defines macros to set and switch languages (it loads part babel.def).

plain.def is not used, and just loads babel.def, for compatibility.

hyphen.cfg is the file to be used when generating the formats to load hyphenation patterns.

There some additional tex, def and lua files.

The babel installer extends docstrip with a few “pseudo-guards” to set “variables” used at installation time. They are used with `<@name@>` at the appropriate places in the source code and defined with either `{(name=value)}`, or with a series of lines between `{(*name)}` and `{(/name)}`. The latter is cumulative (e.g., with *More package options*). That brings a little bit of literate programming. The guards `<-name>` and `<+name>` have been redefined, too. See babel.ins for further details.

2. locale directory

A required component of babel is a set of ini files with basic definitions for about 300 languages. They are distributed as a separate zip file, not packed as dtx. Many of them are essentially finished (except bugs and mistakes, of course). Some of them are still incomplete (but they will be usable), and there are some omissions (e.g., there are no geographic areas in Spanish). Not all include LICR variants.

babel-*.ini files contain the actual data; babel-*.tex files are basically proxies to the corresponding ini files.

See [Keys in ini files](#) in the the babel site.

3. Tools

```
1 <<(version=26.12.135948)>>
2 <<(date=2026/09/20)>>
```

Do not use the following macros in ldf files. They may change in the future. This applies mainly to those recently added for replacing, trimming and looping. The older ones, like `\bbl@afterfi`, will not change. We define some basic macros which just make the code cleaner. `\bbl@add` is now used internally instead of `\addto` because of the unpredictable behavior of the latter. Used in babel.def and in babel.sty, which means in $\LaTeX$ is executed twice, but we need them when defining options and babel.def cannot be load until options have been defined. This does not hurt, but should be fixed somehow.

```
3 <<(*Basic macros)>> ≡
4 \bbl@trace{Basic macros}
5 \def\bbl@stripslash{\expandafter\@gobble\string}
6 \def\bbl@add#1#2{%
7   \bbl@ifunset{\bbl@stripslash#1}%
8   {\def#1{#2}}%
9   {\expandafter\def\expandafter#1\expandafter{#1#2}}}
10 \def\bbl@xin@{\@expandtwoargs\in@}
11 \def\bbl@carg#1#2{\expandafter#1\csname#2\endcsname}%
12 \def\bbl@ncarg#1#2#3{\expandafter#1\expandafter#2\csname#3\endcsname}%
13 \def\bbl@ccarg#1#2#3{%
14   \expandafter#1\csname#2\expandafter\endcsname\csname#3\endcsname}%
15 \def\bbl@carg#1#2{\expandafter#1\csname bbl@#2\endcsname}%
16 \def\bbl@carg#1{\csname bbl@#1\endcsname}
17 \def\bbl@carg#1{\csname bbl@#1\language\endcsname}
18 \def\bbl@loop#1#2#3{\bbl@loop#1{#3}#2,\@nnil,}
```

```

19 \def\bbl@loopx#1#2{\expandafter\bbl@loop\expandafter#1\expandafter{#2}}
20 \def\bbl@loop#1#2#3,{%
21   \ifx\@nnil#3\relax\else
22     \def#1{#3}#2\bbl@afterfi\bbl@loop#1{#2}%
23   \fi}
24 \def\bbl@for#1#2#3{\bbl@loopx#1{#2}{\ifx#1\@empty\else#3\fi}}

```

\bbl@add@list This internal macro adds its second argument to a comma separated list in its first argument. When the list is not defined yet (or empty), it will be initiated. It presumes expandable character strings.

```

25 \def\bbl@add@list#1#2{%
26   \edef#1{%
27     \bbl@ifunset{\bbl@stripslash#1}%
28     }%
29     {\ifx#1\@empty\else#1,\fi}%
30   #2}}

```

\bbl@afterelse

\bbl@afterfi Because the code that is used in the handling of active characters may need to look ahead, we take extra care to ‘throw’ it over the \else and \fi parts of an \if-statement¹. These macros will break if another \if... \fi statement appears in one of the arguments and it is not enclosed in braces.

```

31 \long\def\bbl@afterelse#1\else#2\fi{\fi#1}
32 \long\def\bbl@afterfi#1\fi{\fi#1}

```

\bbl@exp Now, just syntactical sugar, but it makes partial expansion of some code a lot more simple and readable. Here \ stands for \noexpand, \<.> for \noexpand applied to a built macro name (which does not define the macro if undefined to \relax, because it is created locally), and \[. .] for one-level expansion (where . . is the macro name without the backslash). The result may be followed by extra arguments, if necessary.

```

33 \def\bbl@exp#1{%
34   \begingroup
35     \let\<\noexpand
36     \let\<\bbl@exp@en
37     \let\[\bbl@exp@ue
38     \edef\bbl@exp@aux{\endgroup#1}%
39     \bbl@exp@aux}
40 \def\bbl@exp@en#1>{\expandafter\noexpand\csname#1\endcsname}%
41 \def\bbl@exp@ue#1]{%
42   \unexpanded\expandafter\expandafter\expandafter{\csname#1\endcsname}}%

```

\bbl@trim The following piece of code is stolen (with some changes) from keyval, by David Carlisle. It defines two macros: \bbl@trim and \bbl@trim@def. The first one strips the leading and trailing spaces from the second argument and then applies the first argument (a macro, \toks@ and the like). The second one, as its name suggests, defines the first argument as the stripped second argument.

```

43 \def\bbl@tempa#1{%
44   \long\def\bbl@trim##1##2{%
45     \futurelet\bbl@trim@a\bbl@trim@c##2\@nil\@nil#1\@nil\relax{##1}}%
46   \def\bbl@trim@c{%
47     \ifx\bbl@trim@a\@sptoken
48       \expandafter\bbl@trim@b
49     \else
50       \expandafter\bbl@trim@b\expandafter#1%
51     \fi}%
52   \long\def\bbl@trim@b#1##1 \@nil{\bbl@trim@i##1}}
53 \bbl@tempa{ }
54 \long\def\bbl@trim@i#1\@nil#2\relax#3{#3{#1}}
55 \long\def\bbl@trim@def#1{\bbl@trim{\def#1}}

```

¹This code is based on code presented in TUGboat vol. 12, no2, June 1991 in “An expansion Power Lemma” by Sonja Maus.

\bbl@ifunset To check if a macro is defined, we create a new macro, which does the same as `\ifundefined`. However, in an ϵ -tex engine, it is based on `\ifcurname`, which is more efficient, and does not waste memory. Defined inside a group, to avoid `\ifcurname` being implicitly set to `\relax` by the `\curname` test.

```

56 \begingroup
57 \gdef\bbl@ifunset#1{%
58   \expandafter\ifx\curname#1\endcurname\relax
59   \expandafter\@firstoftwo
60   \else
61   \expandafter\@secondoftwo
62   \fi}
63 \bbl@ifunset{ifcurname}%
64 {}%
65 {\gdef\bbl@ifunset#1{%
66   \ifcurname#1\endcurname
67   \expandafter\ifx\curname#1\endcurname\relax
68   \bbl@afterelse\expandafter\@firstoftwo
69   \else
70   \bbl@afterfi\expandafter\@secondoftwo
71   \fi
72   \else
73   \expandafter\@firstoftwo
74   \fi}}
75 \endgroup

```

\bbl@ifblank A tool from url, by Donald Arseneau, which tests if a string is empty or space. The companion macros tests if a macro is defined with some ‘real’ value, i.e., not `\relax` and not empty,

```

76 \def\bbl@ifblank#1{%
77   \bbl@ifblank@i#1\@nil\@nil\@secondoftwo\@firstoftwo\@nil}
78 \long\def\bbl@ifblank@i#1#2\@nil#3#4#5\@nil#4#5%
79 \def\bbl@ifset#1#2#3{%
80   \bbl@ifunset{#1}{#3}{\bbl@exp{\bbl@ifblank{\@nameuse{#1}}}{#3}{#2}}}

```

For each element in the comma separated `<key>=<value>` list, execute `<code>` with #1 and #2 as the key and the value of current item (trimmed). In addition, the item is passed verbatim as #3. With the `<key>` alone, it passes `\@empty` as value (i.e., the macro thus named, not an empty argument, which is what you get with `<key>=` and no value).

```

81 \def\bbl@forkv#1#2{%
82   \def\bbl@kvcmd##1##2##3{#2}%
83   \bbl@kvnext#1,\@nil,}
84 \def\bbl@kvnext#1,{%
85   \ifx\@nil#1\relax\else
86   \bbl@ifblank{#1}{\bbl@forkv@eq#1=\@empty=\@nil{#1}}%
87   \expandafter\bbl@kvnext
88   \fi}
89 \def\bbl@forkv@eq#1=#2=#3\@nil#4{%
90   \bbl@trim\def\bbl@forkv@a{#1}%
91   \bbl@trim{\expandafter\bbl@kvcmd\expandafter{\bbl@forkv@a}}{#2}{#4}}

```

A *for* loop. Each item (trimmed) is #1. It cannot be nested (it’s doable, but we don’t need it).

```

92 \def\bbl@vforeach#1#2{%
93   \def\bbl@forcmd##1{#2}%
94   \bbl@fornext#1,\@nil,}
95 \def\bbl@fornext#1,{%
96   \ifx\@nil#1\relax\else
97   \bbl@ifblank{#1}{\bbl@trim\bbl@forcmd{#1}}%
98   \expandafter\bbl@fornext
99   \fi}
100 \def\bbl@foreach#1{\expandafter\bbl@vforeach\expandafter{#1}}

```

Some code should be executed once. The first argument is a flag.

```

101 \global\let\bbl@done\@empty

```

```

102 \def\bbl@once#1#2{%
103   \bbl@xin@{,#1,}{,\bbl@done,}%
104   \ifin@ \else
105     #2%
106   \xdef\bbl@done{\bbl@done,#1,}%
107   \fi}
108 %   \end{macrodef}
109 %
110 % \macro{\bbl@replace}
111 %
112 % Returns implicitly |\toks@| with the modified string.
113 %
114 %   \begin{macrocode}
115 \def\bbl@replace#1#2#3{% in #1 -> repl #2 by #3
116   \toks@{ }%
117   \def\bbl@replace@aux##1#2##2#2{%
118     \ifx\bbl@nil##2%
119       \toks@\expandafter{\the\toks@##1}%
120     \else
121       \toks@\expandafter{\the\toks@##1#3}%
122       \bbl@afterfi
123       \bbl@replace@aux##2#2%
124     \fi}%
125   \expandafter\bbl@replace@aux#1#2\bbl@nil#2%
126   \edef#1{\the\toks@}}

```

An extension to the previous macro. It takes into account the parameters, and it is string based (i.e., if you replace elax by ho, then \relax becomes \rho). No checking is done at all, because it is not a general purpose macro, and it is used by babel only when it works (an example where it does *not* work is in \bbl@TG@@date, and also fails if there are macros with spaces, because they are retokenized). It may change! (or even merged with \bbl@replace; I'm not sure checking the replacement is really necessary or just paranoia).

```

127 \ifx\detokenize\undefined \else % Unused macros if old Plain TeX
128   \bbl@exp{\def\\bbl@parsedef##1\detokenize{macro:}}#2->#3\relax{%
129     \def\bbl@tempa{#1}%
130     \def\bbl@tempb{#2}%
131     \def\bbl@tempe{#3}}
132   \def\bbl@sreplace#1#2#3{%
133     \begingroup
134       \expandafter\bbl@parsedef\meaning#1\relax
135       \def\bbl@tempc{#2}%
136       \edef\bbl@tempc{\expandafter\strip@prefix\meaning\bbl@tempc}%
137       \def\bbl@tempd{#3}%
138       \edef\bbl@tempd{\expandafter\strip@prefix\meaning\bbl@tempd}%
139       \bbl@xin@{\bbl@tempc}{\bbl@tempe}% If not in macro, do nothing
140       \ifin@
141         \bbl@exp{\\bbl@replace\\bbl@tempe{\bbl@tempc}{\bbl@tempd}}%
142         \def\bbl@tempc{% Expanded an executed below as 'uplevel'
143           \\makeatletter % "internal" macros with @ are assumed
144           \\scantokens{%
145             \bbl@tempa\\@namedef{\bbl@stripslash#1}\bbl@tempb{\bbl@tempe}%
146             \noexpand\noexpand}%
147           \catcode64=\the\catcode64\relax}% Restore @
148       \else
149         \let\bbl@tempc\empty % Not \relax
150       \fi
151       \bbl@exp{% For the 'uplevel' assignments
152     \endgroup
153     \bbl@tempc}} % empty or expand to set #1 with changes
154 \fi

```

Two further tools. \bbl@ifsamestring first expand its arguments and then compare their expansion (sanitized, so that the catcodes do not matter). \bbl@engine takes the following values: 0 is pdf_{La}T_EX, 1 is luatex, and 2 is xetex. You may use the latter it in your language style if you want.

```

155 \def\bb@ifsamestring#1#2{%
156   \begingroup
157   \protected@edef\bb@tempb{#1}%
158   \edef\bb@tempb{\expandafter\strip@prefix\meaning\bb@tempb}%
159   \protected@edef\bb@tempc{#2}%
160   \edef\bb@tempc{\expandafter\strip@prefix\meaning\bb@tempc}%
161   \ifx\bb@tempb\bb@tempc
162     \aftergroup\@firstoftwo
163   \else
164     \aftergroup\@secondoftwo
165   \fi
166 \endgroup}
167 \chardef\bb@engine=%
168 \ifx\directlua\@undefined
169   \ifx\XeTeXinputencoding\@undefined
170     \z@
171   \else
172     \tw@
173   \fi
174 \else
175   \@ne
176 \fi

```

A somewhat hackish tool (hence its name) to avoid spurious spaces in some contexts.

```

177 \def\bb@bsphack{%
178   \ifhmode
179     \hskip\z@skip
180     \def\bb@esphack{\loop\ifdim\lastskip>\z@\unskip\repeat\unskip}%
181   \else
182     \let\bb@esphack\@empty
183   \fi}

```

Another hackish tool, to apply case changes inside a protected macros. It's based on the internal \let's made by \MakeUppercase and \MakeLowercase between things like \oe and \OE.

```

184 \def\bb@cased{%
185   \ifx\oe\OE
186     \expandafter\in@\expandafter
187     {\expandafter\OE\expandafter}\expandafter{\oe}%
188   \ifin@
189     \bb@afterelse\expandafter\MakeUppercase
190   \else
191     \bb@afterfi\expandafter\MakeLowercase
192   \fi
193 \else
194   \expandafter\@firstofone
195 \fi}

```

Ordered key/value lists (aka associative arrays). The setter either replaces the current value if the key exists, or adds the new key/value at the end. The getter returns either the value if the key exists or \relax. The code is based on arguments delimited by the keys. In the getter, note the value is supposed not to start with \@empty.

```

196 \def\bb@array@set#1#2#3{%
197   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux##4{%
198     \ifx\bb@noelt##2%
199       \def#1{##1\bb@elt#2\@{##4}}%
200     \else
201       \def\bb@array@aux####1\bb@elt#2\@####2####3\bb@array@aux####4{%
202         \def#1{####1\bb@elt#2\@{####4}####3}}%
203       \expandafter\bb@array@aux#1\bb@array@aux{#3}%
204     \fi}%
205   \expandafter\bb@array@aux#1\bb@elt#2\@bb@noelt\bb@array@aux{#3}}
206 \def\bb@array@get#1#2#3{%
207   \def\bb@array@aux##1\bb@elt#2\@##2##3\bb@array@aux{%
208     \ifx\@empty##2\let#3\relax\else\def#3{##2}\fi}%

```



```
209 \expandafter\bbbl@array@aux#1\bbbl@elt#2\@empty\bbbl@array@aux}
```

The following adds some code to `\extras...` both before and after, while avoiding doing it twice. It's somewhat convoluted, to deal with #'s. Used to deal with alph, Alph and frenchspacing when there are already changes (with `\babel@save`).

```
210 \def\bbbl@extras@wrap#1#2#3{% 1:in-test, 2:before, 3:after
211 \toks@\expandafter\expandafter\expandafter{%
212 \cscname extras\language\endcsname}%
213 \bbbl@exp{\in@{#1}{\the\toks@}}%
214 \ifin@else
215 \@temptokena{#2}%
216 \edef\bbbl@tempc{\the\@temptokena\the\toks@}%
217 \toks@\expandafter{\bbbl@tempc#3}%
218 \expandafter\edef\cscname extras\language\endcsname{\the\toks@}%
219 \fi}
220 {/Basic macros}}
```

Some files identify themselves with a $\TeX$ macro. The following code is placed before them to define (and then undefine) if not in $\TeX$.

```
221 {(*Make sure ProvidesFile is defined)} ≡
222 \ifx\ProvidesFile\@undefined
223 \def\ProvidesFile#1[#2 #3 #4]{%
224 \wlog{File: #1 #4 #3 <#2>}%
225 \let\ProvidesFile\@undefined}
226 \fi
227 {/Make sure ProvidesFile is defined}}
```

3.1. A few core definitions

\language Just for compatibility, for not to touch `hyphen.cfg`.

```
228 {(*Define core switching macros)} ≡
229 \ifx\language\@undefined
230 \cscname newcount\endcsname\language
231 \fi
232 {/Define core switching macros}}
```

\last@language Another counter is used to keep track of the allocated languages. $\TeX$ and $\LaTeX$ reserves for this purpose the count 19.

\addlanguage This macro was introduced for $\TeX < 2$. Preserved for compatibility.

```
233 {(*Define core switching macros)} ≡
234 \countdef\last@language=19
235 \def\addlanguage{\cscname newlanguage\endcsname}
236 {/Define core switching macros}}
```

Now we make sure all required files are loaded. When the command `\AtBeginDocument` doesn't exist we assume that we are dealing with a plain-based format. In that case the file `plain.def` is needed (which also defines `\AtBeginDocument`, and therefore it is not loaded twice). We need the first part when the format is created, and `\orig@dump` is used as a flag. Otherwise, we need to use the second part, so `\orig@dump` is not defined (`plain.def` undefines it).

Check if the current version of `switch.def` has been previously loaded (mainly, `hyphen.cfg`). If not, load it now. We cannot load `babel.def` here because we first need to declare and process the package options.

3.2. $\LaTeX$: `babel.sty` (start)

Here starts the style file for $\LaTeX$. It also takes care of a number of compatibility issues with other packages.

```
237 {*package}
238 \NeedsTeXFormat{LaTeX2e}
239 \ProvidesPackage{babel}%
240 [%@date@> v<@version@>
241 The multilingual framework for LuaLaTeX, pdfLaTeX and XeLaTeX]
```

Start with some “private” debugging tools, and then define macros for errors. The global lua ‘space’ Babel is declared here, too (inside the test for debug).

```

242 \ifpackagewith{babel}{debug}
243 {\providecommand\bbl@trace[1]{\message{^^J[ #1 ]}}%
244 \let\bbl@debug\@firstofone
245 \ifx\directlua\@undefined\else
246 \directlua{
247     Babel = Babel or {}
248     Babel.debug = true }%
249 \input{babel-debug.tex}%
250 \fi}
251 {\providecommand\bbl@trace[1]{}%
252 \let\bbl@debug\@gobble
253 \ifx\directlua\@undefined\else
254 \directlua{
255     Babel = Babel or {}
256     Babel.debug = false }%
257 \fi}

```

Macros to deal with errors, warnings, etc. Errors are stored in a separate file.

```

258 \def\bbl@error#1{% Implicit #2#3#4
259 \begingroup
260 \catcode`\=0 \catcode`\==12 \catcode`\`=12
261 \input errbabel.def
262 \endgroup
263 \bbl@error{#1}}
264 \def\bbl@warning#1{%
265 \begingroup
266 \def\{\MessageBreak}%
267 \PackageWarning{babel}{#1}%
268 \endgroup}
269 \def\bbl@infowarn#1{%
270 \begingroup
271 \def\{\MessageBreak}%
272 \PackageNote{babel}{#1}%
273 \endgroup}
274 \def\bbl@info#1{%
275 \begingroup
276 \def\{\MessageBreak}%
277 \PackageInfo{babel}{#1}%
278 \endgroup}

```

Many of the following options don’t do anything themselves, they are just defined in order to make it possible for babel and language definition files to check if one of them was specified by the user.

But first, include here the *Basic macros* defined above.

```

279 <@Basic macros>
280 \ifpackagewith{babel}{silent}
281 {\let\bbl@info\@gobble
282 \let\bbl@infowarn\@gobble
283 \let\bbl@warning\@gobble}
284 {}
285 %
286 \def\AfterBabelLanguage#1{%
287 \global\expandafter\bbl@add\csname#1.ldf-h@k\endcsname}%

```

If the format created a list of loaded languages (in \bbl@languages), get the name of the 0-th to show the actual language used. Also available with base, because it just shows info.

```

288 \ifx\bbl@languages\@undefined\else
289 \begingroup
290 \catcode`\^^I=12
291 \@ifpackagewith{babel}{showlanguages}{%
292 \begingroup
293 \def\bbl@elt#1#2#3#4{\wlog{#2^^I#1^^I#3^^I#4}}%

```

```

294      \wlog{<*languages>}%
295      \bbl@languages
296      \wlog{</languages>}%
297      \endgroup{}}
298  \endgroup
299  \def\bbl@elt#1#2#3#4{%
300    \ifnum#2=\z@
301      \gdef\bbl@nulllanguage{#1}%
302      \def\bbl@elt##1##2##3##4{%
303        \fi}%
304    \bbl@languages
305  \fi

```

3.3. base

The first ‘real’ option to be processed is base, which set the hyphenation patterns then resets `ver@babel.sty` so that $\TeX$ forgets about the first loading. After a subset of `babel.def` has been loaded (the old `switch.def`) and `\AfterBabelLanguage` defined, it exits.

Now the base option. With it we can define (and load, with `luatex`) hyphenation patterns, even if we are not interested in the rest of `babel`.

```

306 \bbl@trace{Defining option 'base'}
307 \ifpackagewith{babel}{base}{%
308   \let\bbl@onlyswitch\@empty
309   \let\bbl@provide@locale\relax
310   \input babel.def
311   \let\bbl@onlyswitch\@undefined
312   \ifx\directlua\@undefined
313     \DeclareOption*{\bbl@patterns{\CurrentOption}}%
314   \else
315     \input luababel.def
316     \DeclareOption*{\bbl@patterns@lua{\CurrentOption}}%
317   \fi
318   \DeclareOption{base}{}%
319   \DeclareOption{showlanguages}{}%
320   \ProcessOptions
321   \global\expandafter\let\csname opt@babel.sty\endcsname\relax
322   \global\expandafter\let\csname ver@babel.sty\endcsname\relax
323   \global\let\@ifl@ter@\@ifl@ter
324   \def\@ifl@ter#1#2#3#4#5{\global\let\@ifl@ter\@ifl@ter@}%
325   \endinput}}%

```

3.4. key=value options and other general option

The following macros extract language modifiers, and only real package options are kept in the option list. Modifiers are saved and assigned to `\BabelModifiers` at `\bbl@load@language`; when no modifiers have been given, the former is `\relax`.

```

326 \bbl@trace{key=value and another general options}
327 \bbl@csarg\let{tempa\expandafter}\csname opt@babel.sty\endcsname
328 \def\bbl@tempb#1.#2{% Removes trailing dot
329   #1\ifx\@empty#2\else,\bbl@afterfi\bbl@tempb#2\fi}%
330 \def\bbl@tempe#1=#2\@{
331   \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}}
332 \def\bbl@tempd#1.#2\@nnil{%
333   \ifx\@empty#2%
334     \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
335   \else
336     \in@{,provide=}{, #1}%
337     \ifin@
338       \edef\bbl@tempc{%
339         \ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.\bbl@tempb#2}%
340     \else
341       \in@{$modifiers$}{$#1$}%

```

```

342 \ifin@
343 \bbl@tempe#2\@@
344 \else
345 \in@{=}{#1}%
346 \ifin@
347 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1.#2}%
348 \else
349 \edef\bbl@tempc{\ifx\bbl@tempc\@empty\else\bbl@tempc,\fi#1}%
350 \bbl@csarg\edef{mod@#1}{\bbl@tempb#2}%
351 \fi
352 \fi
353 \fi
354 \fi}
355 \let\bbl@tempc\@empty
356 \bbl@foreach\bbl@tempa{\bbl@tempd#1.\@empty\@nnil}
357 \expandafter\let\csname opt@babel.sty\endcsname\bbl@tempc

```

The next option tells babel to leave shorthand characters active at the end of processing the package. This is *not* the default as it can cause problems with other packages, but for those who want to use the shorthand characters in the preamble of their documents this can help.

```

358 \DeclareOption{KeepShorthandsActive}{}
359 \DeclareOption{activeacute}{}
360 \DeclareOption{activegrave}{}
361 \DeclareOption{debug}{}
362 \DeclareOption{debug-german}{} % Temporary
363 \DeclareOption{noconfigs}{}
364 \DeclareOption{showlanguages}{}
365 \DeclareOption{silent}{}
366 \DeclareOption{shorthands=off}{\bbl@tempa shorthands=\bbl@tempa}
367 \chardef\bbl@iniflag\z@
368 \DeclareOption{provide=*}{\chardef\bbl@iniflag\@ne} % main = 1
369 \DeclareOption{provide+=*}{\chardef\bbl@iniflag\tw@} % second = 2
370 \DeclareOption{provide*=*}{\chardef\bbl@iniflag\thr@@} % second + main
371 \chardef\bbl@ldfflag\z@
372 \DeclareOption{provide=!}{\chardef\bbl@ldfflag\@ne} % main = 1
373 \DeclareOption{provide+=!}{\chardef\bbl@ldfflag\tw@} % second = 2
374 \DeclareOption{provide*=!}{\chardef\bbl@ldfflag\thr@@} % second + main
375 \DeclareOption{licr=extended}{}
376 \DeclareOption{licr=unextended}{}
377 % Don't use. Experimental.
378 \newif\ifbbl@single
379 \DeclareOption{selectors=off}{\bbl@singletrue}
380 <@More package options@>

```

Handling of package options is done in three passes. (I [JBL] am not very happy with the idea, anyway.) The first one processes options which has been declared above or follow the syntax $\langle key \rangle = \langle value \rangle$, the second one loads the requested languages, except the main one if set with the key main, and the third one loads the latter. First, we “flag” valid keys with a nil value.

```

381 \let\bbl@opt@shorthands\@nnil
382 \let\bbl@opt@config\@nnil
383 \let\bbl@opt@main\@nnil
384 \let\bbl@opt@headfoot\@nnil
385 \let\bbl@opt@layout\@nnil
386 \let\bbl@opt@provide\@nnil

```

The following tool is defined temporarily to store the values of options.

```

387 \def\bbl@tempa#1=#2\bbl@tempa{%
388 \bbl@csarg\ifx{opt@#1}\@nnil
389 \bbl@csarg\edef{opt@#1}{#2}%
390 \else
391 \bbl@error{bad-package-option}{#1}{#2}{}%
392 \fi}

```

Now the option list is processed, taking into account only currently declared options (including those declared with a =), and $\langle key \rangle = \langle value \rangle$ options (the former take precedence). Unrecognized options are saved in `\bbl@language@opts`, because they are language options.

```

393 \let\bbl@language@opts\@empty
394 \DeclareOption*{%
395   \bbl@xin@{\string=}{\CurrentOption}%
396   \ifin@
397     \expandafter\bbl@tempa\CurrentOption\bbl@tempa
398   \else
399     \bbl@add@list\bbl@language@opts{\CurrentOption}%
400   \fi}

```

Now we finish the first pass (and start over).

```

401 \ProcessOptions*

```

3.5. Post-process some options

```

402 \ifx\bbl@opt@provide\@nnil
403   \let\bbl@opt@provide\@empty %%% MOVE above
404 \else
405   \chardef\bbl@iniflag\@ne
406   \bbl@exp{\\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
407     \in@{,provide,}{, #1,}%
408     \ifin@
409       \def\bbl@opt@provide{#2}%
410     \fi}
411 \fi

```

If there is no `shorthands=` (*chars*), the original babel macros are left untouched, but if there is, these macros are wrapped (in `babel.def`) to define only those given.

A bit of optimization: if there is no `shorthands=`, then `\bbl@ifshorthand` is always true, and it is always false if `shorthands` is empty. Also, some code makes sense only with `shorthands=...`

```

412 \bbl@trace{Conditional loading of shorthands}
413 \def\bbl@sh@string#1{%
414   \ifx#1\@empty\else
415     \ifx#1t\string~%
416     \else\ifx#1c\string,%
417     \else\string#1%
418     \fi\fi
419     \expandafter\bbl@sh@string
420   \fi}
421 \ifx\bbl@opt@shorthands\@nnil
422   \def\bbl@ifshorthand#1#2#3{#2}%
423 \else\ifx\bbl@opt@shorthands\@empty
424   \def\bbl@ifshorthand#1#2#3{#3}%
425 \else

```

The following macro tests if a shorthand is one of the allowed ones.

```

426 \def\bbl@ifshorthand#1{%
427   \bbl@xin@{\string#1}{\bbl@opt@shorthands}%
428   \ifin@
429     \expandafter\@firstoftwo
430   \else
431     \expandafter\@secondoftwo
432   \fi}

```

We make sure all chars in the string are ‘other’, with the help of an auxiliary macro defined above (which also zaps spaces).

```

433 \edef\bbl@opt@shorthands{%
434   \expandafter\bbl@sh@string\bbl@opt@shorthands\@empty}%

```

The following is ignored with `shorthands=off`, since it is intended to take some additional actions for certain chars.

```

435 \bbl@ifshorthand{'}%
436   {\PassOptionsToPackage{activeacute}{babel}}{}

```

```

437 \bbl@ifshorthand{`}%
438 {\PassOptionsToPackage{activegrave}{babel}}{}
439 \fi\fi

```

With `headfoot=lang` we can set the language used in heads/feet. For example, in `babel/3796` just add `headfoot=english`. It misuses `\resetactivechars`, but seems to work.

```

440 \ifx\bbl@opt@headfoot\@nnil\else
441 \g@addto@macro\@resetactivechars{%
442 \set@typeset@protect
443 \expandafter\select@language@x\expandafter{\bbl@opt@headfoot}%
444 \let\protect\noexpand}
445 \fi

```

For the option `safe` we use a different approach – `\bbl@opt@safe` says which macros are redefined (B for bibs and R for refs). By default, both are currently set, but in a future release it will be set to none.

```

446 \ifx\bbl@opt@safe\@undefined
447 \def\bbl@opt@safe{BR}
448 % \let\bbl@opt@safe\@empty % Pending of \cite
449 \fi

```

For layout an auxiliary macro is provided, available for packages and language styles. Optimization: if there is no layout, just do nothing.

```

450 \bbl@trace{Defining IfBabelLayout}
451 \ifx\bbl@opt@layout\@nnil
452 \newcommand\IfBabelLayout[3]{#3}%
453 \else
454 \bbl@exp{\bbl@forkv{\@nameuse{@raw@opt@babel.sty}}}{%
455 \in@{,layout,}{, #1,}%
456 \ifin@
457 \def\bbl@opt@layout{#2}%
458 \bbl@replace\bbl@opt@layout{ }{.}%
459 \fi}
460 \newcommand\IfBabelLayout[1]{%
461 \@expandtwoargs\in@{.#1.}{.\bbl@opt@layout.}%
462 \ifin@
463 \expandafter\@firstoftwo
464 \else
465 \expandafter\@secondoftwo
466 \fi}
467 \fi
468 \<package>

```

3.6. Plain: `babel.def` (start)

Because of the way `docstrip` works, we need to insert some code for Plain here. However, the tools provided by the `babel` installer for literate programming makes this section a short interlude, because the actual code is below, tagged as *Emulate LaTeX*.

First, exit immediately if previously loaded.

```

469 \<core>
470 \ifx\ldf@quit\@undefined\else
471 \endinput\fi % Same line!
472 \<@Make sure ProvidesFile is defined@>
473 \ProvidesFile{babel.def}[\<@date@> v\<@version@> Babel common definitions]
474 \ifx\AtBeginDocument\@undefined
475 \<@Emulate LaTeX@>
476 \fi
477 \<@Basic macros@>
478 \</core>

```

That is all for the moment. Now follows some common stuff, for both Plain and $\text{\LaTeX}$. After it, we will resume the $\text{\LaTeX}$ -only stuff.

4. `babel.sty` and `babel.def` (common)

```

479 (*package|core)
480 \def\bbl@version{<@version>}
481 \def\bbl@date{<@date>}
482 <@Define core switching macros>

```

\adddialect The macro `\adddialect` can be used to add the name of a dialect or variant language, for which an already defined hyphenation table can be used.

```

483 \def\adddialect#1#2{%
484   \global\chardef#1#2\relax
485   \bbl@usehooks{adddialect}{#1}{#2}}%
486 \begingroup
487   \count@#1\relax
488   \def\bbl@elt##1##2##3##4{%
489     \ifnum\count@=##2\relax
490       \edef\bbl@tempa{\expandafter\@gobbletwo\string#1}%
491       \bbl@info{Hyphen rules for '\expandafter\@gobble\bbl@tempa'
492         set to \expandafter\string\csname l@##1\endcsname\%
493         (\string\language\the\count@). Reported}%
494       \def\bbl@elt####1####2####3####4{%
495         \fi}%
496       \bbl@cs{languages}%
497     \endgroup

```

`\bbl@iflanguage` executes code only if the language `l@` exists. Otherwise raises an error.

The argument of `\bbl@fixname` has to be a macro name, as it may get “fixed” if casing (lc/uc) is wrong. It’s an attempt to fix a long-standing bug when `\foreignlanguage` and the like appear in a `\MakeXXXcase`. However, a lowercase form is not imposed to improve backward compatibility (perhaps you defined a language named MYLANG, but unfortunately mixed case names cannot be trapped). Note `l@` is encapsulated, so that its case does not change.

```

498 \def\bbl@fixname#1{%
499   \begingroup
500   \def\bbl@tempe{l@}%
501   \edef\bbl@tempd{\noexpand\@ifundefined{\noexpand\bbl@tempe#1}}%
502   \bbl@tempd
503     {\lowercase\expandafter{\bbl@tempd}%
504      {\uppercase\expandafter{\bbl@tempd}%
505       \@empty
506        {\edef\bbl@tempd{\def\noexpand#1{#1}}%
507         \uppercase\expandafter{\bbl@tempd}}}%
508       {\edef\bbl@tempd{\def\noexpand#1{#1}}%
509        \lowercase\expandafter{\bbl@tempd}}}%
510     \@empty
511     \edef\bbl@tempd{\endgroup\def\noexpand#1{#1}}%
512   \bbl@tempd
513   \bbl@expf{\bbl@usehooks{language}{\language}{#1}}}%
514 \def\bbl@iflanguage#1{%
515   \@ifundefined{l@#1}{\@nolanerr{#1}\@gobble}\@firstofone}

```

After a name has been ‘fixed’, the selectors will try to load the language. If even the fixed name is not defined, will load it on the fly, either based on its name, or if activated, its BCP 47 code.

We first need a couple of macros for a simple BCP 47 look up. It also makes sure, casing is the usual one, so that `sr-latn-ba` becomes `fr-Latn-BA`.

`\bbl@bcpllookup` returns `\bbl@bcp` with either the found ini tag or `\relax`. Temporary version, to be refactored with the new `\text_bcp_parse:n`, which is not currently fully expandable with invalid BCP 47 tags. The argument may be in some cases either a language name or a tag (e.g., `\foreignlanguage`. This was a huge mistake, because of the potential ambiguities (for example, the name `vai-latin` is also a syntactically valid tag).

```

516 \def\bbl@cntchars#1{\bbl@cntchars@i#1876543210\@@}
517 \def\bbl@cntchars@i#1#2#3#4#5#6#7#8#9\@@{\@car#9\@nil}%
518 \def\bbl@bcpllookup#1{%
519   \let\bbl@templ\@empty % language
520   \let\bbl@temps\@empty % script
521   \let\bbl@tempr\@empty % regions
522   \let\bbl@tempv\@empty % variant

```

```

523 \edef\bbbl@tempa{#1}%
524 \bbbl@replace\bbbl@tempa{-}{,}%
525 \bbbl@replace\bbbl@tempa_{}{,}%
526 \bbbl@foreach\bbbl@tempa{%
527   \ifx\bbbl@templ\@empty
528     \lowercase{\def\bbbl@templ{##1}}%
529   \else\ifnum\bbbl@cntchars{##1}=4
530     \bbbl@xin@{\@car##1\@nil}{0123456789}%
531     \ifin@
532       \def\bbbl@tempv{##1}% eg, 1901
533     \else
534       \uppercase{\edef\bbbl@tempb{\@car##1\@nil}}%
535       \lowercase{\edef\bbbl@tempb{\bbbl@tempb\@gobble##1}}%
536     \fi
537   \else\ifnum\bbbl@cntchars{##1}<4
538     \uppercase{\def\bbbl@tempv{##1}}%
539   \else\ifnum\bbbl@cntchars{##1}>4
540     \lowercase{\def\bbbl@tempv{##1}}%
541   \fi\fi\fi\fi}%
542 \bbbl@expf{\bbbl@bcpllookup@i{\bbbl@templ}{\bbbl@tempb}{\bbbl@tempv}}%
543 \def\bbbl@bcpllookup@try#1{%
544   \ifx\bbbl@bcp\relax
545     \IfFileExists{babel-#1\bbbl@tempe.ini}{\edef\bbbl@bcp{#1\bbbl@tempe}}{}%
546   \fi}
547 \def\bbbl@bcpllookup@i#1#2#3#4{%
548   \let\bbbl@bcp\relax
549   \def\bbbl@tempe{#4}%
550   \ifx\bbbl@tempe\@empty\else
551     \edef\bbbl@tempe{-#4}%
552   \fi
553   \edef\bbbl@tempa{#2#3\bbbl@tempe}% en
554   \ifx\bbbl@tempa\@empty
555     \bbbl@bcpllookup@try{#1}%
556   \else
557     \edef\bbbl@tempa{#3\bbbl@tempe}% en-Latn
558     \ifx\bbbl@tempa\@empty
559       \bbbl@bcpllookup@try{#1-#2}%
560       \bbbl@bcpllookup@try{#1}%
561     \else
562       \edef\bbbl@tempa{#2\bbbl@tempe}% en-US, en-001
563       \ifx\bbbl@tempa\@empty
564         \bbbl@bcpllookup@try{#1-#3}%
565         \bbbl@bcpllookup@try{#1}%
566       \else % en-Latn-US
567         \bbbl@bcpllookup@try{#1-#2-#3}%
568         \bbbl@bcpllookup@try{#1-#2}%
569         \bbbl@bcpllookup@try{#1-#3}%
570         \bbbl@bcpllookup@try{#1}%
571       \fi
572     \fi
573   \fi
574   \ifx\bbbl@bcp\relax
575     \ifx\bbbl@tempe\@empty\else\bbbl@bcpllookup@i{#1}{#2}{#3}\fi%
576   \fi}
577 %
578 \let\bbbl@initoload\relax

```

\iflanguage Users might want to test (in a private package for instance) which language is currently active. For this we provide a test macro, `\iflanguage`, that has three arguments. It checks whether the first argument is a known language. If so, it compares the first argument with the value of `\language`. Then, depending on the result of the comparison, it executes either the second or the third argument.

```

579 \def\iflanguage#1{%

```



```

580 \bbl@iflanguage{#1}{%
581   \ifnum\csname l@#1\endcsname=\language
582     \expandafter\@firstoftwo
583   \else
584     \expandafter\@secondoftwo
585   \fi}}

```

4.1. Selecting the language

\selectlanguage It checks whether the language is already defined before it performs its actual task, which is to update `\language` and activate language-specific definitions.

```

586 \let\bbl@select@type\z@
587 \edef\selectlanguage{%
588   \noexpand\protect
589   \expandafter\noexpand\csname selectlanguage \endcsname}

```

Because the command `\selectlanguage` could be used in a moving argument it expands to `\protect\selectlanguage_`. Therefore, we have to make sure that a macro `\protect` exists. If it doesn't it is `\let` to `\relax`.

```

590 \ifx\@undefined\protect\let\protect\relax\fi

```

The following definition is preserved for backwards compatibility (e.g., `arabi`, `koma`). It is related to a trick for 2.09, now discarded.

```

591 \let\xstring\string

```

Since version 3.5 `babel` writes entries to the auxiliary files in order to typeset table of contents etc. in the correct language environment.

\bbl@pop@language But when the language change happens *inside* a group the end of the group doesn't write anything to the auxiliary files. Therefore we need TeX's `aftergroup` mechanism to help us. The command `\aftergroup` stores the token immediately following it to be executed when the current group is closed. So we define a temporary control sequence `\bbl@pop@language` to be executed at the end of the group. It calls `\bbl@set@language` with the name of the current language as its argument.

\bbl@language@stack The previous solution works for one level of nesting groups, but as soon as more levels are used it is no longer adequate. For that case we need to keep track of the nested languages using a stack mechanism. This stack is called `\bbl@language@stack` and initially empty.

```

592 \def\bbl@language@stack{}

```

When using a stack we need a mechanism to push an element on the stack and to retrieve the information afterwards.

\bbl@push@language

\bbl@pop@language The stack is simply a list of languagenames, separated with a '+' sign; the push function can be simple:

```

593 \def\bbl@push@language{%
594   \ifx\language\@undefined\else
595     \ifx\currentgrouplevel\@undefined
596       \xdef\bbl@language@stack{\language+\bbl@language@stack}%
597     \else
598       \ifnum\currentgrouplevel=\z@
599         \xdef\bbl@language@stack{\language+}%
600       \else
601         \xdef\bbl@language@stack{\language+\bbl@language@stack}%
602       \fi
603     \fi
604   \fi}

```

Retrieving information from the stack is a little bit less simple, as we need to remove the element from the stack while storing it in the macro `\language`. For this we first define a helper function.

\bbl@pop@lang This macro stores its first element (which is delimited by the ‘+’-sign) in \language and stores the rest of the string in \bbl@language@stack.

```
605 \def\bbl@pop@lang#1+#2\@@{%
606   \edef\language{#1}%
607   \xdef\bbl@language@stack{#2}}
```

The reason for the somewhat weird arrangement of arguments to the helper function is the fact it is called in the following way. This means that before \bbl@pop@lang is executed T_EX first *expands* the stack, stored in \bbl@language@stack. The result of that is that the argument string of \bbl@pop@lang contains one or more language names, each followed by a ‘+’-sign (zero language names won’t occur as this macro will only be called after something has been pushed on the stack).

```
608 \let\bbl@ifrestoring\@secondoftwo
609 \def\bbl@pop@language{%
610   \expandafter\bbl@pop@lang\bbl@language@stack\@@
611   \let\bbl@ifrestoring\@firstoftwo
612   \expandafter\bbl@set@language\expandafter{\language}%
613   \let\bbl@ifrestoring\@secondoftwo}
```

Once the name of the previous language is retrieved from the stack, it is fed to \bbl@set@language to do the actual work of switching everything that needs switching.

An alternative way to identify languages (in the babel sense) with a numerical value is introduced in 3.30. This is one of the first steps for a new interface based on the concept of locale, which explains the name of \localeid. This means \l@. . . will be reserved for hyphenation patterns (so that two locales can share the same rules).

```
614 \chardef\localeid\z@
615 \gdef\bbl@id@last{0}    % No real need for a new counter
616 \def\bbl@id@assign{%
617   \bbl@ifunset\bbl@id@\language}%
618   {\count@\bbl@id@last\relax
619    \advance\count@\@ne
620    \global\bbl@csarg\chardef{id@\language}\count@
621    \xdef\bbl@id@last{\the\count@}%
622    \ifcase\bbl@engine\or
623      \directlua{
624        Babel.locale_props[\bbl@id@last] = {}
625        Babel.locale_props[\bbl@id@last].name = '\language'
626        Babel.locale_props[\bbl@id@last].vars = {}
627      }%
628    \fi}%
629   }%
630   \chardef\localeid\bbl@c{id@}}
```

The unprotected part of \selectlanguage. In case it is used as environment, declare \endselectlanguage, just for safety.

```
631 \let\bbl@select@opts\@empty
632 \expandafter\def\csname selectlanguage \endcsname{%
633   \@ifnextchar[\bbl@select@s{\bbl@select@s[]}}
634 \def\bbl@select@s[#1]#2{%
635   \def\bbl@select@opts{#1}%
636   \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\tw\fi
637   \bbl@push@language
638   \aftergroup\bbl@pop@language
639   \bbl@set@language{#2}}
640 \let\endselectlanguage\relax
```

\bbl@set@language The macro \bbl@set@language takes care of switching the language environment *and* of writing entries on the auxiliary files. For historical reasons, language names can be either language of \language. To catch either form a trick is used, but unfortunately as a side effect the catcodes of letters in \language are messed up. This is a bug, but preserved for backwards compatibility. The list of auxiliary files can be extended by redefining \BabelContentsFiles, but make sure they are loaded inside a group (as aux, toc, lof, and lot do) or the last language of the document will remain active afterwards.

We also write a command to change the current language in the auxiliary files.

`\bbl@savelastskip` is used to deal with skips before the write whatsit (as suggested by U Fischer). Adapted from `hyperref`, but it might fail, so I'll consider it a temporary hack, while I study other options (the ideal, but very likely unfeasible except perhaps in `luatex`, is to avoid the `\write` altogether when not needed).

```

641 \def\BabelContentsFiles{toc,lof,lot}
642 \def\bbl@set@language#1{% from selectlanguage, pop@
643 % The old buggy way. Preserved for compatibility, but simplified
644 \edef\language{\expandafter\string#1\@empty}%
645 \select@language{\language}%
646 \bbl@xin@{,main,},{, \bbl@select@opts,}%
647 \ifin@
648 \let\bbl@main@language\localename
649 \let\mainlocalename\localename
650 \fi
651 \let\bbl@select@opts\@empty
652 % write to aux files
653 \expandafter\ifx\csname date\language\endcsname\relax\else
654 \if@filesw
655 \bbl@xin@{,nofiles,},{, \bbl@select@opts,}%
656 \ifin@\else
657 \ifx\babel@aux\@gobbletwo\else % Set if single in the first, redundant
658 \bbl@savelastskip
659 \protected@write\@auxout{\string\babel@aux{\bbl@auxname}{}}%
660 \bbl@restorelastskip
661 \fi
662 \bbl@usehooks{write}{}%
663 \fi
664 \fi
665 \fi}
666 %
667 \let\bbl@restorelastskip\relax
668 \let\bbl@savelastskip\relax
669 %
670 \def\select@language#1{% from set@, babel@aux, babel@toc
671 \ifx\bbl@select@name\@empty
672 \def\bbl@select@name{select}%
673 \fi
674 % set hmap
675 \ifnum\bbl@hmapsel=\@ccclv\chardef\bbl@hmapsel4\relax\fi
676 % set name (when coming from babel@aux)
677 \edef\language{#1}%
678 \bbl@fixname\language
679 % define \localename when coming from set@, with a trick
680 \ifx\scantokens\undefined
681 \def\localename{??}%
682 \else
683 \bbl@exp{\scantokens{\def\localename{\language}\noexpand}\relax}%
684 \fi
685 \bbl@provide@locale
686 \bbl@iflanguage\language{%
687 \let\bbl@select@type\z@
688 \expandafter\bbl@switch\expandafter{\language}}
689 \def\babel@aux#1#2{%
690 \select@language{#1}%
691 \bbl@foreach\BabelContentsFiles{% \relax -> don't assume vertical mode
692 \writefile{##1}{\babel@toc{#1}{#2}\relax}}%
693 \def\babel@toc#1#2{%
694 \select@language{#1}}

```

First, check if the user asks for a known language. If so, update the value of `\language` and call `\originalTeX` to bring `TEX` in a certain pre-defined state.

The name of the language is stored in the control sequence `\language`.

Then we have to *redefine* `\originalTeX` to compensate for the things that have been activated. To

save memory space for the macro definition of `\originalTeX`, we construct the control sequence name for the `\noextras<language>` command at definition time by expanding the `\csname` primitive.

Now activate the language-specific definitions. This is done by constructing the names of three macros by concatenating three words with the argument of `\selectlanguage`, and calling these macros.

The switching of the values of `\lefthyphenmin` and `\righthyphenmin` is somewhat different. First we save their current values, then we check if `\<language>hyphenmins` is defined. If it is not, we set default values (2 and 3), otherwise the values in `\<language>hyphenmins` will be used.

No text is supposed to be added with switching captions and date, so we remove any spurious spaces with `\bbl@bsphack` and `\bbl@esphack`.

```

695 \newif\ifbbl@usedategroup
696 \let\bbl@savextras\@empty
697 \def\bbl@switch#1{% from select@, foreign@
698   % restore
699   \originalTeX
700   \expandafter\def\expandafter\originalTeX\expandafter{%
701     \csname noextras#1\endcsname
702     \let\originalTeX\@empty
703     \babel@beginsave}%
704   \bbl@usehooks{afterreset}{}%
705   \languageshorthands{none}%
706   % set the locale id
707   \bbl@id@assign
708   % switch captions, date
709   \bbl@bsphack
710   \ifcase\bbl@select@type
711     \csname captions#1\endcsname\relax
712     \csname date#1\endcsname\relax
713   \else
714     \bbl@xin@{,captions,}{,\bbl@select@opts,}%
715     \ifin@
716       \csname captions#1\endcsname\relax
717     \fi
718     \bbl@xin@{,date,}{,\bbl@select@opts,}%
719     \ifin@ % if \foreign... within \<language>date
720       \csname date#1\endcsname\relax
721     \fi
722   \fi
723   \bbl@esphack
724   % switch extras
725   \csname bbl@preextras@#1\endcsname
726   \bbl@usehooks{beforeextras}{}%
727   \csname extras#1\endcsname\relax
728   \bbl@usehooks{afterextras}{}%
729   % > babel-ensure
730   % > babel-sh-<short>
731   % > babel-bidi
732   % > babel-fontspec
733   \let\bbl@savextras\@empty
734   % hyphenation - case mapping
735   \ifcase\bbl@opt@hyphenmap\or
736     \def\BabelLower##1##2{\lccode##1=##2\relax}%
737     \ifnum\bbl@hymapsel>4\else
738       \csname\language @bbl@hyphenmap\endcsname
739     \fi
740     \chardef\bbl@opt@hyphenmap\z@
741   \else
742     \ifnum\bbl@hymapsel>\bbl@opt@hyphenmap\else
743       \csname\language @bbl@hyphenmap\endcsname
744     \fi
745   \fi
746   \let\bbl@hymapsel\@cclv
747   % hyphenation - select rules

```

```

748 \ifnum\csname l@\language\endcsname=\l@unhyphenated
749 \edef\bbl@tempa{u}%
750 \else
751 \edef\bbl@tempa{\bbl@cl{\lnbrk}}%
752 \fi
753 % linebreaking - handle u, e, k (v in the future)
754 \bbl@xin@{/u}{/\bbl@tempa}%
755 \ifin@else\bbl@xin@{/e}{/\bbl@tempa}\fi % elongated forms
756 \ifin@else\bbl@xin@{/k}{/\bbl@tempa}\fi % only kashida
757 \ifin@else\bbl@xin@{/p}{/\bbl@tempa}\fi % padding (e.g., Tibetan)
758 \ifin@else\bbl@xin@{/v}{/\bbl@tempa}\fi % variable font
759 % hyphenation - save mins
760 \babel@savevariable\lefthyphenmin
761 \babel@savevariable\righthyphenmin
762 \ifnum\bbl@engine=\@ne
763 \babel@savevariable\hyphenationmin
764 \fi
765 \ifin@
766 % unhyphenated/kashida/elongated/padding = allow stretching
767 \language\l@unhyphenated
768 \babel@savevariable\emergencystretch
769 \emergencystretch\maxdimen
770 \babel@savevariable\hbadness
771 \hbadness\@M
772 \else
773 % other = select patterns
774 \bbl@patterns{#1}%
775 \fi
776 % hyphenation - set mins
777 \expandafter\ifx\csname #1hyphenmins\endcsname\relax
778 \set@hyphenmins\tw@\thr@@\relax
779 \@nameuse{\bbl@hyphenmins}%
780 \else
781 \expandafter\expandafter\expandafter\set@hyphenmins
782 \csname #1hyphenmins\endcsname\relax
783 \fi
784 \@nameuse{\bbl@hyphenmins}%
785 \@nameuse{\bbl@hyphenmins@\language}%
786 \@nameuse{\bbl@hyphenatmin}%
787 \@nameuse{\bbl@hyphenatmin@\language}%
788 \let\bbl@selectorname\@empty}

```

otherlanguage It can be used as an alternative to using the `\selectlanguage` declarative command. The `\ignorespaces` command is necessary to hide the environment when it is entered in horizontal mode.

```

789 \edef\otherlanguage{%
790 \noexpand\protect
791 \expandafter\noexpand\csname otherlanguage \endcsname}
792 \expandafter\def\csname otherlanguage \endcsname{%
793 \@ifstar{\@nameuse{otherlanguage*}}{\bbl@otherlanguage}
794 \def\bbl@otherlanguage#1{%
795 \def\bbl@selectorname{other}%
796 \ifnum\bbl@hymapsel=\@cclv\let\bbl@hymapsel\thr@@\fi
797 \csname selectlanguage \endcsname{#1}%
798 \ignorespaces}

```

The `\endotherlanguage` part of the environment tries to hide itself when it is called in horizontal mode.

```

799 \long\def\endotherlanguage{\@ignoretrue\ignorespaces}

```

otherlanguage* It is meant to be used when a large part of text from a different language needs to be typeset, but without changing the translation of words such as ‘figure’. It makes use of

`\foreign@language.`

```
800 \expandafter\def\csname otherlanguage*\endcsname{%
801   \ifnextchar[\bbl@otherlanguage@s{\bbl@otherlanguage@s[]}}
802 \def\bbl@otherlanguage@s[#1]#2{%
803   \def\bbl@selectorname{other*}%
804   \ifnum\bbl@hymapset=\@ccclv\chardef\bbl@hymapset4\relax\fi
805   \def\bbl@select@opts{#1}%
806   \foreign@language{#2}}
```

At the end of the environment we need to switch off the extra definitions. The grouping mechanism of the environment will take care of resetting the correct hyphenation rules and “extras”.

```
807 \expandafter\let\csname endotherlanguage*\endcsname\relax
```

\foreignlanguage This command takes two arguments, the first argument is the name of the language to use for typesetting the text specified in the second argument.

Unlike `\selectlanguage` this command doesn’t switch *everything*, it only switches the hyphenation rules and the extra definitions for the language specified. It does this within a group and assumes the `\extras<language>` command doesn’t make any `\global` changes. The coding is very similar to part of `\selectlanguage`.

`\bbl@beforeforeign` is a trick to fix a bug in bidi texts. `\foreignlanguage` is supposed to be a ‘text’ command, and therefore it must emit a `\leavevmode`, but it does not, and therefore the indent is placed on the opposite margin. For backward compatibility, however, it is done only if a right-to-left script is requested; otherwise, it is no-op.

(3.11) `\foreignlanguage*` is a temporary, experimental macro for a few lines with a different script direction, while preserving the paragraph format (thank the braces around `\par`, things like `\hangindent` are not reset). Do not use it in production, because its semantics and its syntax may change (and very likely will, or even it could be removed altogether). Currently it enters in vmode and then selects the language (which in turn sets the paragraph direction).

(3.11) Also experimental are the hook `foreign` and `foreign*`. With them you can redefine `\BabelText` which by default does nothing. Its behavior is not well defined yet. So, use it in horizontal mode only if you do not want surprises.

In other words, at the beginning of a paragraph `\foreignlanguage` enters into hmode with the surrounding lang, and with `\foreignlanguage*` with the new lang.

```
808 \providecommand\bbl@beforeforeign{}
809 \edef\foreignlanguage{%
810   \noexpand\protect
811   \expandafter\noexpand\csname foreignlanguage \endcsname}
812 \expandafter\def\csname foreignlanguage \endcsname{%
813   \@ifstar\bbl@foreign@s\bbl@foreign@x}
814 \providecommand\bbl@foreign@x[3][]{%
815   \begingroup
816     \def\bbl@selectorname{foreign}%
817     \def\bbl@select@opts{#1}%
818     \let\BabelText\@firstofone
819     \bbl@beforeforeign
820     \foreign@language{#2}%
821     \bbl@usehooks{foreign}{}%
822     \BabelText{#3}% Now in horizontal mode!
823   \endgroup}
824 \def\bbl@foreign@s#1#2{%
825   \begingroup
826     {\par}%
827     \def\bbl@selectorname{foreign*}%
828     \let\bbl@select@opts\@empty
829     \let\BabelText\@firstofone
830     \foreign@language{#1}%
831     \bbl@usehooks{foreign*}{}%
832     \bbl@dirparastext
833     \BabelText{#2}% Still in vertical mode!
834   {\par}%
835   \endgroup}
836 \providecommand\BabelWrapText[1]{%
```

```

837 \def\bbl@tempa{\def\BabelText####1}%
838 \expandafter\bbl@tempa\expandafter{\BabelText{#1}}

```

\foreign@language This macro does the work for \foreignlanguage and the other language* environment. First we need to store the name of the language and check that it is a known language. Then it just calls bbl@switch.

```

839 \def\foreign@language#1{%
840 % set name
841 \edef\language#1%
842 \ifbbl@usedategroup
843 \bbl@add\bbl@select@opts{,date,}%
844 \bbl@usedategroupfalse
845 \fi
846 \bbl@fixname\language
847 \let\localename\language
848 \bbl@provide@locale
849 \bbl@iflanguage\language{%
850 \let\bbl@select@type@ne
851 \expandafter\bbl@switch\expandafter{\language}}

```

The following macro executes conditionally some code based on the selector being used.

```

852 \def\IfBabelSelectorTF#1{%
853 \bbl@xin{,\bbl@selectorname,}{,\zap@space#1 \@empty,}%
854 \ifin@
855 \expandafter\@firstoftwo
856 \else
857 \expandafter\@secondoftwo
858 \fi}

```

\bbl@patterns This macro selects the hyphenation patterns by changing the \language register. If special hyphenation patterns are available specifically for the current font encoding, use them instead of the default.

It also sets hyphenation exceptions, but only once, because they are global (here language \lccode's has been set, too). \bbl@hyphenation@ is set to relax until the very first \babelhyphenation, so do nothing with this value. If the exceptions for a language (by its number, not its name, so that :ENC is taken into account) has been set, then use \hyphenation with both global and language exceptions and empty the latter to mark they must not be set again.

```

859 \let\bbl@hyphlist\@empty
860 \let\bbl@hyphenation@\relax
861 \let\bbl@pttnlist\@empty
862 \let\bbl@patterns@\relax
863 \let\bbl@hymapsel=\@cclv
864 \def\bbl@patterns#1{%
865 \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
866 \csname l@#1\endcsname
867 \edef\bbl@tempa{#1}%
868 \else
869 \csname l@#1:\f@encoding\endcsname
870 \edef\bbl@tempa{#1:\f@encoding}%
871 \fi
872 \@expandtwoargs\bbl@usehooks{patterns}{#1}{\bbl@tempa}}%
873 % > luatex
874 \@ifundefined{bbl@hyphenation@}{}{% Can be \relax!
875 \begingroup
876 \bbl@xin{,\number\language,}{,\bbl@hyphlist}%
877 \ifin@\else
878 \@expandtwoargs\bbl@usehooks{hyphenation}{#1}{\bbl@tempa}}%
879 \hyphenation%
880 \bbl@hyphenation@
881 \@ifundefined{bbl@hyphenation@#1}%
882 \@empty
883 {\space\csname bbl@hyphenation@#1\endcsname}}%

```

```

884      \xdef\bbl@hyphlist{\bbl@hyphlist\number\language,}%
885      \fi
886      \endgroup}}

```

hyphenrules It can be used to select *just* the hyphenation rules. It does *not* change `\language` and when the hyphenation rules specified were not loaded it has no effect. Note however, `\lccode`'s and font encodings are not set at all, so in most cases you should use `otherlanguage*`.

```

887 \def\hyphenrules#1{%
888   \edef\bbl@tempf{#1}%
889   \bbl@fixname\bbl@tempf
890   \bbl@iflanguage\bbl@tempf{%
891     \expandafter\bbl@patterns\expandafter{\bbl@tempf}%
892     \ifx\languageshorthands\undefined\else
893       \languageshorthands{none}%
894     \fi
895     \expandafter\ifx\csname\bbl@tempf hyphenmins\endcsname\relax
896       \set@hyphenmins\tw@\thr@@\relax
897     \else
898       \expandafter\expandafter\expandafter\set@hyphenmins
899       \csname\bbl@tempf hyphenmins\endcsname\relax
900     \fi}}
901 \let\endhyphenrules\@empty

```

\providehyphenmins The macro `\providehyphenmins` should be used in the language definition files to provide a *default* setting for the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`. If the macro `\(language)hyphenmins` is already defined this command has no effect.

```

902 \def\providehyphenmins#1#2{%
903   \expandafter\ifx\csname #1hyphenmins\endcsname\relax
904     \@namedef{#1hyphenmins}{#2}%
905   \fi}

```

\set@hyphenmins This macro sets the values of `\lefthyphenmin` and `\righthyphenmin`. It expects two values as its argument.

```

906 \def\set@hyphenmins#1#2{%
907   \lefthyphenmin#1\relax
908   \righthyphenmin#2\relax}

```

\ProvidesLanguage The identification code for each file is something that was introduced in $\text{\LaTeX 2}_\epsilon$. When the command `\ProvidesFile` does not exist, a dummy definition is provided temporarily. For use in the language definition file the command `\ProvidesLanguage` is defined by `babel`.

Depending on the format, i.e., or if the former is defined, we use a similar definition or not.

```

909 \ifx\ProvidesFile\@undefined
910   \def\ProvidesLanguage#1[#2 #3 #4]{%
911     \wlog{Language: #1 #4 #3 <#2>}%
912   }
913 \else
914   \def\ProvidesLanguage#1{%
915     \begingroup
916       \catcode`\ 10 %
917       \@makeother\/%
918       \@ifnextchar[%]
919       {\@provideslanguage{#1}}{\@provideslanguage{#1}[]}
920   \def\@provideslanguage#1[#2]{%
921     \wlog{Language: #1 #2}%
922     \expandafter\xdef\csname ver@#1.ldf\endcsname{#2}%
923   \endgroup}
924 \fi

```


\originalTeX The macro `\originalTeX` should be known to $\TeX$ at this moment. As it has to be expandable we `\let` it to `\@empty` instead of `\relax`.

```
925 \ifx\originalTeX\@undefined\let\originalTeX\@empty\fi
```

Because this part of the code can be included in a format, we make sure that the macro which initializes the save mechanism, `\babel@beginsave`, is not considered to be undefined.

```
926 \ifx\babel@beginsave\@undefined\let\babel@beginsave\relax\fi
```

A few macro names are reserved for future releases of babel, which will use the concept of ‘locale’:

```
927 \providecommand\setlocale{\bbl@error{not-yet-available}}{}{}{}
928 \let\uselocale\setlocale
929 \let\locale\setlocale
930 \let\selectlocale\setlocale
931 \let\textlocale\setlocale
932 \let\textlanguage\setlocale
933 \let\languagegettext\setlocale
```

4.2. Errors

\@nolanerr

\@nopatterns The babel package will signal an error when a documents tries to select a language that hasn’t been defined earlier. When a user selects a language for which no hyphenation patterns were loaded into the format he will be given a warning about that fact. We revert to the patterns for `\language=0` in that case. In most formats that will be (US)english, but it might also be empty.

\@noopterr When the package was loaded without options not everything will work as expected. An error message is issued in that case.

When the format knows about `\PackageError` it must be $\LaTeX 2_{\epsilon}$, so we can safely use its error handling interface. Otherwise we’ll have to ‘keep it simple’.

Infos are not written to the console, but on the other hand many people think warnings are errors, so a further message type is defined: an important info which is sent to the console.

```
934 \edef\bbl@nulllanguage{\string\language=0}
935 \def\bbl@nocaption#1#2{\NoCaseChange{\protect\bbl@nocaption@i{#1}{#2}}}
936 \def\bbl@nocaption@i#1#2{% 1: text to be printed 2: caption macro \langXname
937   \global\@namedef{#2}{\textbf{?#1?}}}%
938   \@nameuse{#2}%
939   \edef\bbl@tempa{#1}%
940   \bbl@sreplace\bbl@tempa{name}}}%
941   \bbl@warning{%
942     \@backslashchar#1 not set for '\language'. Please,\\%
943     define it after the language has been loaded\\%
944     (typically in the preamble) with:\\%
945     \string\setlocalecaption{\language}\bbl@tempa{.}\\%
946     Feel free to contribute on github.com/latex3/babel.\\%
947     Reported}}
948 \def\bbl@tentative{\protect\bbl@tentative@i}
949 \def\bbl@tentative@i#1{%
950   \bbl@warning{%
951     Some functions for '#1' are tentative.\\%
952     They might not work as expected and their behavior\\%
953     could change in the future.\\%
954     Reported}}
955 \def\@nolanerr#1{\bbl@error{undefined-language}{#1}}{}{}
956 \def\@nopatterns#1{%
957   \bbl@warning
958     {No hyphenation patterns were preloaded for\\%
959     the language '#1' into the format.\\%
960     Please, configure your TeX system to add them and\\%
961     rebuild the format. Now I will use the patterns\\%
962     preloaded for \bbl@nulllanguage\space instead}}
963 \let\bbl@usehooks@gobbletwo
```

Here ended the now discarded switch.def.
 Here also (currently) ends the base option.
 964 \ifx\babel@onlyswitch\@empty\endinput\fi

4.3. More on selection

\babelensure The user command just parses the optional argument and creates a new macro named `\babel@e<language>`. We register a hook at the `afterextras` event which just executes this macro in a “complete” selection (which, if undefined, is `\relax` and does nothing). This part is somewhat involved because we have to make sure things are expanded the correct number of times.

The macro `\babel@e<language>` contains `\babel@ensure{<include>}{<exclude>}{<fontenc>}`, which in turn loops over the macros names in `\babel@captionslist`, excluding (with the help of `\in@`) those in the exclude list. If the fontenc is given (and not `\relax`), the `\fontencoding` is also added. Then we loop over the include list, but if the macro already contains `\foreignlanguage`, nothing is done. Note this macro (1) is not restricted to the preamble, and (2) changes are local.

```

965 \babel@trace{Defining babelensure}
966 \newcommand\babelensure[2][]{%
967   \AddBabelHook{babel-ensure}{afterextras}{%
968     \ifcase\babel@select@type
969       \babel@cl{e}%
970     \fi}%
971   \begingroup
972     \let\babel@ens@include\@empty
973     \let\babel@ens@exclude\@empty
974     \def\babel@ens@fontenc{\relax}%
975     \def\babel@tempb##1{%
976       \ifx\@empty##1\else\noexpand##1\expandafter\babel@tempb\fi}%
977     \edef\babel@tempa{\babel@tempb#1\@empty}%
978     \def\babel@tempb##1=##2\@{\@namedef{\babel@ens@##1}{##2}}%
979     \babel@foreach\babel@tempa{\babel@tempb##1\@}%
980     \def\babel@tempc{\babel@ensure}%
981     \expandafter\babel@add\expandafter\babel@tempc\expandafter{%
982       \expandafter{\babel@ens@include}}%
983     \expandafter\babel@add\expandafter\babel@tempc\expandafter{%
984       \expandafter{\babel@ens@exclude}}%
985     \toks@\expandafter{\babel@tempc}%
986     \babel@exp{%
987   \endgroup
988   \def\<babel@e#2>{\the\toks@{\babel@ens@fontenc}}}%
989 \def\babel@ensure#1#2#3{% 1: include 2: exclude 3: fontenc
990   \def\babel@tempb##1{% elt for (excluding) \babel@captionslist list
991     \ifx##1\undefined % 3.32 - Don't assume the macro exists
992       \edef##1{\noexpand\babel@nocaption
993         {\babel@stripslash##1}{\language\babel@stripslash##1}}%
994     \fi
995     \ifx##1\@empty\else
996       \in@{##1}{#2}%
997       \ifin@ \else
998         \let\babel@tempc\language\name
999         \babel@replace\babel@tempc{-}{@}%
1000         \babel@ifunset{\babel@ensure@\babel@tempc}%
1001         {\babel@exp{%
1002           \\DeclareRobustCommand\<babel@ensure@\babel@tempc>[1]{%
1003             \\foreignlanguage{\language\name}%
1004             {\ifx\relax#3\else
1005               \\fontencoding{#3}\\selectfont
1006             \fi
1007             #####1}}}%
1008         {}%
1009         \toks@\expandafter{##1}%
1010         \edef##1{%
1011           \babel@csarg\noexpand{ensure@\babel@tempc}%

```

```

1012         {\the\toks@}}}%
1013     \fi
1014     \expandafter\babel@tempb
1015     \fi}%
1016 \expandafter\babel@tempb\babel@captionslist\today\@empty
1017 \def\babel@tempa##1{% elt for include list
1018     \ifx##1\@empty\else
1019         \let\babel@tempc\language
1020         \babel@replace\babel@tempc{-}{@}%
1021         \babel@csarg\in@{ensure@\babel@tempc\expandafter}\expandafter{##1}%
1022         \ifin@\else
1023             \babel@tempb##1\@empty
1024         \fi
1025         \expandafter\babel@tempa
1026     \fi}%
1027 \babel@tempa#1\@empty}
1028 \def\babel@captionslist{%
1029     \prefacename\refname\abstractname\bibname\chaptername\appendixname
1030     \contentsname\listfigurename\listtablename\indexname\figurename
1031     \tablename\partname\enclname\ccname\headtoname\pagename\seename
1032     \alsoname\proofname\glossaryname}

```

4.4. Short tags

\babeltags This macro is straightforward. After zapping spaces, we loop over the list and define the macros `\text<tag>` and `\<tag>`. Definitions are first expanded so that they don't contain `\csname` but the actual macro.

```

1033 \babel@trace{Short tags}
1034 \newcommand\babeltags[1]{%
1035     \edef\babel@tempa{\zap@space#1 \@empty}%
1036     \def\babel@tempb##1=##2\@{
1037         \edef\babel@tempc{
1038             \noexpand\newcommand
1039             \expandafter\noexpand\csname ##1\endcsname{
1040                 \noexpand\protect
1041                 \expandafter\noexpand\csname otherlanguage*\endcsname{##2}}
1042             \noexpand\newcommand
1043             \expandafter\noexpand\csname text##1\endcsname{
1044                 \noexpand\foreignlanguage{##2}}}
1045         \babel@tempc}%
1046     \babel@for\babel@tempa\babel@tempa{
1047         \expandafter\babel@tempb\babel@tempa\@{

```

4.5. Compatibility with language.def

Plain e-TeX doesn't rely on language.dat, but babel can be made compatible with this format easily.

```

1048 \babel@trace{Compatibility with language.def}
1049 \ifx\directlua\@undefined\else
1050     \ifx\babel@luapatterns\@undefined
1051         \input luababel.def
1052     \fi
1053 \fi
1054 \ifx\babel@languages\@undefined
1055     \ifx\directlua\@undefined
1056         \openin0 = language.def
1057         \ifeof0
1058             \closein0
1059             \message{I couldn't find the file language.def}
1060         \else
1061             \closein0
1062             \begingroup
1063                 \def\addlanguage#1#2#3#4#5{

```

```

1064      \expandafter\ifx\csname lang@#1\endcsname\relax\else
1065      \global\expandafter\let\csname l@#1\expandafter\endcsname
1066      \csname lang@#1\endcsname
1067      \fi}%
1068      \def\uselanguage#1{%
1069      \input language.def
1070      \endgroup
1071      \fi
1072      \fi
1073      \chardef\l@english\z@
1074      \fi

```

\addto It takes two arguments, a *(control sequence)* and TeX-code to be added to the *(control sequence)*.

If the *(control sequence)* has not been defined before it is defined now. The control sequence could also expand to `\relax`, in which case a circular definition results. The net result is a stack overflow. Note there is an inconsistency, because the assignment in the last branch is global.

```

1075 \def\addto#1#2{%
1076   \ifx#1\@undefined
1077     \def#1{#2}%
1078   \else
1079     \ifx#1\relax
1080       \def#1{#2}%
1081     \else
1082       {\toks@\expandafter{#1#2}%
1083        \xdef#1{\the\toks@}}%
1084     \fi
1085   \fi}

```

4.6. Hooks

Admittedly, the current implementation is a somewhat simplistic and does very little to catch errors, but it is meant for developers, after all. `\bbl@usehooks` is the commands used by babel to execute hooks defined for an event.

```

1086 \bbl@trace{Hooks}
1087 \newcommand\AddBabelHook[3][]{%
1088   \bbl@iifunset\bbl@hk@#2{\EnableBabelHook{#2}}}%
1089   \def\bbl@tempa##1,##3=\@empty{\def\bbl@tempb{##2}}%
1090   \expandafter\bbl@tempa\bbl@evargs,##3=,\@empty
1091   \bbl@iifunset\bbl@ev@#2@#3@#1{%
1092     {\bbl@csarg\bbl@add{ev@#3@#1}{\bbl@elth{#2}}}%
1093     {\bbl@csarg\let{ev@#2@#3@#1}\relax}%
1094   \bbl@csarg\newcommand{ev@#2@#3@#1}{\bbl@tempb}}
1095 \newcommand\EnableBabelHook[1]{\bbl@csarg\let{hk@#1}\@firstofone}
1096 \newcommand\DisableBabelHook[1]{\bbl@csarg\let{hk@#1}\@gobble}
1097 \def\bbl@usehooks{\bbl@usehooks@lang\language}
1098 \def\bbl@usehooks@lang#1#2#3{% Test for Plain
1099   \ifx\UseHook\@undefined\else\UseHook{babel/*/#2}\fi
1100   \def\bbl@elth##1{%
1101     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#3}}%
1102     \bbl@cs{ev@#2@}%
1103   \ifx\language\@undefined\else % Test required for Plain (?)
1104     \ifx\UseHook\@undefined\else\UseHook{babel/#1/#2}\fi
1105   \def\bbl@elth##1{%
1106     \bbl@cs{hk@##1}{\bbl@cs{ev@##1@#2@#1@#3}}%
1107     \bbl@cs{ev@#2@#1}%
1108   \fi}

```

To ensure forward compatibility, arguments in hooks are set implicitly. So, if a further argument is added in the future, there is no need to change the existing code. Note events intended for `hyphen.cfg` are also loaded (just in case you need them for some reason).

```

1109 \def\bbl@evargs{,% <- don't delete this comma

```

```

1110 everylanguage=1,loadkernel=1,loadpatterns=1,loadexceptions=1,%
1111 adddialect=2,patterns=2,defaultcommands=0,encodedcommands=2,write=0,%
1112 beforeextras=0,afterextras=0,stopcommands=0,stringprocess=0,%
1113 hyphenation=2,initiateactive=3,afterreset=0,foreign=0,foreign*=0,%
1114 beforestart=0,language=2,begindocument=1}
1115 \ifx\NewHook\@undefined\else % Test for Plain (?)
1116 \def\bbl@tempa#1=#2\@{\NewHook{babel/#1}\NewHook{babel/*/#1}}
1117 \bbl@foreach\bbl@evargs{\bbl@tempa#1\@@}
1118 \fi

```

Since the following command is meant for a hook (although a $\LaTeX$ one), it's placed here.

```

1119 \providecommand\PassOptionsToLocale[2]{%
1120 \bbl@csarg\bbl@add@list{passto@#2}{#1}}

```

4.7. Setting up language files

\LdfInit \LdfInit macro takes two arguments. The first argument is the name of the language that will be defined in the language definition file; the second argument is either a control sequence or a string from which a control sequence should be constructed. The existence of the control sequence indicates that the file has been processed before.

At the start of processing a language definition file we always check the category code of the at-sign. We make sure that it is a 'letter' during the processing of the file. We also save its name as the last called option, even if not loaded.

Another character that needs to have the correct category code during processing of language definition files is the equals sign, '=', because it is sometimes used in constructions with the \let primitive. Therefore we store its current catcode and restore it later on.

Now we check whether we should perhaps stop the processing of this file. To do this we first need to check whether the second argument that is passed to \LdfInit is a control sequence. We do that by looking at the first token after passing #2 through string. When it is equal to \@backslashchar we are dealing with a control sequence which we can compare with \@undefined.

If so, we call \ldf@quit to set the main language, restore the category code of the @-sign and call \endinput

When #2 was *not* a control sequence we construct one and compare it with \relax.

Finally we check \originalTeX.

```

1121 \bbl@trace{Macros for setting language files up}
1122 \def\bbl@ldfinit{%
1123 \let\bbl@screset\@empty
1124 \let\BabelStrings\bbl@opt@string
1125 \let\BabelOptions\@empty
1126 \let\BabelLanguages\relax
1127 \ifx\originalTeX\@undefined
1128 \let\originalTeX\@empty
1129 \else
1130 \originalTeX
1131 \fi}
1132 \def\LdfInit#1#2{%
1133 \chardef\atcatcode=\catcode`\@
1134 \catcode`\@=11\relax
1135 \chardef\eqcatcode=\catcode`\=
1136 \catcode`\==12\relax
1137 \ifpackagewith{babel}{ensureinfo=off}{}%
1138 {\ifx\InputIfFileExists\@undefined\else
1139 \bbl@ifunset{\bbl@lname@#1}%
1140 {\let\bbl@ensuring\@empty % Flag used in babel-serbianc.tex
1141 \def\language#1}%
1142 \bbl@id@assign
1143 \bbl@load@info{#1}}}%
1144 {}%
1145 \fi}%
1146 \expandafter\if\expandafter\@backslashchar
1147 \expandafter\@car\string#2\@nil
1148 \ifx#2\@undefined\else
1149 \ldf@quit{#1}%

```

```

1150 \fi
1151 \else
1152 \expandafter\ifx\csname#2\endcsname\relax\else
1153 \ldf@quit{#1}%
1154 \fi
1155 \fi
1156 \bbl@ldfinit}

```

\ldf@quit This macro interrupts the processing of a language definition file. Remember `\endinput` is not executed immediately, but delayed to the end of the current line in the input file.

```

1157 \def\ldf@quit#1{%
1158 \expandafter\main@language\expandafter{#1}%
1159 \catcode`\@=\atcatcode \let\atcatcode\relax
1160 \catcode`\==\eqcatcode \let\eqcatcode\relax
1161 \endinput}

```

\ldf@finish This macro takes one argument. It is the name of the language that was defined in the language definition file.

We load the local configuration file if one is present, we set the main language (taking into account that the argument might be a control sequence that needs to be expanded) and reset the category code of the `@`-sign.

```

1162 \def\bbl@afterldf{%
1163 \bbl@afterlang
1164 \let\bbl@afterlang\relax
1165 \let\BabelModifiers\relax
1166 \let\bbl@screset\relax}%
1167 \def\ldf@finish#1{%
1168 \loadlocalcfg{#1}%
1169 \bbl@afterldf
1170 \expandafter\main@language\expandafter{#1}%
1171 \catcode`\@=\atcatcode \let\atcatcode\relax
1172 \catcode`\==\eqcatcode \let\eqcatcode\relax}

```

After the preamble of the document the commands `\LdfInit`, `\ldf@quit` and `\ldf@finish` are no longer needed. Therefore they are turned into warning messages in $\LaTeX$.

```

1173 \@onlypreamble\LdfInit
1174 \@onlypreamble\ldf@quit
1175 \@onlypreamble\ldf@finish

```

\main@language

\bbl@main@language This command should be used in the various language definition files. It stores its argument in `\bbl@main@language`; to be used to switch to the correct language at the beginning of the document.

```

1176 \def\main@language#1{%
1177 \def\bbl@main@language{#1}%
1178 \let\language\bbl@main@language
1179 \let\localename\bbl@main@language
1180 \let\mainlocalename\bbl@main@language
1181 \bbl@id@assign
1182 \ifcase\bbl@engine\or
1183 \ifx\setattribute\@undefined\else
1184 \setattribute\bbl@attr@locale\localeid
1185 \fi
1186 \fi
1187 \bbl@patterns{\language}

```

We also have to make sure that some code gets executed at the beginning of the document, either when the aux file is read or, if it does not exist, when the `\AtBeginDocument` is executed. Languages do not set `\pagedir`, so we set here for the whole document to the main `\bodydir`.

The code written to the aux file attempts to avoid errors if babel is removed from the document.

```

1188 \def\bbl@beforestart{%

```

```

1189 \def\@nolanerr##1{%
1190   \bbl@carg\chardef{l@##1}\z@
1191   \bbl@warning{Undefined language '##1' in aux.\@Reported}}%
1192 \bbl@usehooks{beforestart}{}%
1193 \global\let\bbl@beforestart\relax
1194 \AtBeginDocument{%
1195   {\@nameuse\bbl@beforestart}}% Group!
1196   \if@filesw
1197     \providecommand\babel@aux[2]{}%
1198     \immediate\write\@mainaux{\unexpanded{%
1199       \providecommand\babel@aux[2]{\global\let\babel@toc\@gobbletwo}}}%
1200     \immediate\write\@mainaux{\string\@nameuse\bbl@beforestart}}%
1201   \fi
1202   \expandafter\selectlanguage\expandafter{\bbl@main@language}%
1203   \ifbbl@single % must go after the line above.
1204     \renewcommand\selectlanguage[1]{}%
1205     \renewcommand\foreignlanguage[2]{#2}%
1206     \global\let\babel@aux\@gobbletwo % Also as flag
1207   \fi}

A bit of optimization. Select in heads/feet the language only if necessary.

1208 \def\select@language@x#1{%
1209   \ifcase\bbl@select@type
1210     \bbl@ifsamestring\language{#1}{\select@language{#1}}%
1211   \else
1212     \select@language{#1}%
1213   \fi}

```

4.8. Shorthands

The macro `\initiate@active@char` below takes all the necessary actions to make its argument a shorthand character. The real work is performed once for each character. But first we define a little tool.

```

1214 \bbl@trace{Shorhands}
1215 \def\bbl@withactive#1#2{%
1216   \begingroup
1217   \lccode`~=#2\relax
1218   \lowercase{\endgroup#1~}}

```

\bbl@add@special The macro `\bbl@add@special` is used to add a new character (or single character control sequence) to the macro `\dospecials` (and `\@sanitize` if $\TeX$ is used). It is used only at one place, namely when `\initiate@active@char` is called (which is ignored if the char has been made active before). Because `\@sanitize` can be undefined, we put the definition inside a conditional.

Items are added to the lists without checking its existence or the original catcode. It does not hurt, but should be fixed. It's already done with `\nfss@catcodes`, added in 3.10.

```

1219 \def\bbl@add@special#1% 1:a macro like "\", \?, etc.
1220 \bbl@add\dospecials{\do#1}% test @sanitize = \relax, for back. compat.
1221 \bbl@ifunset{@sanitize}{\bbl@add\@sanitize{\makeother#1}}%
1222 \ifx\nfss@catcodes\undefined\else
1223   \begingroup
1224     \catcode`#1\active
1225     \nfss@catcodes
1226     \ifnum\catcode`#1=\active
1227       \endgroup
1228       \bbl@add\nfss@catcodes{\@makeother#1}%
1229     \else
1230       \endgroup
1231     \fi
1232   \fi}

```

\initiate@active@char A language definition file can call this macro to make a character active. This macro takes one argument, the character that is to be made active. When the character was already active this macro does nothing. Otherwise, this macro defines the control sequence `\normal@char⟨char⟩` to expand to the character in its ‘normal state’ and it defines the active character to expand to `\normal@char⟨char⟩` by default (`⟨char⟩` being the character to be made active). Later its definition can be changed to expand to `\active@char⟨char⟩` by calling `\bbl@activate{⟨char⟩}`.

For example, to make the double quote character active one could have `\initiate@active@char{"}` in a language definition file. This defines `"` as `\active@prefix "\active@char` (where the first `"` is the character with its original catcode, when the shorthand is created, and `\active@char` is a single token). In protected contexts, it expands to `\protect "` or `\noexpand "` (i.e., with the original `"`); otherwise `\active@char` is executed. This macro in turn expands to `\normal@char` in “safe” contexts (e.g., `\label`), but `\user@active` in normal “unsafe” ones. The latter search a definition in the user, language and system levels, in this order, but if none is found, `\normal@char` is used. However, a deactivated shorthand (with `\bbl@deactivate` defined as `\active@prefix "\normal@char`).

The following macro is used to define shorthands in the three levels. It takes 4 arguments: the (string’ed) character, `\⟨level⟩@group`, `⟨level⟩@active` and `⟨next-level⟩@active` (except in system).

```
1233 \def\bbl@active@def#1#2#3#4{%
1234   \namedef{#3#1}{%
1235     \expandafter\ifx\csname#2@sh@#1@\endcsname\relax
1236       \bbl@afterelse\bbl@sh@select#2#1{#3@arg#1}{#4#1}%
1237     \else
1238       \bbl@afterfi\csname#2@sh@#1@\endcsname
1239     \fi}%
```

When there is also no current-level shorthand with an argument we will check whether there is a next-level defined shorthand for this active character.

```
1240   \long\@namedef{#3@arg#1}##1{%
1241     \expandafter\ifx\csname#2@sh@#1@\string##1@\endcsname\relax
1242       \bbl@afterelse\csname#4#1\endcsname##1%
1243     \else
1244       \bbl@afterfi\csname#2@sh@#1@\string##1@\endcsname
1245     \fi}}%
```

`\initiate@active@char` calls `\@initiate@active@char` with 3 arguments. All of them are the same character with different catcodes: active, other (`\string’ed`) and the original one. This trick simplifies the code a lot.

```
1246 \def\@initiate@active@char#1{%
1247   \bbl@ifunset{active@char\string#1}%
1248   {\bbl@withactive
1249     {\expandafter\@initiate@active@char\expandafter}#1\string#1#1}%
1250   {}}
```

The very first thing to do is saving the original catcode and the original definition, even if not active, which is possible (undefined characters require a special treatment to avoid making them `\relax` and preserving some degree of protection).

```
1251 \def\@initiate@active@char#1#2#3{%
1252   \bbl@csarg\edef{oricat@#2}{\catcode`#2=\the\catcode`#2\relax}%
1253   \ifx#1\@undefined
1254     \bbl@csarg\def{oridef@#2}{\def#1{\active@prefix#1\@undefined}}%
1255   \else
1256     \bbl@csarg\let{oridef@#2}#1%
1257     \bbl@csarg\edef{oridef@#2}{%
1258       \let\noexpand#1%
1259       \expandafter\noexpand\csname bbl@oridef@#2\endcsname}%
1260   \fi
```

If the character is already active we provide the default expansion under this shorthand mechanism. Otherwise we write a message in the transcript file, and define `\normal@char⟨char⟩` to expand to the character in its default state. If the character is mathematically active when babel is loaded (for example `'`) the normal expansion is somewhat different to avoid an infinite loop (but it does not prevent the loop if the mathcode is set to `"8000 a posteriori`).

```
1261   \ifx#1#3\relax
```



```

1262 \expandafter\let\csname normal@char#2\endcsname#3%
1263 \else
1264 \bbl@info{Making #2 an active character}%
1265 \ifnum\mathcode`#2=\ifodd\bbl@engine"1000000 \else"8000 \fi
1266 \@namedef{normal@char#2}{%
1267 \textormath{#3}{\csname bbl@oridef@#2\endcsname}}%
1268 \else
1269 \@namedef{normal@char#2}{#3}%
1270 \fi

```

To prevent problems with the loading of other packages after babel we reset the catcode of the character to the original one at the end of the package and of each language file (except with KeepShorthandsActive). It is re-activate again at `\begin{document}`. We also need to make sure that the shorthands are active during the processing of the aux file. Otherwise some citations may give unexpected results in the printout when a shorthand was used in the optional argument of `\bibitem` for example. Then we make it active (not strictly necessary, but done for backward compatibility).

```

1271 \bbl@restoreactive{#2}%
1272 \AtBeginDocument{%
1273 \catcode`#2\active
1274 \if@files
1275 \immediate\write\@mainaux{\catcode`\string#2\active}%
1276 \fi}%
1277 \expandafter\bbl@add@special\csname#2\endcsname
1278 \catcode`#2\active
1279 \fi

```

Now we have set `\normal@char{char}`, we must define `\active@char{char}`, to be executed when the character is activated. We define the first level expansion of `\active@char{char}` to check the status of the `@safe@actives` flag. If it is set to true we expand to the ‘normal’ version of this character, otherwise we call `\user@active{char}` to start the search of a definition in the user, language and system levels (or eventually `normal@char{char}`).

```

1280 \let\bbl@tempa\@firstoftwo
1281 \if\string^#2%
1282 \def\bbl@tempa{\noexpand\textormath}%
1283 \else
1284 \ifx\bbl@mathnormal\@undefined\else
1285 \let\bbl@tempa\bbl@mathnormal
1286 \fi
1287 \fi
1288 \expandafter\edef\csname active@char#2\endcsname{%
1289 \bbl@tempa
1290 {\noexpand\if@safe@actives
1291 \noexpand\expandafter
1292 \expandafter\noexpand\csname normal@char#2\endcsname
1293 \noexpand\else
1294 \noexpand\expandafter
1295 \expandafter\noexpand\csname bbl@doactive#2\endcsname
1296 \noexpand\fi}%
1297 {\expandafter\noexpand\csname normal@char#2\endcsname}}%
1298 \bbl@csarg\edef{doactive#2}{%
1299 \expandafter\noexpand\csname user@active#2\endcsname}%

```

We now define the default values which the shorthand is set to when activated or deactivated. It is set to the deactivated form (globally), so that the character expands to

$$\text{active@prefix}\langle char\rangle\text{normal@char}\langle char\rangle$$

(where `\active@char{char}` is *one* control sequence!).

```

1300 \bbl@csarg\edef{active@#2}{%
1301 \noexpand\active@prefix\noexpand#1%
1302 \expandafter\noexpand\csname active@char#2\endcsname}%
1303 \bbl@csarg\edef{normal@#2}{%
1304 \noexpand\active@prefix\noexpand#1%
1305 \expandafter\noexpand\csname normal@char#2\endcsname}%
1306 \bbl@ncarg\let#1\bbl@normal@#2}%

```

The next level of the code checks whether a user has defined a shorthand for himself with this character. First we check for a single character shorthand. If that doesn't exist we check for a shorthand with an argument.

```
1307 \bbl@active@def#2\user@group{user@active}{language@active}%
1308 \bbl@active@def#2\language@group{language@active}{system@active}%
1309 \bbl@active@def#2\system@group{system@active}{normal@char}%
```

In order to do the right thing when a shorthand with an argument is used by itself at the end of the line we provide a definition for the case of an empty argument. For that case we let the shorthand character expand to its non-active self. Also, When a shorthand combination such as ' ' ends up in a heading $\TeX$ would see `\protect'\protect'`. To prevent this from happening a couple of shorthand needs to be defined at user level.

```
1310 \expandafter\edef\csname\user@group @sh#2@@\endcsname
1311 {\expandafter\noexpand\csname normal@char#2\endcsname}%
1312 \expandafter\edef\csname\user@group @sh#2@\string\protect\endcsname
1313 {\expandafter\noexpand\csname user@active#2\endcsname}%
```

Finally, a couple of special cases are taken care of. (1) If we are making the right quote (') active we need to change `\pr@m@s` as well. Also, make sure that a single ' in math mode 'does the right thing'. (2) If we are using the caret (^) as a shorthand character special care should be taken to make sure math still works. Therefore an extra level of expansion is introduced with a check for math mode on the upper level.

```
1314 \if\string'#2%
1315 \let\prim@s\bbl@prim@s
1316 \let\active@math@prime#1%
1317 \fi
1318 \bbl@usehooks{initiateactive}{\{#1\}{#2\}{#3\}}
```

The following package options control the behavior of shorthands in math mode.

```
1319 <(*More package options)> ≡
1320 \DeclareOption{math=active}{}
1321 \DeclareOption{math=normal}{\def\bbl@mathnormal{\noexpand\textormath}}
1322 <(/More package options)>
```

Initiating a shorthand makes active the char. That is not strictly necessary but it is still done for backward compatibility. So we need to restore the original catcode at the end of package *and* the end of the *ldf*.

```
1323 \ifpackagewith{babel}{KeepShorthandsActive}%
1324 {\let\bbl@restoreactive@gobble}%
1325 {\def\bbl@restoreactive#1{%
1326 \bbl@exp{%
1327 \\\AfterBabelLanguage\\CurrentOption
1328 {\catcode`#1=\the\catcode`#1\relax}%
1329 \\\AtEndOfPackage
1330 {\catcode`#1=\the\catcode`#1\relax}}}%
1331 \AtEndOfPackage{\let\bbl@restoreactive@gobble}}
```

\bbl@sh@select This command helps the shorthand supporting macros to select how to proceed.

Note that this macro needs to be expandable as do all the shorthand macros in order for them to work in expansion-only environments such as the argument of `\hyphenation`.

This macro expects the name of a group of shorthands in its first argument and a shorthand character in its second argument. It will expand to either `\bbl@firstcs` or `\bbl@scndcs`. Hence two more arguments need to follow it.

```
1332 \def\bbl@sh@select#1#2{%
1333 \expandafter\ifx\csname#1@sh#2@sel\endcsname\relax
1334 \bbl@afterelse\bbl@scndcs
1335 \else
1336 \bbl@afterfi\csname#1@sh#2@sel\endcsname
1337 \fi}
```

\active@prefix Used in the expansion of active characters has a function similar to \OT1-cmd in that it \protects the active character whenever \protect is *not* \@typeset@protect. The \gobble is needed to remove a token such as \activechar: (when the double colon was the active character to be dealt with). There are two definitions, depending on \ifincsname is available. If there is, the expansion will be more robust.

```

1338 \begingroup
1339 \bbl@ifunset{ifincsname}
1340 {\gdef\active@prefix#1{%
1341   \ifx\protect\@typeset@protect
1342   \else
1343     \ifx\protect\@unexpandable@protect
1344       \noexpand#1%
1345     \else
1346       \protect#1%
1347     \fi
1348     \expandafter\@gobble
1349   \fi}}
1350 {\gdef\active@prefix#1{%
1351   \ifincsname
1352     \string#1%
1353     \expandafter\@gobble
1354   \else
1355     \ifx\protect\@typeset@protect
1356     \else
1357       \ifx\protect\@unexpandable@protect
1358         \noexpand#1%
1359       \else
1360         \protect#1%
1361       \fi
1362       \expandafter\expandafter\expandafter\@gobble
1363     \fi
1364   \fi}}
1365 \endgroup

```

if@safe@actives In some circumstances it is necessary to be able to reset the shorthand to its ‘normal’ value (usually the character with catcode ‘other’) on the fly. For this purpose the switch @safe@actives is available. The setting of this switch should be checked in the first level expansion of \active@char⟨char⟩. When this expansion mode is active (with \@safe@activetrue), something like "13"13 becomes "12"12 in an \edef (in other words, shorthands are \string'ed). This contrasts with \protected@edef, where catcodes are always left unchanged. Once converted, they can be used safely even after this expansion mode is deactivated (with \@safe@activefalse).

```

1366 \newif\if@safe@actives
1367 \@safe@activesfalse

```

\bbl@restore@actives When the output routine kicks in while the active characters were made “safe” this must be undone in the headers to prevent unexpected typeset results. For this situation we define a command to make them “unsafe” again.

```

1368 \def\bbl@restore@actives{\if@safe@actives\@safe@activessfalse\fi}

```

\bbl@activate

\bbl@deactivate Both macros take one argument, like \initiate@active@char. The macro is used to change the definition of an active character to expand to \active@char⟨char⟩ in the case of \bbl@activate, or \normal@char⟨char⟩ in the case of \bbl@deactivate.

```

1369 \chardef\bbl@activated\z@
1370 \def\bbl@activate#1{%
1371   \chardef\bbl@activated\@ne
1372   \bbl@withactive{\expandafter\let\expandafter}#1%
1373   \csname bbl@active@\string#1\endcsname}
1374 \def\bbl@deactivate#1{%
1375   \chardef\bbl@activated\tw@
1376   \bbl@withactive{\expandafter\let\expandafter}#1%

```

```
1377 \csname bbl@normal@\string#1\endcsname}
```

\bbl@firstcs

\bbl@scndcs These macros are used only as a trick when declaring shorthands.

```
1378 \def\bbl@firstcs#1#2{\csname#1\endcsname}
1379 \def\bbl@scndcs#1#2{\csname#2\endcsname}
```

\declare@shorthand Used to declare a shorthand on a certain level. It takes three arguments:

1. a name for the collection of shorthands, i.e., ‘system’, or ‘dutch’;
2. the character (sequence) that makes up the shorthand, i.e., ~ or "a;
3. the code to be executed when the shorthand is encountered.

The auxiliary macro `\babel@texpdf` improves the interoperativity with `hyperref` and takes 4 arguments: (1) The $\TeX$ code in text mode, (2) the string for `hyperref`, (3) the $\TeX$ code in math mode, and (4), which is currently ignored, but it’s meant for a string in math mode, like a minus sign instead of an hyphen (currently `hyperref` doesn’t discriminate the mode). This macro may be used in `ldf` files.

```
1380 \def\babel@texpdf#1#2#3#4{%
1381 \ifx\texorpdfstring\undefined
1382 \textormath{#1}{#3}%
1383 \else
1384 \texorpdfstring{\textormath{#1}{#3}}{#2}%
1385 % \texorpdfstring{\textormath{#1}{#3}}{\textormath{#2}{#4}}%
1386 \fi}
1387 %
1388 \def\declare@shorthand#1#2{\@decl@short{#1}#2@nil}
1389 \def\@decl@short#1#2#3@nil#4{%
1390 \def\bbl@tempa{#3}%
1391 \ifx\bbl@tempa\empty
1392 \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@scndcs
1393 \bbl@ifunset{#1@sh@\string#2@}{}%
1394 {\def\bbl@tempa{#4}%
1395 \expandafter\ifx\csname#1@sh@\string#2@endcsname\bbl@tempa
1396 \else
1397 \bbl@info
1398 {Redefining #1 shorthand \string#2\}%
1399 in language \CurrentOption}%
1400 \fi}%
1401 \@namedef{#1@sh@\string#2@}{#4}%
1402 \else
1403 \expandafter\let\csname #1@sh@\string#2@sel\endcsname\bbl@firstcs
1404 \bbl@ifunset{#1@sh@\string#2@\string#3@}{}%
1405 {\def\bbl@tempa{#4}%
1406 \expandafter\ifx\csname#1@sh@\string#2@\string#3@endcsname\bbl@tempa
1407 \else
1408 \bbl@info
1409 {Redefining #1 shorthand \string#2\string#3\}%
1410 in language \CurrentOption}%
1411 \fi}%
1412 \@namedef{#1@sh@\string#2@\string#3@}{#4}%
1413 \fi}
```

\textormath Some of the shorthands that will be declared by the language definition files have to be usable in both text and mathmode. To achieve this the helper macro `\textormath` is provided.

```
1414 \def\textormath{%
1415 \ifmmode
1416 \expandafter\@secondoftwo
1417 \else
1418 \expandafter\@firstoftwo
1419 \fi}
```

\user@group

\language@group

\system@group The current concept of ‘shorthands’ supports three levels or groups of shorthands. For each level the name of the level or group is stored in a macro. The default is to have a user group; use language group ‘english’ and have a system group called ‘system’.

```
1420 \def\user@group{user}
1421 \def\language@group{english}
1422 \def\system@group{system}
```

\useshorthands This is the user level macro. It initializes and activates the character for use as a shorthand character (i.e., it’s active in the preamble). Languages can deactivate shorthands, so a starred version is also provided which activates them always after the language has been switched.

```
1423 \def\useshorthands{%
1424   \ifstar\bb@usesh@s{\bb@usesh@x{}}
1425 \def\bb@usesh@s#1{%
1426   \bb@usesh@x
1427   {\AddBabelHook{babel-sh-\string#1}{afterextras}{\bb@activate{#1}}}%
1428   {#1}}
1429 \def\bb@usesh@x#1#2{%
1430   \bb@ifshorthand{#2}%
1431   {\def\user@group{user}%
1432     \initiate@active@char{#2}%
1433     #1%
1434     \bb@activate{#2}}%
1435   {\bb@error{shorthand-is-off}{#2}}}
```

\defineshorthand Currently we only support two groups of user level shorthands, named internally user and user@(\language) (language-dependent user shorthands). By default, only the first one is taken into account, but if the former is also used (in the optional argument of \defineshorthand) a new level is inserted for it (user@generic, done by \bb@set@user@generic); we make also sure {} and \protect are taken into account in this new top level.

```
1436 \def\user@language@group{user@\language@group}
1437 \def\bb@set@user@generic#1#2{%
1438   \bb@ifunset{user@generic@active#1}%
1439   {\bb@active@def#1\user@language@group{user@active}{user@generic@active}%
1440     \bb@active@def#1\user@group{user@generic@active}{language@active}%
1441     \expandafter\edef\csname#2@sh@#1@\endcsname{%
1442       \expandafter\noexpand\csname normal@char#1\endcsname}%
1443     \expandafter\edef\csname#2@sh@#1@\string\protect\endcsname{%
1444       \expandafter\noexpand\csname user@active#1\endcsname}}%
1445   \@empty}
1446 \newcommand\defineshorthand[3][user]{%
1447   \edef\bb@tempa{\zap@space#1 \@empty}%
1448   \bb@for\bb@tempb\bb@tempa{%
1449     \if*\expandafter\@car\bb@tempb\@nil
1450       \edef\bb@tempb{user@\expandafter\@gobble\bb@tempb}%
1451       \@expandtwoargs
1452       \bb@set@user@generic{\expandafter\string\@car#2\@nil}\bb@tempb
1453     \fi
1454     \declare@shorthand{\bb@tempb}{#2}{#3}}}
```

\languageshorthands A user level command to change the language from which shorthands are used. Unfortunately, babel currently does not keep track of defined groups, and therefore there is no way to catch a possible change in casing to fix it in the same way languages names are fixed.

```
1455 \def\languageshorthands#1{%
1456   \bb@ifsamestring{none}{#1}{}%
1457   \bb@once{short-\localename-#1}{%
1458     \bb@info{'\localename' activates '#1' shorthands.\@Reported}}%
1459   \def\language@group{#1}}
```

\aliasshorthand *Deprecated.* First the new shorthand needs to be initialized. Then, we define the new shorthand in terms of the original one, but note with `\aliasshandands{"}{/}` is `\active@prefix / \active@char/`, so we still need to let the latter to `\active@char`".

```

1460 \def\aliasshorthand#1#2{%
1461   \bbl@ifshorthand{#2}%
1462   {\expandafter\ifx\csname active@char\string#2\endcsname\relax
1463     \ifx\document\@notprerr
1464       \@notshorthand{#2}%
1465     \else
1466       \initiate@active@char{#2}%
1467       \bbl@ccarg\let{active@char\string#2}{active@char\string#1}%
1468       \bbl@ccarg\let{normal@char\string#2}{normal@char\string#1}%
1469       \bbl@activate{#2}%
1470     \fi
1471   \fi}%
1472   {\bbl@error{shorthand-is-off}{#2}{}}}

```

\@notshorthand

```

1473 \def\@notshorthand#1{\bbl@error{not-a-shorthand}{#1}{}}

```

\shorthandon

\shorthandoff The first level definition of these macros just passes the argument on to `\bbl@switch@sh`, adding `\@nil` at the end to denote the end of the list of characters.

```

1474 \newcommand*\shorthandon[1]{\bbl@switch@sh\@ne#1\@nnil}
1475 \DeclareRobustCommand*\shorthandoff{%
1476   \@ifstar{\bbl@shorthandoff\tw@}{\bbl@shorthandoff\z@}}
1477 \def\bbl@shorthandoff#1#2{\bbl@switch@sh#1#2\@nnil}

```

\bbl@switch@sh The macro `\bbl@switch@sh` takes the list of characters apart one by one and subsequently switches the category code of the shorthand character according to the first argument of `\bbl@switch@sh`.

But before any of this switching takes place we make sure that the character we are dealing with is known as a shorthand character. If it is, a macro such as `\active@char`" should exist.

Switching off and on is easy – we just set the category code to ‘other’ (12) and `\active`. With the starred version, the original catcode and the original definition, saved in `@initiate@active@char`, are restored.

```

1478 \def\bbl@switch@sh#1#2{%
1479   \ifx#2\@nnil\else
1480     \bbl@ifunset{bbl@active@\string#2}%
1481     {\bbl@error{not-a-shorthand-b}{#2}{}}%
1482     {\ifcase#1 off, on, off*
1483       \catcode`#2\relax
1484     \or
1485       \catcode`#2\active
1486       \bbl@ifunset{bbl@shdef@\string#2}%
1487       {}%
1488       {\bbl@withactive{\expandafter\let\expandafter}#2%
1489         \csname bbl@shdef@\string#2\endcsname
1490         \bbl@csarg\let{shdef@\string#2}\relax}%
1491       \ifcase\bbl@activated\or
1492         \bbl@activate{#2}%
1493       \else
1494         \bbl@deactivate{#2}%
1495       \fi
1496     \or
1497       \bbl@ifunset{bbl@shdef@\string#2}%
1498       {\bbl@withactive{\bbl@csarg\let{shdef@\string#2}}#2}%
1499       {}%
1500       \csname bbl@oricat@\string#2\endcsname
1501       \csname bbl@oridef@\string#2\endcsname
1502     \fi}%

```

```

1503 \bbl@afterfi\bbl@switch@sh#1%
1504 \fi}

```

Note the value is that at the expansion time; e.g., in the preamble shorthands are usually deactivated.

```

1505 \def\babelshorthand{\active@prefix\babelshorthand\bbl@putsh}
1506 \def\bbl@putsh#1{%
1507 \bbl@ifunset{\bbl@active@\string#1}%
1508 {\bbl@putsh@i#1\@empty\@nnil}%
1509 {\csname bbl@active@\string#1\endcsname}}
1510 \def\bbl@putsh@i#1#2\@nnil{%
1511 \csname\language@group @sh@\string#1@%
1512 \ifx\@empty#2\else\string#2@\fi\endcsname}
1513 %
1514 \ifx\bbl@opt@shorthands\@nnil\else
1515 \let\bbl@s@initiate@active@char@initiate@active@char
1516 \def\initiate@active@char#1{%
1517 \bbl@ifshorthand{#1}{\bbl@s@initiate@active@char{#1}}{}}
1518 \let\bbl@s@switch@sh\bbl@switch@sh
1519 \def\bbl@switch@sh#1#2{%
1520 \ifx#2\@nnil\else
1521 \bbl@afterfi
1522 \bbl@ifshorthand{#2}{\bbl@s@switch@sh#1{#2}}{\bbl@switch@sh#1}%
1523 \fi}
1524 \let\bbl@s@activate\bbl@activate
1525 \def\bbl@activate#1{%
1526 \bbl@ifshorthand{#1}{\bbl@s@activate{#1}}{}}
1527 \let\bbl@s@deactivate\bbl@deactivate
1528 \def\bbl@deactivate#1{%
1529 \bbl@ifshorthand{#1}{\bbl@s@deactivate{#1}}{}}
1530 \fi

```

You may want to test if a character is a shorthand. Note it does not test whether the shorthand is on or off.

```

1531 \newcommand\ifbabelshorthand[3]{\bbl@ifunset{\bbl@active@\string#1}{#3}{#2}}

```

\bbl@prim@s

\bbl@pr@m@s One of the internal macros that are involved in substituting `\prime` for each right quote in mathmode is `\prim@s`. This checks if the next character is a right quote. When the right quote is active, the definition of this macro needs to be adapted to look also for an active right quote; the hat could be active, too.

```

1532 \def\bbl@prim@s{%
1533 \prime\futurelet\@let@token\bbl@pr@m@s}
1534 \def\bbl@if@primes#1#2{%
1535 \ifx#1\@let@token
1536 \expandafter\@firstoftwo
1537 \else\ifx#2\@let@token
1538 \bbl@afterelse\expandafter\@firstoftwo
1539 \else
1540 \bbl@afterfi\expandafter\@secondoftwo
1541 \fi\fi}
1542 \begingroup
1543 \catcode`\^=7 \catcode`\*=\active \lccode`\*='\^
1544 \catcode`\'=12 \catcode`\\"=\active \lccode`\\"='\^
1545 \lowercase{%
1546 \gdef\bbl@pr@m@s{%
1547 \bbl@if@primes" '%
1548 \pr@@@s
1549 {\bbl@if@primes*\^{\pr@@@t\egroup}}}
1550 \endgroup

```

Usually the `~` is active and expands to `\penalty\@M\_\_`. When it is written to the aux file it is written expanded. To prevent that and to be able to use the character `~` as a start character for a shorthand, it

is redefined here as a one character shorthand on system level. The system declaration is in most cases redundant (when ~ is still a non-break space), and in some cases is inconvenient (if ~ has been redefined); however, for backward compatibility it is maintained (some existing documents may rely on the babel value).

```
1551 \initiate@active@char{~}
1552 \declare@shorthand{system}{~}{\leavevmode\nobreak\ }
1553 \bbl@activate{~}
```

\OT1dqpos

\T1dqpos The position of the double quote character is different for the OT1 and T1 encodings. It will later be selected using the \f@encoding macro. Therefore we define two macros here to store the position of the character in these encodings.

```
1554 \expandafter\def\csname OT1dqpos\endcsname{127}
1555 \expandafter\def\csname T1dqpos\endcsname{4}
```

When the macro \f@encoding is undefined (as it is in plain T_EX) we define it here to expand to OT1

```
1556 \ifx\f@encoding\undefined
1557   \def\f@encoding{OT1}
1558 \fi
```

4.9. Language attributes

Language attributes provide a means to give the user control over which features of the language definition files he wants to enable.

\languageattribute The macro \languageattribute checks whether its arguments are valid and then activates the selected language attribute. First check whether the language is known, and then process each attribute in the list.

To make sure each attribute is selected only once, we store the already selected attributes in \bbl@known@attrs. When that control sequence is not yet defined this attribute is certainly not selected before.

When we end up here the attribute is not selected before. So, we add it to the list of selected attributes and execute the associated T_EX code.

```
1559 \bbl@trace{Language attributes}
1560 \newcommand\languageattribute[2]{%
1561   \def\bbl@tempc{#1}%
1562   \bbl@fixname\bbl@tempc
1563   \bbl@iflanguage\bbl@tempc{%
1564     \bbl@vforeach{#2}{%
1565       \ifx\bbl@known@attrs\undefined
1566         \in@false
1567       \else
1568         \bbl@xin@{,\bbl@tempc-##1,}{,\bbl@known@attrs,}%
1569       \fi
1570       \ifin@
1571         \bbl@warning{%
1572           You have more than once selected the attribute '##1'\%
1573           for language #1. Reported}%
1574       \else
1575         \bbl@info{Activated '##1' attribute for\\%
1576           '\bbl@tempc'. Reported}%
1577         \bbl@exp{%
1578           \\ \bbl@add@list\\ \bbl@known@attrs{\bbl@tempc-##1}}%
1579         \edef\bbl@tempa{\bbl@tempc-##1}%
1580         \expandafter\bbl@ifknown@ttrib\expandafter{\bbl@tempa}\bbl@attributes
1581         {\csname\bbl@tempc @attr##1\endcsname}%
1582         {\bbl@error{unknown-attribute}{\bbl@tempc}{##1}}}%
1583       \fi}}}
1584 \@onlypreamble\languageattribute
```


\bbl@declare@ttribute This command adds the new language/attribute combination to the list of known attributes.

Then it defines a control sequence to be executed when the attribute is used in a document. The result of this should be that the macro `\extras...` for the current language is extended, otherwise the attribute will not work as its code is removed from memory at `\begin{document}`.

```
1585 \def\bbl@declare@ttribute#1#2#3{%
1586   \bbl@xin@{,#2,}{,\BabelModifiers,}%
1587   \ifin@
1588     \AfterBabelLanguage{#1}{\languageattribute{#1}{#2}}%
1589   \fi
1590   \bbl@add@list\bbl@attributes{#1-#2}%
1591   \expandafter\def\csname#1@attr@#2\endcsname{#3}}
```

\bbl@ifattributeset This internal macro has 4 arguments. It can be used to interpret $\TeX$ code based on whether a certain attribute was set. This command should appear inside the argument to `\AtBeginDocument` because the attributes are set in the document preamble, *after* babel is loaded.

The first argument is the language, the second argument the attribute being checked, and the third and fourth arguments are the true and false clauses.

```
1592 \def\bbl@ifattributeset#1#2#3#4{%
1593   \ifx\bbl@known@attribs\@undefined
1594     \in@false
1595   \else
1596     \bbl@xin@{,#1-#2,}{,\bbl@known@attribs,}%
1597   \fi
1598   \ifin@
1599     \bbl@afterelse#3%
1600   \else
1601     \bbl@afterfi#4%
1602   \fi}
```

\bbl@ifknown@ttrib An internal macro to check whether a given language/attribute is known. The macro takes 4 arguments, the language/attribute, the attribute list, the $\TeX$ -code to be executed when the attribute is known and the $\TeX$ -code to be executed otherwise.

We first assume the attribute is unknown. Then we loop over the list of known attributes, trying to find a match.

```
1603 \def\bbl@ifknown@ttrib#1#2{%
1604   \let\bbl@tempa\@secondoftwo
1605   \bbl@loopx\bbl@tempb{#2}{%
1606     \expandafter\in@\expandafter{\expandafter,\bbl@tempb,}{,#1,}%
1607   \ifin@
1608     \let\bbl@tempa\@firstoftwo
1609   \else
1610     \fi}%
1611   \bbl@tempa}
```

\bbl@clear@ttribs This macro removes all the attribute code from $\TeX$'s memory at `\begin{document}` time (if any is present).

```
1612 \def\bbl@clear@ttribs{%
1613   \ifx\bbl@attributes\@undefined\else
1614     \bbl@loopx\bbl@tempa{\bbl@attributes}{%
1615       \expandafter\bbl@clear@ttrib\bbl@tempa.}%
1616     \let\bbl@attributes\@undefined
1617   \fi}
1618 \def\bbl@clear@ttrib#1-#2.{%
1619   \expandafter\let\csname#1@attr@#2\endcsname\@undefined}
1620 \AtBeginDocument{\bbl@clear@ttribs}
```

4.10. Support for saving and redefining macros

To save the meaning of control sequences using `\babel@save`, we use temporary control sequences. To save hash table entries for these control sequences, we don't use the name of the control sequence

to be saved to construct the temporary name. Instead we simply use the value of a counter, which is reset to zero each time we begin to save new values. This works well because we release the saved meanings before we begin to save a new set of control sequence meanings (see `\selectlanguage` and `\originalTeX`). Note undefined macros are not undefined any more when saved – they are `\relax`’ed.

`\babel@savecnt`

`\babel@beginsave` The initialization of a new save cycle: reset the counter to zero.

```
1621 \bbl@trace{Macros for saving definitions}
1622 \def\babel@beginsave{\babel@savecnt\z@}
```

Before it’s forgotten, allocate the counter and initialize all.

```
1623 \newcount\babel@savecnt
1624 \babel@beginsave
```

`\babel@save`

`\babel@savevariable` The macro `\babel@save{csname}` saves the current meaning of the control sequence `(csname)` to `\originalTeX` (which has to be expandable, i.e., you shouldn’t let it to `\relax`). To do this, we let the current meaning to a temporary control sequence, the restore commands are appended to `\originalTeX` and the counter is incremented. The macro `\babel@savevariable{variable}` saves the value of the variable. `(variable)` can be anything allowed after the `\the` primitive. To avoid messing saved definitions up, they are saved only the very first time.

```
1625 \def\babel@save#1{%
1626   \def\bbl@tempa{,{#1,}}% Clumsy, for Plain
1627   \expandafter\bbl@add\expandafter\bbl@tempa\expandafter{%
1628     \expandafter{\expandafter,\bbl@savextras,}}%
1629   \expandafter\in@\bbl@tempa
1630   \ifin@%else
1631     \bbl@add\bbl@savextras{,{#1,}}%
1632     \bbl@carg\let\babel@number\babel@savecnt\#1\relax
1633     \toks@{\expandafter{\originalTeX\let#1=}}%
1634     \bbl@exp{%
1635       \def\\\originalTeX{\the\toks@<\babel@number\babel@savecnt>\relax}}%
1636     \advance\babel@savecnt@ne
1637   \fi}
1638 \def\babel@savevariable#1{%
1639   \toks@{\expandafter{\originalTeX #1=}}%
1640   \bbl@exp{\def\\\originalTeX{\the\toks@the#1\relax}}}
```

`\bbl@redefine` To redefine a command, we save the old meaning of the macro. Then we redefine it to call the original macro with the ‘sanitized’ argument. The reason why we do it this way is that we don’t want to redefine the $\TeX$ macros completely in case their definitions change (they have changed in the past). A macro named `\macro` will be saved new control sequences named `\org@macro`.

```
1641 \def\bbl@redefine#1{%
1642   \edef\bbl@tempa{\bbl@stripslash#1}%
1643   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1644   \expandafter\def\csname\bbl@tempa\endcsname}
1645 \@onlypreamble\bbl@redefine
```

`\bbl@redefine@long` This version of `\babel@redefine` can be used to redefine `\long` commands such as `\ifthenelse`.

```
1646 \def\bbl@redefine@long#1{%
1647   \edef\bbl@tempa{\bbl@stripslash#1}%
1648   \expandafter\let\csname org@\bbl@tempa\endcsname#1%
1649   \long\expandafter\def\csname\bbl@tempa\endcsname}
1650 \@onlypreamble\bbl@redefine@long
```

\bbl@redefineroobust For commands that are redefined, but which *might* be robust we need a slightly more intelligent macro. A robust command `foo` is defined to expand to `\protect\foo`. So it is necessary to check whether `\foo` exists. The result is that the command that is being redefined is always robust afterwards. Therefore all we need to do now is define `\foo`.

```

1651 \def\bbl@redefineroobust#1{%
1652   \edef\bbl@tempa{\bbl@stripslash#1}%
1653   \bbl@iifunset{\bbl@tempa\space}%
1654   {\expandafter\let\csname org@\bbl@tempa\endcsname#1%
1655    \bbl@exp{\def\#1{\protect\<\bbl@tempa\space>}}}%
1656   {\bbl@exp{\let\<org@\bbl@tempa>\<\bbl@tempa\space>}}}%
1657   \@namedef{\bbl@tempa\space}}
1658 \onlypreamble\bbl@redefineroobust

```

4.11. French spacing

\bbl@frenchspacing

\bbl@nonfrenchspacing Some languages need to have `\frenchspacing` in effect. Others don't want that. The command `\bbl@frenchspacing` switches it on when it isn't already in effect and `\bbl@nonfrenchspacing` switches it off if necessary.

```

1659 \def\bbl@frenchspacing{%
1660   \ifnum\the\sfcode`\.=\@m
1661     \let\bbl@nonfrenchspacing\relax
1662   \else
1663     \frenchspacing
1664     \let\bbl@nonfrenchspacing\nonfrenchspacing
1665   \fi}
1666 \let\bbl@nonfrenchspacing\nonfrenchspacing

```

A more refined way to switch the catcodes is done with `ini` files. Here an auxiliary macro is defined, but the main part is in `\babelprovide`. This new method should be ideally the default one.

```

1667 \let\bbl@elt\relax
1668 \edef\bbl@fs@chars{%
1669   \bbl@elt{\string.}\@m{3000}\bbl@elt{\string?}\@m{3000}%
1670   \bbl@elt{\string!}\@m{3000}\bbl@elt{\string:}\@m{2000}%
1671   \bbl@elt{\string;}\@m{1500}\bbl@elt{\string,}\@m{1250}}
1672 \def\bbl@pre@fs{%
1673   \def\bbl@elt##1##2##3{\sfcode`##1=\the\sfcode`##1\relax}%
1674   \edef\bbl@save@sfcodes{\bbl@fs@chars}%
1675   \def\bbl@post@fs{%
1676     \bbl@save@sfcodes
1677     \edef\bbl@tempa{\bbl@cl{frspc}}%
1678     \edef\bbl@tempa{\expandafter\@car\bbl@tempa\@nil}%
1679     \if u\bbl@tempa      % do nothing
1680     \else\if n\bbl@tempa % non french
1681       \def\bbl@elt##1##2##3{%
1682         \ifnum\sfcode`##1=##2\relax
1683         \babel@savevariable{\sfcode`##1}%
1684         \sfcode`##1=##3\relax
1685       \fi}%
1686       \bbl@fs@chars
1687     \else\if y\bbl@tempa % french
1688       \def\bbl@elt##1##2##3{%
1689         \ifnum\sfcode`##1=##3\relax
1690         \babel@savevariable{\sfcode`##1}%
1691         \sfcode`##1=##2\relax
1692       \fi}%
1693       \bbl@fs@chars
1694     \fi\fi\fi}

```

4.12. Hyphens

\babelhyphenation This macro saves hyphenation exceptions. Two macros are used to store them: `\bbl@hyphenation@` for the global ones and `\bbl@hyphenation@(<language>)` for language ones. See

\bbl@patterns above for further details. We make sure there is a space between words when multiple commands are used.

```

1695 \bbl@trace{Hyphens}
1696 \@onlypreamble\babelhyphenation
1697 \AtEndOfPackage{%
1698   \newcommand\babelhyphenation[2][\@empty]{%
1699     \ifx\bbl@hyphenation@\relax
1700       \let\bbl@hyphenation@\@empty
1701     \fi
1702     \ifx\bbl@hyphlist@\@empty\else
1703       \bbl@warning{%
1704         You must not intermingle \string\selectlanguage\space and\\%
1705         \string\babelhyphenation\space or some exceptions will not\\%
1706         be taken into account. Reported}%
1707       \fi
1708       \ifx\@empty#1%
1709         \protected@edef\bbl@hyphenation@{\bbl@hyphenation@\space#2}%
1710       \else
1711         \bbl@vforeach{#1}{%
1712           \def\bbl@tempa{##1}%
1713           \bbl@fixname\bbl@tempa
1714           \bbl@iflanguage\bbl@tempa{%
1715             \bbl@csarg\protected@edef{hyphenation@\bbl@tempa}{%
1716               \bbl@ifunset{bbl@hyphenation@\bbl@tempa}%
1717               {}%
1718               {\csname bbl@hyphenation@\bbl@tempa\endcsname\space}%
1719               #2}}}%
1720         \fi}}

```

\babelhyphenmins Only L^AT_EX (basically because it's defined with a L^AT_EX tool).

```

1721 \ifx\NewDocumentCommand\@undefined\else
1722   \NewDocumentCommand\babelhyphenmins{sommo}{%
1723     \IfNoValueTF{#2}%
1724     {\protected@edef\bbl@hyphenmins@{\set@hyphenmins{#3}{#4}}}%
1725     \IfValueT{#5}{%
1726       \protected@edef\bbl@hyphenatmin@{\hyphenationmin=#5\relax}}%
1727     \IfBooleanT{#1}{%
1728       \lefthyphenmin=#3\relax
1729       \righthyphenmin=#4\relax
1730       \IfValueT{#5}{\hyphenationmin=#5\relax}}}%
1731     {\edef\bbl@tempb{\zap@space#2 \@empty}%
1732      \bbl@for\bbl@tempa\bbl@tempb{%
1733        \@namedef{bbl@hyphenmins@\bbl@tempa}{\set@hyphenmins{#3}{#4}}}%
1734      \IfValueT{#5}{%
1735        \@namedef{bbl@hyphenatmin@\bbl@tempa}{\hyphenationmin=#5\relax}}}%
1736      \IfBooleanT{#1}{\bbl@error{hyphenmins-args}{}}}%
1737 \fi

```

\bbl@allowhyphens This macro makes hyphenation possible. Basically its definition is nothing more than \nobreak \hskip 0pt plus 0pt. T_EX begins and ends a word for hyphenation at a glue node. The penalty prevents a linebreak at this glue node.

```

1738 \def\bbl@allowhyphens{\ifvmode\else\nobreak\hskip\z@skip\fi}
1739 \def\bbl@t@one{Tl}
1740 \def\allowhyphens{\ifx\cf@encoding\bbl@t@one\else\bbl@allowhyphens\fi}

```

\babelhyphen Macros to insert common hyphens. Note the space before @ in \babelhyphen. Instead of protecting it with \DeclareRobustCommand, which could insert a \relax, we use the same procedure as shorthands, with \active@prefix.

```

1741 \newcommand\babelnullhyphen{\char\hyphenchar\font}
1742 \def\babelhyphen{\active@prefix\babelhyphen\bbl@hyphen}
1743 \def\bbl@hyphen{%

```

```

1744 \ifstar{\bbl@hyphen@i @}{\bbl@hyphen@i\@empty}}
1745 \def\bbl@hyphen@i#1#2{%
1746 \lowercase{\bbl@ifunset{\bbl@hy#1#2\@empty}}}%
1747 {\csname bbl@lusehyphen\endcsname{\discretionary{#2}{}{#2}}}%
1748 {\lowercase{\csname bbl@hy#1#2\@empty\endcsname}}}

```

The following two commands are used to wrap the “hyphen” and set the behavior of the rest of the word – the version with a single @ is used when further hyphenation is allowed, while that with @@ if no more hyphens are allowed. In both cases, if the hyphen is preceded by a positive space, breaking after the hyphen is disallowed.

There should not be a discretionary after a hyphen at the beginning of a word, so it is prevented if preceded by a skip. Unfortunately, this does handle cases like “(-suffix)”. \nobreak is always preceded by \leavevmode, in case the shorthand starts a paragraph.

```

1749 \def\bbl@usehyphen#1{%
1750 \leavevmode
1751 \ifdim\lastskip>z@\mbox{#1}\else\nobreak#1\fi
1752 \nobreak\hskip\z@skip}
1753 \def\bbl@usehyphen#1{%
1754 \leavevmode\ifdim\lastskip>z@\mbox{#1}\else#1\fi}

```

The following macro inserts the hyphen char.

```

1755 \def\bbl@hyphenchar{%
1756 \ifnum\hyphenchar\font=\m@ne
1757 \babe\nullhyphen
1758 \else
1759 \char\hyphenchar\font
1760 \fi}

```

Finally, we define the hyphen “types”. Their names will not change, so you may use them in ldf’s. After a space, the \mbox in \bbl@hy@nobreak is redundant.

```

1761 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1762 \def\bbl@hy@soft{\bbl@usehyphen{\discretionary{\bbl@hyphenchar}{}}{}}
1763 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1764 \def\bbl@hy@hard{\bbl@usehyphen\bbl@hyphenchar}
1765 \def\bbl@hy@nobreak{\bbl@usehyphen{\mbox{\bbl@hyphenchar}}}
1766 \def\bbl@hy@nobreak{\mbox{\bbl@hyphenchar}}
1767 \def\bbl@hy@repeat{%
1768 \bbl@usehyphen{%
1769 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1770 \def\bbl@hy@repeat{%
1771 \bbl@usehyphen{%
1772 \discretionary{\bbl@hyphenchar}{\bbl@hyphenchar}{\bbl@hyphenchar}}}
1773 \def\bbl@hy@empty{\hskip\z@skip}
1774 \def\bbl@hy@empty{\discretionary{}{}{}}

```

\bbl@disc For some languages the macro \bbl@disc is used to ease the insertion of discretionaries for letters that behave ‘abnormally’ at a breakpoint.

```

1775 \def\bbl@disc#1#2{\nobreak\discretionary{#2-}{}{#1}\bbl@allowhyphens}

```

4.13. Multiencoding strings

The aim following commands is to provide a common interface for strings in several encodings. They also contains several hooks which can be used by luatex and xetex. The code is organized here with pseudo-guards, so we start with the basic commands.

Tools But first, a tool. It makes global a local variable. This is not the best solution, but it works.

```

1776 \bbl@trace{Multiencoding strings}
1777 \def\bbl@tglobal#1{\global\let#1#1}

```

The following option is currently no-op. It was meant for the deprecated \SetCase.

```

1778 <(*More package options)> ≡
1779 \DeclareOption{nocase}{}
1780 <(/More package options)>

```

The following package options control the behavior of `\SetString`.

```

1781 ({*More package options}) ≡
1782 \let\bbl@opt@strings\@nnil % accept strings=value
1783 \DeclareOption{strings}{\def\bbl@opt@strings{\BabelStringsDefault}}
1784 \DeclareOption{strings=encoded}{\let\bbl@opt@strings\relax}
1785 \def\BabelStringsDefault{generic}
1786 ({/More package options})

```

Main command This is the main command. With the first use it is redefined to omit the basic setup in subsequent blocks. We make sure strings contain actual letters in the range 128-255, not active characters.

```

1787 \@onlypreamble\StartBabelCommands
1788 \def\StartBabelCommands{%
1789   \begingroup
1790   \@tempcnta="7F
1791   \def\bbl@tempa{%
1792     \ifnum\@tempcnta>"FF\else
1793       \catcode\@tempcnta=11
1794       \advance\@tempcnta\@ne
1795       \expandafter\bbl@tempa
1796     \fi}%
1797   \bbl@tempa
1798   <@Macros local to BabelCommands@>
1799   \def\bbl@provstring##1##2{%
1800     \providecommand##1{##2}%
1801     \bbl@tglobal##1}%
1802   \global\let\bbl@scafter\@empty
1803   \let\StartBabelCommands\bbl@startcmds
1804   \ifx\BabelLanguages\relax
1805     \let\BabelLanguages\CurrentOption
1806   \fi
1807   \begingroup
1808   \let\bbl@screset\@nnil % local flag - disable 1st stopcommands
1809   \StartBabelCommands}
1810 \def\bbl@startcmds{%
1811   \ifx\bbl@screset\@nnil\else
1812     \bbl@usehooks{stopcommands}{}%
1813   \fi
1814   \endgroup
1815   \begingroup
1816   \@ifstar
1817   {\ifx\bbl@opt@strings\@nnil
1818     \let\bbl@opt@strings\BabelStringsDefault
1819     \fi
1820     \bbl@startcmds@i}%
1821   \bbl@startcmds@i}
1822 \def\bbl@startcmds@i#1#2{%
1823   \edef\bbl@L{\zap@space#1 \@empty}%
1824   \edef\bbl@G{\zap@space#2 \@empty}%
1825   \bbl@startcmds@ii}
1826 \let\bbl@startcommands\StartBabelCommands

```

Parse the encoding info to get the label, input, and font parts.

Select the behavior of `\SetString`. There are two main cases, depending of if there is an optional argument: without it and `strings=encoded`, strings are defined always; otherwise, they are set only if they are still undefined (i.e., fallback values). With labelled blocks and `strings=encoded`, define the strings, but with another value, define strings only if the current label or font encoding is the value of strings; otherwise (i.e., no strings or a block whose label is not in `strings=`) do nothing.

We presume the current block is not loaded, and therefore set (above) a couple of default values to gobble the arguments. Then, these macros are redefined if necessary according to several parameters.

```

1827 \newcommand\bbl@startcmds@ii[1][\@empty]{%

```

```

1828 \let\SetString\@gobbletwo
1829 \let\bbl@stringdef\@gobbletwo
1830 \let\AfterBabelCommands\@gobble
1831 \ifx\@empty#1%
1832   \def\bbl@sc@label{generic}%
1833   \def\bbl@encstring##1##2{%
1834     \ProvideTextCommandDefault##1{##2}%
1835     \bbl@tglobal##1%
1836     \expandafter\bbl@tglobal\csname\string?\string##1\endcsname}%
1837   \let\bbl@sctest\in@true
1838 \else
1839   \let\bbl@sc@charset\space % <- zapped below
1840   \let\bbl@sc@fontenc\space % <- " "
1841   \def\bbl@tempa##1=##2\@nil{%
1842     \bbl@csarg\edef{sc@zap@space##1 \@empty}{##2 }}%
1843   \bbl@vforeach{label=#1}{\bbl@tempa##1\@nil}%
1844   \def\bbl@tempa##1 ##2{% space -> comma
1845     ##1%
1846     \ifx\@empty##2\else\ifx,##1,\else,\fi\bbl@afterfi\bbl@tempa##2\fi}%
1847   \edef\bbl@sc@fontenc{\expandafter\bbl@tempa\bbl@sc@fontenc\@empty}%
1848   \edef\bbl@sc@label{\expandafter\zap@space\bbl@sc@label\@empty}%
1849   \edef\bbl@sc@charset{\expandafter\zap@space\bbl@sc@charset\@empty}%
1850   \def\bbl@encstring##1##2{%
1851     \bbl@foreach\bbl@sc@fontenc{%
1852       \bbl@ifunset{T@###1}%
1853       {}%
1854       {\ProvideTextCommand##1{###1}{##2}%
1855         \bbl@tglobal##1%
1856         \expandafter
1857         \bbl@tglobal\csname###1\string##1\endcsname}}}%
1858   \def\bbl@sctest{%
1859     \bbl@xin@{\bbl@opt@strings,}{\bbl@sc@label,\bbl@sc@fontenc,}%
1860 \fi
1861 \ifx\bbl@opt@strings\@nnil % i.e., no strings key -> defaults
1862 \else\ifx\bbl@opt@strings\relax % i.e., strings=encoded
1863   \let\AfterBabelCommands\bbl@aftercmds
1864   \let\SetString\bbl@setstring
1865   \let\bbl@stringdef\bbl@encstring
1866 \else % i.e., strings=value
1867   \bbl@sctest
1868 \fin@
1869   \let\AfterBabelCommands\bbl@aftercmds
1870   \let\SetString\bbl@setstring
1871   \let\bbl@stringdef\bbl@provstring
1872 \fi\fi\fi
1873 \bbl@scswitch
1874 \ifx\bbl@G\@empty
1875   \def\SetString##1##2{%
1876     \bbl@error{missing-group}{##1}{}}%
1877 \fi
1878 \ifx\@empty#1%
1879   \bbl@usehooks{defaultcommands}{}%
1880 \else
1881   \@expandtwoargs
1882   \bbl@usehooks{encodedcommands}{\bbl@sc@charset}\bbl@sc@fontenc}%
1883 \fi}

```

There are two versions of `\bbl@scswitch`. The first version is used when `ldfs` are read, and it makes sure `\langle group \rangle \langle language \rangle` is reset, but only once (`\bbl@screset` is used to keep track of this). The second version is used in the preamble and packages loaded after `babel` and does nothing.

The macro `\bbl@forlang` loops `\bbl@L` but its body is executed only if the value is in `\BabelLanguages` (inside `babel`) or `\date\langle language \rangle` is defined (after `babel` has been loaded). There are also two version of `\bbl@forlang`. The first one skips the current iteration if the language is not in `\BabelLanguages` (used in `ldfs`), and the second one skips undefined languages (after `babel` has

been loaded).

```

1884 \def\bbl@forlang#1#2{%
1885   \bbl@for#1\bbl@L{%
1886     \bbl@xin@{,#1,},{,\BabelLanguages,}%
1887     \ifin@#2\relax\fi}}
1888 \def\bbl@scswitch{%
1889   \bbl@forlang\bbl@tempa{%
1890     \ifx\bbl@G\@empty\else
1891       \ifx\SetString@gobbletwo\else
1892         \edef\bbl@GL{\bbl@G\bbl@tempa}%
1893         \bbl@xin@{,\bbl@GL,}{,\bbl@screset,}%
1894         \ifin@\else
1895           \global\expandafter\let\csname\bbl@GL\endcsname\@undefined
1896           \xdef\bbl@screset{\bbl@screset,\bbl@GL}%
1897         \fi
1898       \fi
1899     \fi}}
1900 \AtEndOfPackage{%
1901   \def\bbl@forlang#1#2{\bbl@for#1\bbl@L{\bbl@ifunset{date#1}{\#2}}}%
1902   \let\bbl@scswitch\relax}
1903 \@onlypreamble\EndBabelCommands
1904 \def\EndBabelCommands{%
1905   \bbl@usehooks{stopcommands}{}%
1906   \endgroup
1907   \endgroup
1908   \bbl@scafter}
1909 \let\bbl@endcommands\EndBabelCommands

```

Now we define commands to be used inside \StartBabelCommands.

Strings The following macro is the actual definition of \SetString when it is “active”

First save the “switcher”. Create it if undefined. Strings are defined only if undefined (i.e., like \providescommand). With the event stringprocess you can preprocess the string by manipulating the value of \BabelString. If there are several hooks assigned to this event, preprocessing is done in the same order as defined. Finally, the string is set.

```

1910 \def\bbl@setstring#1#2{% e.g., \prefacename{<string>}
1911   \bbl@forlang\bbl@tempa{%
1912     \edef\bbl@LC{\bbl@tempa\bbl@stripslash#1}%
1913     \bbl@ifunset{\bbl@LC}% e.g., \germanchaptername
1914     {\bbl@exp{%
1915       \global\\bbl@add\<\bbl@G\bbl@tempa>{\\bbl@scset\\#1\<\bbl@LC>}}}%
1916     }%
1917     \def\BabelString{#2}%
1918     \bbl@usehooks{stringprocess}{}%
1919     \expandafter\bbl@stringdef
1920     \csname\bbl@LC\expandafter\endcsname\expandafter{\BabelString}}

```

A little auxiliary command sets the string. Formerly used with casing. Very likely no longer necessary, although it’s used in \setlocalecaption.

```

1921 \def\bbl@scset#1#2{\def#1{#2}}

```

Define \SetStringLoop, which is actually set inside \StartBabelCommands. The current definition is somewhat complicated because we need a count, but \count@ is not under our control (remember \SetString may call hooks). Instead of defining a dedicated count, we just “pre-expand” its value.

```

1922 <(*Macros local to BabelCommands)> ≡
1923 \def\SetStringLoop##1#2{%
1924   \def\bbl@templ####1{\expandafter\noexpand\csname##1\endcsname}%
1925   \count@\z@
1926   \bbl@loop\bbl@tempa{##2}{% empty items and spaces are ok
1927     \advance\count@\@ne
1928     \toks@\expandafter{\bbl@tempa}%
1929     \bbl@exp{%
1930       \\SetString\bbl@templ{\romannumeral\count@}{\the\toks@}%

```



```

1931 \count@=\the\count@\relax}}}%
1932 <</Macros local to BabelCommands>>

```

Delaying code Now the definition of `\AfterBabelCommands` when it is activated.

```

1933 \def\bbl@aftercmds#1{%
1934 \toks@\expandafter{\bbl@scafter#1}%
1935 \xdef\bbl@scafter{\the\toks@}}

```

Case mapping The command `\SetCase` is deprecated. Currently it consists in a definition with a hack just for backward compatibility in the macro mapping.

```

1936 <<*Macros local to BabelCommands>> ≡
1937 \newcommand\SetCase[3][]{%
1938 \def\bbl@tempa####1####2{%
1939 \ifx####1\@empty\else
1940 \bbl@carg\bbl@add{extras\CurrentOption}{%
1941 \bbl@carg\babel@save{c__text_uppercase_\string####1_tl}%
1942 \bbl@carg\def{c__text_uppercase_\string####1_tl}{####2}%
1943 \bbl@carg\babel@save{c__text_lowercase_\string####2_tl}%
1944 \bbl@carg\def{c__text_lowercase_\string####2_tl}{####1}}%
1945 \expandafter\bbl@tempa
1946 \fi}%
1947 \bbl@tempa##1\@empty\@empty
1948 \bbl@carg\bbl@tglobal{extras\CurrentOption}}%
1949 <</Macros local to BabelCommands>>

```

Macros to deal with case mapping for hyphenation. To decide if the document is monolingual or multilingual, we make a rough guess – just see if there is a comma in the languages list, built in the first pass of the package options.

```

1950 <<*Macros local to BabelCommands>> ≡
1951 \newcommand\SetHyphenMap[1]{%
1952 \bbl@forlang\bbl@tempa{%
1953 \expandafter\bbl@stringdef
1954 \csname\bbl@tempa @bbl@hyphenmap\endcsname{##1}}}%
1955 <</Macros local to BabelCommands>>

```

There are 3 helper macros which do most of the work for you.

```

1956 \newcommand\BabelLower[2]{% one to one.
1957 \ifnum\lccode#1=#2\else
1958 \babel@savevariable{\lccode#1}%
1959 \lccode#1=#2\relax
1960 \fi}
1961 \newcommand\BabelLowerMM[4]{% many-to-many
1962 \@tempcnta=#1\relax
1963 \@tempcntb=#4\relax
1964 \def\bbl@tempa{%
1965 \ifnum\@tempcnta>#2\else
1966 \@expandtwoargs\BabelLower{\the\@tempcnta}{\the\@tempcntb}%
1967 \advance\@tempcnta#3\relax
1968 \advance\@tempcntb#3\relax
1969 \expandafter\bbl@tempa
1970 \fi}%
1971 \bbl@tempa}
1972 \newcommand\BabelLowerM0[4]{% many-to-one
1973 \@tempcnta=#1\relax
1974 \def\bbl@tempa{%
1975 \ifnum\@tempcnta>#2\else
1976 \@expandtwoargs\BabelLower{\the\@tempcnta}{#4}%
1977 \advance\@tempcnta#3
1978 \expandafter\bbl@tempa
1979 \fi}%
1980 \bbl@tempa}

```

The following package options control the behavior of hyphenation mapping.

```

1981 (<{*More package options}>) ≡
1982 \DeclareOption{hyphenmap=off}{\chardef\bbl@opt@hyphenmap\z@}
1983 \DeclareOption{hyphenmap=first}{\chardef\bbl@opt@hyphenmap\@ne}
1984 \DeclareOption{hyphenmap=select}{\chardef\bbl@opt@hyphenmap\tw@}
1985 \DeclareOption{hyphenmap=other}{\chardef\bbl@opt@hyphenmap\thr@@}
1986 \DeclareOption{hyphenmap=other*}{\chardef\bbl@opt@hyphenmap4\relax}
1987 (<{/More package options}>)

```

Initial setup to provide a default behavior if hyphenmap is not set.

```

1988 \AtEndOfPackage{%
1989   \ifx\bbl@opt@hyphenmap\undefined
1990     \bbl@xin@{,}{\bbl@language@opts}%
1991     \chardef\bbl@opt@hyphenmap\ifin@4\else\@ne\fi
1992   \fi}

```

4.14. Tailor captions

A general tool for resetting the caption names with a unique interface. With the old way, which mixes the switcher and the string, we convert it to the new one, which separates these two steps.

```

1993 \newcommand\setlocalecaption{%
1994   \ifstar\bbl@setcaption@s\bbl@setcaption@x}
1995 \def\bbl@setcaption@x#1#2#3{% language caption-name string
1996   \bbl@trim@def\bbl@tempa{#2}%
1997   \bbl@xin@{.template}{\bbl@tempa}%
1998   \ifin@
1999     \bbl@ini@captions@template{#3}{#1}%
2000   \else
2001     \edef\bbl@tempd{%
2002       \expandafter\expandafter\expandafter
2003       \strip@prefix\expandafter\meaning\csname captions#1\endcsname}%
2004     \bbl@xin@
2005       {\expandafter\string\csname #2name\endcsname}%
2006       {\bbl@tempd}%
2007     \ifin@ % Renew caption
2008       \bbl@xin@{\string\bbl@scset}{\bbl@tempd}%
2009       \ifin@
2010         \bbl@exp{%
2011           \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2012           {\bbl@scset\<#2name>\<#1#2name>}%
2013           {}}%
2014         \else % Old way converts to new way
2015           \bbl@ifunset{#1#2name}%
2016             {\bbl@exp{%
2017               \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2018               \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2019               {\def\<#2name>\<#1#2name>}}%
2020               {}}}%
2021           {}%
2022         \fi
2023       \else
2024         \bbl@xin@{\string\bbl@scset}{\bbl@tempd}% New
2025         \ifin@ % New way
2026           \bbl@exp{%
2027             \\bbl@add\<captions#1>\bbl@scset\<#2name>\<#1#2name>}%
2028             \\bbl@ifsamestring{\bbl@tempa}{\language name}%
2029             {\bbl@scset\<#2name>\<#1#2name>}%
2030             {}}%
2031           \else % Old way, but defined in the new way
2032             \bbl@exp{%
2033               \\bbl@add\<captions#1>\def\<#2name>\<#1#2name>}}%
2034               \\bbl@ifsamestring{\bbl@tempa}{\language name}%

```

```

2035         {\def\<#2name>{\<#1#2name>}}}%
2036         {}}%
2037     \fi%
2038 \fi
2039 \@namedef{#1#2name}{#3}%
2040 \toks@{\expandafter{\bbl@captionslist}%
2041 \bbl@exp{\in{\<#2name>}{\the\toks@}}}%
2042 \ifin@else
2043     \bbl@exp{\bbl@add{\bbl@captionslist{\<#2name>}}}%
2044     \bbl@tglobal\bbl@captionslist
2045 \fi
2046 \fi}

```

4.15. Making glyphs available

This section makes a number of glyphs available that either do not exist in the OT1 encoding and have to be ‘faked’, or that are not accessible through Tlenc.def.

\set@low@box The following macro is used to lower quotes to the same level as the comma. It prepares its argument in box register 0.

```

2047 \bbl@trace{Macros related to glyphs}
2048 \def\set@low@box#1{\setbox\tw@{\hbox{,}}\setbox\z@{\hbox{#1}}%
2049 \dimen\z@{\ht\z@ \advance\dimen\z@ -\ht\tw@}%
2050 \setbox\z@{\hbox{\lower\dimen\z@ \box\z@}\ht\z@{\ht\tw@ \dp\z@\dp\tw@}}

```

\save@sf@q The macro \save@sf@q is used to save and reset the current space factor.

```

2051 \def\save@sf@q#1{\leavevmode
2052 \begingroup
2053 \edef\@SF{\spacefactor\the\spacefactor}#1\@SF
2054 \endgroup}

```

4.15.1. Quotation marks

\quotedblbase In the T1 encoding the opening double quote at the baseline is available as a separate character, accessible via \quotedblbase. In the OT1 encoding it is not available, therefore we make it available by lowering the normal open quote character to the baseline.

```

2055 \ProvideTextCommand{\quotedblbase}{OT1}{%
2056 \save@sf@q{\set@low@box{\textquotedblright/}}%
2057 \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2058 \ProvideTextCommandDefault{\quotedblbase}{%
2059 \UseTextSymbol{OT1}{\quotedblbase}}

```

\quotesinglbase We also need the single quote character at the baseline.

```

2060 \ProvideTextCommand{\quotesinglbase}{OT1}{%
2061 \save@sf@q{\set@low@box{\textquoteright/}}%
2062 \box\z@\kern-.04em\bbl@allowhyphens}}

```

Make sure that when an encoding other than OT1 or T1 is used this glyph can still be typeset.

```

2063 \ProvideTextCommandDefault{\quotesinglbase}{%
2064 \UseTextSymbol{OT1}{\quotesinglbase}}

```

\guillemetleft

\guillemetright The guillemet characters are not available in OT1 encoding. They are faked. (Wrong names with o preserved for compatibility.)

```
2065 \ProvideTextCommand{\guillemetleft}{OT1}{%
2066   \ifmmode
2067     \ll
2068   \else
2069     \save@sf@q{\nobreak
2070       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2071   \fi}
2072 \ProvideTextCommand{\guillemetright}{OT1}{%
2073   \ifmmode
2074     \gg
2075   \else
2076     \save@sf@q{\nobreak
2077       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2078   \fi}
2079 \ProvideTextCommand{\guillemotleft}{OT1}{%
2080   \ifmmode
2081     \ll
2082   \else
2083     \save@sf@q{\nobreak
2084       \raise.2ex\hbox{$\scriptscriptstyle\ll$}\bbl@allowhyphens}%
2085   \fi}
2086 \ProvideTextCommand{\guillemotright}{OT1}{%
2087   \ifmmode
2088     \gg
2089   \else
2090     \save@sf@q{\nobreak
2091       \raise.2ex\hbox{$\scriptscriptstyle\gg$}\bbl@allowhyphens}%
2092   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2093 \ProvideTextCommandDefault{\guillemetleft}{%
2094   \UseTextSymbol{OT1}{\guillemetleft}}
2095 \ProvideTextCommandDefault{\guillemetright}{%
2096   \UseTextSymbol{OT1}{\guillemetright}}
2097 \ProvideTextCommandDefault{\guillemotleft}{%
2098   \UseTextSymbol{OT1}{\guillemotleft}}
2099 \ProvideTextCommandDefault{\guillemotright}{%
2100   \UseTextSymbol{OT1}{\guillemotright}}
```

\guilsinglleft

\guilsinglright The single guillemets are not available in OT1 encoding. They are faked.

```
2101 \ProvideTextCommand{\guilsinglleft}{OT1}{%
2102   \ifmmode
2103     <%
2104   \else
2105     \save@sf@q{\nobreak
2106       \raise.2ex\hbox{$\scriptscriptstyle<$}\bbl@allowhyphens}%
2107   \fi}
2108 \ProvideTextCommand{\guilsinglright}{OT1}{%
2109   \ifmmode
2110     >%
2111   \else
2112     \save@sf@q{\nobreak
2113       \raise.2ex\hbox{$\scriptscriptstyle>$}\bbl@allowhyphens}%
2114   \fi}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2115 \ProvideTextCommandDefault{\guilsinglleft}{%
2116   \UseTextSymbol{OT1}{\guilsinglleft}}
2117 \ProvideTextCommandDefault{\guilsinglright}{%
2118   \UseTextSymbol{OT1}{\guilsinglright}}
```

4.15.2. Letters

\ij

\IJ The dutch language uses the letter 'ij'. It is available in T1 encoded fonts, but not in the OT1 encoded fonts. Therefore we fake it for the OT1 encoding.

```
2119 \DeclareTextCommand{\ij}{OT1}{%
2120   i\kern-0.02em\bbI\allowhyphens j}
2121 \DeclareTextCommand{\IJ}{OT1}{%
2122   I\kern-0.02em\bbI\allowhyphens J}
2123 \DeclareTextCommand{\ij}{T1}{\char188}
2124 \DeclareTextCommand{\IJ}{T1}{\char156}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2125 \ProvideTextCommandDefault{\ij}{%
2126   \UseTextSymbol{OT1}{\ij}}
2127 \ProvideTextCommandDefault{\IJ}{%
2128   \UseTextSymbol{OT1}{\IJ}}
```

\dj

\DJ The croatian language needs the letters \dj and \DJ; they are available in the T1 encoding, but not in the OT1 encoding by default.

Some code to construct these glyphs for the OT1 encoding was made available to me by Stipčević Mario, (stipcevic@olimp.irb.hr).

```
2129 \def\crrtic@{\hrule height0.1ex width0.3em}
2130 \def\crrtic@{\hrule height0.1ex width0.33em}
2131 \def\ddj@{%
2132   \setbox0\hbox{d}\dimen@=\ht0
2133   \advance\dimen@lex
2134   \dimen@.45\dimen@
2135   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2136   \advance\dimen@ii.5ex
2137   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2138 \def\DDJ@{%
2139   \setbox0\hbox{D}\dimen@=.55\ht0
2140   \dimen@ii\expandafter\rem@pt\the\fontdimen\@ne\font\dimen@
2141   \advance\dimen@ii.15ex % correction for the dash position
2142   \advance\dimen@ii-.15\fontdimen7\font % correction for cmtt font
2143   \dimen\thr@@\expandafter\rem@pt\the\fontdimen7\font\dimen@
2144   \leavevmode\rlap{\raise\dimen@\hbox{\kern\dimen@ii\vbox{\crrtic@}}}}
2145 %
2146 \DeclareTextCommand{\dj}{OT1}{\ddj@ d}
2147 \DeclareTextCommand{\DJ}{OT1}{\DDJ@ D}
```

Make sure that when an encoding other than OT1 or T1 is used these glyphs can still be typeset.

```
2148 \ProvideTextCommandDefault{\dj}{%
2149   \UseTextSymbol{OT1}{\dj}}
2150 \ProvideTextCommandDefault{\DJ}{%
2151   \UseTextSymbol{OT1}{\DJ}}
```

\SS For the T1 encoding \SS is defined and selects a specific glyph from the font, but for other encodings it is not available. Therefore we make it available here.

```
2152 \DeclareTextCommand{\SS}{OT1}{SS}
2153 \ProvideTextCommandDefault{\SS}{\UseTextSymbol{OT1}{\SS}}
```

4.15.3. Shorthands for quotation marks

Shorthands are provided for a number of different quotation marks, which make them usable both outside and inside mathmode. They are defined with \ProvideTextCommandDefault, but this is very likely not required because their definitions are based on encoding-dependent macros.

\glq

\grq The ‘german’ single quotes.

```
2154 \ProvideTextCommandDefault{\glq}{%  
2155 \textormath{\quotesinglbase}{\mbox{\quotesinglbase}}}
```

The definition of \grq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2156 \ProvideTextCommand{\grq}{T1}{%  
2157 \textormath{\kern\z@\textquoteleft}{\mbox{\textquoteleft}}}  
2158 \ProvideTextCommand{\grq}{TU}{%  
2159 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}  
2160 \ProvideTextCommand{\grq}{OT1}{%  
2161 \save@sf@q{\kern-.0125em  
2162 \textormath{\textquoteleft}{\mbox{\textquoteleft}}}%  
2163 \kern.07em\relax}}  
2164 \ProvideTextCommandDefault{\grq}{\UseTextSymbol{OT1}\grq}
```

\glqq

\grqq The ‘german’ double quotes.

```
2165 \ProvideTextCommandDefault{\glqq}{%  
2166 \textormath{\quotedblbase}{\mbox{\quotedblbase}}}
```

The definition of \grqq depends on the fontencoding. With T1 encoding no extra kerning is needed.

```
2167 \ProvideTextCommand{\grqq}{T1}{%  
2168 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}  
2169 \ProvideTextCommand{\grqq}{TU}{%  
2170 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}  
2171 \ProvideTextCommand{\grqq}{OT1}{%  
2172 \save@sf@q{\kern-.07em  
2173 \textormath{\textquotedblleft}{\mbox{\textquotedblleft}}}%  
2174 \kern.07em\relax}}  
2175 \ProvideTextCommandDefault{\grqq}{\UseTextSymbol{OT1}\grqq}
```

\flq

\frq The ‘french’ single guillemets.

```
2176 \ProvideTextCommandDefault{\flq}{%  
2177 \textormath{\guilsinglleft}{\mbox{\guilsinglleft}}}  
2178 \ProvideTextCommandDefault{\frq}{%  
2179 \textormath{\guilsinglright}{\mbox{\guilsinglright}}}
```

\flqq

\frqq The ‘french’ double guillemets.

```
2180 \ProvideTextCommandDefault{\flqq}{%  
2181 \textormath{\guillemetleft}{\mbox{\guillemetleft}}}  
2182 \ProvideTextCommandDefault{\frqq}{%  
2183 \textormath{\guillemetright}{\mbox{\guillemetright}}}
```

4.15.4. Umlauts and tremas

The command \ " needs to have a different effect for different languages. For German for instance, the ‘umlaut’ should be positioned lower than the default position for placing it over the letters a, o, u, A, O and U. When placed over an e, i, E or I it can retain its normal position. For Dutch the same glyph is always placed in the lower position.

\umlauthigh

\umlautlow To be able to provide both positions of `\` we provide two commands to switch the positioning, the default will be `\umlauthigh` (the normal positioning).

```

2184 \def\umlauthigh{%
2185   \def\bbbl@umlauta##1{\leavevmode\bgroup%
2186     \accent\csname\f@encoding dqpos\endcsname
2187     ##1\bbbl@allowhyphens\egroup}%
2188   \let\bbbl@umlaute\bbbl@umlauta}
2189 \def\umlautlow{%
2190   \def\bbbl@umlauta{\protect\lower@umlaut}}
2191 \def\umlautelow{%
2192   \def\bbbl@umlaute{\protect\lower@umlaut}}
2193 \umlauthigh

```

\lower@umlaut Used to position the `\` closer to the letter. We want the umlaut character lowered, nearer to the letter. To do this we need an extra (*dimen*) register.

```

2194 \expandafter\ifx\csname U@D\endcsname\relax
2195   \csname newdimen\endcsname U@D
2196 \fi

```

The following code fools $\TeX$'s `make\_accent` procedure about the current x-height of the font to force another placement of the umlaut character. First we have to save the current x-height of the font, because we'll change this font dimension and this is always done globally.

Then we compute the new x-height in such a way that the umlaut character is lowered to the base character. The value of `.45ex` depends on the METAFONT parameters with which the fonts were built. (Just try out, which value will look best.) If the new x-height is too low, it is not changed. Finally we call the `\accent` primitive, reset the old x-height and insert the base character in the argument.

```

2197 \def\lower@umlaut#1{%
2198   \leavevmode\bgroup
2199   \U@D lex%
2200   {\setbox\z@\hbox{%
2201     \char\csname\f@encoding dqpos\endcsname}%
2202     \dimen@ -.45ex\advance\dimen@ht\z@
2203     \ifdim lex<\dimen@ \fontdimen5\font\dimen@ \fi}%
2204   \accent\csname\f@encoding dqpos\endcsname
2205   \fontdimen5\font\U@D #1%
2206   \egroup}

```

For all vowels we declare `\` to be a composite command which uses `\bbbl@umlauta` or `\bbbl@umlaute` to position the umlaut character. We need to be sure that these definitions override the ones that are provided when the package `fontenc` with option `OT1` is used. Therefore these declarations are postponed until the beginning of the document. Note these definitions only apply to some languages, but `babel` sets them for *all* languages – you may want to redefine `\bbbl@umlauta` and/or `\bbbl@umlaute` for a language in the corresponding `ldf` (using the `babel` switching mechanism, of course).

```

2207 \AtBeginDocument{%
2208   \DeclareTextCompositeCommand{\}{OT1}{a}{\bbbl@umlauta{a}}%
2209   \DeclareTextCompositeCommand{\}{OT1}{e}{\bbbl@umlaute{e}}%
2210   \DeclareTextCompositeCommand{\}{OT1}{i}{\bbbl@umlaute{i}}%
2211   \DeclareTextCompositeCommand{\}{OT1}{\i}{\bbbl@umlaute{i}}%
2212   \DeclareTextCompositeCommand{\}{OT1}{o}{\bbbl@umlauta{o}}%
2213   \DeclareTextCompositeCommand{\}{OT1}{u}{\bbbl@umlauta{u}}%
2214   \DeclareTextCompositeCommand{\}{OT1}{A}{\bbbl@umlauta{A}}%
2215   \DeclareTextCompositeCommand{\}{OT1}{E}{\bbbl@umlaute{E}}%
2216   \DeclareTextCompositeCommand{\}{OT1}{I}{\bbbl@umlaute{I}}%
2217   \DeclareTextCompositeCommand{\}{OT1}{O}{\bbbl@umlauta{O}}%
2218   \DeclareTextCompositeCommand{\}{OT1}{U}{\bbbl@umlauta{U}}%

```

Finally, make sure the default hyphenrules are defined (even if empty). For internal use, another empty `\language` is defined. Currently used in `Amharic`.

```

2219 \ifx\l@english\undefined
2220   \chardef\l@english\z@
2221 \fi

```

```

2222 % The following is used to cancel rules in ini files (see Amharic).
2223 \ifx\l@unhyphenated\@undefined
2224   \newlanguage\l@unhyphenated
2225 \fi

```

4.16. Layout

Layout is mainly intended to set bidi documents, but there is at least a tool useful in general.

```

2226 \bbl@trace{Bidi layout}
2227 \providecommand\IfBabelLayout[3]{#3}%

```

4.17. Load engine specific macros

Some macros are not defined in all engines, so, after loading the files define them if necessary to raise an error.

```

2228 \bbl@trace{Input engine specific macros}
2229 \ifcase\bbl@engine
2230   \input txtbabel.def
2231 \or
2232   \input luababel.def
2233 \or
2234   \input xebabel.def
2235 \fi
2236 \providecommand\babelfont{\bbl@error{only-lua-xe}{}}{}{}{}
2237 \providecommand\babelprehyphenation{\bbl@error{only-lua}{}}{}{}{}
2238 \ifx\babelposthyphenation\@undefined
2239   \let\babelposthyphenation\babelprehyphenation
2240   \let\babelpatterns\babelprehyphenation
2241   \let\babelcharproperty\babelprehyphenation
2242 \fi
2243 (/package|core)

```

4.18. Creating and modifying languages

Continue with $\LaTeX$ only.

`\babelprovide` is a general purpose tool for creating and modifying languages. It creates the language infrastructure, and loads, if requested, an ini file. It may be used in conjunction to previously loaded `ldf` files.

```

2244 (*package)
2245 \bbl@trace{Creating languages and reading ini files}
2246 \let\bbl@extend@ini\@gobble
2247 \newcommand\babelprovide[2][]{%
2248   \let\bbl@savelangname\languagename
2249   \edef\bbl@savelocaleid{\the\localeid}%
2250   \global\let\bbl@afterload\@empty
2251   % Set name and locale id
2252   \edef\languagename{#2}%
2253   \bbl@id@assign
2254   % Initialize keys
2255   \bbl@vforeach{captions,date,import,main,script,language,%
2256     hyphenrules,linebreaking,justification,mapfont,maparabic,%
2257     mapdigits,intraspaces,intrapenalty,onchar,transforms,alph,%
2258     Alph,labels,labels*,mapdot,calendar,date,casing,interchar,%
2259     @import}%
2260     {\bbl@csarg\let{KVP@##1}\@nnil}%
2261   \global\let\bbl@release@transforms\@empty
2262   \global\let\bbl@release@casing\@empty
2263   \let\bbl@calendars\@empty
2264   \global\let\bbl@inidata\@empty
2265   \global\let\bbl@extend@ini\@gobble
2266   \global\let\bbl@included@inis\@empty
2267   \gdef\bbl@key@list{;}%

```



```

2268 \bbl@ifunset{bbl@passto@#2}%
2269 {\def\bbl@tempa{#1}}%
2270 {\bbl@exp{\def\\bbl@tempa{[bbl@passto@#2],\unexpanded{#1}}}%
2271 \expandafter\bbl@forkv\expandafter\bbl@tempa}%
2272 \in@{/}{##1}% With /, (re)sets a value in the ini
2273 \ifin@
2274 \bbl@renewinikey##1\@{##2}%
2275 \else
2276 \bbl@csarg\ifx{KVP@##1}\@nnil\else
2277 \bbl@error{unknown-provide-key}{##1}{}%
2278 \fi
2279 \bbl@csarg\def{KVP@##1}{##2}%
2280 \fi}%
2281 \chardef\bbl@howloaded= 0:none; 1:ldf without ini; 2:ini
2282 \bbl@ifunset{date#2}\z@{\bbl@ifunset{bbl@llevel@#2}\@ne\tw@}%
2283 % == init ==
2284 \ifx\bbl@screset\undefined
2285 \bbl@ldfinit
2286 \fi
2287 % ==
2288 % If there is no import (last wins), use @import (internal, there
2289 % must be just one). To consider any order (because
2290 % \PassOptionsToLocale).
2291 \ifx\bbl@KVP@import\@nnil
2292 \let\bbl@KVP@import\bbl@KVP@@import
2293 \fi
2294 % == date (as option) ==
2295 % \ifx\bbl@KVP@date\@nnil\else
2296 % \fi
2297 % ==
2298 \let\bbl@lbkflag\relax % \@empty = do setup linebreak, only in 3 cases:
2299 \ifcase\bbl@howloaded
2300 \let\bbl@lbkflag\@empty % new
2301 \else
2302 \ifx\bbl@KVP@hyphenrules\@nnil\else
2303 \let\bbl@lbkflag\@empty
2304 \fi
2305 \ifx\bbl@KVP@import\@nnil\else
2306 \let\bbl@lbkflag\@empty
2307 \fi
2308 \fi
2309 % == import, captions ==
2310 \ifx\bbl@KVP@import\@nnil\else
2311 \bbl@exp{\\bbl@ifblank{\bbl@KVP@import}}%
2312 {\ifx\bbl@initoload\relax
2313 \begingroup
2314 \def\BabelBeforeIni##1##2{\gdef\bbl@KVP@import{##1}\endinput}%
2315 \bbl@input@texini{##2}%
2316 \endgroup
2317 \else
2318 \xdef\bbl@KVP@import{\bbl@initoload}%
2319 \fi}%
2320 {}%
2321 \let\bbl@KVP@date\@empty
2322 \fi
2323 \let\bbl@KVP@captions@@\bbl@KVP@captions
2324 \ifx\bbl@KVP@captions\@nnil
2325 \let\bbl@KVP@captions\bbl@KVP@import
2326 \fi
2327 % ==
2328 \ifx\bbl@KVP@transforms\@nnil\else
2329 \bbl@replace\bbl@KVP@transforms{ }{,}%
2330 \fi

```

```

2331 % ==
2332 \ifx\bbk@KVP@mapdot\@nnil\else
2333 \def\bbk@tempa{\@empty}%
2334 \ifx\bbk@KVP@mapdot\bbk@tempa\else
2335 \bbk@exp{\gdef\<bbk@map@. @\language\name>{\bbk@KVP@mapdot}}}%
2336 \fi
2337 \fi
2338 % Load ini
2339 % -----
2340 \ifcase\bbk@howloaded
2341 \bbk@provide@new{#2}%
2342 \else
2343 \bbk@ifblank{#1}%
2344 {}% With \bbk@load@basic below
2345 {\bbk@provide@renew{#2}}}%
2346 \fi
2347 % Post tasks
2348 % -----
2349 % == subsequent calls after the first provide for a locale ==
2350 \ifx\bbk@inidata\@empty\else
2351 \bbk@extend@ini{#2}%
2352 \fi
2353 % == ensure captions ==
2354 \ifx\bbk@KVP@captions\@nnil\else
2355 \bbk@ifunset{\bbk@extracaps@#2}%
2356 {\bbk@exp{\bbk@babelensure[exclude=\\today]{#2}}}%
2357 {\bbk@exp{\bbk@babelensure[exclude=\\today,
2358 include=\bbk@extracaps@#2]{#2}}}%
2359 \let\bbk@tempc\language\name
2360 \bbk@replace\bbk@tempc{-}{@}%
2361 \bbk@ifunset{\bbk@ensure@\bbk@tempc}%
2362 {\bbk@exp{%
2363 \\\DeclareRobustCommand\<bbk@ensure@\bbk@tempc>[1]{%
2364 \\\foreignlanguage{\language\name}%
2365 {###1}}}%
2366 }%
2367 \bbk@exp{%
2368 \\\bbk@tglobal\<bbk@ensure@\bbk@tempc>%
2369 \\\bbk@tglobal\<bbk@ensure@\bbk@tempc\space>%
2370 \fi

```

At this point all parameters are defined if 'import'. Now we execute some code depending on them. But what about if nothing was imported? We just set the basic parameters, but still loading the whole ini file.

```

2371 \bbk@load@basic{#2}%
2372 % == script, language ==
2373 % Override the values from ini or defines them
2374 \ifx\bbk@KVP@script\@nnil\else
2375 \bbk@csarg\edef{sname@#2}{\bbk@KVP@script}%
2376 \fi
2377 \ifx\bbk@KVP@language\@nnil\else
2378 \bbk@csarg\edef{lname@#2}{\bbk@KVP@language}%
2379 \fi
2380 \ifcase\bbk@engine\or
2381 \bbk@ifunset{\bbk@chrng@\language\name}}}%
2382 {\directlua{
2383 Babel.set_chranges_b('\bbk@cl{sbc}', '\bbk@cl{chrng}') }}%
2384 \fi
2385 % == Line breaking: intraspace, intrapenalty ==
2386 % For CJK, East Asian, Southeast Asian, if interspace in ini
2387 \ifx\bbk@KVP@intraspace\@nnil\else % We can override the ini or set
2388 \bbk@csarg\edef{intsp@#2}{\bbk@KVP@intraspace}%
2389 \fi

```

```

2390 \bbl@provide@intraspace
2391 % == Line breaking: justification ==
2392 \ifx\bbl@KVP@justification\@nnil\else
2393   \let\bbl@KVP@linebreaking\bbl@KVP@justification
2394 \fi
2395 \ifx\bbl@KVP@linebreaking\@nnil\else
2396   \bbl@xin@{\,\bbl@KVP@linebreaking,}%
2397   {,elongated,kashida,cjk,padding,unhyphenated,}%
2398   \ifin@
2399     \bbl@csarg\xdef
2400       {\lnbrk@{\language\name}}{\expandafter\@car\bbl@KVP@linebreaking\@nil}%
2401   \fi
2402 \fi
2403 \bbl@xin@{/e}{/\bbl@cl{\lnbrk}}%
2404 \ifin@\\else\bbl@xin@{/k}{/\bbl@cl{\lnbrk}}\fi
2405 \ifin@\bbl@arabicjust\fi
2406 \bbl@xin@{/p}{/\bbl@cl{\lnbrk}}%
2407 \ifin@\\AtBeginDocument{\@nameuse{\bbl@tibetanjust}}\fi
2408 % == Line breaking: hyphenate.other.(locale|script) ==
2409 \ifx\bbl@lbfkflag\@empty
2410   \bbl@ifunset{\bbl@hyotl@{\language\name}}{%
2411     {\bbl@csarg\bbl@replace{\hyotl@{\language\name}}{ }{,}%
2412     \bbl@startcommands*{\language\name}}{%
2413       \bbl@csarg\bbl@foreach{\hyotl@{\language\name}}{%
2414         \ifcase\bbl@engine
2415           \ifnum##1<257
2416             \SetHyphenMap{\BabelLower{##1}{##1}}%
2417           \fi
2418           \else
2419             \SetHyphenMap{\BabelLower{##1}{##1}}%
2420           \fi}%
2421     \bbl@endcommands}%
2422   \bbl@ifunset{\bbl@hyots@{\language\name}}{%
2423     {\bbl@csarg\bbl@replace{\hyots@{\language\name}}{ }{,}%
2424     \bbl@csarg\bbl@foreach{\hyots@{\language\name}}{%
2425       \ifcase\bbl@engine
2426         \ifnum##1<257
2427           \global\lccode##1=##1\relax
2428         \fi
2429         \else
2430           \global\lccode##1=##1\relax
2431         \fi}}}%
2432 \fi
2433 % == Counters: maparabic ==
2434 % Native digits, if provided in ini (TeX level, xe and lua)
2435 \ifcase\bbl@engine\else
2436   \bbl@ifunset{\bbl@dgnat@{\language\name}}{%
2437     {\expandafter\ifx\csname\bbl@dgnat@{\language\name}\endcsname\@empty\else
2438       \expandafter\expandafter\expandafter
2439       \bbl@setdigits\csname\bbl@dgnat@{\language\name}\endcsname
2440     \ifx\bbl@KVP@maparabic\@nnil\else
2441       \ifx\bbl@latinarabic\@undefined
2442         \expandafter\let\expandafter\@arabic
2443         \csname\bbl@counter@{\language\name}\endcsname
2444       \else % i.e., if layout=counters, which redefines \@arabic
2445         \expandafter\let\expandafter\bbl@latinarabic
2446         \csname\bbl@counter@{\language\name}\endcsname
2447       \fi
2448     \fi
2449   \fi}%
2450 \fi
2451 % == Counters: mapdigits ==
2452 % > luababel.def

```

```

2453 % == Counters: alph, Alph ==
2454 \ifx\bbbl@KVP@alph\@nnil\else
2455   \bbbl@exp{%
2456     \\bbbl@add\<bbbl@preextras@\language\name>{%
2457       \\babel@save\\@alph
2458       \let\\@alph\<bbbl@cntr@\bbbl@KVP@alph @\language\name>}}%
2459 \fi
2460 \ifx\bbbl@KVP@Alph\@nnil\else
2461   \bbbl@exp{%
2462     \\bbbl@add\<bbbl@preextras@\language\name>{%
2463       \\babel@save\\@Alph
2464       \let\\@Alph\<bbbl@cntr@\bbbl@KVP@Alph @\language\name>}}%
2465 \fi
2466 % == Counters: mapdot ==
2467 \ifx\bbbl@KVP@mapdot\@nnil\else
2468   \bbbl@foreach\bbbl@list@the{%
2469     \bbbl@ifunset{the##1}{}%
2470     {{\bbbl@ncarg\let\bbbl@tempd{the##1}%
2471       \bbbl@carg\bbbl@sreplace{the##1}{.}{\bbbl@map@lbl{.}}%
2472       \expandafter\ifx\csname the##1\endcsname\bbbl@tempd\else
2473         \bbbl@exp{\gdef\<the##1>{{\the##1}}}%
2474         \fi}}%
2475   \edef\bbbl@tempb{enumi,enumii,enumiii,enumiv}%
2476   \bbbl@foreach\bbbl@tempb{%
2477     \bbbl@ifunset{label##1}{}%
2478     {{\bbbl@ncarg\let\bbbl@tempd{label##1}%
2479       \bbbl@carg\bbbl@sreplace{label##1}{.}{\bbbl@map@lbl{.}}%
2480       \expandafter\ifx\csname label##1\endcsname\bbbl@tempd\else
2481         \bbbl@exp{\gdef\<label##1>{{\label##1}}}%
2482         \fi}}%
2483 \fi
2484 % == Casing ==
2485 \bbbl@release@casing
2486 \ifx\bbbl@KVP@casing\@nnil\else
2487   \bbbl@csarg\xdef{casing@\language\name}%
2488   {\@nameuse{bbbl@casing@\language\name}\bbbl@maybextx\bbbl@KVP@casing}%
2489 \fi
2490 % == Calendars ==
2491 \ifx\bbbl@KVP@calendar\@nnil
2492   \edef\bbbl@KVP@calendar{\bbbl@cl{calpr}}%
2493 \fi
2494 \def\bbbl@tempe##1 ##2\@{% Get first calendar
2495   \def\bbbl@tempa{##1}%
2496   \bbbl@exp{\bbbl@tempe\bbbl@KVP@calendar\space\@}%
2497 \def\bbbl@tempe##1.##2.##3.##4\@{%
2498   \def\bbbl@tempf{##3}% First
2499   \let\bbbl@tempc\@empty \let\bbbl@tempb\@empty
2500   \bbbl@cal@getconvert\bbbl@tempc{##1}%
2501   \bbbl@cal@getconvert\bbbl@tempb{##2}%
2502   \expandafter\bbbl@tempe\bbbl@tempa...\@
2503   \bbbl@csarg\edef{calpr@\language\name}{%
2504     \ifx\bbbl@tempc\@empty\else
2505       calendar=\bbbl@tempc
2506     \fi
2507     \ifx\bbbl@tempb\@empty\else
2508       ,variant=\bbbl@tempb
2509     \fi
2510     \ifx\bbbl@tempf\@empty\else
2511       ,convert=\bbbl@tempf
2512     \fi}%
2513 % == engine specific extensions ==
2514 % Defined in XXXbabel.def
2515 \bbbl@provide@extra{##2}%

```

```

2516 % == require.babel in ini ==
2517 % To load or reload the babel-*.tex, if require.babel in ini
2518 \ifx\bbl@beforestart\relax\else % But not in doc aux or body
2519   \bbl@ifunset{bbl@rqtex@\language\name}{}%
2520     {\expandafter\ifx\csname bbl@rqtex@\language\name\endcsname\@empty\else
2521       \let\BabelBeforeIni@gobbletwo
2522       \chardef\atcatcode=\catcode\@
2523       \catcode\@=11\relax
2524       \def\CurrentOption{#2}%
2525       \bbl@input@texini{\bbl@cs{rqtex@\language\name}}%
2526       \catcode\@=\atcatcode
2527       \let\atcatcode\relax
2528       \global\bbl@csarg\let{rqtex@\language\name}\relax
2529     \fi}%
2530 \bbl@foreach\bbl@calendars{%
2531   \bbl@ifunset{bbl@ca##1}{%
2532     \chardef\atcatcode=\catcode\@
2533     \catcode\@=11\relax
2534     \InputIfFileExists{babel-ca-##1.tex}{}{}%
2535     \catcode\@=\atcatcode
2536     \let\atcatcode\relax}%
2537   {}}%
2538 \fi
2539 % == frenchspacing ==
2540 \ifcase\bbl@howloaded\in@true\else\in@false\fi
2541 \ifin@else\bbl@xin@{typography/frenchspacing}{\bbl@key@list}\fi
2542 \ifin@
2543   \bbl@extras@wrap{\bbl@pre@fs}%
2544   {\bbl@pre@fs}%
2545   {\bbl@post@fs}%
2546 \fi
2547 % == transforms ==
2548 % > luababel.def
2549 \def\CurrentOption{#2}%
2550 \@nameuse{bbl@icsave#2}%
2551 % == main ==
2552 \ifx\bbl@KVP@main\@nnil % Restore only if not 'main'
2553   \let\language\bbl@savelangname
2554   \chardef\localeid\bbl@savelocaleid\relax
2555 \fi
2556 % == ==
2557 \ifx\ldf@finish\@onlypreamble\else
2558   \bbl@afterload
2559 \fi
2560 % == hyphenrules (apply if current) ==
2561 \ifx\bbl@KVP@hyphenrules\@nnil\else
2562   \ifnum\bbl@savelocaleid=\localeid
2563     \language\@nameuse{l@\language\name}%
2564   \fi
2565 \fi}

```

Depending on whether or not the language exists (based on `\date<language>`), we define two macros. Remember `\bbl@startcommands` opens a group.

```

2566 \def\bbl@provide@new#1{%
2567   \@namedef{date#1}{}% marks lang exists - required by \StartBabelCommands
2568   \@namedef{extras#1}{}%
2569   \@namedef{noextras#1}{}%
2570   \bbl@startcommands*{#1}{captions}%
2571   \ifx\bbl@KVP@captions\@nnil % and also if import, implicit
2572     \def\bbl@tempb##1{% elt for \bbl@captionslist
2573       \ifx##1\@nnil\else
2574         \bbl@exp{%
2575           \\SetString\\##1}%

```

```

2576         \\bbl@nocaption{\bbl@stripslash##1}{#1\bbl@stripslash##1}}}%
2577         \expandafter\bbl@tempb
2578     \fi}%
2579     \expandafter\bbl@tempb\bbl@captionslist\@nnil
2580 \else
2581     \ifx\bbl@initoload\relax
2582         \bbl@read@ini{\bbl@KVP@captions}2%   % Here letters cat = 11
2583     \else
2584         \bbl@read@ini{\bbl@initoload}2%       % Same
2585     \fi
2586 \fi
2587 \StartBabelCommands*{#1}{date}%
2588 \ifx\bbl@KVP@date\@nnil
2589     \bbl@exp{%
2590         \\SetString\\today{\\bbl@nocaption{today}{#1today}}}%
2591 \else
2592     \bbl@savetoday
2593     \bbl@savedate
2594 \fi
2595 \bbl@endcommands
2596 \bbl@load@basic{#1}%
2597 % == hyphenmins == (only if new)
2598 \bbl@exp{%
2599     \gdef<#1hyphenmins>{%
2600         {\bbl@ifunset{\bbl@lfthm@#1}{2}{\bbl@cs{lfthm@#1}}}%
2601         {\bbl@ifunset{\bbl@rgthm@#1}{3}{\bbl@cs{rgthm@#1}}}}}%
2602 % == hyphenrules (also in renew) ==
2603 \bbl@provide@hyphens{#1}%
2604 % == main ==
2605 \ifx\bbl@KVP@main\@nnil\else
2606     \expandafter\main@language\expandafter{#1}%
2607 \fi}
2608 %
2609 \def\bbl@provide@renew#1{%
2610     \ifx\bbl@KVP@captions\@nnil\else
2611         \StartBabelCommands*{#1}{captions}%
2612         \bbl@read@ini{\bbl@KVP@captions}2%   % Here all letters cat = 11
2613     \EndBabelCommands
2614 \fi
2615 \ifx\bbl@KVP@date\@nnil\else
2616     \StartBabelCommands*{#1}{date}%
2617     \bbl@savetoday
2618     \bbl@savedate
2619 \EndBabelCommands
2620 \fi
2621 % == hyphenrules (also in new) ==
2622 \ifx\bbl@lbkflag\@empty
2623     \bbl@provide@hyphens{#1}%
2624 \fi
2625 % == main ==
2626 \ifx\bbl@KVP@main\@nnil\else
2627     \expandafter\main@language\expandafter{#1}%
2628 \fi}

```

Load the basic parameters (ids, typography, counters, and a few more), while captions and dates are left out. But it may happen some data has been loaded before automatically, so we first discard the saved values.

```

2629 \def\bbl@load@basic#1{%
2630     \ifcase\bbl@howloaded\or\or
2631         \ifcase\csname bbl@llevel@\language\endcsname
2632             \bbl@csarg\let{lname@\language}\relax
2633         \fi
2634     \fi

```

```

2635 \bbl@ifunset{bbl@lname@#1}%
2636 {\def\BabelBeforeIni##1##2{%
2637   \begingroup
2638     \let\bbl@ini@captions@aux\@gobbletwo
2639     \def\bbl@inidate ####1.####2.####3.####4\relax ####5####6}%
2640     \bbl@read@ini{##1}l%
2641     \ifx\bbl@initoload\relax\endinput\fi
2642   \endgroup}%
2643   \begingroup      % boxed, to avoid extra spaces:
2644   \ifx\bbl@initoload\relax
2645     \bbl@input@texini{#1}%
2646   \else
2647     \setbox\z@\hbox{\BabelBeforeIni{\bbl@initoload}{}}%
2648   \fi
2649   \endgroup}%
2650 {}%

```

The following ini reader ignores everything but the identification section. It is called when a font is defined (i.e., when the language is first selected) to know which script/language must be enabled. This means we must make sure a few characters are not active. The ini is not read directly, but with a proxy tex file named as the language (which means any code in it must be skipped, too).

```

2651 \def\bbl@load@info#1{%
2652   \def\BabelBeforeIni##1##2{%
2653     \begingroup
2654       \bbl@read@ini{##1}0%
2655       \endinput      % babel- .tex may contain onlypreamble's
2656     \endgroup}%
2657     {\bbl@input@texini{#1}}%

```

The hyphenrules option is handled with an auxiliary macro. This macro is called in three cases: when a language is first declared with \babelprovide, with hyphenrules and with import.

```

2658 \def\bbl@provide@hyphens#1{%
2659   \@tempcnta@m@ne % a flag
2660   \ifx\bbl@KVP@hyphenrules\@nnil\else
2661     \bbl@replace\bbl@KVP@hyphenrules{ }{,}%
2662     \bbl@foreach\bbl@KVP@hyphenrules{%
2663       \ifnum\@tempcnta=\m@ne % if not yet found
2664         \bbl@ifsamestring{##1}{+}%
2665         {\bbl@carg\addlanguage{l@#1}}%
2666         {}%
2667         \bbl@ifunset{l@#1}% After a possible +
2668         {}%
2669         {\@tempcnta\@nameuse{l@#1}}%
2670       \fi}%
2671   \ifnum\@tempcnta=\m@ne
2672     \bbl@warning{%
2673       Requested 'hyphenrules' for '\language' not found:\\%
2674       \bbl@KVP@hyphenrules.\\%
2675       Using the default value. Reported}%
2676   \fi
2677   \fi
2678   \ifnum\@tempcnta=\m@ne % if no opt or no language in opt found
2679     \ifx\bbl@KVP@captions@\@nnil
2680       \bbl@ifunset{bbl@hyphr@#1}{}% use value in ini, if exists
2681       {\bbl@exp{\bbl@ifblank{\bbl@cs{hyphr@#1}}}%
2682        {}%
2683        {\bbl@ifunset{l@\bbl@cl{hyphr}}}%
2684        {}%
2685        {\@tempcnta\@nameuse{l@\bbl@cl{hyphr}}}}}%
2686   \fi
2687   \fi
2688   \bbl@ifunset{l@#1}%
2689   {\ifnum\@tempcnta=\m@ne
2690     \bbl@carg\adddialect{l@#1}\language

```

```

2691 \else
2692 \bbl@carg\adddialect{l@#1}\@tempcnta
2693 \fi}%
2694 {\ifnum\@tempcnta=\m@ne\else
2695 \global\bbl@carg\chardef{l@#1}\@tempcnta
2696 \fi}}

```

The reader of babel-...tex files. We reset temporarily some catcodes (and make sure no space is accidentally inserted).

```

2697 \def\bbl@input@texini#1{%
2698 \bbl@bsphack
2699 \bbl@exp{%
2700 \catcode`\\=\14 \catcode`\\=\0
2701 \catcode`\\=\1 \catcode`\\=\2
2702 \lowercase{\InputIfFileExists{babel-#1.tex}}{}}%
2703 \catcode`\\=\the\catcode`\relax
2704 \catcode`\\=\the\catcode`\\relax
2705 \catcode`\\=\the\catcode`\relax
2706 \catcode`\\=\the\catcode`\relax}%
2707 \bbl@esphack}

```

The following macros read and store ini files (but don't process them). For each line, there are 3 possible actions: ignore if starts with ;, switch section if starts with [, and store otherwise. There are used in the first step of \bbl@read@ini.

```

2708 \def\bbl@iniline#1\bbl@iniline{%
2709 \ifnextchar[\bbl@iniset{\ifnextchar;\bbl@iniskip\bbl@inistore}#1\@@}% ]
2710 \def\bbl@iniset[#1]#2\@@{\def\bbl@section{#1}}
2711 \def\bbl@iniskip#1\@@{% if starts with ;
2712 \def\bbl@inistore#1=#2\@@{% full (default)
2713 \bbl@trim@def\bbl@tempa{#1}%
2714 \bbl@trim\toks@{#2}%
2715 \bbl@ifsamestring{\bbl@tempa}{\include}%
2716 {\bbl@read@subini{\the\toks@}}%
2717 {\bbl@xin@;\bbl@section/\bbl@tempa;}{\bbl@key@list}%
2718 \ifin@ \else
2719 \bbl@xin@{,identification/include.}%
2720 {,\bbl@section/\bbl@tempa}%
2721 \ifin@\xdef\bbl@included@inis{\the\toks@}\fi
2722 \bbl@exp{%
2723 \\g@addto@macro\\bbl@inidata{%
2724 \\bbl@elt{\bbl@section}{\bbl@tempa}{\the\toks@}}}%
2725 \fi}}
2726 \def\bbl@inistore@min#1=#2\@@{% minimal (maybe set in \bbl@read@ini)
2727 \bbl@trim@def\bbl@tempa{#1}%
2728 \bbl@trim\toks@{#2}%
2729 \bbl@xin@{.identification.}{.\bbl@section.}%
2730 \ifin@
2731 \bbl@exp{\\g@addto@macro\\bbl@inidata{%
2732 \\bbl@elt{identification}{\bbl@tempa}{\the\toks@}}}%
2733 \fi}

```

4.19. Main loop in 'provide'

Now, the 'main loop', \bbl@read@ini, which **must be executed inside a group**. At this point, \bbl@inidata may contain data declared in \babelprovide, with 'slashed' keys. There are 3 steps: first read the ini file and store it; then traverse the stored values, and process some groups if required (date, captions, labels, counters); finally, 'export' some values by defining global macros (identification, typography, characters, numbers). The second argument is 0 when called to read the minimal data for fonts; with \babelprovide it's either 1 (without import) or 2 (which import). The value -1 is used with \DocumentMetadata.

\bbl@loop@ini is the reader, line by line (1: stream), and calls \bbl@iniline to save the key/value pairs. If \bbl@inistore finds the @include directive, the input stream is switched temporarily and \bbl@read@subini is called.

When the language is being set based on the document metadata (#2 in \bbl@read@ini is -1), there is an interlude to get the name, after the data have been collected, and before it's processed.

```

2734 \def\bbl@loop@ini#1{%
2735   \loop
2736     \if T\ifeof#1 F\fi T\relax % Trick, because inside \loop
2737     \endlinechar\m@ne
2738     \read#1 to \bbl@line
2739     \endlinechar\^^M
2740     \ifx\bbl@line\empty\else
2741       \expandafter\bbl@iniline\bbl@line\bbl@iniline
2742     \fi
2743   \repeat}
2744 %
2745 \def\bbl@read@subini#1{%
2746   \ifx\bbl@readsubstream\undefined
2747     \csname newread\endcsname\bbl@readsubstream
2748   \fi
2749   \openin\bbl@readsubstream=babel-#1.ini
2750   \ifeof\bbl@readsubstream
2751     \bbl@error{no-ini-file}{#1}{}{}%
2752   \else
2753     {\bbl@loop@ini\bbl@readsubstream}%
2754   \fi
2755   \closein\bbl@readsubstream}
2756 %
2757 \ifx\bbl@readstream\undefined
2758   \csname newread\endcsname\bbl@readstream
2759 \fi
2760 \def\bbl@read@ini#1#2{%
2761   \global\let\bbl@extend@ini@gobble
2762   \openin\bbl@readstream=babel-#1.ini
2763   \ifeof\bbl@readstream
2764     \bbl@error{no-ini-file}{#1}{}{}%
2765   \else
2766     % == Store ini data in \bbl@inidata ==
2767     \catcode\ =10 \catcode\ =12 \catcode\#=12
2768     \catcode\[ =12 \catcode\] =12 \catcode\&=12 \catcode\&=12
2769     \catcode\;=12 \catcode\| =12 \catcode\% =14 \catcode\-=12
2770     \ifnum#2=\m@ne % Just for the info
2771       \edef\language{tag \bbl@metalang}%
2772     \fi
2773     \ifnum#2=\m@ne
2774       \bbl@info{Metadata: '\bbl@metalang' requested, '#1' provided.\\%
2775         Fetching the locale name from babel-#1.ini\@gobble}%
2776     \else
2777       \bbl@info{Importing
2778         \ifcase#2font and identification \or basic \fi
2779         data for '\language'\@%
2780         from babel-#1.ini. Reported}%
2781     \fi
2782     \ifnum#2<\@ne
2783       \global\let\bbl@inidata\empty
2784       \let\bbl@inistore\bbl@inistore@min % Remember it's local
2785     \fi
2786     \def\bbl@section{identification}%
2787     \bbl@exp{%
2788       \\bbl@inistore tag.ini=#1\\@@
2789       \\bbl@inistore load.level=\ifnum#2<\@ne 0\else #2\fi\\@@}%
2790     \bbl@loop@ini\bbl@readstream
2791     % == Process stored data ==
2792     \ifnum#2=\m@ne
2793       \def\bbl@tempa##1 ##2\@{##1}% Get first name
2794       \def\bbl@elt##1##2##3{%

```

```

2795 \bbl@ifsamestring{identification/name.babel}{##1/##2}%
2796 {\edef\language\language{\bbl@tempa##3 \@@}%
2797 \let\localename\language
2798 \bbl@id@assign
2799 \def\bbl@elt###1###2###3{}}%
2800 {}}%
2801 \bbl@inidata
2802 \fi
2803 \bbl@csarg\xdef\l@ini{\language}{#1}%
2804 \bbl@read@ini@aux
2805 % == 'Export' data ==
2806 \bbl@ini@exports{#2}%
2807 \global\bbl@csarg\let\inidata@\language\bbl@inidata
2808 \global\let\bbl@inidata\empty
2809 \bbl@xin@{\language},{\bbl@ini@loaded},%
2810 \ifin@else
2811 \bbl@exp{\bbl@add@list\bbl@ini@loaded{\language}}%
2812 \bbl@tglobal\bbl@ini@loaded
2813 \fi
2814 \@ifpackagewith{babel}{licr=unextended}{%
2815 {\ifcase\bbl@engine\else % Find a better place
2816 \bbl@xin@{\bbl@cl{sbcpr}},{\Cyr},}%
2817 \ifin@
2818 \bbl@once{licr-cryl}{\g@addto@macro\bbl@afterload{\input{cyr2uni.def}}}%
2819 \fi
2820 \fi}%
2821 \fi
2822 \closein\bbl@readstream}
2823 \def\bbl@read@ini@aux{%
2824 \let\bbl@savestrings\empty
2825 \let\bbl@savetoday\empty
2826 \let\bbl@savestate\empty
2827 \def\bbl@elt###1##2###3{%
2828 \def\bbl@section{##1}%
2829 \in@{=date.}{=##1}% Find a better place
2830 \ifin@
2831 \bbl@ifunset\bbl@inikv@##1{%
2832 {\bbl@ini@calendar{##1}}%
2833 {}}%
2834 \fi
2835 \bbl@ifunset\bbl@inikv@##1{%
2836 {\csname bbl@inikv@##1\endcsname{##2}{##3}}}%
2837 \bbl@inidata}

```

A variant to be used when the ini file has been already loaded, because it's not the first \babelprovide for this language.

```

2838 \def\bbl@extend@ini@aux#1{%
2839 \bbl@startcommands*{#1}{captions}%
2840 % Activate captions/... and modify exports
2841 \bbl@csarg\def\inikv@captions.licr{##1##2}%
2842 \setlocalecaption{#1}{##1}{##2}%
2843 \def\bbl@inikv@captions##1##2{%
2844 \bbl@ini@captions@aux{##1}{##2}}%
2845 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2846 \def\bbl@exportkey##1##2##3{%
2847 \bbl@ifunset\bbl@kv@##2{%
2848 {\expandafter\ifx\csname bbl@kv@##2\endcsname\empty\else
2849 \bbl@exp{\global\let<bbl@##1\language>\<bbl@kv@##2>}}%
2850 \fi}}%
2851 % As with \bbl@read@ini, but with some changes
2852 \bbl@read@ini@aux
2853 \bbl@ini@exports\tw@
2854 % Update inidata@lang by pretending the ini is read.

```

```

2855 \def\bbl@elt##1##2##3{%
2856 \def\bbl@section{##1}%
2857 \bbl@iniline##2=##3\bbl@iniline}%
2858 \csname bbl@inidata@#1\endcsname
2859 \global\bbl@csarg\let{inidata@#1}\bbl@inidata
2860 \StartBabelCommands*{#1}{date}% And from the import stuff
2861 \def\bbl@stringdef##1##2{\gdef##1{##2}}%
2862 \bbl@savetoday
2863 \bbl@savestate
2864 \bbl@endcommands}

```

A somewhat hackish tool to handle calendar sections.

```

2865 \def\bbl@ini@calendar#1{%
2866 \lowercase{\def\bbl@tempa{=#1=}}%
2867 \bbl@replace\bbl@tempa{=date.gregorian}{}%
2868 \bbl@replace\bbl@tempa{=date.}{}%
2869 \in@{.licr=}{#1=}%
2870 \ifin@
2871 \ifcase\bbl@engine
2872 \bbl@replace\bbl@tempa{.licr=}{}%
2873 \else
2874 \let\bbl@tempa\relax
2875 \fi
2876 \fi
2877 \ifx\bbl@tempa\relax\else
2878 \bbl@replace\bbl@tempa{=}{}%
2879 \ifx\bbl@tempa@empty\else
2880 \xdef\bbl@calendars{\bbl@calendars,\bbl@tempa}%
2881 \fi
2882 \bbl@exp{%
2883 \def\<bbl@inikv@#1>####1####2{%
2884 \\\bbl@inidate####1...\relax{####2}{\bbl@tempa}}}%
2885 \fi}

```

A key with a slash in \babelprovide replaces the value in the ini file (which is ignored altogether). The mechanism is simple (but suboptimal): add the data to the ini one (at this point the ini file has not yet been read), and define a dummy macro. When the ini file is read, just skip the corresponding key and reset the macro (in \bbl@inistore above).

```

2886 \def\bbl@renewinikey#1/#2\@#3{%
2887 \global\let\bbl@extend@ini\bbl@extend@ini@aux
2888 \edef\bbl@tempa{\zap@space #1 \@empty}% section
2889 \edef\bbl@tempb{\zap@space #2 \@empty}% key
2890 \bbl@trim\toks@{#3}% value
2891 \bbl@exp{%
2892 \edef\\bbl@key@list{\bbl@key@list \bbl@tempa/\bbl@tempb;}%
2893 \\g@addto@macro\\bbl@inidata{%
2894 \\\bbl@elt{\bbl@tempa}{\bbl@tempb}{\the\toks@}}}%

```

The previous assignments are local, so we need to export them. If the value is empty, we can provide a default value.

```

2895 \def\bbl@exportkey#1#2#3{%
2896 \bbl@ifunset{bbl@kv@#2}%
2897 {\bbl@csarg\gdef{#1@language}{#3}}%
2898 {\expandafter\ifx\csname bbl@kv@#2\endcsname\@empty
2899 \bbl@csarg\gdef{#1@language}{#3}%
2900 \else
2901 \bbl@exp{\global\let\<bbl@#1@language>\<bbl@kv@#2>}}%
2902 \fi}}

```

Key-value pairs are treated differently depending on the section in the ini file. The following macros are the readers for identification and typography. Note \bbl@ini@exports is called always (via \bbl@inisec), while \bbl@after@ini must be called explicitly after \bbl@read@ini if necessary.

Although BCP 47 doesn't treat '-x-' as an extension, the CLDR and many other sources do (as a *private use extension*). For consistency with other single-letter subtags or 'singletons', here is considered an extension, too.

The identification section is used internally by babel in the following places [to be completed]: BCP 47 script tag in the Unicode ranges, which is in turn used by onchar; the language system is set with the names, and then fontspec maps them to the opentype tags, but if the latter package doesn't define them, then babel does it; encodings are used in pdfTeX to select a font encoding valid (and preloaded) for a language loaded on the fly.

```

2903 \def\bbl@iniwarning#1{%
2904   \bbl@ifunset{\bbl@kv@identification.warning#1}{}%
2905     {\bbl@warning{%
2906       From babel-\bbl@cs{lini@language}.ini:\\%
2907       \bbl@cs{@kv@identification.warning#1}\\%
2908       Reported}}}
2909 %
2910 \let\bbl@release@transforms\@empty
2911 \let\bbl@release@casing\@empty

```

Relevant keys are 'exported', i.e., global macros with short names are created with values taken from the corresponding keys. The number of exported keys depends on the loading level (#1): -1 and 0 only info (the identification section), 1 also basic (like linebreaking or character ranges), 2 also (re)new (with date and captions).

```

2912 \def\bbl@ini@exports#1{%
2913   % Identification always exported
2914   \bbl@iniwarning{%
2915     \ifcase\bbl@engine
2916       \bbl@iniwarning{.pdfLatex}%
2917     \or
2918       \bbl@iniwarning{.luaLatex}%
2919     \or
2920       \bbl@iniwarning{.XeLatex}%
2921     \fi%
2922     \bbl@exportkey{lllevel}{identification.load.level}{}%
2923     \bbl@exportkey{elname}{identification.name.english}{}%
2924     \bbl@expf{\bbl@exportkey{lname}{identification.name.opentype}%
2925       {\csname bbl@elname@language\endcsname}}%
2926     \bbl@exportkey{tbcp}{identification.tag.bcp47}{}%
2927     \bbl@exportkey{casing}{identification.tag.bcp47}{}%
2928     \bbl@exportkey{lbcp}{identification.language.tag.bcp47}{}%
2929     \bbl@exportkey{lotf}{identification.tag.opentype}{dflt}%
2930     \bbl@exportkey{esname}{identification.script.name}{}%
2931     \bbl@expf{\bbl@exportkey{sname}{identification.script.name.opentype}%
2932       {\csname bbl@esname@language\endcsname}}%
2933     \bbl@exportkey{sbcpr}{identification.script.tag.bcp47}{}%
2934     \bbl@exportkey{sotf}{identification.script.tag.opentype}{DFLT}%
2935     \bbl@exportkey{rbcp}{identification.region.tag.bcp47}{}%
2936     \bbl@exportkey{vbcp}{identification.variant.tag.bcp47}{}%
2937     \bbl@exportkey{extt}{identification.extension.t.tag.bcp47}{}%
2938     \bbl@exportkey{extu}{identification.extension.u.tag.bcp47}{}%
2939     \bbl@exportkey{extx}{identification.extension.x.tag.bcp47}{}%
2940     % Also maps bcp47 -> language
2941     \bbl@csarg\edef{bcp@map@bbl@cl{tbcp}}{\language}%
2942     \ifcase\bbl@engine\or
2943       \directlua{%
2944         Babel.locale_props[\the\bbl@cs{id@language}].script
2945           = '\bbl@cl{sbcpr}'}%
2946     \fi
2947     % Conditional
2948     \ifnum#1>\z@ % -1 or 0 = only info, 1 = basic, 2 = (re)new
2949       \bbl@exportkey{calpr}{date.calendar.preferred}{}%
2950       \bbl@exportkey{lnbrk}{typography.linebreaking}{h}%
2951       \bbl@exportkey{hyphr}{typography.hyphenrules}{}%
2952       \bbl@exportkey{lftm}{typography.lefthyphenmin}{2}%

```

```

2953 \bbl@exportkey{rgthm}{typography.righthyphenmin}{3}%
2954 \bbl@exportkey{prehc}{typography.prehyphenchar}{}%
2955 \bbl@exportkey{hyotl}{typography.hyphenate.other.locale}{}%
2956 \bbl@exportkey{hyots}{typography.hyphenate.other.script}{}%
2957 \bbl@exportkey{intsp}{typography.intraspace}{}%
2958 \bbl@exportkey{frspc}{typography.frenchspacing}{u}%
2959 \bbl@exportkey{chrng}{characters.ranges}{}%
2960 \bbl@exportkey{quote}{characters.delimiters.quotes}{}%
2961 \bbl@exportkey{dgnat}{numbers.digits.native}{}%
2962 \ifnum#1=\tw@ % only (re)new
2963 \bbl@exportkey{rqtex}{identification.require.babel}{}%
2964 \bbl@tglobal\bbl@savetoday
2965 \bbl@tglobal\bbl@savedate
2966 \bbl@savestrings
2967 \fi
2968 \fi}

```

4.20. Processing keys in ini

A shared handler for key=val lines to be stored in \bbl@kv@<section>.<key>.

```

2969 \def\bbl@inikv#1#2{%      key=value
2970 \toks@{#2}%              This hides #'s from ini values
2971 \bbl@csarg\edef{@kv@\bbl@section.#1}{\the\toks@}}

```

By default, the following sections are just read. Actions are taken later.

```

2972 \let\bbl@inikv@identification\bbl@inikv
2973 \let\bbl@inikv@date\bbl@inikv
2974 \let\bbl@inikv@typography\bbl@inikv
2975 \let\bbl@inikv@numbers\bbl@inikv

```

The characters section also stores the values, but casing is treated in a different fashion. Much like transforms, a set of commands calling the parser are stored in \bbl@release@casing, which is executed in \babelprovide.

```

2976 \def\bbl@maybextx{-\bbl@csarg\ifx{extx@\language}\empty x-\fi}
2977 \def\bbl@inikv@characters#1#2{%
2978 \bbl@ifsamestring{#1}{casing}% e.g., casing = uV
2979 {\bbl@exp{%
2980 \\\g@addto@macro{\bbl@release@casing{%
2981 \\\bbl@casemapping}{\language}\unexpanded{#2}}}%
2982 {\in@{$casing.}{$#1}% e.g., casing.Uv = uV
2983 \ifin@
2984 \lowercase{\def\bbl@tempb{#1}%
2985 \bbl@replace\bbl@tempb{casing.}}}%
2986 \bbl@exp{\\\g@addto@macro{\bbl@release@casing{%
2987 \\\bbl@casemapping
2988 {\\\bbl@maybextx\bbl@tempb}{\language}\unexpanded{#2}}}%
2989 \else
2990 \bbl@inikv{#1}{#2}%
2991 \fi}}

```

Additive numerals require an additional definition. When .1 is found, two macros are defined – the basic one, without .1 called by \localenumeral, and another one preserving the trailing .1 for the ‘units’. The asterisk is a ‘terminator’.

```

2992 \def\bbl@inikv@counters#1#2{%
2993 \bbl@ifsamestring{#1}{digits}%
2994 {\bbl@error{digits-is-reserved}{}}}%
2995 {}%
2996 \def\bbl@tempc{#1}%
2997 \bbl@trim@def{\bbl@tempb*}{#2}%
2998 \in@{.1$}{#1$}%
2999 \ifin@
3000 \bbl@replace\bbl@tempc{.1}{}%
3001 \bbl@csarg\protected@xdef{cntr@\bbl@tempc @\language}{%

```

```

3002 \noexpand\bbl@alphanumeric{\bbl@tempc}}%
3003 \fi
3004 \in@{.F.}{#1}%
3005 \ifin@else\in@{.S.}{#1}\fi
3006 \ifin@
3007 \bbl@csarg\protected@xdef{cnt@#1@language}{\bbl@tempb*}%
3008 \else
3009 \toks@{}% Required by \bbl@buildifcase, which returns \bbl@tempa
3010 \expandafter\bbl@buildifcase\bbl@tempb* \ \ % Space after \
3011 \bbl@csarg\global\expandafter\let{cnt@#1@language}\bbl@tempa
3012 \fi
3013 \in@{.D$}{#1$}%
3014 \ifin@else\in@{.C$}{#1$}\fi
3015 \ifin@
3016 \bbl@replace\bbl@tempc{.D}{}%
3017 \bbl@replace\bbl@tempc{.C}{}%
3018 \bbl@exp\gdef\<bbl@cnt@bbl@tempc @language>####1{%
3019 \expandafter\\bbl@zhnumeral\\expandafter{\\number####1}{\bbl@tempc}}}%
3020 \fi}

```

Now captions and captions.licr, depending on the engine. And below also for dates. They rely on a few auxiliary macros. It is expected the ini file provides the complete set in Unicode and LICR, in that order.

```

3021 \ifcase\bbl@engine
3022 \bbl@csarg\def{inikv@captions.licr}#1#2{%
3023 \bbl@ini@captions@aux{#1}{#2}}
3024 \else
3025 \def\bbl@inikv@captions#1#2{%
3026 \bbl@ini@captions@aux{#1}{#2}}
3027 \fi

```

The auxiliary macro for captions define \<caption>name.

```

3028 \def\bbl@ini@captions@template#1#2{% string language tempa=capt-name
3029 \bbl@replace\bbl@tempa{.template}{}%
3030 \def\bbl@toreplace{#1}{}%
3031 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace}}%
3032 \bbl@replace\bbl@toreplace{[ ]}{\csname}%
3033 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3034 \bbl@replace\bbl@toreplace{[ ]}{\name\endcsname}}%
3035 \bbl@replace\bbl@toreplace{[ ]}{\endcsname}}%
3036 \bbl@xin@{,\bbl@tempa,}{,chapter,appendix,part,}%
3037 \ifin@
3038 \@nameuse{\bbl@patch\bbl@tempa}%
3039 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3040 \fi
3041 \bbl@xin@{,\bbl@tempa,}{,figure,table,}%
3042 \ifin@
3043 \global\bbl@csarg\let{\bbl@tempa fmt@#2}\bbl@toreplace
3044 \bbl@exp\gdef\<fnum@\bbl@tempa>{%
3045 \\bbl@ifunset{\bbl@bbl@tempa fmt@\\language}%
3046 {[fnum@\bbl@tempa]}%
3047 {\\@nameuse{\bbl@bbl@tempa fmt@\\language}}}%
3048 \fi}
3049 %
3050 \def\bbl@ini@captions@aux#1#2{%
3051 \bbl@trim\def\bbl@tempa{#1}%
3052 \bbl@xin@{.template}{\bbl@tempa}%
3053 \ifin@
3054 \bbl@ini@captions@template{#2}\language
3055 \else
3056 \bbl@ifblank{#2}%
3057 {\bbl@exp{%
3058 \toks@{\\bbl@nocaption{\bbl@tempa name}{\language\bbl@tempa name}}}%
3059 {\bbl@trim\toks@{#2}}}%

```

```

3060 \bbl@exp{%
3061   \\\bbl@add\\\bbl@savestrings{%
3062     \\\SetString<\bbl@tempa name>{\the\toks@}}}%
3063 \toks@ \expandafter{\bbl@captionslist}%
3064 \bbl@exp{\\\in@{\<\bbl@tempa name>}{\the\toks@}}%
3065 \ifin@ \else
3066   \bbl@exp{%
3067     \\\bbl@add<\bbl@extracaps@ \language name>{\<\bbl@tempa name>}}%
3068     \\\bbl@tglobal \<\bbl@extracaps@ \language name>}%
3069 \fi
3070 \fi}

```

Labels. Captions must contain just strings, no format at all, so there is new group in ini files.

```

3071 \def\bbl@list@the{%
3072   part,chapter,section,subsection,subsubsection,paragraph,%
3073   subparagraph,enumi,enumii,enumiii,enumiv,equation,figure,%
3074   table,page,footnote,mpfootnote,mpfn}
3075 %
3076 \def\bbl@map@cnt#1{% #1:roman,etc, // #2:enumi,etc
3077   \bbl@iifunset{\bbl@map@#1@ \language name}%
3078   {\@nameuse{#1}}%
3079   {\@nameuse{\bbl@map@#1@ \language name}}}
3080 %
3081 \def\bbl@map@lbl#1{% #1:a sign, eg, .
3082   \ifin@csname#1\else
3083     \bbl@iifunset{\bbl@map@#1@ \language name}%
3084     {#1}%
3085     {\@nameuse{\bbl@map@#1@ \language name}}}
3086 \fi}
3087 %
3088 \def\bbl@inikv@labels#1#2{%
3089   \in@{.map}{#1}%
3090   \ifin@
3091     \in@{,dot.map,}{, #1,}%
3092     \ifin@
3093       \global\@namedef{\bbl@map@. @ \language name}{#2}%
3094     \fi
3095   \ifx\bbl@KVP@labels\@nnil\else
3096     \bbl@xin@{ map }{ \bbl@KVP@labels \space}%
3097     \ifin@
3098       \def\bbl@tempc{#1}%
3099       \bbl@replace\bbl@tempc{.map}{}%
3100       \in@{, #2,}{,arabic,roman,Roman,alph,Alph,fnsymbol,}%
3101       \bbl@exp{%
3102         \gdef\<\bbl@map@\bbl@tempc @ \language name>%
3103         {\ifin@ \<#2> \else \\\localecounter{#2} \fi}}%
3104       \bbl@foreach\bbl@list@the{%
3105         \bbl@iifunset{the##1}{}%
3106         {\bbl@ncarg\let\bbl@tempd{the##1}%
3107         \bbl@exp{%
3108           \\\bbl@sreplace\<the##1>%
3109           {\<\bbl@tempc>{##1}}%
3110           {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3111           \\\bbl@sreplace\<the##1>%
3112           {\<\@empty @\bbl@tempc>\<c@##1>%
3113           {\\\bbl@map@cnt{\bbl@tempc}{##1}}%
3114           \\\bbl@sreplace\<the##1>%
3115           {\\\csname @\bbl@tempc\\endcsname\<c@##1>%
3116           {\\\bbl@map@cnt{\bbl@tempc}{##1}}}%
3117         \expandafter\ifx\csname the##1\endcsname\bbl@tempd\else
3118         \bbl@exp{\gdef\<the##1>{\[the##1]}}%
3119         \fi}}%
3120       \fi

```

```

3121 \fi
3122 %
3123 \else
3124 % The following code is still under study. You can test it and make
3125 % suggestions. E.g., enumerate.2 = ([enumi]).([enumii]). It's
3126 % language dependent.
3127 \in@{enumerate.}{#1}%
3128 \ifin@
3129 \def\bbl@tempa{#1}%
3130 \bbl@replace\bbl@tempa{enumerate.}{}%
3131 \def\bbl@toreplace{#2}%
3132 \bbl@replace\bbl@toreplace{[ ]}{\nobreakspace{}}%
3133 \bbl@replace\bbl@toreplace{[ ]}{\csname the}%
3134 \bbl@replace\bbl@toreplace{[ ]}{\endcsname{}}%
3135 \toks@{\expandafter\bbl@toreplace}%
3136 \bbl@exp{%
3137 \\\bbl@add<extras\language>{%
3138 \\\babel@save<labelenum\romannumeral\bbl@tempa>%
3139 \def<labelenum\romannumeral\bbl@tempa>{\the\toks@}%
3140 \\\bbl@tglobal<extras\language>}%
3141 \fi
3142 \fi}

```

To show correctly some captions in a few languages, we need to patch some internal macros, because the order is hardcoded. For example, in Japanese the chapter number is surrounded by two string, while in Hungarian is placed after. These replacement works in many classes, but not all. Actually, the following lines are somewhat tentative.

```

3143 \def\bbl@chapttype{chapter}
3144 \ifx\@makechapterhead\@undefined
3145 \let\bbl@patchchapter\relax
3146 \else\ifx\thechapter\@undefined
3147 \let\bbl@patchchapter\relax
3148 \else\ifx\ps@headings\@undefined
3149 \let\bbl@patchchapter\relax
3150 \else
3151 \def\bbl@patchchapter{%
3152 \global\let\bbl@patchchapter\relax
3153 \gdef\bbl@chfmt{%
3154 \bbl@ifunset{bbl@\bbl@chapttype fmt@\language}%
3155 {\@chapapp\space\thechapter}%
3156 {\@nameuse{bbl@\bbl@chapttype fmt@\language}}}%
3157 \bbl@add\appendix{\def\bbl@chapttype{appendix}}% Not harmful, I hope
3158 \bbl@sreplace\ps@headings{\@chapapp\ \thechapter}{\bbl@chfmt}%
3159 \bbl@sreplace\chaptermark{\@chapapp\ \thechapter}{\bbl@chfmt}%
3160 \bbl@sreplace\@makechapterhead{\@chapapp\space\thechapter}{\bbl@chfmt}%
3161 \bbl@tglobal\appendix
3162 \bbl@tglobal\ps@headings
3163 \bbl@tglobal\chaptermark
3164 \bbl@tglobal\@makechapterhead}
3165 \let\bbl@patchappendix\bbl@patchchapter
3166 \fi\fi\fi
3167 \ifx\@part\@undefined
3168 \let\bbl@patchpart\relax
3169 \else
3170 \def\bbl@patchpart{%
3171 \global\let\bbl@patchpart\relax
3172 \gdef\bbl@partformat{%
3173 \bbl@ifunset{bbl@partfmt@\language}%
3174 {\@partname\nobreakspace\thepart}%
3175 {\@nameuse{bbl@partfmt@\language}}}%
3176 \bbl@sreplace\@part{\@partname\nobreakspace\thepart}{\bbl@partformat}%
3177 \bbl@tglobal\@part}
3178 \fi

```


Date. Arguments (year, month, day) are *not* protected, on purpose. In \today, arguments are always gregorian, and therefore always converted with other calendars.

```

3179 \def\bbl@cal@getconvert#1#2{% Used in provide
3180   \in@{:}{#2}%
3181   \ifin@{\def\bbl@tempf{#2}\else\def#1{#2}\fi}
3182 \let\bbl@calendar@empty
3183 \DeclareRobustCommand\localedate[1][\bbl@localedate{#1}]
3184 \def\bbl@localedate#1#2#3#4{%
3185   \begingroup
3186     \edef\bbl@they{#2}%
3187     \edef\bbl@them{#3}%
3188     \edef\bbl@thed{#4}%
3189     \edef\bbl@tempe{%
3190       \bbl@ifunset{\bbl@calpr\language\language}\bbl@cl{calpr}},%
3191       #1}%
3192     \bbl@exp{\lowercase{\edef\\bbl@tempe{\bbl@tempe}}}%
3193     \let\bbl@ld@calendar@empty
3194     \let\bbl@ld@variant@empty
3195     \let\bbl@ld@convert@relax
3196     \let\bbl@ld@@convert@relax
3197     \expandafter\bbl@forkv\expandafter{\bbl@tempe}{\bbl@csarg\edef{ld##1}{##2}}%
3198     \bbl@replace\bbl@ld@calendar{gregorian}{}%
3199     \ifx\bbl@ld@convert@relax
3200       \let\bbl@ld@convert\bbl@ld@@convert
3201     \fi
3202     \ifx\bbl@ld@convert@empty
3203       \ifx\bbl@ld@calendar@empty
3204         \let\bbl@ld@convert@relax
3205       \else
3206         \let\bbl@ld@convert\bbl@ld@calendar
3207       \fi
3208     \fi
3209     \ifx\bbl@ld@convert@relax\else
3210       \babelcalendar[\bbl@they-\bbl@them-\bbl@thed]%
3211       {\bbl@ld@convert}\bbl@they\bbl@them\bbl@thed
3212     \fi
3213     \@nameuse{\bbl@precalendar}% Remove, e.g., +, -civil (-ca-islamic)
3214     \edef\bbl@calendar{% Used in \month..., too
3215       \bbl@ld@calendar
3216       \ifx\bbl@ld@variant@empty\else
3217         .\bbl@ld@variant
3218       \fi}%
3219     \bbl@cased
3220     {\@nameuse{\bbl@date@\language @\bbl@calendar}%
3221      \bbl@they\bbl@them\bbl@thed}%
3222   \endgroup}
3223 %
3224 \def\bbl@printdate#1{%
3225   \ifnextchar[{\bbl@printdate@i{#1}}{\bbl@printdate@i{#1}[]}}
3226 \def\bbl@printdate@i#1[#2]#3#4#5{%
3227   \bbl@usedategrouptrue
3228   \def\bbl@tempc{#1}%
3229   \bbl@replace\bbl@tempc{-}{@}%
3230   \@nameuse{\bbl@ensure@\bbl@tempc}{\localedate[#2][#3][#4][#5]}
3231 %
3232 % e.g.: 1=months, 2=wide, 3=1, 4=dummy, 5=value, 6=calendar
3233 \def\bbl@inidate#1.#2.#3.#4\relax#5#6{%
3234   \bbl@trim@def\bbl@tempa{#1.#2}%
3235   \bbl@ifsamestring{\bbl@tempa}{months.wide}%      to savedate
3236   {\bbl@trim@def\bbl@tempa{#3}%
3237    \bbl@trim\toks@{#5}%
3238    \@temptokena\expandafter{\bbl@savedate}%
3239    \bbl@exp{% Reverse order - in ini last wins

```

```

3240 \def\\bbl@savestate{%
3241   \\SetString\<month\romannumeral\bbl@tempa#6name>{\the\toks@}%
3242   \the\@temptokena}}}%
3243 {\bbl@ifsamestring{\bbl@tempa}{date.long}% defined now
3244 {\lowercase{\def\bbl@tempb{#6}}}%
3245 \bbl@trim@def\bbl@toreplace{#5}%
3246 \bbl@TG@@date
3247 \global\bbl@csarg\let{date@\language name @\bbl@tempb}\bbl@toreplace
3248 \ifx\bbl@savestate\empty
3249 \bbl@exp{%
3250   \\AfterBabelCommands{%
3251     \gdef\<\language name date>{\\protect\<\language name date >}%
3252     \gdef\<\language name date >{\\bbl@printdate{\language name}}}%
3253   \def\\bbl@savestate{%
3254     \\SetString\\today{%
3255       \<\language name date>[@convert]%
3256       {\the\year}{\the\month}{\the\day}}}%
3257   \fi}%
3258 {}}}

```

Dates will require some macros for the basic formatting. They may be redefined by language, so “semi-public” names (camel case) are used. Oddly enough, the CLDR places particles like “de” inconsistently in either in the date or in the month name. Note after `\bbl@replace \toks@` contains the resulting string, which is used by `\bbl@replace@finish@iii` (this implicit behavior doesn’t seem a good idea, but it’s efficient).

```

3259 \let\bbl@calendar\empty
3260 \newcommand\babelcalendar[2][\the\year-\the\month-\the\day]{%
3261   \bbl@xin@{\$calendrica:}{\$#2}%
3262   \ifin@
3263     \directlua{Babel.calendrica = Babel.calendrica or require("calendrica")}%
3264     \edef\bbl@tempa{\$#2}%
3265     \bbl@replace\bbl@tempa{\$calendrica:}{}%
3266     \bbl@replace\bbl@tempa{-}{_}%
3267     \bbl@afterelse
3268     \bbl@exp{\\bbl@calendrica#1\\@@{\bbl@tempa}}%
3269   \else
3270     \bbl@afterfi
3271     \@nameuse{\bbl@ca#2}#1\@@
3272   \fi}
3273 \def\bbl@calendrica#1-#2-#3\@@#4#5#6#7{%
3274   \directlua{
3275     local d = Babel.calendrica.new_date(
3276       {year=\number#1, month=\number#2, day=\number#3},
3277       Babel.calendrica.cal_gregorian)
3278     local r = Babel.calendrica.as_date(d, Babel.calendrica.cal_#4)
3279     token.set_macro('\bbl@stripslash#5', r.year)
3280     token.set_macro('\bbl@stripslash#6', r.month)
3281     token.set_macro('\bbl@stripslash#7', r.day) }}
3282 \newcommand\BabelDateSpace{\nobreakspace}
3283 \newcommand\BabelDateDot{.\@}
3284 \newcommand\BabelDated[1]{\number#1}
3285 \newcommand\BabelDatedd[1]{\ifnum#1<10 0\fi\number#1}
3286 \newcommand\BabelDateM[1]{\number#1}
3287 \newcommand\BabelDateMM[1]{\ifnum#1<10 0\fi\number#1}
3288 \newcommand\BabelDateMMM[1]{%
3289   \csname month\romannumeral#1\bbl@calendar name\endcsname}%
3290 \newcommand\BabelDatey[1]{\number#1}%
3291 \newcommand\BabelDateyy[1]{%
3292   \ifnum#1<10 0\number#1 %
3293   \else\ifnum#1<100 \number#1 %
3294   \else\ifnum#1<1000 \expandafter\@gobble\number#1 %
3295   \else\ifnum#1<10000 \expandafter\@gobbletwo\number#1 %
3296   \else

```

```

3297 \bbl@error{limit-two-digits}{\f}{\f}%
3298 \fi\fi\fi\fi}}
3299 \newcommand\BabelDateyyy[1]{\number#1}}
3300 \newcommand\BabelDateU[1]{\number#1}}%
3301 \def\bbl@replace@finish@iii#1{%
3302 \bbl@exp{\def\#1###1###2###3{\the\toks@}}
3303 \def\bbl@TG@date{%
3304 \bbl@replace\bbl@toreplace{[ ]}{\BabelDateSpace{}}}%
3305 \bbl@replace\bbl@toreplace{[. ]}{\BabelDateDot{}}}%
3306 \bbl@replace\bbl@toreplace{[y]}{\BabelDatey{###1}}}%
3307 \bbl@replace\bbl@toreplace{[y|]}{\bbl@datecctr{###1|}}%
3308 \bbl@replace\bbl@toreplace{[yy]}{\BabelDateyy{###1}}}%
3309 \bbl@replace\bbl@toreplace{[yyyy]}{\BabelDateyyy{###1}}}%
3310 \bbl@replace\bbl@toreplace{[M]}{\BabelDateM{###2}}}%
3311 \bbl@replace\bbl@toreplace{[M|]}{\bbl@datecctr{###2|}}%
3312 \bbl@replace\bbl@toreplace{[MM]}{\BabelDateMM{###2}}}%
3313 \bbl@replace\bbl@toreplace{[MMM]}{\BabelDateMMM{###2}}}%
3314 \bbl@replace\bbl@toreplace{[d]}{\BabelDated{###3}}}%
3315 \bbl@replace\bbl@toreplace{[d|]}{\bbl@datecctr{###3|}}%
3316 \bbl@replace\bbl@toreplace{[dd]}{\BabelDatedd{###3}}}%
3317 \bbl@replace\bbl@toreplace{[U]}{\BabelDateU{###1}}}%
3318 \bbl@replace\bbl@toreplace{[U|]}{\bbl@datecctr{###1|}}%
3319 \bbl@replace@finish@iii\bbl@toreplace}
3320 \def\bbl@datecctr{\expandafter\bbl@xdatecctr\expandafter}
3321 \def\bbl@xdatecctr[#1|#2]{\localenumeral{#2}{#1}}

```

4.21. French spacing (again)

For the following declarations, see issue #240. `\nonfrenchspacing` is set by document too early, so it's a hack.

```

3322 \AddToHook{begindocument/before}{%
3323 \let\bbl@normalsf\normalsfcodes
3324 \let\normalsfcodes\relax}
3325 \AtBeginDocument{%
3326 \ifx\bbl@normalsf\empty
3327 \ifnum\sfcodes\@m
3328 \let\normalsfcodes\frenchspacing
3329 \else
3330 \let\normalsfcodes\nonfrenchspacing
3331 \fi
3332 \else
3333 \let\normalsfcodes\bbl@normalsf
3334 \fi}

```

Transforms.

Process the transforms read from ini files, converts them to a form close to the user interface (with `\babelprehyphenation` and `\babelposthyphenation`), wrapped with `\bbl@transforms@aux` ...`\relax`, and stores them in `\bbl@release@transforms`. However, since building a list enclosed in braces isn't trivial, the replacements are added after a comma, and then `\bbl@transforms@aux` adds the braces.

```

3335 \bbl@csarg\let{inikv@transforms.prehyphenation}\bbl@inikv
3336 \bbl@csarg\let{inikv@transforms.posthyphenation}\bbl@inikv
3337 \def\bbl@transforms@aux#1#2#3#4,#5\relax{%
3338 #1[#2]{#3}{#4}{#5}}
3339 \begingroup
3340 \catcode\%=12
3341 \catcode\&=14
3342 \gdef\bbl@transforms#1#2#3{\&%
3343 \directlua{
3344 local str = [=[#2]=]
3345 str = str:gsub('%.%d+%.%d+$', '')
3346 token.set_macro('babeltempa', str)
3347 }&%

```

```

3348 \def\babeltempc{}&%
3349 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,}&%
3350 \ifin@else
3351 \bbl@xin@{\babeltempa,}{,\bbl@KVP@transforms,}&%
3352 \fi
3353 \ifin@
3354 \bbl@foreach\bbl@KVP@transforms{&%
3355 \bbl@xin@{\babeltempa,}{,##1,}&%
3356 \ifin@ &% font:font:transform syntax
3357 \directlua{
3358 local t = {}
3359 for m in string.gmatch('##1'..'':', '(.):') do
3360 table.insert(t, m)
3361 end
3362 table.remove(t)
3363 token.set_macro('babeltempc', ', fonts=' .. table.concat(t, ' '))
3364 }&%
3365 \fi}&%
3366 \in@{.0$}{#2$}&%
3367 \ifin@
3368 \directlua{&% (\attribute) syntax
3369 local str = string.match([[ \bbl@KVP@transforms]],
3370 '%([^(%[]-)%)[^%)]-\babeltempa')
3371 if str == nil then
3372 token.set_macro('babeltempb', '')
3373 else
3374 token.set_macro('babeltempb', ', attribute=' .. str)
3375 end
3376 }&%
3377 \toks@{#3}&%
3378 \bbl@exp{&%
3379 \\g@addto@macro\\bbl@release@transforms{&%
3380 \relax &% Closes previous \bbl@transforms@aux
3381 \\bbl@transforms@aux
3382 \\#1{label=\babeltempa\babeltempb\babeltempc}&%
3383 {\language\the\toks@}}&%
3384 \else
3385 \g@addto@macro\bbl@release@transforms{, {#3}}&%
3386 \fi
3387 \fi}
3388 \endgroup

```

4.22. Handle language system

The language system (i.e., Language and Script) to be used when defining a font or setting the direction are set with the following macros. It also deals with unhyphenated line breaking in xetex (e.g., Thai and traditional Sanskrit), which is done with a hack at the font level because this engine doesn't support it.

```

3389 \def\bbl@provide@lsys#1{%
3390 \bbl@ifunset\bbl@lname@#1{%
3391 {\bbl@load@info{#1}}%
3392 }%
3393 \bbl@csarg\let{lsys@#1}\@empty
3394 \bbl@ifunset\bbl@sname@#1{\bbl@csarg\gdef{sname@#1}{Default}}{%
3395 \bbl@ifunset\bbl@sotf@#1{\bbl@csarg\gdef{sotf@#1}{DFLT}}{%
3396 \bbl@csarg\bbl@add@list{lsys@#1}{Script=\bbl@cs{sname@#1}}%
3397 \bbl@ifunset\bbl@lname@#1{%
3398 {\bbl@csarg\bbl@add@list{lsys@#1}{Language=\bbl@cs{lname@#1}}}%
3399 \ifcase\bbl@engine\or\or
3400 \bbl@ifunset\bbl@prehc@#1{%
3401 {\bbl@exp{\\bbl@ifblank{\bbl@cs{prehc@#1}}}%
3402 }%
3403 {\ifx\bbl@xenohyph\undefined

```

```

3404      \global\let\bbl@xenoxyph\bbl@xenoxyph@d
3405      \ifx\AtBeginDocument\@notprerr
3406        \expandafter\@secondoftwo % to execute right now
3407      \fi
3408      \AtBeginDocument{%
3409        \bbl@patchfont{\bbl@xenoxyph}%
3410        {\expandafter\select@language\expandafter{\language}}}%
3411    \fi}%
3412  \fi
3413  \bbl@csarg\bbl@toglobal{\sys@#1}}

```

4.23. Numerals

A tool to define the macros for native digits from the list provided in the `ini` file. Somewhat convoluted because there are 10 digits, but only 9 arguments in $\text{T}_{\text{E}}\text{X}$. Non-digits characters are kept. The first macro is the generic “localized” command.

[illegible]

Alphabetic counters must be converted from a space separated list to an \ifcase structure.

```

3445 \def\bbl@buildifcase#1 {% Returns \bbl@tempa, requires \toks@={}%
3446 \ifx\#1% % \ before, in case #1 is multiletter
3447 \bbl@exp{%
3448 \def\\bbl@tempa####1{%
3449 \<ifcase>####1\space\the\toks@\<else>\\@ctrerrr\<fi>}}%
3450 \else
3451 \toks@\expandafter{\the\toks@\or #1}%
3452 \expandafter\bbl@buildifcase
3453 \fi}

```

The code for additive counters is somewhat tricky and it's based on the fact the arguments just before \@@ collect digits which have been left 'unused' in previous arguments, the first of them being the number of digits in the number to be converted. This explains the reverse set 76543210.

Digits above 10000 are not handled yet. When the key contains the subkey .F., the number after is treated as an special case, for a fixed form (see babel-he.ini, for example).

```

3454 \newcommand\localenumberal[2]{%
3455   \bbl@ifunset{\bbl@cntr@#1@\language\language}%
3456   {#2}%
3457   {\bbl@cs{cntr@#1@\language}{#2}}}
3458 \def\bbl@localecntr#1#2{\localenumberal{#2}{#1}}
3459 \newcommand\localecounter[2]{%
3460   \expandafter\bbl@localecntr
3461   \expandafter{\number\csname c@#2\endcsname}{#1}}
3462 \def\bbl@alphnumer#1#2{%
3463   \expandafter\bbl@alphnumer@i\number#2 76543210\@@{#1}}
3464 \def\bbl@alphnumer@i#1#2#3#4#5#6#7#8\@@#9{%
3465   \ifcase\@car#8\@nil\or   % Currently <10000, but prepared for bigger
3466     \bbl@alphnumer@ii{#9}00000#1\or
3467     \bbl@alphnumer@ii{#9}00000#1#2\or
3468     \bbl@alphnumer@ii{#9}00000#1#2#3\or
3469     \bbl@alphnumer@ii{#9}00000#1#2#3#4\else
3470     \bbl@error{counter-too-large}{>9999}{}}%
3471   \fi}
3472 \def\bbl@alphnumer@ii#1#2#3#4#5#6#7#8{%
3473   \bbl@ifunset{\bbl@cntr@#1.F.\number#5#6#7#8@\language}%
3474   {\bbl@cs{cntr@#1.4@\language}{#5}%
3475     \bbl@cs{cntr@#1.3@\language}{#6}%
3476     \bbl@cs{cntr@#1.2@\language}{#7}%
3477     \bbl@cs{cntr@#1.1@\language}{#8}%
3478     \ifnum#6#7#8>\z@
3479       \bbl@ifunset{\bbl@cntr@#1.S.321@\language}{}}%
3480     {\bbl@cs{cntr@#1.S.321@\language}{}}%
3481     \fi}%
3482   {\bbl@cs{cntr@#1.F.\number#5#6#7#8@\language}}}
```

Some Chinese counters follow special rules.

```

3483 \def\bbl@zhnumer#1#2{%
3484   \ifnum#1<10 \bbl@zhnum000#1{#2}%
3485   \else\ifnum#1<100 \bbl@zhnum00#1{#2}%
3486   \else\ifnum#1<1000 \bbl@zhnum0#1{#2}%
3487   \else\ifnum#1<10000 \bbl@zhnum#1{#2}%
3488   \else\bbl@error{counter-too-large}{#1}{}}%
3489   \fi\fi\fi\fi}
3490 \def\bbl@zhnum#1#2#3#4#5{%
3491   \ifnum\numexpr#1#2#3#4\relax=\z@
3492   \localenumberal{#5.C}\@ne
3493   \else
3494     \ifnum#1>\z@\localenumberal{#5.D}{#1}\localenumberal{#5.C}5\fi % Thousands
3495     \ifnum#2>\z@\localenumberal{#5.D}{#2}\localenumberal{#5.C}4\fi % Hundreds
3496     \else\ifnum#1>\z@\ifnum#3#4>\z@\localenumberal{#5.C}\@ne\fi
3497     \fi\fi
3498     \ifnum#3>\z@ % Tens
3499       \ifnum#1#2=\z@\ifnum#3=\@ne\else\localenumberal{#5.D}{#3}\fi % No  for 10-19:
3500       \else\localenumberal{#5.D}{#3}%
3501       \fi
3502       \localenumberal{#5.C}\thr@@
3503     \else
3504       \ifnum#2>\z@\ifnum#4>\z@\localenumberal{#5.C}\@ne\fi\fi
3505       \fi
3506       \ifnum#4>\z@\localenumberal{#5.D}{#4}\fi % Units
3507     \fi}
```

4.24. Casing

```

3508 \newcommand\BabelUppercaseMapping[3]{%
3509   \DeclareUppercaseMapping[\@nameuse{\bbl@casing@#1}]{#2}{#3}}
```

```

3510 \newcommand\BabelTitlecaseMapping[3]{%
3511   \DeclareTitlecaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}
3512 \newcommand\BabelLowercaseMapping[3]{%
3513   \DeclareLowercaseMapping[\@nameuse{bbl@casing@#1}]{#2}{#3}}

  The parser for casing and casing. (variant).
3514 \ifcase\bbl@engine % Converts utf8 to its code (expandable)
3515   \def\bbl@uftocode#1{\the\numexpr\decode@UTFviii#1\relax}
3516 \else
3517   \def\bbl@uftocode#1{\expandafter`\string#1}
3518 \fi
3519 \def\bbl@casemapping#1#2#3{% 1:variant
3520   \def\bbl@tempa##1 ##2{% Loop
3521     \bbl@casemapping@i{##1}%
3522     \ifx\@empty##2\else\bbl@afterfi\bbl@tempa##2\fi}%
3523   \edef\bbl@templ{\@nameuse{bbl@casing@#2}#1}% Language code
3524   \def\bbl@tempe{0}% Mode (upper/lower...)
3525   \def\bbl@tempc{#3}% Casing list
3526   \expandafter\bbl@tempa\bbl@tempc\@empty}
3527 \def\bbl@casemapping@i#1{%
3528   \def\bbl@tempb{#1}%
3529   \ifcase\bbl@engine % Handle utf8 in pdftex, by surrounding chars with {}
3530     \@nameuse{regex_replace_all:nnN}%
3531     {[\\x{c0}-\\x{ff}][\\x{80}-\\x{bf}]*}{\\0}}\bbl@tempb
3532   \else
3533     \@nameuse{regex_replace_all:nnN}{.}{\\0}}\bbl@tempb
3534   \fi
3535   \expandafter\bbl@casemapping@ii\bbl@tempb\@{
3536 \def\bbl@casemapping@ii#1#2#3\@{
3537   \in@{#1#3}{<>}% i.e., if <u>, <l>, <t>
3538   \ifin@
3539     \edef\bbl@tempe{%
3540       \if#2u1 \else\if#2l2 \else\if#2t3 \fi\fi\fi}%
3541   \else
3542     \ifcase\bbl@tempe\relax
3543       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3544       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#2}}{#1}%
3545     \or
3546       \DeclareUppercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3547     \or
3548       \DeclareLowercaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3549     \or
3550       \DeclareTitlecaseMapping[\bbl@templ]{\bbl@uftocode{#1}}{#2}%
3551     \fi
3552   \fi}

```

4.25. Getting info

The information in the identification section can be useful, so the following macro just exposes it with a user command.

```

3553 \def\bbl@localeinfo#1#2{%
3554   \bbl@ifunset{bbl@info@#2}{#1}%
3555   {\bbl@ifunset{bbl@csname bbl@info@#2\endcsname @\languagename}{#1}%
3556    {\bbl@cs{csname bbl@info@#2\endcsname @\languagename}}}%
3557 \newcommand\localeinfo[1]{%
3558   \ifx*#1\@empty
3559     \bbl@afterelse\bbl@localeinfo{}%
3560   \else
3561     \bbl@localeinfo
3562     {\bbl@error{no-ini-info}{}}{}{}%
3563     {#1}%
3564   \fi}
3565 % \namedef{bbl@info@name.locale}{lcname}
3566 \namedef{bbl@info@tag.ini}{lini}

```

```

3567 \@namedef{bbl@info@name.english}{elname}
3568 \@namedef{bbl@info@name.opentype}{lname}
3569 \@namedef{bbl@info@tag.bcp47}{tbc}
3570 \@namedef{bbl@info@language.tag.bcp47}{lbc}
3571 \@namedef{bbl@info@tag.opentype}{lotf}
3572 \@namedef{bbl@info@script.name}{esname}
3573 \@namedef{bbl@info@script.name.opentype}{sname}
3574 \@namedef{bbl@info@script.tag.bcp47}{sbcp}
3575 \@namedef{bbl@info@script.tag.opentype}{sotf}
3576 \@namedef{bbl@info@region.tag.bcp47}{rbcp}
3577 \@namedef{bbl@info@variant.tag.bcp47}{vbcp}
3578 \@namedef{bbl@info@extension.t.tag.bcp47}{extt}
3579 \@namedef{bbl@info@extension.u.tag.bcp47}{extu}
3580 \@namedef{bbl@info@extension.x.tag.bcp47}{extx}

```

With version 3.75 \BabelEnsureInfo is executed always, but there is an option to disable it. Since the info in ini files are always loaded, it has been made no-op in version 25.8.

```

3581 <(*More package options)> ≡
3582 \DeclareOption{ensureinfo=off}{}
3583 <(/More package options)>
3584 \let\BabelEnsureInfo\relax

```

More general, but non-expandable, is \getlocaleproperty.

```

3585 \newcommand\getlocaleproperty{%
3586   \@ifstar\bbl@getproperty@s\bbl@getproperty@x}
3587 \def\bbl@getproperty@s#1#2#3{%
3588   \let#1\relax
3589   \def\bbl@elt##1##2##3{%
3590     \bbl@ifsamestring{##1/##2}{#3}%
3591     {\providecommand#1{##3}%
3592     \def\bbl@elt####1####2####3{}}}%
3593   {}}%
3594   \bbl@cs{inidata@#2}}%
3595 \def\bbl@getproperty@x#1#2#3{%
3596   \bbl@getproperty@s{#1}{#2}{#3}%
3597   \ifx#1\relax
3598     \bbl@error{unknown-locale-key}{#1}{#2}{#3}%
3599   \fi}

```

To inspect every possible loaded ini, we define \LocaleForEach, where \bbl@ini@loaded is a comma-separated list of locales, built by \bbl@read@ini.

```

3600 \let\bbl@ini@loaded\@empty
3601 \newcommand\LocaleForEach{\bbl@foreach\bbl@ini@loaded}
3602 \def\ShowLocaleProperties#1{%
3603   \typeout{}}%
3604   \typeout{*** Properties for language '#1' ***}%
3605   \def\bbl@elt##1##2##3{\typeout{##1/##2 = \unexpanded{##3}}}%
3606   \@nameuse{bbl@inidata@#1}%
3607   \typeout{*****}}

```

4.26. BCP 47 related commands

This macro is called by language selectors when the language isn't recognized. So, it's the core for (1) mapping from a BCP 27 tag to the actual language, if bcp47.toname is enabled (i.e., if bbl@bcptoname is true), and (2) lazy loading. With autoload.bcp47 enabled *and* lazy loading, we must first build a name for the language, with the help of autoload.bcp47.prefix. Then we use \provideprovide passing the options set with autoload.bcp47.options (by default import). Finally, and if the locale has not been loaded before, we use \provideprovide with the language name as passed to the selector.

```

3608 \newif\ifbbl@bcpallowed
3609 \bbl@bcpallowedfalse
3610 \def\bbl@autoload@options{@import}
3611 \def\bbl@provide@locale{%

```



```

3612 \ifx\babelprovide\undefined
3613   \bbl@error{base-on-the-fly}{\}\}%
3614 \fi
3615 \let\bbl@auxname\language
3616 \ifbbl@bcptoname
3617   \bbl@ifunset{bbl@bcp@map@\language}{\}% Move uplevel??
3618   {\edef\language{\@nameuse{bbl@bcp@map@\language}}\%
3619     \let\localname\language}%
3620 \fi
3621 \ifbbl@bcpallowed
3622   \expandafter\ifx\csname date\language\endcsname\relax
3623     \expandafter
3624     \bbl@bcplookup{\language}%
3625     \ifx\bbl@bcp\relax\else % Returned by \bbl@bcplookup
3626       \edef\language{\bbl@bcp@prefix\bbl@bcp}%
3627       \let\localname\language
3628       \expandafter\ifx\csname date\language\endcsname\relax
3629         \let\bbl@initoload\bbl@bcp
3630         \bbl@exp{\babelprovide[\bbl@autoload@bcptoptions]{\language}}%
3631         \let\bbl@initoload\relax
3632       \fi
3633       \bbl@csarg\xdef{bcp@map@\bbl@bcp}{\localname}%
3634     \fi
3635   \fi
3636 \fi
3637 \expandafter\ifx\csname date\language\endcsname\relax
3638   \IfFileExists{babel-\language.tex}%
3639   {\bbl@exp{\babelprovide[\bbl@autoload@options]{\language}}}%
3640   {\}%
3641 \fi}

```

$\text{\LaTeX}$ needs to know the BCP 47 codes for some features. For that, it expects `\BCPdata` to be defined. While language, region, script, and variant are recognized, extension `.<s>` for singletons may change.

Still somewhat hackish. Note `\str_if_eq:nnTF` is fully expandable (`\bbl@ifsamestring` isn't). The argument is the prefix to tag.bcp47.

```

3642 \providecommand\BCPdata{}
3643 \ifx\renewcommand\undefined\else
3644   \renewcommand\BCPdata[1]{\bbl@bcpdata@i#1\@empty\@empty\@empty}
3645   \def\bbl@bcpdata@i#1#2#3#4#5#6\@empty{%
3646     \@nameuse{str_if_eq:nnTF}{#1#2#3#4#5}{main.}%
3647     {\bbl@bcpdata@ii{#6}\bbl@main@language}%
3648     {\bbl@bcpdata@ii{#1#2#3#4#5#6}\language}}%
3649   \def\bbl@bcpdata@ii#1#2{%
3650     \bbl@ifunset{bbl@info@#1.tag.bcp47}%
3651     {\bbl@error{unknown-ini-field}{#1}{\}\}%
3652     {\bbl@ifunset{bbl@\csname bbl@info@#1.tag.bcp47\endcsname @#2}{\}%
3653       {\bbl@cs{\csname bbl@info@#1.tag.bcp47\endcsname @#2}}}}
3654 \fi
3655 \@namedef{bbl@info@casing.tag.bcp47}{casing}
3656 \@namedef{bbl@info@tag.tag.bcp47}{tbc} % For \BCPdata

```

5. Adjusting the Babel behavior

A generic high level interface is provided to adjust some global and general settings.

```

3657 \newcommand\babeladjust[1]{%
3658   \bbl@forkv{#1}{%
3659     \bbl@ifunset{bbl@ADJ@##1@##2}%
3660     {\bbl@cs{ADJ@##1}{##2}}%
3661     {\bbl@cs{ADJ@##1@##2}}}%
3662 %
3663 \def\bbl@adjust@lua#1#2{%

```

```

3664 \ifvmode
3665 \ifnum\currentgrouplevel=\z@
3666 \directlua{ Babel.#2 }%
3667 \expandafter\expandafter\expandafter\@gobble
3668 \fi
3669 \fi
3670 {\bbl@error{adjust-only-vertical}{#1}{}}}% Gobbled if everything went ok.
3671 \@namedef{bbl@ADJ@bidi.mirroring@on}{%
3672 \bbl@adjust@lua{bidi}{mirroring_enabled=true}}
3673 \@namedef{bbl@ADJ@bidi.mirroring@off}{%
3674 \bbl@adjust@lua{bidi}{mirroring_enabled=false}}
3675 \@namedef{bbl@ADJ@bidi.text@on}{%
3676 \bbl@adjust@lua{bidi}{bidi_enabled=true}}
3677 \@namedef{bbl@ADJ@bidi.text@off}{%
3678 \bbl@adjust@lua{bidi}{bidi_enabled=false}}
3679 \@namedef{bbl@ADJ@bidi.math@on}{%
3680 \let\bbl@noamsmath\@empty}
3681 \@namedef{bbl@ADJ@bidi.math@off}{%
3682 \let\bbl@noamsmath\relax}
3683 %
3684 \@namedef{bbl@ADJ@bidi.mapdigits@on}{%
3685 \bbl@adjust@lua{bidi}{digits_mapped=true}}
3686 \@namedef{bbl@ADJ@bidi.mapdigits@off}{%
3687 \bbl@adjust@lua{bidi}{digits_mapped=false}}
3688 %
3689 \@namedef{bbl@ADJ@linebreak.sea@on}{%
3690 \bbl@adjust@lua{linebreak}{sea_enabled=true}}
3691 \@namedef{bbl@ADJ@linebreak.sea@off}{%
3692 \bbl@adjust@lua{linebreak}{sea_enabled=false}}
3693 \@namedef{bbl@ADJ@linebreak.cjk@on}{%
3694 \bbl@adjust@lua{linebreak}{cjk_enabled=true}}
3695 \@namedef{bbl@ADJ@linebreak.cjk@off}{%
3696 \bbl@adjust@lua{linebreak}{cjk_enabled=false}}
3697 \@namedef{bbl@ADJ@justify.arabic@on}{%
3698 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=true}}
3699 \@namedef{bbl@ADJ@justify.arabic@off}{%
3700 \bbl@adjust@lua{linebreak}{arabic.justify_enabled=false}}
3701 %
3702 \def\bbl@adjust@layout#1{%
3703 \ifvmode
3704 #1%
3705 \expandafter\@gobble
3706 \fi
3707 {\bbl@error{layout-only-vertical}{}}}% Gobbled if everything went ok.
3708 \@namedef{bbl@ADJ@layout.tabular@on}{%
3709 \ifnum\bbl@tabular@mode=\tw@
3710 \bbl@adjust@layout{\let\@tabular\bbl@NL@@tabular}%
3711 \else
3712 \chardef\bbl@tabular@mode\@ne
3713 \fi}
3714 \@namedef{bbl@ADJ@layout.tabular@off}{%
3715 \ifnum\bbl@tabular@mode=\tw@
3716 \bbl@adjust@layout{\let\@tabular\bbl@OL@@tabular}%
3717 \else
3718 \chardef\bbl@tabular@mode\z@
3719 \fi}
3720 \@namedef{bbl@ADJ@layout.lists@on}{%
3721 \bbl@adjust@layout{\let\list\bbl@NL@list}}
3722 \@namedef{bbl@ADJ@layout.lists@off}{%
3723 \bbl@adjust@layout{\let\list\bbl@OL@list}}
3724 %
3725 \@namedef{bbl@ADJ@autoload.bcp47@on}{%
3726 \bbl@bcpallowedtrue}

```

```

3727 \@namedef{bbl@ADJ@autoload.bcp47@off}{%
3728   \bbl@bcppallowedfalse}
3729 \@namedef{bbl@ADJ@autoload.bcp47.prefix}#1{%
3730   \def\bbl@bcpp@prefix{#1}}
3731 \def\bbl@bcpp@prefix{bcp47-}
3732 \@namedef{bbl@ADJ@autoload.options}#1{%
3733   \def\bbl@autoload@options{#1}}
3734 \def\bbl@autoload@bcppoptions{import}
3735 \@namedef{bbl@ADJ@autoload.bcp47.options}#1{%
3736   \def\bbl@autoload@bcppoptions{#1}}
3737 \newif\ifbbl@bcptoname
3738 %
3739 \@namedef{bbl@ADJ@bcp47.toname@on}{%
3740   \bbl@bcptonametrue}
3741 \@namedef{bbl@ADJ@bcp47.toname@off}{%
3742   \bbl@bcptonamefalse}
3743 %
3744 \@namedef{bbl@ADJ@prehyphenation.disable@nohyphenation}{%
3745   \directlua{ Babel.ignore_pre_char = function(node)
3746     return (node.lang == \the\csname \@nohyphenation\endcsname)
3747   end }}
3748 \@namedef{bbl@ADJ@prehyphenation.disable@off}{%
3749   \directlua{ Babel.ignore_pre_char = function(node)
3750     return false
3751   end }}
3752 %
3753 \@namedef{bbl@ADJ@interchar.disable@nohyphenation}{%
3754   \def\bbl@ignoreinterchar{%
3755     \ifnum\language=\@nohyphenation
3756       \expandafter\@gobble
3757     \else
3758       \expandafter\@firstofone
3759     \fi}}
3760 \@namedef{bbl@ADJ@interchar.disable@off}{%
3761   \let\bbl@ignoreinterchar\@firstofone}
3762 %
3763 \@namedef{bbl@ADJ@select.write@shift}{%
3764   \let\bbl@restorelastskip\relax
3765   \def\bbl@savelastskip{%
3766     \let\bbl@restorelastskip\relax
3767     \ifvmode
3768       \ifdim\lastskip=\z@
3769         \let\bbl@restorelastskip\nobreak
3770       \else
3771         \bbl@exp{%
3772           \def\\bbl@restorelastskip{%
3773             \skip@=\the\lastskip
3774             \\nobreak \vskip-\skip@ \vskip\skip@}}%
3775         \fi
3776       \fi}}
3777 \@namedef{bbl@ADJ@select.write@keep}{%
3778   \let\bbl@restorelastskip\relax
3779   \let\bbl@savelastskip\relax}
3780 \@namedef{bbl@ADJ@select.write@omit}{%
3781   \AddBabelHook{babel-select}{beforestart}{%
3782     \expandafter\babel@aux\expandafter{\bbl@main@language}{}}%
3783   \let\bbl@restorelastskip\relax
3784   \def\bbl@savelastskip##1\bbl@restorelastskip{}}
3785 \@namedef{bbl@ADJ@select.encoding@off}{%
3786   \let\bbl@encoding@select@off\@empty}

```

5.1. Cross referencing macros

The $\LaTeX$ book states:

The *key* argument is any sequence of letters, digits, and punctuation symbols; upper- and lowercase letters are regarded as different.

When the above quote should still be true when a document is typeset in a language that has active characters, special care has to be taken of the category codes of these characters when they appear in an argument of the cross referencing macros.

When a cross referencing command processes its argument, all tokens in this argument should be character tokens with category ‘letter’ or ‘other’.

The following package options control which macros are to be redefined.

```
3787 ({*More package options}) ≡
3788 \DeclareOption{safe=none}{\let\bbl@opt@safe\empty}
3789 \DeclareOption{safe=bib}{\def\bbl@opt@safe{B}}
3790 \DeclareOption{safe=ref}{\def\bbl@opt@safe{R}}
3791 \DeclareOption{safe=refbib}{\def\bbl@opt@safe{BR}}
3792 \DeclareOption{safe=bibref}{\def\bbl@opt@safe{BR}}
3793 ({/More package options})
```

\@newl@bel First we open a new group to keep the changed setting of `\protect` local and then we set the `@safe@actives` switch to true to make sure that any shorthand that appears in any of the arguments immediately expands to its non-active self.

```
3794 \bbl@trace{Cross referencing macros}
3795 \ifx\bbl@opt@safe\empty\else % i.e., if 'ref' and/or 'bib'
3796   \def\@newl@bel#1#2#3{%
3797     {\@safe@activestrue
3798       \bbl@ifunset{#1@#2}%
3799       \relax
3800       {\gdef\@multiplelabels{%
3801         \@latex@warning@no@line{There were multiply-defined labels}}%
3802         \@latex@warning@no@line{Label `#2' multiply defined}}%
3803       \global\@namedef{#1@#2}{#3}}}
```

\@testdef An internal $\LaTeX$ macro used to test if the labels that have been written on the aux file have changed. It is called by the `\enddocument` macro.

```
3804 \CheckCommand*\@testdef[3]{%
3805   \def\reserved@a{#3}%
3806   \expandafter\ifx\csname#1@#2\endcsname\reserved@a
3807   \else
3808     \@tempswatrue
3809   \fi}
```

Now that we made sure that `\@testdef` still has the same definition we can rewrite it. First we make the shorthands ‘safe’. Then we use `\bbl@tempa` as an ‘alias’ for the macro that contains the label which is being checked. Then we define `\bbl@tempb` just as `\@newl@bel` does it. When the label is defined we replace the definition of `\bbl@tempa` by its meaning. If the label didn’t change, `\bbl@tempa` and `\bbl@tempb` should be identical macros.

```
3810 \def\@testdef#1#2#3{%
3811   \@safe@activestrue
3812   \expandafter\let\expandafter\bbl@tempa\csname #1@#2\endcsname
3813   \def\bbl@tempb{#3}%
3814   \@safe@activesfalse
3815   \ifx\bbl@tempa\relax
3816   \else
3817     \edef\bbl@tempa{\expandafter\strip@prefix\meaning\bbl@tempa}%
3818   \fi
3819   \edef\bbl@tempb{\expandafter\strip@prefix\meaning\bbl@tempb}%
3820   \ifx\bbl@tempa\bbl@tempb
3821   \else
3822     \@tempswatrue
3823   \fi}
3824 \fi
```

\ref

\pageref The same holds for the macro `\ref` that references a label and `\pageref` to reference a page. We make them robust as well (if they weren't already) to prevent problems if they should become expanded at the wrong moment.

```
3825 \bbl@xin@{R}\bbl@opt@safe
3826 \ifin@
3827 \edef\bbl@tempc{\expandafter\string\csname ref_code\endcsname}%
3828 \bbl@xin@{\expandafter\strip@prefix\meaning\bbl@tempc}%
3829 {\expandafter\strip@prefix\meaning\ref}%
3830 \ifin@
3831 \bbl@redefine\@kernel@ref#1{%
3832   \@safe@activetrue\org@@kernel@ref{#1}\@safe@activfalse}
3833 \bbl@redefine\@kernel@pageref#1{%
3834   \@safe@activetrue\org@@kernel@pageref{#1}\@safe@activfalse}
3835 \bbl@redefine\@kernel@sref#1{%
3836   \@safe@activetrue\org@@kernel@sref{#1}\@safe@activfalse}
3837 \bbl@redefine\@kernel@spageref#1{%
3838   \@safe@activetrue\org@@kernel@spageref{#1}\@safe@activfalse}
3839 \else
3840 \bbl@redefinero bust\ref#1{%
3841   \@safe@activetrue\org@ref{#1}\@safe@activfalse}
3842 \bbl@redefinero bust\pageref#1{%
3843   \@safe@activetrue\org@pageref{#1}\@safe@activfalse}
3844 \fi
3845 \else
3846 \let\org@ref\ref
3847 \let\org@pageref\pageref
3848 \fi
```

\@citex The macro used to cite from a bibliography, `\cite`, uses an internal macro, `\@citex`. It is this internal macro that picks up the argument(s), so we redefine this internal macro and leave `\cite` alone. The first argument is used for typesetting, so the shorthands need only be deactivated in the second argument.

```
3849 \bbl@xin@{B}\bbl@opt@safe
3850 \ifin@
3851 \bbl@redefine\@citex[#1]#2{%
3852   \@safe@activetrue\edef\bbl@tempa{#2}\@safe@activfalse
3853   \org@@citex[#1]{\bbl@tempa}}
```

Unfortunately, the packages `natbib` and `cite` need a different definition of `\@citex`... To begin with, `natbib` has a definition for `\@citex` with *three* arguments... We only know that a package is loaded when `\begin{document}` is executed, so we need to postpone the different redefinition.

Notice that we use `\def` here instead of `\bbl@redefine` because `\org@@citex` is already defined and we don't want to overwrite that definition (it would result in parameter stack overflow because of a circular definition).

(Recent versions of `natbib` change dynamically `\@citex`, so PR4087 doesn't seem fixable in a simple way. Just load `natbib` before.)

```
3854 \AtBeginDocument{%
3855   \@ifpackageloaded{natbib}{%
3856     \def\@citex[#1][#2]#3{%
3857       \@safe@activetrue\edef\bbl@tempa{#3}\@safe@activfalse
3858       \org@@citex[#1][#2]{\bbl@tempa}}%
3859   }}
```

The package `cite` has a definition of `\@citex` where the shorthands need to be turned off in both arguments.

```
3860 \AtBeginDocument{%
3861   \@ifpackageloaded{cite}{%
3862     \def\@citex[#1]#2{%
3863       \@safe@activetrue\org@@citex[#1][#2]\@safe@activfalse}%
3864     }}
```

\nocite The macro \nocite which is used to instruct BiB_T_EX to extract uncited references from the database.

```
3865 \bbl@redefine\nocite#1{%
3866   \@safe@activestrue\org@nocite{#1}\@safe@activesfalse}
```

\bibcite The macro that is used in the aux file to define citation labels. When packages such as natbib or cite are not loaded its second argument is used to typeset the citation label. In that case, this second argument can contain active characters but is used in an environment where \@safe@activestrue is in effect. This switch needs to be reset inside the \hbox which contains the citation label. In order to determine during aux file processing which definition of \bibcite is needed we define \bibcite in such a way that it redefines itself with the proper definition. We call \bbl@cite@choice to select the proper definition for \bibcite. This new definition is then activated.

```
3867 \bbl@redefine\bibcite{%
3868   \bbl@cite@choice
3869   \bibcite}
```

\bbl@bibcite The macro \bbl@bibcite holds the definition of \bibcite needed when neither natbib nor cite is loaded.

```
3870 \def\bbl@bibcite#1#2{%
3871   \org@bibcite{#1}{\@safe@activesfalse#2}}
```

\bbl@cite@choice The macro \bbl@cite@choice determines which definition of \bibcite is needed. First we give \bibcite its default definition.

```
3872 \def\bbl@cite@choice{%
3873   \global\let\bibcite\bbl@bibcite
3874   \@ifpackageloaded{natbib}{\global\let\bibcite\org@bibcite}{}%
3875   \@ifpackageloaded{cite}{\global\let\bibcite\org@bibcite}{}%
3876   \global\let\bbl@cite@choice\relax}
```

When a document is run for the first time, no aux file is available, and \bibcite will not yet be properly defined. In this case, this has to happen before the document starts.

```
3877 \AtBeginDocument{\bbl@cite@choice}
```

\@bibitem One of the two internal L^AT_EX macros called by \bibitem that write the citation label on the aux file.

```
3878 \bbl@redefine\@bibitem#1{%
3879   \@safe@activestrue\org@@bibitem{#1}\@safe@activesfalse}
3880 \else
3881   \let\org@nocite\nocite
3882   \let\org@@citex\@citex
3883   \let\org@bibcite\bibcite
3884   \let\org@@bibitem\@bibitem
3885 \fi
```

5.2. Layout

```
3886 \newcommand\BabelPatchSection[1]{%
3887   \@ifundefined{#1}{}{%
3888     \bbl@exp{\let\bbl@ss@#1>\<#1>}%
3889     \@namedef{#1}{%
3890       \ifstar{\bbl@presec@#1}%
3891       {\@dblarg{\bbl@presec@x{#1}}}}%
3892   \def\bbl@presec@x#1[#2]#3{%
3893     \bbl@exp{%
3894       \\select@language@x{\bbl@main@language}%
3895       \\bbl@cs{sspre@#1}%
3896       \\bbl@cs{ss@#1}%
3897       [\\foreignlanguage{\language}\unexpanded{#2}}}%
3898       {\\foreignlanguage{\language}\unexpanded{#3}}}%
3899     \\select@language@x{\language}}}
```

```

3900 \def\bbl@presec@s#1#2{%
3901   \bbl@exp{%
3902     \\\select@language@x{\bbl@main@language}%
3903     \\\bbl@cs{sspre@#1}%
3904     \\\bbl@cs{ss@#1}*%
3905     {\\\foreignlanguage{\language}{\unexpanded{#2}}}%
3906     \\\select@language@x{\language}}%
3907 %
3908 \IfBabelLayout{sectioning}%
3909   {\BabelPatchSection{part}%
3910    \BabelPatchSection{chapter}%
3911    \BabelPatchSection{section}%
3912    \BabelPatchSection{subsection}%
3913    \BabelPatchSection{subsubsection}%
3914    \BabelPatchSection{paragraph}%
3915    \BabelPatchSection{subparagraph}%
3916    \def\babel@toc#1{%
3917      \select@language@x{\bbl@main@language}}}%
3918 \IfBabelLayout{captions}%
3919   {\BabelPatchSection{caption}}}%

```

\BabelFootnote Footnotes.

```

3920 \bbl@trace{Footnotes}
3921 \def\bbl@footnote#1#2#3{%
3922   \ifnextchar[%
3923     {\bbl@footnote@o{#1}{#2}{#3}}%
3924     {\bbl@footnote@x{#1}{#2}{#3}}}
3925 \long\def\bbl@footnote@x#1#2#3#4{%
3926   \bgroup
3927     \select@language@x{\bbl@main@language}%
3928     \bbl@fn@footnote{#2#1{\ignorespaces#4}#3}%
3929   \egroup}
3930 \long\def\bbl@footnote@o#1#2#3[#4]#5{%
3931   \bgroup
3932     \select@language@x{\bbl@main@language}%
3933     \bbl@fn@footnote[#4]{#2#1{\ignorespaces#5}#3}%
3934   \egroup}
3935 \def\bbl@footnotetext#1#2#3{%
3936   \ifnextchar[%
3937     {\bbl@footnotetext@o{#1}{#2}{#3}}%
3938     {\bbl@footnotetext@x{#1}{#2}{#3}}}
3939 \long\def\bbl@footnotetext@x#1#2#3#4{%
3940   \bgroup
3941     \select@language@x{\bbl@main@language}%
3942     \bbl@fn@footnotetext{#2#1{\ignorespaces#4}#3}%
3943   \egroup}
3944 \long\def\bbl@footnotetext@o#1#2#3[#4]#5{%
3945   \bgroup
3946     \select@language@x{\bbl@main@language}%
3947     \bbl@fn@footnotetext[#4]{#2#1{\ignorespaces#5}#3}%
3948   \egroup}
3949 \def\BabelFootnote#1#2#3#4{%
3950   \ifx\bbl@fn@footnote\undefined
3951     \let\bbl@fn@footnote\footnote
3952   \fi
3953   \ifx\bbl@fn@footnotetext\undefined
3954     \let\bbl@fn@footnotetext\footnotetext
3955   \fi
3956   \bbl@iifblank{#2}%
3957     {\def#1{\bbl@footnote{\@firstofone}{#3}{#4}}
3958      \namedef{\bbl@stripslash#1text}%
3959      {\bbl@footnotetext{\@firstofone}{#3}{#4}}}%
3960     {\def#1{\bbl@exp{\bbl@footnote{\foreignlanguage{#2}}}{#3}{#4}}}%

```

```

3961 \namedef{\bbl@stripslash#1text}%
3962 {\bbl@exp{\bbl@footnotetext{\foreignlanguage{#2}}{#3}{#4}}}%
3963 \IfBabelLayout{footnotes}%
3964 {\let\bbl@OL@footnote\footnote
3965 \BabelFootnote\footnote\languagesname{}}%
3966 \BabelFootnote\localfootnote\languagesname{}}%
3967 \BabelFootnote\mainfootnote{}}%
3968 {}

```

5.3. Marks

\markright Because the output routine is asynchronous, we must pass the current language attribute to the head lines. To achieve this we need to adapt the definition of `\markright` and `\markboth` somewhat. However, headlines and footlines can contain text outside marks; for that we must take some actions in the output routine if the 'headfoot' options is used.

We need to make some redefinitions to the output routine to avoid an endless loop and to correctly handle the page number in bidi documents.

```

3969 \bbl@trace{Marks}
3970 \IfBabelLayout{sectioning}
3971 {\ifx\bbl@opt@headfoot\@nnil
3972 \g@addto@macro\resetactivechars{%
3973 \set@typeset@protect
3974 \expandafter\select@language@x\expandafter{\bbl@main@language}%
3975 \let\protect\noexpand
3976 \ifcase\bbl@bidimode\else % Only with bidi. See also above
3977 \edef\thepage{%
3978 \noexpand\babelsublr{\unexpanded\expandafter{\thepage}}}%
3979 \fi}%
3980 \fi}
3981 {\ifbbl@single\else
3982 \bbl@ifunset{markright } \bbl@redefine\bbl@redefineroobust
3983 \markright#1{%
3984 \bbl@ifblank{#1}%
3985 {\org@markright{}}%
3986 {\toks@{#1}%
3987 \bbl@exp{%
3988 \org@markright{\protect\foreignlanguage{\languagesname}%
3989 \protect\bbl@restore@actives\the\toks@}}}%

```

\markboth

\@mkboth The definition of `\markboth` is equivalent to that of `\markright`, except that we need two token registers. The documentclasses report and book define and set the headings for the page. While doing so they also store a copy of `\markboth` in `\@mkboth`. Therefore we need to check whether `\@mkboth` has already been set. If so we need to do that again with the new definition of `\markboth`. (As of Oct 2019, ~~La~~TeX stores the definition in an intermediate macro, so it's not necessary anymore, but it's preserved for older versions.)

```

3990 \ifx\@mkboth\markboth
3991 \def\bbl@tempc{\let\@mkboth\markboth}%
3992 \else
3993 \def\bbl@tempc{%
3994 \fi
3995 \bbl@ifunset{markboth } \bbl@redefine\bbl@redefineroobust
3996 \markboth#1#2{%
3997 \protected@edef\bbl@tempb##1{%
3998 \protect\foreignlanguage
3999 {\languagesname}{\protect\bbl@restore@actives##1}}%
4000 \bbl@ifblank{#1}%
4001 {\toks@{}}%
4002 {\toks@\expandafter{\bbl@tempb{#1}}}%
4003 \bbl@ifblank{#2}%
4004 {\@temptokena{}}%
4005 {\@temptokena\expandafter{\bbl@tempb{#2}}}%

```



```

4006      \bbl@exp{\org@markboth{\the\toks@}{\the\temptokena}}}%
4007      \bbl@tempc
4008      \fi} % end ifbbl@single, end \IfBabelLayout

```

5.4. Other packages

5.4.1. ifthen

\ifthenelse Sometimes a document writer wants to create a special effect depending on the page a certain fragment of text appears on. This can be achieved by the following piece of code:

```

% \ifthenelse{\isodd{\pageref{some-label}}}{
%      {code for odd pages}
%      {code for even pages}
%

```

In order for this to work the argument of `\isodd` needs to be fully expandable. With the above redefinition of `\pageref` it is not in the case of this example. To overcome that, we add some code to the definition of `\ifthenelse` to make things work.

We want to revert the definition of `\pageref` and `\ref` to their original definition for the first argument of `\ifthenelse`, so we first need to store their current meanings.

Then we can set the `\@safe@actives` switch and call the original `\ifthenelse`. In order to be able to use shorthands in the second and third arguments of `\ifthenelse` the resetting of the switch *and* the definition of `\pageref` happens inside those arguments.

```

4009 \bbl@trace{Preventing clashes with other packages}
4010 \ifx\org@ref\undefined\else
4011   \bbl@xin@{R}\bbl@opt@safe
4012   \ifin@
4013     \AtBeginDocument{%
4014       \@ifpackageloaded{ifthen}{%
4015         \bbl@redefine@long\ifthenelse#1#2#3{%
4016           \let\bbl@temp@pref\pageref
4017           \let\pageref\org@pageref
4018           \let\bbl@temp@ref\ref
4019           \let\ref\org@ref
4020           \@safe@activestrue
4021           \org@ifthenelse{#1}%
4022             {\let\pageref\bbl@temp@pref
4023              \let\ref\bbl@temp@ref
4024              \@safe@activesfalse
4025              #2}%
4026             {\let\pageref\bbl@temp@pref
4027              \let\ref\bbl@temp@ref
4028              \@safe@activesfalse
4029              #3}%
4030           }%
4031         }{}%
4032       }
4033 \fi

```

5.4.2. varioref

\@vpageref

\vrefpagemum

\Ref When the package `varioref` is in use we need to modify its internal command `\@vpageref` in order to prevent problems when an active character ends up in the argument of `\vref`. The same needs to happen for `\vrefpagemum`.

```

4034 \AtBeginDocument{%
4035   \@ifpackageloaded{varioref}{%
4036     \bbl@redefine\@vpageref#1[#2]#3{%
4037       \@safe@activestrue
4038       \org@@@vpageref{#1}[#2]{#3}%
4039       \@safe@activesfalse}%

```

```

4040 \bbl@redefine\vrefpagemum#1#2{%
4041 \@safe@activestrue
4042 \org\vrefpagemum{#1}{#2}%
4043 \@safe@activesfalse}%

```

The package `varioref` defines `\Ref` to be a robust command which uppercases the first character of the reference text. In order to be able to do that it needs to access the expandable form of `\ref`. So we employ a little trick here. We redefine the (internal) command `\Ref_` to call `\org@ref` instead of `\ref`. The disadvantage of this solution is that whenever the definition of `\Ref` changes, this definition needs to be updated as well.

```

4044 \expandafter\def\csname Ref \endcsname#1{%
4045 \protected@edef\@tempa{\org@ref{#1}}\expandafter\MakeUppercase\@tempa}
4046 }{}%
4047 }
4048 \fi

```

5.4.3. `hhline`

`\hhline` Delaying the activation of the shorthand characters has introduced a problem with the `hhline` package. The reason is that it uses the “`:`” character which is made active by the french support in `babel`. Therefore we need to *reload* the package when the “`:`” is an active character. Note that this happens *after* the category code of the `@`-sign has been changed to other, so we need to temporarily change it to letter again.

```

4049 \AtEndOfPackage{%
4050 \AtBeginDocument{%
4051 \@ifpackageloaded{hhline}%
4052 {\expandafter\ifx\csname normal@char\string\endcsname\relax
4053 \else
4054 \makeatletter
4055 \def\@currname{hhline}\input{hhline.sty}\makeatother
4056 \fi}%
4057 {}}}

```

`\substitutefontfamily` *Deprecated.* It creates an `fd` file on the fly. The first argument is an encoding mnemonic, the second and third arguments are font family names. Use the tools provided by $\TeX$ (`\DeclareFontFamilySubstitution`).

```

4058 \def\substitutefontfamily#1#2#3{%
4059 \lowercase{\immediate\openout15=#1#2.fd\relax}%
4060 \immediate\write15{%
4061 \string\ProvidesFile{#1#2.fd}%
4062 [\the\year/\two@digits{\the\month}/\two@digits{\the\day}
4063 \space generated font description file]^J
4064 \string\DeclareFontFamily{#1}{#2}{}}^J
4065 \string\DeclareFontShape{#1}{#2}{m}{n}{<->ssub * #3/m/n}{}}^J
4066 \string\DeclareFontShape{#1}{#2}{m}{it}{<->ssub * #3/m/it}{}}^J
4067 \string\DeclareFontShape{#1}{#2}{m}{sl}{<->ssub * #3/m/sl}{}}^J
4068 \string\DeclareFontShape{#1}{#2}{m}{sc}{<->ssub * #3/m/sc}{}}^J
4069 \string\DeclareFontShape{#1}{#2}{b}{n}{<->ssub * #3/bx/n}{}}^J
4070 \string\DeclareFontShape{#1}{#2}{b}{it}{<->ssub * #3/bx/it}{}}^J
4071 \string\DeclareFontShape{#1}{#2}{b}{sl}{<->ssub * #3/bx/sl}{}}^J
4072 \string\DeclareFontShape{#1}{#2}{b}{sc}{<->ssub * #3/bx/sc}{}}^J
4073 }%
4074 \closeout15
4075 }
4076 \@onlypreamble\substitutefontfamily

```

5.5. Encoding and fonts

Because documents may use non-ASCII font encodings, we make sure that the logos of $\TeX$ and $\LaTeX$ always come out in the right encoding. There is a list of non-ASCII encodings. Requested encodings are currently stored in `\@fontenc@load@list`. If a non-ASCII has been loaded, we define versions of `\TeX` and `\LaTeX` for them using `\ensureascii`. The default ASCII encoding is set, too (in reverse order): the “main” encoding (when the document begins), the last loaded, or OT1.

\ensureascii

```
4077 \bbl@trace{Encoding and fonts}
4078 \newcommand\BabelNonASCII{LGR,LGI,X2,OT2,OT3,OT6,LHE,LWN,LMA,LMC,LMS,LMU}
4079 \newcommand\BabelNonText{TS1,T3,TS3}
4080 \let\org@TeX\TeX
4081 \let\org@LaTeX\LaTeX
4082 \let\ensureascii\@firstofone
4083 \let\asciienencoding\@empty
4084 \AtBeginDocument{%
4085   \def\@elt#1{,#1,}%
4086   \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%
4087   \let\@elt\relax
4088   \let\bbl@tempb\@empty
4089   \def\bbl@tempc{OT1}%
4090   \bbl@foreach\BabelNonASCII{% LGR loaded in a non-standard way
4091     \bbl@ifunset{T@#1}{\def\bbl@tempb{#1}}}%
4092   \bbl@foreach\bbl@tempa{%
4093     \bbl@xin@{,#1,}{,\BabelNonASCII,}%
4094     \ifin@
4095       \def\bbl@tempb{#1}% Store last non-ascii
4096     \else\bbl@xin@{,#1,}{,\BabelNonText,}% Pass
4097       \ifin@
4098         \def\bbl@tempc{#1}% Store last ascii
4099       \fi
4100     \fi}%
4101   \ifx\bbl@tempb\@empty\else
4102     \bbl@xin@{,\cf@encoding,}{,\BabelNonASCII,\BabelNonText,}%
4103     \ifin@
4104       \edef\bbl@tempc{\cf@encoding}% The default if ascii wins
4105     \fi
4106     \let\asciienencoding\bbl@tempc
4107     \renewcommand\ensureascii[1]{%
4108       {\fontencoding{\asciienencoding}\selectfont#1}}%
4109     \DeclareTextCommandDefault{\TeX}{\ensureascii{\org@TeX}}%
4110     \DeclareTextCommandDefault{\LaTeX}{\ensureascii{\org@LaTeX}}%
4111   \fi}
```

Now comes the old deprecated stuff (with a little change in 3.9l, for fontspec). The first thing we need to do is to determine, at `\begin{document}`, which latin fontencoding to use.

\latinencoding When text is being typeset in an encoding other than 'latin' (OT1 or T1), it would be nice to still have Roman numerals come out in the Latin encoding. So we first assume that the current encoding at the end of processing the package is the Latin encoding.

```
4112 \AtEndOfPackage{\edef\latinencoding{\cf@encoding}}
```

But this might be overruled with a later loading of the package fontenc. Therefore we check at the execution of `\begin{document}` whether it was loaded with the T1 option. The normal way to do this (using `\@ifpackageloaded`) is disabled for this package. Now we have to revert to parsing the internal macro `\@filelist` which contains all the filenames loaded.

```
4113 \AtBeginDocument{%
4114   \@ifpackageloaded{fontspec}%
4115   {\xdef\latinencoding{%
4116     \ifx\UTFencname\undefined
4117       EU\ifcase\bbl@engine\or2\or1\fi
4118     \else
4119       \UTFencname
4120     \fi}}%
4121   {\gdef\latinencoding{OT1}%
4122     \ifx\cf@encoding\bbl@t@one
4123       \xdef\latinencoding{\bbl@t@one}%
4124     \else
4125       \def\@elt#1{,#1,}%
4126       \edef\bbl@tempa{\expandafter\@gobbletwo\@fontenc@load@list}%

```

```

4127      \let\@elt\relax
4128      \bbl@xin@{,Tl,}\bbl@tempa
4129      \ifin@
4130      \xdef\latinencoding{\bbl@t@one}%
4131      \fi
4132      \fi}}

```

\latintext Then we can define the command `\latintext` which is a declarative switch to a latin font-encoding. Usage of this macro is deprecated.

```

4133 \DeclareRobustCommand{\latintext}{%
4134   \fontencoding{\latinencoding}\selectfont
4135   \def\encodingdefault{\latinencoding}}

```

\textlatin This command takes an argument which is then typeset using the requested font encoding. In order to avoid many encoding switches it operates in a local scope.

```

4136 \ifx\@undefined\DeclareTextFontCommand
4137   \DeclareRobustCommand{\textlatin}[1]{\leavevmode{\latintext #1}}
4138 \else
4139   \DeclareTextFontCommand{\textlatin}{\latintext}
4140 \fi

```

For several functions, we need to execute some code with `\selectfont`. With $\text{\LaTeX}$ 2021-06-01, there is a hook for this purpose.

```

4141 \def\bbl@patchfont#1{\AddToHook{selectfont}{#1}}

```

5.6. Basic bidi support

This code is currently placed here for practical reasons. It will be moved to the correct place soon, I hope.

It is loosely based on `rlbabel.def`, but most of it has been developed from scratch. This babel module (by Johannes Braams and Boris Lavva) has served the purpose of typesetting R documents for two decades, and despite its flaws I think it is still a good starting point (some parts have been copied here almost verbatim), partly thanks to its simplicity. I’ve also looked at ARABI (by Youssef Jabri), which is compatible with babel.

There are two ways of modifying macros to make them “bidi”, namely, by patching the internal low-level macros (which is what I have done with lists, columns, counters, tocs, much like `rlbabel` did), and by introducing a “middle layer” just below the user interface (sectioning, footnotes).

- `pdfTeX` provides a minimal support for bidi text, and it must be done by hand. Vertical typesetting is not possible.
- `xetex` is somewhat better, thanks to its font engine (even if not always reliable) and a few additional tools. However, very little is done at the paragraph level. Another challenging problem is text direction does not honour $\text{\TeX}$ grouping.
- `luatex` can provide the most complete solution, as we can manipulate almost freely the node list, the generated lines, and so on, but bidi text does not work out of the box and some development is necessary. It also provides tools to properly set left-to-right and right-to-left page layouts. As `Lua $\text{\TeX}$ -ja` shows, vertical typesetting is possible, too.

```

4142 \bbl@trace{Loading basic (internal) bidi support}
4143 \ifodd\bbl@engine
4144 \else % Any xe+lua bidi
4145   \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
4146     \bbl@error{bidi-only-lua}{\}\}\}\}%
4147     \let\bbl@beforeforeign\leavevmode
4148     \AtEndOfPackage{%
4149       \EnableBabelHook{babel-bidi}%
4150       \bbl@xebidipar}
4151   \fi\fi
4152   \def\bbl@loadxebidi#1{%
4153     \ifx\RTLfootnotetext\@undefined
4154       \AtEndOfPackage{%
4155         \EnableBabelHook{babel-bidi}%

```

```

4156 \ifx\fontspec\undefined
4157 \usepackage{fontspec}% bidi needs fontspec
4158 \fi
4159 \usepackage#1{bidi}%
4160 \let\bbl@digitsdotdash\DigitsDotDashInterCharToks
4161 \def\DigitsDotDashInterCharToks{% See the 'bidi' package
4162 \ifnum\@nameuse{bbl@wdir\language}\tw@ % 'AL' bidi
4163 \bbl@digitsdotdash % So ignore in 'R' bidi
4164 \fi}%
4165 \fi}
4166 \ifnum\bbl@bidimode>200 % Any xe bidi=
4167 \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
4168 \bbl@tentative{bidi=bidi}
4169 \bbl@loadxebidi{}
4170 \or
4171 \bbl@loadxebidi{[rldocument]}
4172 \or
4173 \bbl@loadxebidi{}
4174 \fi
4175 \fi
4176 \fi
4177 \ifnum\bbl@bidimode=\@ne % bidi=default
4178 \let\bbl@beforeforeign\leavevmode
4179 \ifodd\bbl@engine % lua
4180 \newattribute\bbl@attr@dir
4181 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
4182 \bbl@exp{\output{\bodydir\pagedir\the\output}}
4183 \fi
4184 \AtEndOfPackage{%
4185 \EnableBabelHook{babel-bidi}% pdf/lua/xe
4186 \ifodd\bbl@engine\else % pdf/xe
4187 \bbl@xebidipar
4188 \fi}
4189 \fi

```

Now come the macros used to set the direction when a language is switched. Testing are based on script names, because it's the user interface (including language and script in \babelprovide. First the (mostly) common macros.

```

4190 \bbl@trace{Macros to switch the text direction}
4191 \def\bbl@alscripts{%
4192 ,Arabic,Syriac,Thaana,Hanifi Rohingya,Hanifi,Sogdian,}
4193 \def\bbl@rscripts{%
4194 Adlam,Avestan,Chorasmian,Cypriot,Elymaic,Garay,%
4195 Hatran,Hebrew,Imperial Aramaic,Inscriptional Pahlavi,%
4196 Inscriptional Parthian,Kharoshthi,Lydian,Mandaic,Manichaeen,%
4197 Mende Kikakui,Meroitic Cursive,Meroitic Hieroglyphs,Nabataean,%
4198 Nko,Old Hungarian,Old North Arabian,Old Sogdian,%
4199 Old South Arabian,Old Turkic,Old Uyghur,Palmyrene,Phoenician,%
4200 Psalter Pahlavi,Samaritan,Yezidi,Mandaean,%
4201 Meroitic,N'Ko,Orkhon,Todhri}
4202 %
4203 \def\bbl@provide@dirs#1{%
4204 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts\bbl@rscripts}%
4205 \ifin@
4206 \global\bbl@csarg\chardef{wdir@#1}\@ne
4207 \bbl@xin@{\csname bbl@sname@#1\endcsname}{\bbl@alscripts}%
4208 \ifin@
4209 \global\bbl@csarg\chardef{wdir@#1}\tw@
4210 \fi
4211 \else
4212 \global\bbl@csarg\chardef{wdir@#1}\z@
4213 \fi
4214 \ifodd\bbl@engine

```

```

4215 \bbl@csarg\ifcase{wdir@#1}%
4216 \directlua{ Babel.locale_props[\the\localeid].texmdir = 'l' }%
4217 \or
4218 \directlua{ Babel.locale_props[\the\localeid].texmdir = 'r' }%
4219 \or
4220 \directlua{ Babel.locale_props[\the\localeid].texmdir = 'al' }%
4221 \fi
4222 \fi}
4223 %
4224 \def\bbl@switchdir{%
4225 \bbl@ifunset{\bbl@sys{\languagename}}{\bbl@provide@sys{\languagename}}{}%
4226 \bbl@ifunset{\bbl@wdir{\languagename}}{\bbl@provide@dirs{\languagename}}{}%
4227 \bbl@exp{\bbl@setdirs\bbl@cl{wdir}}}%
4228 \def\bbl@setdirs#1{%
4229 \ifcase\bbl@select@type
4230 \bbl@bodydir{#1}%
4231 \bbl@pardir{#1}% <- Must precede \bbl@texmdir
4232 \fi
4233 \bbl@texmdir{#1}}
4234 \ifnum\bbl@bidimode>\z@
4235 \AddBabelHook{babel-bidi}{afterextras}{\bbl@switchdir}
4236 \DisableBabelHook{babel-bidi}
4237 \fi

```

Now the engine-dependent macros.

```

4238 \ifodd\bbl@engine % luatex=1
4239 \else % pdftex=0, xetex=2
4240 \newcount\bbl@dirlevel
4241 \chardef\bbl@thetexmdir\z@
4242 \chardef\bbl@thepardir\z@
4243 \def\bbl@texmdir#1{%
4244 \ifcase#1\relax
4245 \chardef\bbl@thetexmdir\z@
4246 \@nameuse{setlatin}%
4247 \bbl@texmdir@i\beginL\endL
4248 \else
4249 \chardef\bbl@thetexmdir@ne
4250 \@nameuse{setnonlatin}%
4251 \bbl@texmdir@i\beginR\endR
4252 \fi}
4253 \def\bbl@texmdir@i#1#2{%
4254 \ifhmode
4255 \ifnum\currentgrouplevel>\z@
4256 \ifnum\currentgrouplevel=\bbl@dirlevel
4257 \bbl@error{multiple-bidi}{}{}%
4258 \bgroup\aftergroup#2\aftergroup\egroup
4259 \else
4260 \ifcase\currentgrouptype\or % 0 bottom
4261 \aftergroup#2% 1 simple {}
4262 \or
4263 \bgroup\aftergroup#2\aftergroup\egroup % 2 hbox
4264 \or
4265 \bgroup\aftergroup#2\aftergroup\egroup % 3 adj hbox
4266 \or\or\or % vbox vtop align
4267 \or
4268 \bgroup\aftergroup#2\aftergroup\egroup % 7 noalign
4269 \or\or\or\or\or\or % output math disc insert vcent mathchoice
4270 \or
4271 \aftergroup#2% 14 \begingroup
4272 \else
4273 \bgroup\aftergroup#2\aftergroup\egroup % 15 adj
4274 \fi
4275 \fi

```

```

4276      \bbl@dirlevel\currentgrouplevel
4277      \fi
4278      #1%
4279      \fi}
4280      \def\bbl@pardir#1{\chardef\bbl@thepardir#1\relax}
4281      \let\bbl@bodydir\@gobble
4282      \def\bbl@dirparastext{\chardef\bbl@thepardir\bbl@thetextdir}

```

The following command is executed only if there is a right-to-left script (once). It activates the `\everypar` hack for xetex, to properly handle the par direction. Note text and par dirs are decoupled to some extent (although not completely).

```

4283      \def\bbl@xebidipar{%
4284      \let\bbl@xebidipar\relax
4285      \TeXeTstate\@ne
4286      \def\bbl@xeeverypar{%
4287      \ifcase\bbl@thepardir
4288      \ifcase\bbl@thetextdir\else\beginR\fi
4289      \else
4290      {\setbox\z@\lastbox\beginR\box\z@}%
4291      \fi}%
4292      \AddToHook{para/begin}{\bbl@xeeverypar}}
4293      \ifnum\bbl@bidimode>200 % Any xe bidi=
4294      \let\bbl@textdir\i\@gobbletwo
4295      \let\bbl@xebidipar\@empty
4296      \AddBabelHook{bidi}{foreign}{%
4297      \ifcase\bbl@thetextdir
4298      \BabelWrapText{\LR{##1}}%
4299      \else
4300      \BabelWrapText{\RL{##1}}%
4301      \fi}
4302      \def\bbl@pardir#1{\ifcase#1\relax\setLR\else\setRL\fi}
4303      \fi
4304      \fi

```

A tool for weak L (mainly digits). We also disable warnings with hyperref.

```

4305      \DeclareRobustCommand\babelsublr[1]{\leavevmode{\bbl@textdir\z@#1}}
4306      \AtBeginDocument{%
4307      \ifx\pdfstringdefDisableCommands\undefined\else
4308      \ifx\pdfstringdefDisableCommands\relax\else
4309      \pdfstringdefDisableCommands{\let\babelsublr\@firstofone}%
4310      \fi
4311      \fi}

```

5.7. Local Language Configuration

\loadlocalcfg At some sites it may be necessary to add site-specific actions to a language definition file. This can be done by creating a file with the same name as the language definition file, but with the extension `.cfg`. For instance the file `norsk.cfg` will be loaded when the language definition file `norsk.ldf` is loaded.

For plain-based formats we don't want to override the definition of `\loadlocalcfg` from `plain.def`.

```

4312      \bbl@trace{Local Language Configuration}
4313      \ifx\loadlocalcfg\undefined
4314      \ifpackagewith{babel}{noconfigs}%
4315      {\let\loadlocalcfg\@gobble}%
4316      {\def\loadlocalcfg#1{%
4317      \InputIfFileExists{#1.cfg}%
4318      {\typeout{*****^J%
4319      * Local config file #1.cfg used^^J%
4320      *}}%
4321      \@empty}}
4322      \fi

```

5.8. Language options

Languages are loaded when processing the corresponding option *except* if a main language has been set. In such a case, it is not loaded until all options has been processed. The following macro inputs the ldf file and does some additional checks (\input works, too, but possible errors are not caught).

```
4323 \bbl@trace{Language options}
4324 \def\BabelDefinitionFile#1#2#3{}
4325 \let\bbl@afterlang\relax
4326 \let\BabelModifiers\relax
4327 \let\bbl@loaded\@empty
4328 \def\bbl@load@language#1{%
4329   \InputIfFileExists{#1.ldf}%
4330   {\edef\bbl@loaded{\CurrentOption
4331     \ifx\bbl@loaded\@empty\else,\bbl@loaded\fi}%
4332     \expandafter\let\expandafter\bbl@afterlang
4333     \csname\CurrentOption.ldf-h@k\endcsname
4334     \expandafter\let\expandafter\BabelModifiers
4335     \csname\bbl@mod@\CurrentOption\endcsname
4336     \bbl@exp{\AtBeginDocument{%
4337       \bbl@usehooks@lang{\CurrentOption}{\begin{document}}{\CurrentOption}}}%
4338     {\bbl@error{unknown-package-option}}{}}{}}
```

Another way to extend the list of ‘known’ options for babel was to create the file `bblopts.cfg` in which one can add option declarations. However, this mechanism is deprecated – if you want an alternative name for a language, just create a new ldf file loading the actual one. You can also set the name of the file with the package option `config=<name>`, which will load `<name>.cfg` instead.

If the language has been set as metadata, read the info from the corresponding ini file and extract the babel name. Then added it as a package option at the end, so that it becomes the main language. The behavior of a metatag with a global language option is not well defined, so if there is not a main option we set here explicitly.

Tagging PDF Span elements requires horizontal mode. With DocumentMetadata we also force it with `\foreignlanguage` (this is also done in bidi texts).

```
4339 \ifx\GetDocumentProperties\undefined\else
4340   \let\bbl@beforeforeign\leavevmode
4341   \edef\bbl@tempa{\GetDocumentProperties{document/other-languages}}%
4342   \let\bbl@collect@other\@empty
4343   \bbl@foreach\bbl@tempa{%
4344     \edef\bbl@metalang{#1}%
4345     \ifx\bbl@metalang\@empty\else
4346       \begin{group}
4347         \expandafter
4348         \bbl@bcpllookup{\bbl@metalang}%
4349         \ifx\bbl@bcp\relax
4350           \ifx\bbl@opt@main\@nnil
4351             \bbl@error{no-locale-for-meta}{\bbl@metalang}{}%
4352           \fi
4353         \else
4354           \bbl@read@ini{\bbl@bcp}\m@ne
4355           \xdef\bbl@collect@other{\bbl@collect@other,\language\name}%
4356           \GenericInfo{(babel) \spaces\spaces\spaces
4357             Passing '\language' to babel\@gobble}%
4358         \fi
4359       \end{group}
4360     \fi}
4361   \ifx\bbl@collect@other\@empty\else
4362     \xdef\bbl@language@opts{\bbl@collect@other,\bbl@language@opts}%
4363   \fi
4364   \edef\bbl@metalang{\GetDocumentProperties{document/language}}%
4365   \ifx\bbl@metalang\@empty\else
4366     \begin{group}
4367       \expandafter
4368       \bbl@bcpllookup{\bbl@metalang}%
4369       \ifx\bbl@bcp\relax
```



```

4370     \ifx\bbl@opt@main\@nnil
4371         \bbl@error{no-locale-for-meta}{\bbl@metalang}{}}}%
4372     \fi
4373     \else
4374         \bbl@read@ini{\bbl@bcp}\m@ne
4375         \xdef\bbl@language@opts{\bbl@language@opts,\language}%
4376         \ifx\bbl@opt@main\@nnil
4377             \global\let\bbl@opt@main\language
4378         \fi
4379         \GenericInfo{{(babel) \@spaces\@spaces\@spaces
4380             Passing '\language' to babel\@gobble}}}%
4381     \fi
4382 \endgroup
4383 \fi
4384 \fi
4385 \ifx\bbl@opt@config\@nnil
4386     \@ifpackagewith{babel}{noconfigs}}}%
4387     {\InputIfFileExists{bblopts.cfg}%
4388         {\bbl@info{Configuration files are deprecated, as\\%
4389             they can break document portability.\\%
4390             Reported}%
4391             \typeout{*****^J%
4392                 * Local config file bblopts.cfg used^^J%
4393                 *}}}%
4394     }{}%
4395 \else
4396     \InputIfFileExists{\bbl@opt@config.cfg}%
4397     {\bbl@info{Configuration files are deprecated, as\\%
4398         they can break document portability.\\%
4399         Reported}%
4400         \typeout{*****^J%
4401             * Local config file \bbl@opt@config.cfg used^^J%
4402             *}}}%
4403     {\bbl@error{config-not-found}}{}%
4404 \fi

```

Recognizing global options in packages not having a closed set of them is not trivial, as for them to be processed they must be defined explicitly. So, package options not yet taken into account and stored in `\bbl@language@opts` are assumed to be languages. If not declared above, the names of the option and the file are the same. We first pre-process the class and package options to determine the available locales, and which version (l`df` or i`ni`) will be loaded. This is done by first loading the corresponding `babel-⟨name⟩.tex` file.

The second argument of `\BabelBeforeIni` may contain a `\BabelDefinitionFile` which defines `\bbl@tempa` and `\bbl@tempb` and saves the third argument for the moment of the actual loading. If there is no `\BabelDefinitionFile` the last element is usually empty, and the `ini` file is loaded. The values are used to build a list in the form ‘main-or-not’ / ‘l`df`-or-l`dfini`-flag’ // ‘option-name’ // ‘bcp-tag’ / ‘l`df`-name-or-none’. The ‘main-or-not’ element is 0 by default and set to 10 later if necessary (by prepending 1). The ‘bcp-tag’ is stored here so that the corresponding `ini` file can be loaded directly (with `@import`).

```

4405 \def\BabelBeforeIni#1#2{%
4406     \def\bbl@tempa{\@m}% <- Default if no \BDefFile
4407     \let\bbl@tempb\@empty
4408     #2%
4409     \edef\bbl@toload{%
4410         \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4411         \bbl@toload@last}%
4412     \edef\bbl@toload@last{0/\bbl@tempa//\CurrentOption//\#1/\bbl@tempb}}
4413 \def\BabelDefinitionFile#1#2#3{%
4414     \def\bbl@tempa{#1}\def\bbl@tempb{#2}%
4415     \@namedef{bbl@preldf@CurrentOption}{#3}%
4416     \endinput}%

```

For efficiency, first preprocess the class options to remove those with `=`, which are becoming increasingly frequent (no language should contain this character). Here we use the more robust

macro to traverse a clist from the $\TeX$ 3 layer.

```

4417 \def\bbl@tempf{,}
4418 \@nameuse{clist_map_inline:Nn}\@raw@classoptionslist{%
4419   \in@{=}{#1}%
4420   \ifin@else
4421     \edef\bbl@tempf{\bbl@tempf\zap@space#1 \@empty,}%
4422   \fi}

```

Store the class/package options in a list. If there is an explicit main, it's placed as the last option. Then loop it to read the tex files, which can have a `\BabelDefinitionFile`. If there is no tex file, we attempt loading the ldf for the option name; if it fails, an error is raised. Note the option name is surrounded by `//...//`. Class and package options are separated with `@`, because errors and info are dealt with in different ways. Consecutive identical languages count as one.

```

4423 \let\bbl@toload\@empty
4424 \let\bbl@toload@last\@empty
4425 \let\bbl@unkopt\@gobble %% <- Ugly
4426 \edef\bbl@tempc{%
4427   \bbl@tempf,@,\bbl@language@opts
4428   \ifx\bbl@opt@main\@nnil\else,\bbl@opt@main\fi}
4429 \let\BabelLocalesTentative\bbl@tempc
4430 %
4431 \bbl@foreach\bbl@tempc{%
4432   \in@{@@}{#1}% <- Ugly
4433   \ifin@
4434     \def\bbl@unkopt##1{%
4435       \DeclareOption{##1}{\bbl@error{unknown-package-option}}{}}}%
4436   \else
4437     \def\CurrentOption{#1}%
4438     \bbl@xin@{//#1//}{\bbl@toload@last}% Collapse consecutive
4439     \ifin@else
4440     \lowercase{\InputIfFileExists{babel-#1.tex}}{ }{%
4441       \IfFileExists{#1.ldf}%
4442       {\edef\bbl@toload{%
4443         \ifx\bbl@toload\@empty\else\bbl@toload,\fi
4444         \bbl@toload@last}%
4445         \edef\bbl@toload@last{0/0//\CurrentOption//und/#1}}%
4446       {\bbl@unkopt{#1}}}%
4447     \fi
4448   \fi}

```

We have to determine (1) if no language has been loaded (in which case we fallback to 'nil', with a special tag), and (2) the main language. With an explicit 'main' language, remove repeated elements. The number 1 flags it as the main language (relevant in *ini* locales), because with 0 becomes 10.

```

4449 \ifx\bbl@opt@main\@nnil
4450   \ifx\bbl@toload@last\@empty
4451     \def\bbl@toload@last{0/0//nil//und-x-nil/nil}
4452     \bbl@info{%
4453       You haven't specified a language as a class or package\%
4454       option. I'll load 'nil'. Reported}
4455   \fi
4456 \else
4457   \let\bbl@tempc\@empty
4458   \bbl@foreach\bbl@toload{%
4459     \bbl@xin@{//#1//}{\bbl@opt@main//}{#1}%
4460     \ifin@else
4461       \bbl@add@list\bbl@tempc{#1}%
4462     \fi}
4463   \let\bbl@toload\bbl@tempc
4464 \fi
4465 \edef\bbl@toload{\bbl@toload,1\bbl@toload@last}

```

Finally, load the 'ini' file or the pair 'ini'/'ldf' file. Babel resorts to its own mechanism, not the default one based on `\ProcessOptions` (which is still present to make some internal clean-up). First,

handle provide=! and friends (with a recursive call if they are present), and then provide=* and friend. \count@ is used as flag: 0 if 'ini', 1 if 'ldf'.

```

4466 \def\AfterBabelLanguage#1{%
4467   \bbl@ifsamestring\CurrentOption{#1}{\global\bbl@add\bbl@afterlang{}}%
4468 \NewHook{babel/presets}
4469 \UseHook{babel/presets}
4470 %
4471 \let\bbl@tempb\empty
4472 \def\bbl@tempc#1/#2/#3/#4/#5\@@{%
4473   \count@z@
4474   \ifnum#2=\@m % if no \BabelDefinitionFile
4475     \ifnum#1=z@ % not main. -- % if provide+=!, provide*=!
4476       \ifnum\bbl@ldfflag>\@ne\bbl@tempc 0/0/#3/#4/#3\@@
4477       \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4478       \fi
4479     \else % 10 = main -- % if provide=!, provide*=!
4480       \ifodd\bbl@ldfflag\bbl@tempc 10/0/#3/#4/#3\@@
4481       \else\bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4482       \fi
4483     \fi
4484   \else
4485     \ifnum#1=z@ % not main
4486       \ifnum\bbl@iniflag>\@ne\else % if 0, provide
4487         \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4488         \fi
4489       \else % 10 = main
4490         \ifodd\bbl@iniflag\else % if provide+, provide*
4491           \ifcase#2\count@\@ne\else\ifcase\bbl@engine\count@\@ne\fi\fi
4492           \fi
4493         \fi
4494         \bbl@tempd{#1}{#2}{#3}{#4}{#5}%
4495       \fi}

```

Based on the value of \count@, do the actual loading. If 'ldf', we load the basic info from the 'ini' file before.

```

4496 \def\bbl@tempd#1#2#3#4#5{%
4497   \DeclareOption{#3}{}%
4498   \ifcase\count@
4499     \bbl@exp{\bbl@add\bbl@tempb{%
4500       \global\let\bbl@afterload\empty
4501       \bbl@nameuse\bbl@preini{#3}%
4502       \bbl@ldfinit
4503       \def\CurrentOption{#3}%
4504       \bbl@babelprovide[@import=#4,\ifnum#1=z@\else\bbl@opt@provide,main\fi]{#3}%
4505       \bbl@afterldf
4506       \bbl@afterload}}%
4507   \else
4508     \bbl@add\bbl@tempb{%
4509       \global\let\bbl@afterload\empty
4510       \def\CurrentOption{#3}%
4511       \let\localename\CurrentOption
4512       \let\language\localename
4513       \def\BabelIniTag{#4}%
4514       \bbl@nameuse\bbl@preldf{#3}%
4515       \begingroup
4516         \bbl@id@assign
4517         \bbl@read@ini{\BabelIniTag}0%
4518       \endgroup
4519       \bbl@load@language{#5}%
4520       \bbl@afterload}%
4521   \fi}
4522 %
4523 \bbl@foreach\bbl@to load{\bbl@tempc#1\@@}

```

```

4524 \bbl@tempb
4525 \DeclareOption*{}
4526 \ProcessOptions
4527 %
4528 \bbl@exp{%
4529   \\AtBeginDocument{\\bbl@usehooks@lang{/}{begindocument}}{}}}%
4530 \def\AfterBabelLanguage{\bbl@error{late-after-babel}}{}{}
4531 \AddToHook{begindocument/end}{%
4532   \bbl@info{Main language set to '\mainlocalename'\@gobble}}
4533 (/package)

```

6. The kernel of Babel

The kernel of the babel system is currently stored in `babel.def`. The file `babel.def` contains most of the code. The file `hyphen.cfg` is a file that can be loaded into the format, which is necessary when you want to be able to switch hyphenation patterns.

Because plain $\TeX$ users might want to use some of the features of the babel system too, care has to be taken that plain $\TeX$ can process the files. For this reason the current format will have to be checked in a number of places. Some of the code below is common to plain $\TeX$ and $\LaTeX$, some of it is for the $\LaTeX$ case only.

Plain formats based on `etex` (`etex`, `xetex`, `luatex`) don't load `hyphen.cfg` but `etex.src`, which follows a different naming convention, so we need to define the babel names. It presumes `language.def` exists and it is the same file used when formats were created.

A proxy file for `switch.def`

```

4534 (*kernel)
4535 \let\bbl@onlyswitch\@empty
4536 \input babel.def
4537 \let\bbl@onlyswitch\@undefined
4538 (/kernel)

```

7. Error messages

They are loaded when `\bbl@error` is first called. To save space, the main code just identifies them with a tag, and messages are stored in a separate file. Since it can be loaded anywhere, you make sure some catcodes have the right value, although those for `\`, ```, `^M`, `%` and `=` are reset before loading the file.

```

4539 (*errors)
4540 \catcode`\{=1 \catcode`\}=2 \catcode`\#=6
4541 \catcode`\:=12 \catcode`\,=12 \catcode`\.=12 \catcode`\-=12
4542 \catcode`\'=12 \catcode`\(=12 \catcode`\)=12
4543 \catcode`\@=11 \catcode`\^=7
4544 %
4545 \ifx\MessageBreak\@undefined
4546   \gdef\bbl@error@i#1#2{%
4547     \begingroup
4548       \newlinechar=`^^J
4549       \def\{^^J(babel) }%
4550       \errhelp{#2}\errmessage{\{#1}%
4551     \endgroup}
4552 \else
4553   \gdef\bbl@error@i#1#2{%
4554     \begingroup
4555       \def\{\MessageBreak}%
4556       \PackageError{babel}{#1}{#2}%
4557     \endgroup}
4558 \fi
4559 \def\bbl@errmessage#1#2#3{%
4600   \expandafter\gdef\csname bbl@err@#1\endcsname##1##2##3{%
4601     \bbl@error@i{#2}{#3}}
4602 % Implicit #2#3#4:
4603 \gdef\bbl@error#1{\csname bbl@err@#1\endcsname}

```

```

4564 %
4565 \bbl@errmessage{not-yet-available}
4566     {Not yet available}%
4567     {Find an armchair, sit down and wait}
4568 \bbl@errmessage{bad-package-option}%
4569     {Bad option '#1=#2'. Either you have misspelled the\\%
4570     key or there is a previous setting of '#1'. Valid\\%
4571     keys are, among others, 'shorthands', 'main', 'bidi',\\%
4572     'strings', 'config', 'headfoot', 'safe', 'math'.}%
4573     {See the manual for further details.}
4574 \bbl@errmessage{base-on-the-fly}
4575     {For a language to be defined on the fly 'base'\\%
4576     is not enough, and the whole package must be\\%
4577     loaded. Either delete the 'base' option or\\%
4578     request the languages explicitly}%
4579     {See the manual for further details.}
4580 \bbl@errmessage{undefined-language}
4581     {You haven't defined the language '#1' yet.\\%
4582     Perhaps you misspelled it or your installation\\%
4583     is not complete}%
4584     {Your command will be ignored, type <return> to proceed}
4585 \bbl@errmessage{invalid-ini-name}
4586     {'#1' not valid with the 'ini' mechanism.\\%
4587     I think you want '#2' instead. You may continue,\\%
4588     but you should fix the name. See the babel manual\\%
4589     for the available locales with 'provide'}%
4590     {See the manual for further details.}
4591 \bbl@errmessage{shorthand-is-off}
4592     {I can't declare a shorthand turned off (\string#2)}
4593     {Sorry, but you can't use shorthands which have been\\%
4594     turned off in the package options}
4595 \bbl@errmessage{not-a-shorthand}
4596     {The character '\string #1' should be made a shorthand character;\\%
4597     add the command \string\usesshorthands\string{#1\string} to
4598     the preamble.\\%
4599     I will ignore your instruction}%
4600     {You may proceed, but expect unexpected results}
4601 \bbl@errmessage{not-a-shorthand-b}
4602     {I can't switch '\string#2' on or off--not a shorthand\\%
4603     This character is not a shorthand. Maybe you made\\%
4604     a typing mistake?}%
4605     {I will ignore your instruction.}
4606 \bbl@errmessage{unknown-attribute}
4607     {The attribute #2 is unknown for language #1.}%
4608     {Your command will be ignored, type <return> to proceed}
4609 \bbl@errmessage{missing-group}
4610     {Missing group for string \string#1}%
4611     {You must assign strings to some category, typically\\%
4612     captions or extras, but you set none}
4613 \bbl@errmessage{only-lua-xe}
4614     {This macro is available only in LuaLaTeX and XeLaTeX.}%
4615     {Consider switching to these engines.}
4616 \bbl@errmessage{only-lua}
4617     {This macro is available only in LuaLaTeX}%
4618     {Consider switching to that engine.}
4619 \bbl@errmessage{unknown-provide-key}
4620     {Unknown key '#1' in \string\babelprovide}%
4621     {See the manual for valid keys}%
4622 \bbl@errmessage{unknown-mapfont}
4623     {Option '\bbl@KVP@mapfont' unknown for\\%
4624     mapfont. Use 'direction'}%
4625     {See the manual for details.}
4626 \bbl@errmessage{no-ini-file}

```

```

4627 {There is no ini file for the requested language\\%
4628 (#1: \language\name). Perhaps you misspelled it or your\\%
4629 installation is not complete}%
4630 {Fix the name or reinstall babel.}
4631 \bbl@errmessage{digits-is-reserved}
4632 {The counter name 'digits' is reserved for mapping\\%
4633 decimal digits}%
4634 {Use another name.}
4635 \bbl@errmessage{limit-two-digits}
4636 {Currently two-digit years are restricted to the\\5
4637 range 0-9999}%
4638 {There is little you can do. Sorry.}
4639 \bbl@errmessage{counter-too-large}
4640 {Counter too large (#1)}%
4641 {Currently this is the limit.}
4642 \bbl@errmessage{no-ini-info}
4643 {I've found no info for the current locale.\\%
4644 The corresponding ini file has not been loaded\\%
4645 Perhaps it doesn't exist}%
4646 {See the manual for details.}
4647 \bbl@errmessage{unknown-ini-field}
4648 {Unknown field '#1' in \string\BCPdata.\\%
4649 Perhaps you misspelled it}%
4650 {See the manual for details.}
4651 \bbl@errmessage{unknown-locale-key}
4652 {Unknown key for locale '#2':\\%
4653 #3\\%
4654 \string#1 will be set to \string\relax}%
4655 {Perhaps you misspelled it.}%
4656 \bbl@errmessage{adjust-only-vertical}
4657 {Currently, #1 related features can be adjusted only\\%
4658 in the main vertical list}%
4659 {Maybe things change in the future, but this is what it is.}
4660 \bbl@errmessage{layout-only-vertical}
4661 {Currently, layout related features can be adjusted only\\%
4662 in vertical mode}%
4663 {Maybe things change in the future, but this is what it is.}
4664 \bbl@errmessage{bidi-only-lua}
4665 {The bidi method 'basic' is available only in\\%
4666 luatex. I'll continue with 'bidi=default', so\\%
4667 expect wrong results.\\%
4668 Suggested actions:\\%
4669 * If possible, switch to luatex, as xetex is not\\%
4670 recommend anymore.\\
4671 * If you can't, try 'bidi=bidi' with xetex.\\%
4672 * With pdftex, only 'bidi=default' is available.}%
4673 {See the manual for further details.}
4674 \bbl@errmessage{multiple-bidi}
4675 {Multiple bidi settings inside a group\\%
4676 I'll insert a new group, but expect wrong results.\\%
4677 Suggested action:\\%
4678 * Add a new group where appropriate.}
4679 {See the manual for further details.}
4680 \bbl@errmessage{unknown-package-option}
4681 {Unknown option '\CurrentOption'.\\%
4682 Suggested actions:\\%
4683 * Make sure you haven't misspelled it\\%
4684 * Check in the babel manual that it's supported\\%
4685 * If supported and it's a language, you may\\%
4686 \space\space need in some distributions a separate\\%
4687 \space\space installation\\%
4688 * If installed, check there isn't an old\\%
4689 \space\space version of the required files in your system\\%

```

```

4690     * If it's an unsupported language, create it with\\%
4691     \string\babelprovide (see the manual)}
4692     {Valid options are, among others: shorthands=, KeepShorthandsActive,\\%
4693     activeacute, activegrave, noconfigs, safe=, main=, math=\\%
4694     headfoot=, strings=, config=, hyphenmap=, or a language name.}
4695 \bbl@errmessage{config-not-found}
4696     {Local config file '\bbl@opt@config.cfg' not found.\\%
4697     Suggested actions:\\%
4698     * Make sure you haven't misspelled it in config=\\%
4699     * Check it exists and it's in the correct path}%
4700     {Perhaps you misspelled it.}
4701 \bbl@errmessage{late-after-babel}
4702     {Too late for \string\AfterBabelLanguage}%
4703     {Languages have been loaded, so I can do nothing}
4704 \bbl@errmessage{double-hyphens-class}
4705     {Double hyphens aren't allowed in \string\babelcharclass\\%
4706     because it's potentially ambiguous}%
4707     {See the manual for further info}
4708 \bbl@errmessage{unknown-interchar}
4709     {'#1' for '\language' cannot be enabled.\\%
4710     Maybe there is a typo}%
4711     {See the manual for further details.}
4712 \bbl@errmessage{unknown-interchar-b}
4713     {'#1' for '\language' cannot be disabled.\\%
4714     Maybe there is a typo}%
4715     {See the manual for further details.}
4716 \bbl@errmessage{charproperty-only-vertical}
4717     {\string\babelcharproperty\space can be used only in\\%
4718     vertical mode (preamble or between paragraphs)}%
4719     {See the manual for further info}
4720 \bbl@errmessage{unknown-char-property}
4721     {No property named '#2'. Allowed values are\\%
4722     direction (bc), mirror (bmg), and linebreak (lb)}%
4723     {See the manual for further info}
4724 \bbl@errmessage{bad-transform-option}
4725     {Bad option '#1' in a transform.\\%
4726     I'll ignore it but expect more errors}%
4727     {See the manual for further info.}
4728 \bbl@errmessage{font-conflict-transforms}
4729     {Transforms cannot be re-assigned to different\\%
4730     fonts. The conflict is in '\bbl@kv@label'.\\%
4731     Apply the same fonts or use a different label}%
4732     {See the manual for further details.}
4733 \bbl@errmessage{transform-not-available}
4734     {'#1' for '\language' cannot be enabled.\\%
4735     Maybe there is a typo or it's a font-dependent transform}%
4736     {See the manual for further details.}
4737 \bbl@errmessage{transform-not-available-b}
4738     {'#1' for '\language' cannot be disabled.\\%
4739     Maybe there is a typo or it's a font-dependent transform}%
4740     {See the manual for further details.}
4741 \bbl@errmessage{year-out-range}
4742     {Year out of range.\\%
4743     The allowed range is #1}%
4744     {See the manual for further details.}
4745 \bbl@errmessage{only-pdfTeX-lang}
4746     {The '#1' ldf style doesn't work with #2,\\%
4747     but you can use the ini locale instead.\\%
4748     Try adding 'provide=' to the option list. You may\\%
4749     also want to set 'bidi=' to some value}%
4750     {See the manual for further details.}
4751 \bbl@errmessage{hyphenmins-args}
4752     {\string\babelhyphenmins\ accepts either the optional\\%

```

```

4753 argument or the star, but not both at the same time}%
4754 {See the manual for further details.}
4755 \bbl@errmessage{no-locale-for-meta}
4756 {There isn't currently a locale for the 'lang' requested\\%
4757 in the PDF metadata ('#1'). To fix it, you can\\%
4758 set explicitly a similar language (using the same\\%
4759 script) with the key main= when loading babel. If you\\%
4760 continue, I'll fallback to the 'nil' language, with\\%
4761 tag 'und' and script 'Latn', but expect a bad font\\%
4762 rendering with other scripts. You may also need set\\%
4763 explicitly captions and date, too}%
4764 {See the manual for further details.}
4765 (/errors)
4766 (*patterns)

```

8. Loading hyphenation patterns

The following code is meant to be read by $\text{\texttt{iniT\TeX}}$ because it should instruct $\text{\texttt{T\TeX}}$ to read hyphenation patterns. To this end the `docstrip` option `patterns` is used to include this code in the file `hyphen.cfg`. Code is written with lower level macros.

```

4767 <@Make sure ProvidesFile is defined@>
4768 \ProvidesFile{hyphen.cfg}[<@date@> v<@version@> Babel hyphens]
4769 \xdef\bbl@format{\jobname}
4770 \def\bbl@version{<@version@>}
4771 \def\bbl@date{<@date@>}
4772 \ifx\AtBeginDocument\@undefined
4773 \def\@empty{}
4774 \fi
4775 <@Define core switching macros@>

```

\process@line Each line in the file `language.dat` is processed by `\process@line` after it is read. The first thing this macro does is to check whether the line starts with `=`. When the first token of a line is an `=`, the macro `\process@synonym` is called; otherwise the macro `\process@language` will continue.

```

4776 \def\process@line#1#2 #3 #4 {%
4777 \ifx=#1%
4778 \process@synonym{#2}%
4779 \else
4780 \process@language{#1#2}{#3}{#4}%
4781 \fi
4782 \ignorespaces}

```

\process@synonym This macro takes care of the lines which start with an `=`. It needs an empty token register to begin with. `\bbl@languages` is also set to empty.

```

4783 \toks@{}
4784 \def\bbl@languages{}

```

When no languages have been loaded yet, the name following the `=` will be a synonym for hyphenation register 0. So, it is stored in a token register and executed when the first pattern file has been processed. (The `\relax` just helps to the `\if` below catching synonyms without a language.)

Otherwise the name will be a synonym for the language loaded last.

We also need to copy the `hyphenmins` parameters for the synonym.

```

4785 \def\process@synonym#1{%
4786 \ifnum\last@language=\m@ne
4787 \toks@\expandafter{\the\toks@\relax\process@synonym{#1}}%
4788 \else
4789 \expandafter\chardef\csname l@#1\endcsname\last@language
4790 \wlog{\string\l@#1=\string\language\the\last@language}%
4791 \expandafter\let\csname #1hyphenmins\expandafter\endcsname
4792 \csname\language\hyphenmins\endcsname
4793 \let\bbl@elt\relax
4794 \edef\bbl@languages{\bbl@languages\bbl@elt{#1}{\the\last@language}}}%

```


\process@language The macro \process@language is used to process a non-empty line from the ‘configuration file’. It has three arguments, each delimited by white space. The first argument is the ‘name’ of a language; the second is the name of the file that contains the patterns. The optional third argument is the name of a file containing hyphenation exceptions.

The first thing to do is call \addlanguage to allocate a pattern register and to make that register ‘active’. Then the pattern file is read.

For some hyphenation patterns it is needed to load them with a specific font encoding selected. This can be specified in the file language.dat by adding for instance ‘:T1’ to the name of the language. The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. The latter can be used in hyphenation files if you need to set a behavior depending on the given encoding (it is set to empty if no encoding is given).

Pattern files may contain assignments to \lefthyphenmin and \righthyphenmin. T_EX does not keep track of these assignments. Therefore we try to detect such assignments and store them in the \<language>hyphenmins macro. When no assignments were made we provide a default setting.

Some pattern files contain changes to the \lccode en \uccode arrays. Such changes should remain local to the language; therefore we process the pattern file in a group; the \patterns command acts globally so its effect will be remembered.

Then we globally store the settings of \lefthyphenmin and \righthyphenmin and close the group. When the hyphenation patterns have been processed we need to see if a file with hyphenation exceptions needs to be read. This is the case when the third argument is not empty and when it does not contain a space token. (Note however there is no need to save hyphenation exceptions into the format.)

\bbl@languages saves a snapshot of the loaded languages in the form \bbl@elt{<language-name>}{<number>}{<patterns-file>}{<exceptions-file>}. Note the last 2 arguments are empty in ‘dialects’ defined in language.dat with =. Note also the language name can have encoding info.

Finally, if the counter \language is equal to zero we execute the synonyms stored.

```

4796 \def\process@language#1#2#3{%
4797   \expandafter\addlanguage\csname l@#1\endcsname
4798   \expandafter\language\csname l@#1\endcsname
4799   \edef\language#1#2#3{%
4800     \bbl@hook@everylanguage{#1}%
4801     % > luatex
4802     \bbl@get@enc#1::\@@@
4803     \begingroup
4804       \lefthyphenmin\m@ne
4805       \bbl@hook@loadpatterns{#2}%
4806       % > luatex
4807       \ifnum\lefthyphenmin=\m@ne
4808         \else
4809           \expandafter\xdef\csname #1hyphenmins\endcsname{%
4810             \the\lefthyphenmin\the\righthyphenmin}%
4811         \fi
4812       \endgroup
4813     \def\bbl@tempa{#3}%
4814     \ifx\bbl@tempa\@empty\else
4815       \bbl@hook@loadexceptions{#3}%
4816       % > luatex
4817     \fi
4818     \let\bbl@elt\relax
4819     \edef\bbl@languages{%
4820       \bbl@languages\bbl@elt{#1}{\the\language}{#2}{\bbl@tempa}}%
4821     \ifnum\the\language=z@
4822       \expandafter\ifx\csname #1hyphenmins\endcsname\relax
4823         \set@hyphenmins\tw@\thr@{\relax
4824         \else
4825           \expandafter\expandafter\expandafter\set@hyphenmins
4826           \csname #1hyphenmins\endcsname
4827         \fi
4828         \the\toks@

```

```

4829 \toks@{}%
4830 \fi}

```

\bbl@get@enc

\bbl@hyph@enc The macro \bbl@get@enc extracts the font encoding from the language name and stores it in \bbl@hyph@enc. It uses delimited arguments to achieve this.

```

4831 \def\bbl@get@enc#1:#2:#3\@@@{\def\bbl@hyph@enc{#2}}

```

Now, hooks are defined. For efficiency reasons, they are dealt here in a special way. Besides luatex, format-specific configuration files are taken into account. loadkernel currently loads nothing, but define some basic macros instead.

```

4832 \def\bbl@hook@everylanguage#1{}
4833 \def\bbl@hook@loadpatterns#1{\input #1\relax}
4834 \let\bbl@hook@loadexceptions\bbl@hook@loadpatterns
4835 \def\bbl@hook@loadkernel#1{%
4836   \def\addlanguage{\csname newlanguage\endcsname}%
4837   \def\adddialect##1##2{%
4838     \global\chardef##1##2\relax
4839     \wlog{\string##1 = a dialect from \string\language##2}}%
4840   \def\iflanguage##1{%
4841     \expandafter\ifx\csname l@##1\endcsname\relax
4842       \@nolanerr{##1}%
4843     \else
4844       \ifnum\csname l@##1\endcsname=\language
4845         \expandafter\expandafter\expandafter\@firstoftwo
4846       \else
4847         \expandafter\expandafter\expandafter\@secondoftwo
4848       \fi
4849     \fi}%
4850   \def\providehyphenmins##1##2{%
4851     \expandafter\ifx\csname ##1hyphenmins\endcsname\relax
4852       \@namedef{##1hyphenmins}{##2}%
4853     \fi}%
4854   \def\set@hyphenmins##1##2{%
4855     \lefthyphenmin##1\relax
4856     \righthyphenmin##2\relax}%
4857   \def\selectlanguage{%
4858     \errhelp{Selecting a language requires a package supporting it}%
4859     \errmessage{No multilingual package has been loaded}}%
4860   \let\foreignlanguage\selectlanguage
4861   \let\otherlanguage\selectlanguage
4862   \expandafter\let\csname otherlanguage*\endcsname\selectlanguage
4863   \def\bbl@usehooks##1##2{%
4864     \def\setlocale{%
4865       \errhelp{Find an armchair, sit down and wait}%
4866       \errmessage{(babel) Not yet available}}%
4867     \let\uselocale\setlocale
4868     \let\locale\setlocale
4869     \let\selectlocale\setlocale
4870     \let\localename\setlocale
4871     \let\textlocale\setlocale
4872     \let\textlanguage\setlocale
4873     \let\languagetext\setlocale}
4874   \begingroup
4875     \def\AddBabelHook#1#2{%
4876       \expandafter\ifx\csname bbl@hook@#2\endcsname\relax
4877         \def\next{\toks1}%
4878       \else
4879         \def\next{\expandafter\gdef\csname bbl@hook@#2\endcsname###1}%
4880       \fi
4881       \next}
4882   \ifx\directlua\@undefined
4883     \ifx\XeTeXinputencoding\@undefined\else

```

```

4884     \input xebabel.def
4885     \fi
4886 \else
4887     \input luababel.def
4888     \fi
4889 \openin1 = babel-\bbl@format.cfg
4890 \ifeof1
4891 \else
4892     \input babel-\bbl@format.cfg\relax
4893     \fi
4894 \closein1
4895 \endgroup
4896 \bbl@hook@loadkernel{switch.def}

```

\readconfigfile The configuration file can now be opened for reading.

```

4897 \openin1 = language.dat

```

See if the file exists, if not, use the default hyphenation file `hyphen.tex`. The user will be informed about this.

```

4898 \def\language{english}%
4899 \ifeof1
4900 \message{I couldn't find the file language.dat,\space
4901         I will try the file hyphen.tex}
4902 \input hyphen.tex\relax
4903 \chardef\l@english\z@
4904 \else

```

Pattern registers are allocated using count register `\last@language`. Its initial value is 0. The definition of the macro `\newlanguage` is such that it first increments the count register and then defines the language. In order to have the first patterns loaded in pattern register number 0 we initialize `\last@language` with the value `-1`.

```

4905 \last@language\m@ne

```

We now read lines from the file until the end is found. While reading from the input, it is useful to switch off recognition of the end-of-line character. This saves us stripping off spaces from the contents of the control sequence.

```

4906 \loop
4907   \endlinechar\m@ne
4908   \read1 to \bbl@line
4909   \endlinechar\^M

```

If the file has reached its end, exit from the loop here. If not, empty lines are skipped. Add 3 space characters to the end of `\bbl@line`. This is needed to be able to recognize the arguments of `\process@line` later on. The default language should be the very first one.

```

4910   \if T\ifeof1F\fi T\relax
4911   \ifx\bbl@line\@empty\else
4912     \edef\bbl@line{\bbl@line\space\space\space}%
4913     \expandafter\process@line\bbl@line\relax
4914   \fi
4915 \repeat

```

Check for the end of the file. We must reverse the test for `\ifeof` without `\else`. Then reactivate the default patterns, and close the configuration file.

```

4916 \begingroup
4917 \def\bbl@elt#1#2#3#4{%
4918   \global\language=#2\relax
4919   \gdef\language{#1}%
4920   \def\bbl@elt##1##2##3##4{}}%
4921 \bbl@languages
4922 \endgroup
4923 \fi
4924 \closein1

```

We add a message about the fact that babel is loaded in the format and with which language patterns to the `\everyjob` register.

```
4925 \if/\the\toks@\else
4926 \errhelp{language.dat loads no language, only synonyms}
4927 \errmessage{Orphan language synonym}
4928 \fi
```

Also remove some macros from memory and raise an error if `\toks@` is not empty. Finally load `switch.def`, but the latter is not required and the line inputting it may be commented out.

```
4929 \let\bbl@line\@undefined
4930 \let\process@line\@undefined
4931 \let\process@synonym\@undefined
4932 \let\process@language\@undefined
4933 \let\bbl@get@enc\@undefined
4934 \let\bbl@hyph@enc\@undefined
4935 \let\bbl@tempa\@undefined
4936 \let\bbl@hook@loadkernel\@undefined
4937 \let\bbl@hook@everylanguage\@undefined
4938 \let\bbl@hook@loadpatterns\@undefined
4939 \let\bbl@hook@loadexceptions\@undefined
4940 (/patterns)
```

Here the code for `initTeX` ends.

9. luatex + xetex: common stuff

Add the bidi handler just before `luaotfload`, which is loaded by default by LaTeX. Just in case, consider the possibility it has not been loaded. First, a couple of definitions related to bidi (although default also applies to `pdftex`).

```
4941 ((*More package options)) ≡
4942 \chardef\bbl@bidimode\z@
4943 \DeclareOption{bidi=default}{\chardef\bbl@bidimode=\@ne}
4944 \DeclareOption{bidi=basic}{\chardef\bbl@bidimode=101 }
4945 \DeclareOption{bidi=basic-r}{\chardef\bbl@bidimode=102 }
4946 \DeclareOption{bidi=bidi}{\chardef\bbl@bidimode=201 }
4947 \DeclareOption{bidi=bidi-r}{\chardef\bbl@bidimode=202 }
4948 \DeclareOption{bidi=bidi-l}{\chardef\bbl@bidimode=203 }
4949 (/More package options)
```

\babbelfont With explicit languages, we could define the font at once, but we don't. Just wait and see if the language is actually activated. `bbl@font` replaces hardcoded font names inside `\. . family` by the corresponding macro `\. . default`.

```
4950 ((*Font selection)) ≡
4951 \bbl@trace{Font handling with fontspec}
4952 \AddBabelHook{babel-fontspec}{afterextras}{\bbl@switchfont}
4953 \AddBabelHook{babel-fontspec}{beforestart}{\bbl@cckstdfont}
4954 \DisableBabelHook{babel-fontspec}
4955 @onlypreamble\babbelfont
4956 \ifx\NewDocumentCommand\@undefined\else % Not plain
4957 \NewDocumentCommand\babbelfont{0}{m0}{m0}{}%
4958 \bbl@bblfont@o[#1]{#2}[#3,#5]{#4}}
4959 \fi
4960 \newcommand\bbl@bblfont@o[2][]{% 1=langs/scripts 2=fam
4961 \ifx\fontspec\@undefined
4962 \usepackage{fontspec}%
4963 \fi
4964 \EnableBabelHook{babel-fontspec}%
4965 \edef\bbl@tempa{#1}%
4966 \def\bbl@tempb{#2}% Used by \bbl@bblfont
4967 \bbl@bblfont}
4968 \newcommand\bbl@bblfont[2][]{% 1=features 2=fontname, @font=rm|sf|tt
```

```

4969 \bbl@ifunset{\bbl@tempb family}%
4970 {\bbl@providfam{\bbl@tempb}}%
4971 {}%
4972 % For the default font, just in case:
4973 \bbl@ifunset{\bbl@lsys@\language}\bbl@provide@lsys{\language}}{}%
4974 \expandafter\bbl@ifblank\expandafter{\bbl@tempa}%
4975 {\bbl@csarg\edef{\bbl@tempb dflt@}{<#1>#2}}% save bbl@rmdflt@
4976 \bbl@exp{%
4977 \let<\bbl@tempb dflt@\language><\bbl@tempb dflt@>%
4978 \\\bbl@font@set<\bbl@tempb dflt@\language>%
4979 \<\bbl@tempb default>\<\bbl@tempb family>}}%
4980 {\bbl@foreach\bbl@tempa{% i.e., bbl@rmdflt@lang / *scrt
4981 \bbl@csarg\def{\bbl@tempb dflt@##1}{<#1>#2}}}%

```

If the family in the previous command does not exist, it must be defined. Here is how:

```

4982 \def\bbl@providfam#1{%
4983 \bbl@exp{%
4984 \\\newcommand<#1default>{}% Just define it
4985 \\\bbl@add@list\\bbl@font@fams{#1}%
4986 \\\NewHook{#1family}%
4987 \\\DeclareRobustCommand<#1family>{%
4988 \\\not@math@alphabet<#1family>\relax
4989 % \\\prepare@family@series@update{#1}<#1default>% TODO. Fails
4990 \\\fontfamily<#1default>%
4991 \\\UseHook{#1family}%
4992 \\\selectfont}%
4993 \\\DeclareTextFontCommand{\<text#1>}{<#1family>}}%

```

The following macro is activated when the hook babel - fontspec is enabled. But before, we define a macro for a warning, which sets a flag to avoid duplicate them.

```

4994 \def\bbl@nostdfont#1{%
4995 \bbl@once{nostdfam-\f@family}%
4996 {\bbl@infowarn{The current font is not a babel standard family:\\%
4997 #1%
4998 \fontname\font\\%
4999 There is nothing intrinsically wrong, and you can\\%,
5000 ignore this message altogether if you do not need\\%,
5001 this font. If they are used in the document, be aware\\%,
5002 'babel' will not set Script and Language for it, so\\%,
5003 you may consider defining a new family with \string\babelfont.\\%
5004 See the manual for further details about \string\babelfont.
5005 Reported}}%
5006 {}}%
5007 \gdef\bbl@switchfont{%
5008 \bbl@ifunset{\bbl@lsys@\language}\bbl@provide@lsys{\language}}{}%
5009 \bbl@exp{% e.g., Arabic -> arabic
5010 \lowercase{\edef\\bbl@tempa{\bbl@c1{sname}}}}%
5011 \bbl@foreach\bbl@font@fams{%
5012 \bbl@ifunset{\bbl@##1dflt@\language}% (1) language?
5013 {\bbl@ifunset{\bbl@##1dflt*@\bbl@tempa}% (2) from script?
5014 {\bbl@ifunset{\bbl@##1dflt@}% 2=F - (3) from generic?
5015 {}% 123=F - nothing!
5016 {\bbl@exp{% 3=T - from generic
5017 \global\let<\bbl@##1dflt@\language>%
5018 \<\bbl@##1dflt@>}}}%
5019 {\bbl@exp{% 2=T - from script
5020 \global\let<\bbl@##1dflt@\language>%
5021 \<\bbl@##1dflt*@\bbl@tempa>}}}%
5022 {}}% 1=T - language, already defined
5023 \def\bbl@tempa{\bbl@nostdfont{}}%
5024 \bbl@foreach\bbl@font@fams{% don't gather with prev for
5025 \bbl@ifunset{\bbl@##1dflt@\language}%
5026 {\bbl@cs{famrst@##1}%
5027 \global\bbl@csarg\let{famrst@##1}\relax}%

```

```

5028      {\bbl@exp{% order is relevant.
5029          \\bbl@add\\originalTeX{%
5030              \\bbl@font@rst{\bbl@cfl{##1dflt}}}%
5031                  \<##1default>\<##1family>{##1}}%
5032          \\bbl@font@set\<bbl@##1dflt@\language\>% the main part!
5033              \<##1default>\<##1family>}}}%
5034      \bbl@ifrestoring{}\bbl@tempa}%

```

The following is executed at the beginning of the aux file or the document to warn about fonts not defined with `\babelfont`.

```

5035 \ifx\f@family\undefined\else      % if latex
5036 \ifcase\bbl@engine                  % if pdftex
5037     \let\bbl@cckstdfont\relax
5038 \else
5039     \def\bbl@cckstdfont{%
5040         \begingroup
5041         \global\let\bbl@cckstdfont\relax
5042         \let\bbl@tempa\empty
5043         \bbl@foreach\bbl@font@fams{%
5044             \bbl@ifunset{\bbl@##1dflt@}%
5045                 {\nameuse{##1family}}%
5046                 \bbl@csarg\gdef{WFF@\f@family}}}% Flag
5047             \bbl@exp{\\bbl@add\\bbl@tempa{* \<##1family>= \f@family\\}%
5048                 \space\space\fontname\font\\}%
5049             \bbl@csarg\xdef{##1dflt@}{\f@family}}%
5050             \expandafter\xdef\csname ##1default\endcsname{\f@family}}%
5051             {}}%
5052     \ifx\bbl@tempa\empty\else
5053         \bbl@infowarn{The following font families will use the default\\%
5054             settings for all or some languages:\\%
5055             \bbl@tempa
5056             There is nothing intrinsically wrong with it, but\\%
5057             'babel' will no set Script and Language, which could\\%
5058             be relevant in some languages. If your document uses\\%
5059             these families, consider redefining them with \string\babelfont.\\%
5060             Reported}%
5061     \fi
5062 \endgroup
5063 \fi
5064 \fi

```

Now the macros defining the font with `fontspec`.

When there are repeated keys in `fontspec`, the last value wins. So, we just place the ini settings at the beginning, and user settings will take precedence. We must deactivate temporarily `\bbl@mapselect` because `\selectfont` is called internally when a font is defined.

For historical reasons, $\text{\LaTeX}$ can select two different series (bx and b), for what is conceptually a single one. This can lead to problems when a single family requires several fonts, depending on the language, mainly because ‘substitutions’ with some combinations are not done consistently – sometimes `bx/sc` is the correct font, but sometimes points to `b/n`, even if `b/sc` exists. So, some substitutions are redefined (in a somewhat hackish way, by inspecting if the variant declaration contains `>ssub*`).

```

5065 \def\bbl@font@set#1#2#3{% e.g., \bbl@rmdflt@lang \rmdefault \rmfamily
5066     \bbl@xin@{<>}{#1}%
5067     \ifin@
5068         \bbl@exp{\\bbl@fontspec@set\\#1\expandafter\@gobbletwo#1\\#3}%
5069     \fi
5070     \bbl@exp{%
5071         \def\\#2{#1}% e.g., \rmdefault{\bbl@rmdflt@lang}
5072         \\bbl@ifsamestring{#2}{\f@family}%
5073         {\#3
5074             \\bbl@ifsamestring{\f@series}{\bfdefault}{\\bfseries}}}%
5075     \let\\bbl@tempa\relax}%
5076     {}%

```

Loaded locally, which does its job, but very must be global. The problem is how. This actually defines a font predeclared with `\babelfont`, making sure Script and Language names are defined. If they are not, the corresponding data in the ini file is used. The font is actually set temporarily to get the family name (`\f@family`). There is also a hack because by default some replacements related to the bold series are sometimes assigned to the wrong font (see issue #92).

```

5077 \def\bbl@fontspec@set#1#2#3#4{% eg \bbl@rmdflt@lang fnt-opt fnt-nme \xxfamily
5078 \let\bbl@tempe\bbl@mapselect
5079 \edef\bbl@tempb{\bbl@stripslash#4/}% Catcodes hack (better pass it).
5080 \bbl@exp{\bbl@replace\bbl@tempb{\bbl@stripslash\family/}}}%
5081 \let\bbl@mapselect\relax
5082 \let\bbl@temp@fam#4% e.g., '\rmfamily', to be restored below
5083 \let#4@empty % Make sure \renewfontfamily is valid
5084 \bbl@set@renderer
5085 \bbl@exp{%
5086 \let\bbl@temp@pfam<\bbl@stripslash#4\space> e.g., '\rmfamily '
5087 \<keys_if_exist:nnF>{fontspec-opentype}{Script/\bbl@cl{sname}}}%
5088 {\bbl@newfontscript{\bbl@cl{sname}}{\bbl@cl{sotf}}}%
5089 \<keys_if_exist:nnF>{fontspec-opentype}{Language/\bbl@cl{lname}}}%
5090 {\bbl@newfontlanguage{\bbl@cl{lname}}{\bbl@cl{lotf}}}%
5091 \bbl@renewfontfamily\#4%
5092 [\bbl@cl{lsys},% xetex removes unknown features :-(
5093 \ifcase\bbl@engine\or RawFeature={family=\bbl@tempb},\fi
5094 #2]}{#3}% i.e., \bbl@exp{..}{#3}
5095 \bbl@unset@renderer
5096 \begingroup
5097 #4%
5098 \xdef#1{\f@family}% e.g., \bbl@rmdflt@lang{FreeSerif(0)}
5099 \endgroup
5100 \bbl@xin@{\detokenize{>ssub*}}}%
5101 {\expandafter\meaning\csname TU/#1/bx/sc\endcsname}%
5102 \ifin@
5103 \global\bbl@ccarg\let{TU/#1/bx/sc}{TU/#1/b/sc}%
5104 \fi
5105 \bbl@xin@{\detokenize{>ssub*}}}%
5106 {\expandafter\meaning\csname TU/#1/bx/scit\endcsname}%
5107 \ifin@
5108 \global\bbl@ccarg\let{TU/#1/bx/scit}{TU/#1/b/scit}%
5109 \fi
5110 \let#4\bbl@temp@fam
5111 \bbl@exp{\let<\bbl@stripslash#4\space>\bbl@temp@pfam
5112 \let\bbl@mapselect\bbl@tempe}%

```

`font@rst` and `famrst` are only used when there is no global settings, to save and restore de previous families. Not really necessary, but done for optimization.

```

5113 \def\bbl@font@rst#1#2#3#4{%
5114 \bbl@ccarg\def{famrst@#4}{\bbl@font@set{#1}#2#3}}

```

The default font families. They are eurocentric, but the list can be expanded easily with `\babelfont`.

```

5115 \def\bbl@font@fams{rm,sf,tt}
5116 ({/Font selection})

```

10. Hooks for XeTeX and LuaTeX

10.1. XeTeX

Unfortunately, the current encoding cannot be retrieved and therefore it is reset always to `utf8`, which seems a sensible default.

Now, the code.

```

5117 (*xetex)
5118 \def\BabelStringsDefault{unicode}
5119 \let\xebbl@stop\relax

```

```

5120 \AddBabelHook{xetex}{encodedcommands}{%
5121   \def\bbl@tempa{#1}%
5122   \ifx\bbl@tempa\@empty
5123     \XeTeXinputencoding"bytes"%
5124   \else
5125     \XeTeXinputencoding"#1"%
5126   \fi
5127   \def\xebbl@stop{\XeTeXinputencoding"utf8"}}
5128 \AddBabelHook{xetex}{stopcommands}{%
5129   \xebbl@stop
5130   \let\xebbl@stop\relax}
5131 \def\bbl@input@classes{% Used in CJK intraspaces
5132   \input{load-unicode-xetex-classes.tex}%
5133   \let\bbl@input@classes\relax}
5134 \def\bbl@intraspace#1 #2 #3\@@{%
5135   \bbl@csarg\gdef{xeisp@\language}%
5136   {\XeTeXlinebreakskip #1em plus #2em minus #3em\relax}}
5137 \def\bbl@intrapenalty#1\@@{%
5138   \bbl@csarg\gdef{xeipn@\language}%
5139   {\XeTeXlinebreakpenalty #1\relax}}
5140 \def\bbl@provide@intraspace{%
5141   \bbl@xin@{/s}{/\bbl@cl{lnbrk}}}%
5142   \ifin@ \else \bbl@xin@{/c}{/\bbl@cl{lnbrk}} \fi
5143   \ifin@
5144     \bbl@ifunset{bbl@intsp@\language}{}%
5145     {\expandafter\ifx\csname bbl@intsp@\language\endcsname\@empty\else
5146       \ifx\bbl@KVP@intraspace\@nnil
5147         \bbl@exp{%
5148           \\bbl@intraspace\bbl@cl{intsp}\@@}%
5149         \fi
5150         \ifx\bbl@KVP@intrapenalty\@nnil
5151           \bbl@intrapenalty0\@@
5152         \fi
5153         \fi
5154         \ifx\bbl@KVP@intraspace\@nnil\else % We may override the ini
5155           \expandafter\bbl@intraspace\bbl@KVP@intraspace\@@
5156         \fi
5157         \ifx\bbl@KVP@intrapenalty\@nnil\else
5158           \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty\@@
5159         \fi
5160         \bbl@exp{%
5161           \\bbl@add\<extras\language>{%
5162             \XeTeXlinebreaklocale "\bbl@cl{tbcpr}"%
5163             \<bbl@xeisp@\language>%
5164             \<bbl@xeipn@\language>}%
5165           \\bbl@tglobal\<extras\language>%
5166           \\bbl@add\<noextras\language>{%
5167             \XeTeXlinebreaklocale ""}%
5168           \\bbl@tglobal\<noextras\language>}%
5169           \ifx\bbl@ispace\@undefined
5170             \gdef\bbl@ispace{\bbl@cl{xeisp}}%
5171             \ifx\AtBeginDocument\@notprerr
5172               \expandafter\@secondoftwo % to execute right now
5173             \fi
5174             \AtBeginDocument{\bbl@patchfont{\bbl@ispace}}%
5175           \fi}%
5176   \fi}
5177 \ifx\DisableBabelHook\@undefined\endinput\fi
5178 \let\bbl@set@renderer\relax
5179 \let\bbl@unset@renderer\relax
5180 <@Font selection@>
5181 \def\bbl@provide@extra#1{}

```


Hack for unhyphenated line breaking. See `\bbl@provide@lsys` in the common code.

```

5182 \def\bbl@xenoxyph@d{%
5183   \bbl@ifset{bbl@prehc@\languagename}%
5184     {\ifnum\hyphenchar\font=\defaultshyphenchar
5185       \iffontchar\font\bbl@cl{prehc}\relax
5186       \hyphenchar\font\bbl@cl{prehc}\relax
5187       \else\iffontchar\font"200B
5188         \hyphenchar\font"200B
5189       \else
5190         \bbl@warning
5191           {Neither 0 nor ZERO WIDTH SPACE are available\\%
5192            in the current font, and therefore the hyphen\\%
5193            will be printed. Try changing the fontspec's\\%
5194            'HyphenChar' to another value, but be aware\\%
5195            this setting is not safe (see the manual).\\%
5196            Reported}%
5197         \hyphenchar\font\defaultshyphenchar
5198       \fi\fi
5199     \fi}%
5200   {\hyphenchar\font\defaultshyphenchar}}

```

10.2. Support for interchar

xetex reserves some values for CJK (although they are not set in XELATEX), so we make sure they are skipped. Define some user names for the global classes, too.

```

5201 \ifnum\xe@alloc@intercharclass<\thr@@
5202   \xe@alloc@intercharclass\thr@@
5203 \fi
5204 \chardef\bbl@xe@class@default@=\z@
5205 \chardef\bbl@xe@class@cjkideogram@=\@ne
5206 \chardef\bbl@xe@class@cjkleftpunctuation@=\tw@
5207 \chardef\bbl@xe@class@cjkrightpunctuation@=\thr@@
5208 \chardef\bbl@xe@class@boundary@=4095
5209 \chardef\bbl@xe@class@ignore@=4096

```

The machinery is activated with a hook (enabled only if actually used). Here `\bbl@tempc` is pre-set with `\bbl@usingxe@class`, defined below. The standard mechanism based on `\originalTeX` to save, set and restore values is used. `\count@` stores the previous char to be set, except at the beginning (0) and after `\bbl@upto`, which is the previous char negated, as a flag to mark a range.

```

5210 \AddBabelHook{babel-interchar}{beforeextras}{%
5211   \@nameuse{bbl@xechars@\languagename}}
5212 \DisableBabelHook{babel-interchar}
5213 \protected\def\bbl@charclass#1{%
5214   \ifnum\count@<\z@
5215     \count@-\count@
5216     \loop
5217       \bbl@exp{%
5218         \\babel@savevariable{\XeTeXcharclass`\Uchar\count@}}%
5219         \XeTeXcharclass\count@ \bbl@tempc
5220       \ifnum\count@<`#1\relax
5221         \advance\count@ \@ne
5222       \repeat
5223   \else
5224     \babel@savevariable{\XeTeXcharclass`#1}%
5225     \XeTeXcharclass`#1 \bbl@tempc
5226   \fi
5227   \count@`#1\relax}

```

Now the two user macros. Char classes are declared implicitly, and then the macro to be executed at the `babel-interchar` hook is created. The list of chars to be handled by the hook defined above has internally the form `\bbl@usingxe@class\bbl@xe@class@punct@english\bbl@charclass{.}` `\bbl@charclass{,}` (etc.), where `\bbl@usingxe@class` stores the class to be applied to the

subsequent characters. The `\ifcat` part deals with the alternative way to enter characters as macros (e.g., `\`). As a special case, hyphens are stored as `\bbl@upto`, to deal with ranges.

```

5228 \newcommand\bbl@ifinterchar[1]{%
5229   \let\bbl@tempa\@gobble      % Assume to ignore
5230   \edef\bbl@tempb{\zap@space#1 \@empty}%
5231   \ifx\bbl@KVP@interchar\@nnil\else
5232     \bbl@replace\bbl@KVP@interchar{ }{,}%
5233     \bbl@foreach\bbl@tempb{%
5234       \bbl@xin@{,##1,}{, \bbl@KVP@interchar,}%
5235       \ifin@
5236         \let\bbl@tempa\@firstofone
5237       \fi}%
5238   \fi
5239   \bbl@tempa}
5240 \newcommand\IfBabelIntercharT[2]{%
5241   \bbl@carg\bbl@add{\bbl@icsave\CurrentOption}{\bbl@ifinterchar{#1}{#2}}}%
5242 \newcommand\babelcharclass[3]{%
5243   \EnableBabelHook{babel-interchar}%
5244   \bbl@csarg\newXeTeXintercharclass{xeclass@#2@#1}%
5245   \def\bbl@tempb##1{%
5246     \ifx##1\@empty\else
5247       \ifx##1-%
5248         \bbl@upto
5249       \else
5250         \bbl@charclass{%
5251           \ifcat\noexpand##1\relax\bbl@stripslash##1\else\string##1\fi}%
5252         \fi
5253         \expandafter\bbl@tempb
5254       \fi}%
5255   \bbl@ifunset{\bbl@xechars@#1}%
5256   {\toks@{%
5257     \babel@savevariable\XeTeXinterchartokenstate
5258     \XeTeXinterchartokenstate\@ne
5259   }}%
5260   {\toks@\expandafter\expandafter\expandafter{%
5261     \csname bbl@xechars@#1\endcsname}}%
5262   \bbl@csarg\edef{xechars@#1}{%
5263     \the\toks@
5264     \bbl@usingxeclass\csname bbl@xeclass@#2@#1\endcsname
5265     \bbl@tempb#3\@empty}}
5266 \protected\def\bbl@usingxeclass#1{\count@ \z@ \let\bbl@tempc#1}
5267 \protected\def\bbl@upto{%
5268   \ifnum\count@>\z@
5269     \advance\count@\@ne
5270     \count@-\count@
5271   \else\ifnum\count@=\z@
5272     \bbl@charclass{-}%
5273   \else
5274     \bbl@error{double-hyphens-class}{\count@}{\count@}%
5275   \fi\fi}

```

And finally, the command with the code to be inserted. If the language doesn't define a class, then use the global one, as defined above. For the definition there is an intermediate macro, which can be 'disabled' with `\bbl@ic@<label>@<language>`.

```

5276 \def\bbl@ignoreinterchar{%
5277   \ifnum\language=\l@nohyphenation
5278     \expandafter\@gobble
5279   \else
5280     \expandafter\@firstofone
5281   \fi}
5282 \newcommand\babelinterchar[5][{}]{%
5283   \let\bbl@kv@label\@empty
5284   \bbl@forkv{#1}{\bbl@csarg\edef{kv@##1}{##2}}%

```

```

5285 \namedef{\zap@space bbl@xeinter@bbl@kv@label @#3@#4@#2 \@empty}%
5286 {\bbl@ignoreinterchar{#5}}%
5287 \bbl@csarg\let{ic@bbl@kv@label @#2}\@firstofone
5288 \bbl@exp{\bbl@for{\bbl@tempa{\zap@space#3 \@empty}}{%
5289 \bbl@exp{\bbl@for{\bbl@tempb{\zap@space#4 \@empty}}{%
5290 \XeTeXinterchartoks
5291 \@nameuse{bbl@xeclasse@bbl@tempa @#2}%
5292 \bbl@ifunset{bbl@xeclasse@bbl@tempa @#2}{#2}} %
5293 \@nameuse{bbl@xeclasse@bbl@tempb @#2}%
5294 \bbl@ifunset{bbl@xeclasse@bbl@tempb @#2}{#2}} %
5295 = \expandafter{%
5296 \csname bbl@ic@bbl@kv@label @#2\expandafter\endcsname
5297 \csname\zap@space bbl@xeinter@bbl@kv@label
5298 @#3@#4@#2 \@empty\endcsname}}}%
5299 \DeclareRobustCommand\enablelocaleinterchar[1]{%
5300 \bbl@ifunset{bbl@ic@#1@language}%
5301 {\bbl@error{unknown-interchar}{#1}}}%
5302 {\bbl@csarg\let{ic@#1@language}\@firstofone}}
5303 \DeclareRobustCommand\disablelocaleinterchar[1]{%
5304 \bbl@ifunset{bbl@ic@#1@language}%
5305 {\bbl@error{unknown-interchar-b}{#1}}}%
5306 {\bbl@csarg\let{ic@#1@language}\@gobble}}
5307 (/xetex)

```

10.3. Layout

Note elements like headlines and margins can be modified easily with packages like fancyhdr, typearea or titleps, and geometry.

\bbl@startskip and \bbl@endskip are available to package authors. Thanks to the T_EX expansion mechanism the following constructs are valid: \adim\bbl@startskip, \advance\bbl@startskip\adim, \bbl@startskip\adim.

Consider txtbabel as a shorthand for *tex-xet babel*, which is the bidi model in both pdftex and xetex.

```

5308 (*xetex|texxet)
5309 \providecommand\bbl@provide@intraspace{}
5310 \bbl@trace{Redefinitions for bidi layout}

```

Finish here if there is no layout.

```

5311 \ifx\bbl@opt@layout\@nnil\else % if layout=..
5312 \ifx\bbl@opt@layout\@undefined\else % Plain
5313 \IfBabelLayout{nopars}
5314 {}
5315 {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
5316 \fi
5317 \def\bbl@startskip{\ifcase\bbl@thepardir\leftskip\else\rightskip\fi}
5318 \def\bbl@endskip{\ifcase\bbl@thepardir\rightskip\else\leftskip\fi}
5319 \ifnum\bbl@bidimode>\z@
5320 \IfBabelLayout{pars}
5321 {\def\hangfrom#1{%
5322 \setbox\@tempboxa\hbox{#1}}%
5323 \hangindent\ifcase\bbl@thepardir\wd\@tempboxa\else-\wd\@tempboxa\fi
5324 \noindent\box\@tempboxa}
5325 \def\raggedright{%
5326 \let\\\@centercr
5327 \bbl@startskip\z@skip
5328 \@rightskip\@flushglue
5329 \bbl@endskip\@rightskip
5330 \parindent\z@
5331 \parfillskip\bbl@startskip}
5332 \def\raggedleft{%
5333 \let\\\@centercr
5334 \bbl@startskip\@flushglue
5335 \bbl@endskip\z@skip

```

```

5336     \parindent\z@
5337     \parfillskip\bbl@endskip}}
5338 {}
5339 \fi
5340 \IfBabelLayout{lists}
5341 {\bbl@sreplace\list
5342   {\@totalleftmargin\leftmargin}{\@totalleftmargin\bbl@listleftmargin}%
5343   \def\bbl@listleftmargin{%
5344     \ifcase\bbl@thepardir\leftmargin\else\rightmargin\fi}%
5345   \ifcase\bbl@engine
5346     \def\labelenumii{}\theenumii{}\pdfTeX doesn't reverse ()
5347     \def\p@enumiii{\p@enumii}\theenumii}%
5348   \fi
5349   \bbl@sreplace\@verbatim
5350   {\leftskip\@totalleftmargin}%
5351   {\bbl@startskip\textwidth
5352     \advance\bbl@startskip-\linewidth}%
5353   \bbl@sreplace\@verbatim
5354   {\rightskip\z@skip}%
5355   {\bbl@endskip\z@skip}}%
5356 {}
5357 \IfBabelLayout{contents}
5358 {\bbl@sreplace\@dottedtocline{\leftskip}{\bbl@startskip}%
5359   \bbl@sreplace\@dottedtocline{\rightskip}{\bbl@endskip}}
5360 {}
5361 \IfBabelLayout{columns}
5362 {\bbl@sreplace\@outputdblcol{\hb@xt@\textwidth}{\bbl@outputbox}%
5363   \def\bbl@outputbox#1{%
5364     \hb@xt@\textwidth{%
5365       \hskip\columnwidth
5366       \hfil
5367       {\normalcolor\vrule \@width\columnseprule}%
5368       \hfil
5369       \hb@xt@\columnwidth{\box\@leftcolumn \hss}%
5370       \hskip-\textwidth
5371       \hb@xt@\columnwidth{\box\@outputbox \hss}%
5372       \hskip\columnsep
5373       \hskip\columnwidth}}}%
5374 {}

```

Implicitly reverses sectioning labels in `bidibasic`, because the full stop is not in contact with L numbers any more. I think there must be a better way.

```

5375 \IfBabelLayout{counters*}%
5376 {\bbl@add\bbl@opt@layout{.counters.}%
5377   \AddToHook{shipout/before}{%
5378     \let\bbl@tempa\babelsublr
5379     \let\babelsublr\@firstofone
5380     \let\bbl@save@thepage\thepage
5381     \protected@edef\thepage{\thepage}%
5382     \let\babelsublr\bbl@tempa}%
5383   \AddToHook{shipout/after}{%
5384     \let\thepage\bbl@save@thepage}}{}
5385 \IfBabelLayout{counters}%
5386 {\let\bbl@latinarabic=\@arabic
5387   \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
5388   \let\bbl@asciroman=\@roman
5389   \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
5390   \let\bbl@asciiRoman=\@Roman
5391   \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}}{}
5392 \fi % end if layout
5393 (/xetex|texet)

```

10.4. 8-bit TeX

Which start just above, because some code is shared with xetex. Now, 8-bit specific stuff. If just one encoding has been declared, then assume no switching is necessary (1).

```
5394 {*texxet}
5395 \def\bbl@provide@extra#1{%
5396   % == auto-select encoding ==
5397   \ifx\bbl@encoding@select@off\@empty\else
5398     \bbl@ifunset\bbl@encoding@#1{%
5399       {\def\elt##1{,##1,}%
5400        \edef\bbl@tempe{\expandafter\@gobbletwo\@fontenc@load@list}%
5401        \count@z@
5402        \bbl@foreach\bbl@tempe{%
5403          \def\bbl@tempd{##1}% Save last declared
5404          \advance\count@\@ne}%
5405        \ifnum\count@>\@ne % (1)
5406          \getlocaleproperty*\bbl@tempa{#1}{identification/encodings}%
5407          \ifx\bbl@tempa\relax \let\bbl@tempa\@empty \fi
5408          \bbl@replace\bbl@tempa{ }{,}%
5409          \global\bbl@csarg\let{encoding@#1}\@empty
5410          \bbl@xin@{,\bbl@tempd,}{,\bbl@tempa,}%
5411          \ifin\@else % if main encoding included in ini, do nothing
5412            \let\bbl@tempb\relax
5413            \bbl@foreach\bbl@tempa{%
5414              \ifx\bbl@tempb\relax
5415                \bbl@xin@{,##1,}{,\bbl@tempe,}%
5416                \ifin\def\bbl@tempb{##1}\fi
5417              \fi}%
5418            \ifx\bbl@tempb\relax\else
5419              \bbl@exp{%
5420                \global\<bbl@add>\<bbl@preextras@#1>\<bbl@encoding@#1>%
5421                \gdef\<bbl@encoding@#1>{%
5422                  \\babel@save\\f@encoding
5423                  \\bbl@add\\originalTeX{\\selectfont}%
5424                  \\fontencoding{\bbl@tempb}%
5425                  \\selectfont}}%
5426              \fi
5427            \fi
5428          \fi}%
5429        }%
5430      \fi}
5431 {/texxet}
```

10.5. LuaTeX

The loader for luatex is based solely on language.dat, which is read on the fly. The code shouldn't be executed when the format is build, so we check if \AddBabelHook is defined. Then comes a modified version of the loader in hyphen.cfg (without the hyphenmins stuff, which is under the direct control of babel).

The names \l@<language> are defined and take some value from the beginning because all ldf files assume this for the corresponding language to be considered valid, but patterns are not loaded (except the first one). This is done later, when the language is first selected (which usually means when the ldf finishes). If a language has been loaded, \bbl@hyphendata@<num> exists (with the names of the files read).

The default setup preloads the first language into the format. This is intended mainly for 'english', so that it's available without further intervention from the user. To avoid duplicating it, the following rule applies: if the "0th" language and the first language in language.dat have the same name then just ignore the latter. If there are new synonymous, they are added, but note if the language patterns have not been preloaded they won't at run time.

Other preloaded languages could be read twice, if they have been preloaded into the format. This is not optimal, but it shouldn't happen very often – with luatex patterns are best loaded when the document is typeset, and the "0th" language is preloaded just for backwards compatibility.

As of 1.1b, lua(e)tex is taken into account. Formerly, loading of patterns on the fly didn't work in this format, but with the new loader it does. Unfortunately, the format is not based on babel, and data could be duplicated, because languages are reassigned above those in the format (nothing serious, anyway). Note even with this format language.dat is used (under the principle of a single source), instead of language.def.

Of course, there is room for improvements, like tools to read and reassign languages, which would require modifying the language list, and better error handling.

We need catcode tables, but no format (targeted by babel) provide a command to allocate them (although there are packages like ctablestack). FIX - This isn't true anymore. For the moment, a dangerous approach is used - just allocate a high random number and cross the fingers. To complicate things, etex.sty changes the way languages are allocated.

This files is read at three places: (1) when plain.def, babel.sty starts, to read the list of available languages from language.dat (for the base option); (2) at hyphen.cfg, to modify some macros; (3) in the middle of plain.def and babel.sty, by babel.def, with the commands and other definitions for luatex (e.g., \babelpatterns).

```

5432 (*luatex)
5433 \directlua{ Babel = Babel or {} } % DL2
5434 \ifx\AddBabelHook\undefined % When plain.def, babel.sty starts
5435 \bbl@trace{Read language.dat}
5436 \ifx\bbl@readstream\undefined
5437 \csname newread\endcsname\bbl@readstream
5438 \fi
5439 \begingroup
5440 \toks@{}
5441 \count@\z@ % 0=start, 1=0th, 2=normal
5442 \def\bbl@process@line#1#2 #3 #4 {%
5443 \ifx=#1%
5444 \bbl@process@synonym{#2}%
5445 \else
5446 \bbl@process@language{#1#2}{#3}{#4}%
5447 \fi
5448 \ignorespaces}
5449 \def\bbl@manylang{%
5450 \ifnum\bbl@last>\@ne
5451 \bbl@info{Non-standard hyphenation setup}%
5452 \fi
5453 \let\bbl@manylang\relax}
5454 \def\bbl@process@language#1#2#3{%
5455 \ifcase\count@
5456 \ifundefined{zth@#1}{\count@\tw@}{\count@\@ne}%
5457 \or
5458 \count@\tw@
5459 \fi
5460 \ifnum\count@=\tw@
5461 \expandafter\addlanguage\csname l@#1\endcsname
5462 \language\allocationnumber
5463 \chardef\bbl@last\allocationnumber
5464 \bbl@manylang
5465 \let\bbl@elt\relax
5466 \xdef\bbl@languages{%
5467 \bbl@languages\bbl@elt{#1}{\the\language}{#2}{#3}}%
5468 \fi
5469 \the\toks@
5470 \toks@{}}
5471 \def\bbl@process@synonym@aux#1#2{%
5472 \global\expandafter\chardef\csname l@#1\endcsname#2\relax
5473 \let\bbl@elt\relax
5474 \xdef\bbl@languages{%
5475 \bbl@languages\bbl@elt{#1}{#2}{}}}%
5476 \def\bbl@process@synonym#1{%
5477 \ifcase\count@
5478 \toks@\expandafter{\the\toks@\relax\bbl@process@synonym{#1}}%
5479 \or

```

```

5480 \ifundefined{zth@#1}{\bbl@process@synonym@aux{#1}{0}}{}%
5481 \else
5482 \bbl@process@synonym@aux{#1}{\the\bbl@last}%
5483 \fi}
5484 \ifx\bbl@languages\undefined % Just a (sensible?) guess
5485 \chardef\l@english\z@
5486 \chardef\l@USenglish\z@
5487 \chardef\bbl@last\z@
5488 \global\@namedef{bbl@hyphendata@0}{{hyphen.tex}}{}
5489 \gdef\bbl@languages{%
5490 \bbl@elt{english}{0}{hyphen.tex}}{}%
5491 \bbl@elt{USenglish}{0}{}{}
5492 \else
5493 \global\let\bbl@languages@format\bbl@languages
5494 \def\bbl@elt#1#2#3#4{% Remove all except language 0
5495 \ifnum#2>\z@\else
5496 \noexpand\bbl@elt{#1}{#2}{#3}{#4}%
5497 \fi}%
5498 \xdef\bbl@languages{\bbl@languages}%
5499 \fi
5500 \def\bbl@elt#1#2#3#4{\@namedef{zth@#1}}{} % Define flags
5501 \bbl@languages
5502 \openin\bbl@readstream=language.dat
5503 \ifeof\bbl@readstream
5504 \bbl@warning{I couldn't find language.dat. No additional\\%
5505 patterns loaded. Reported}%
5506 \else
5507 \loop
5508 \endlinechar\m@ne
5509 \read\bbl@readstream to \bbl@line
5510 \endlinechar\^^M
5511 \if T\ifeof\bbl@readstream F\fi T\relax
5512 \ifx\bbl@line\empty\else
5513 \edef\bbl@line{\bbl@line\space\space\space}%
5514 \expandafter\bbl@process@line\bbl@line\relax
5515 \fi
5516 \repeat
5517 \fi
5518 \closein\bbl@readstream
5519 \endgroup
5520 \bbl@trace{Macros for reading patterns files}
5521 \def\bbl@get@enc#1:#2:#3\@@{\def\bbl@hyph@enc{#2}}
5522 \ifx\babelcatcodetablenum\undefined
5523 \ifx\newcatcodetable\undefined
5524 \def\babelcatcodetablenum{5211}
5525 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5526 \else
5527 \newcatcodetable\babelcatcodetablenum
5528 \newcatcodetable\bbl@pattcodes
5529 \fi
5530 \else
5531 \def\bbl@pattcodes{\numexpr\babelcatcodetablenum+1\relax}
5532 \fi
5533 \def\bbl@luapatterns#1#2{%
5534 \bbl@get@enc#1:.\@@@
5535 \setbox\z@\hbox\bgroup
5536 \begin{group}
5537 \savecatcodetable\babelcatcodetablenum\relax
5538 \initcatcodetable\bbl@pattcodes\relax
5539 \catcodetable\bbl@pattcodes\relax
5540 \catcode\#=6 \catcode\$=3 \catcode\&=4 \catcode\^=7
5541 \catcode\_ =8 \catcode\{=1 \catcode\}=2 \catcode\~=13
5542 \catcode\@=11 \catcode\^^I=10 \catcode\^^J=12

```

```

5543      \catcode`\<=12 \catcode`\>=12 \catcode`\*=12 \catcode`\.=12
5544      \catcode`\-=12 \catcode`\/=12 \catcode`\[=12 \catcode`\]=12
5545      \catcode`\`=12 \catcode`\'=12 \catcode`\\"=12
5546      \input #1\relax
5547      \catcodetable\babelcatcodetablenum\relax
5548      \endgroup
5549      \def\bbl@tempa{#2}%
5550      \ifx\bbl@tempa\@empty\else
5551          \input #2\relax
5552      \fi
5553      \egroup}%
5554      \def\bbl@patterns@lua#1{%
5555      \language=\expandafter\ifx\csname l@#1:\f@encoding\endcsname\relax
5556          \csname l@#1\endcsname
5557          \edef\bbl@tempa{#1}%
5558      \else
5559          \csname l@#1:\f@encoding\endcsname
5560          \edef\bbl@tempa{#1:\f@encoding}%
5561      \fi\relax
5562      \namedef{lu@texhyphen@loaded@the\language}{}% Temp
5563      \ifundefined{bbl@hyphendata@the\language}%
5564          {\def\bbl@elt##1##2###4{%
5565              \ifnum##2=\csname l@bbl@tempa\endcsname % #2=spanish, dutch:OT1...
5566              \def\bbl@tempb{##3}%
5567              \ifx\bbl@tempb\@empty\else % if not a synonymous
5568                  \def\bbl@tempc{{##3}{##4}}%
5569              \fi
5570              \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5571              \fi}%
5572      \bbl@languages
5573      \ifundefined{bbl@hyphendata@the\language}%
5574          {\bbl@info{No hyphenation patterns were set for\%
5575              language '\bbl@tempa'. Reported}}%
5576          {\expandafter\expandafter\expandafter\bbl@luapatterns
5577              \csname bbl@hyphendata@the\language\endcsname}}}%
5578      \endinput\fi

```

Here ends \ifx\AddBabelHook\@undefined. A few lines are only read by HYPHEN.CFG.

```

5579      \ifx\DisableBabelHook\@undefined
5580          \AddBabelHook{luatex}{everylanguage}{%
5581              \def\process@language##1##2##3{%
5582                  \def\process@line####1####2 ####3 ####4 {}%
5583              \AddBabelHook{luatex}{loadpatterns}{%
5584                  \input #1\relax
5585                  \expandafter\gdef\csname bbl@hyphendata@the\language\endcsname
5586                      {{#1}}}%
5587              \AddBabelHook{luatex}{loadexceptions}{%
5588                  \input #1\relax
5589                  \def\bbl@tempb##1##2{{##1}{##2}}%
5590                  \expandafter\xdef\csname bbl@hyphendata@the\language\endcsname
5591                      {\expandafter\expandafter\expandafter\bbl@tempb
5592                      \csname bbl@hyphendata@the\language\endcsname}}%
5593              \endinput\fi

```

Here stops reading code for HYPHEN.CFG. The following is read the 2nd time it's loaded. First, global declarations for lua.

```

5594      \begingroup
5595      \catcode`\%=12
5596      \catcode`\'=12
5597      \catcode`\\"=12
5598      \catcode`\:=12
5599      \directlua{
5600          Babel.locale_props = Babel.locale_props or {}
5601

```



```

5602 function Babel.lua_error(e, a)
5603     tex.print([[noexpand\csname bbl@error\endcsname{]] ..
5604         e .. '}' .. (a or '') .. '}{'}])
5605 end
5606
5607 function Babel.bytes(line)
5608     return line:gsub(".",
5609         function (chr) return unicode.utf8.char(string.byte(chr)) end)
5610 end
5611
5612 function Babel.priority_in_callback(name,description)
5613     for i,v in ipairs(luatexbase.callback_descriptions(name)) do
5614         if v == description then return i end
5615     end
5616     return false
5617 end
5618
5619 function Babel.begin_process_input()
5620     if luatexbase and luatexbase.add_to_callback then
5621         luatexbase.add_to_callback('process_input_buffer',
5622             Babel.bytes,'Babel.bytes')
5623     else
5624         Babel.callback = callback.find('process_input_buffer')
5625         callback.register('process_input_buffer',Babel.bytes)
5626     end
5627 end
5628 function Babel.end_process_input ()
5629     if luatexbase and luatexbase.remove_from_callback then
5630         luatexbase.remove_from_callback('process_input_buffer','Babel.bytes')
5631     else
5632         callback.register('process_input_buffer',Babel.callback)
5633     end
5634 end
5635
5636 function Babel.str_to_nodes(fn, matches, base)
5637     local n, head, last
5638     if fn == nil then return nil end
5639     for s in string.utfvalues(fn(matches)) do
5640         if base.id == 7 then
5641             base = base.replace
5642         end
5643         n = node.copy(base)
5644         n.char = s
5645         if not head then
5646             head = n
5647         else
5648             last.next = n
5649         end
5650         last = n
5651     end
5652     return head
5653 end
5654
5655 Babel.linebreaking = Babel.linebreaking or {}
5656 Babel.linebreaking.before = {}
5657 Babel.linebreaking.after = {}
5658 Babel.locale = {}
5659 function Babel.linebreaking.add_before(func, pos)
5660     tex.print([[noexpand\csname bbl@luahyphenate\endcsname]])
5661     if pos == nil then
5662         table.insert(Babel.linebreaking.before, func)
5663     else
5664         table.insert(Babel.linebreaking.before, pos, func)

```

```

5665     end
5666 end
5667 function Babel.linebreaking.add_after(func)
5668     tex.print([[\\noexpand\\csname bbl@lua hyphenate\\endcsname]])
5669     table.insert(Babel.linebreaking.after, func)
5670 end
5671
5672 function Babel.addpatterns(pp, lg)
5673     local lg = lang.new(lg)
5674     local pats = lang.patterns(lg) or ''
5675     lang.clear_patterns(lg)
5676     for p in pp:gmatch('[^%s]+') do
5677         ss = ''
5678         for i in string.utfcharacters(p:gsub('%d', '')) do
5679             ss = ss .. '%d?' .. i
5680         end
5681         ss = ss:gsub('^%d%?%.', '%%.') .. '%d?'
5682         ss = ss:gsub('%.%d%?$', '%%.')
5683         pats, n = pats:gsub('%s' .. ss .. '%s', ' ' .. p .. ' ')
5684         if n == 0 then
5685             tex.sprint(
5686                 [[\\string\\csname\\space bbl@info\\endcsname{New pattern: }
5687                 .. p .. [{}]])
5688             pats = pats .. ' ' .. p
5689         else
5690             tex.sprint(
5691                 [[\\string\\csname\\space bbl@info\\endcsname{Renew pattern: }
5692                 .. p .. [{}]])
5693         end
5694     end
5695     lang.patterns(lg, pats)
5696 end
5697
5698 Babel.characters = Babel.characters or {}
5699 Babel.ranges = Babel.ranges or {}
5700 function Babel.hlist_has_bidi(head)
5701     local has_bidi = false
5702     local ranges = Babel.ranges
5703     for item in node.traverse(head) do
5704         if item.id == node.id'glyph' then
5705             local itemchar = item.char
5706             local chardata = Babel.characters[itemchar]
5707             local dir = chardata and chardata.d or nil
5708             if not dir then
5709                 for nn, et in ipairs(ranges) do
5710                     if itemchar < et[1] then
5711                         break
5712                     elseif itemchar <= et[2] then
5713                         dir = et[3]
5714                         break
5715                     end
5716                 end
5717             end
5718             if dir and (dir == 'al' or dir == 'r') then
5719                 has_bidi = true
5720             end
5721         end
5722     end
5723     return has_bidi
5724 end
5725 function Babel.set_chranges_b (script, chrng)
5726     if chrng == '' then return end
5727     texio.write('Package babel Info: Replacing ' .. script .. ' script ranges')

```

```

5728     Babel.script_blocks[script] = {}
5729     for s, e in string.gmatch(chrng..' ', '(.-%.(-)%s') do
5730         table.insert(
5731             Babel.script_blocks[script], {tonumber(s,16), tonumber(e,16)})
5732     end
5733 end
5734
5735 function Babel.discard_sublr(str)
5736     if str:find( [[\string\indexentry]] ) and
5737         str:find( [[\string\babelsublr]] ) then
5738         str = str:gsub( [[\string\babelsublr%s*(%b{})]],
5739             function(m) return m:sub(2,-2) end )
5740     end
5741     return str
5742 end
5743 }
5744 \endgroup
5745 \ifx\newattribute\@undefined\else % Test for plain
5746     \newattribute\bbl@attr@locale % DL4
5747     \directlua{ Babel.attr_locale = luatexbase.registernumber'bbl@attr@locale' }
5748     \AddBabelHook{luatex}{beforeextras}{%
5749         \setattribute\bbl@attr@locale\localeid}
5750 \fi
5751 %
5752 \def\BabelStringsDefault{unicode}
5753 \let\luabbl@stop\relax
5754 \AddBabelHook{luatex}{encodedcommands}{%
5755     \def\bbl@tempa{utf8}\def\bbl@tempb{#1}%
5756     \ifx\bbl@tempa\bbl@tempb\else
5757         \directlua{Babel.begin_process_input()}%
5758         \def\luabbl@stop{%
5759             \directlua{Babel.end_process_input()}}%
5760     \fi}%
5761 \AddBabelHook{luatex}{stopcommands}{%
5762     \luabbl@stop
5763     \let\luabbl@stop\relax}
5764 %
5765 \AddBabelHook{luatex}{patterns}{%
5766     \@ifundefined{bbl@hyphendata@the\language}%
5767     {\def\bbl@elt##1##2##3##4{%
5768         \ifnum##2=\csname l@##2\endcsname % #2=spanish, dutch:OT1...
5769         \def\bbl@tempb{##3}%
5770         \ifx\bbl@tempb\empty\else % if not a synonymous
5771             \def\bbl@tempc{{##3}{##4}}%
5772             \fi
5773             \bbl@csarg\xdef{hyphendata@##2}{\bbl@tempc}%
5774             \fi}%
5775     \bbl@languages
5776     \@ifundefined{bbl@hyphendata@the\language}%
5777     {\bbl@info{No hyphenation patterns were set for\%
5778         language '#2'. Reported}}%
5779     {\expandafter\expandafter\expandafter\bbl@luapatterns
5780         \csname bbl@hyphendata@the\language\endcsname}}}%
5781 \@ifundefined{bbl@patterns@}{}%
5782 \begingroup
5783     \bbl@xin@{,\number\language,}{,\bbl@pttnlist}%
5784     \ifin@else
5785         \ifx\bbl@patterns@\empty\else
5786             \directlua{ Babel.addpatterns(
5787                 [[\bbl@patterns@]], \number\language) }%
5788             \fi
5789             \@ifundefined{bbl@patterns@#1}%
5790             \@empty

```

```

5791         {\directlua{ Babel.addpatterns(
5792             [[\space\csname bbl@patterns@#1\endcsname]],
5793             \number\language) }}%
5794         \xdef\bbl@pttnlist{\bbl@pttnlist\number\language,}%
5795         \fi
5796     \endgroup}%
5797 \bbl@exp{%
5798     \bbl@ifunset{\bbl@prehc@\languagename}{}%
5799     {\bbl@ifblank{\bbl@cs{\prehc@\languagename}}{}}%
5800     {\prehyphenchar=\bbl@cl{\prehc}\relax}}

```

\babelpatterns This macro adds patterns. Two macros are used to store them: `\bbl@patterns@` for the global ones and `\bbl@patterns@<language>` for language ones. We make sure there is a space between words when multiple commands are used.

```

5801 \@onlypreamble\babelpatterns
5802 \AtEndOfPackage{%
5803     \newcommand\babelpatterns[2][\@empty]{%
5804         \ifx\bbl@patterns@\relax
5805             \let\bbl@patterns@\@empty
5806         \fi
5807         \ifx\bbl@pttnlist@\@empty\else
5808             \bbl@warning{%
5809                 You must not intermingle \string\selectlanguage\space and\%
5810                 \string\babelpatterns\space or some patterns will not\%
5811                 be taken into account. Reported}%
5812             \fi
5813             \ifx\@empty#1%
5814                 \protected@edef\bbl@patterns@{\bbl@patterns@\space#2}%
5815             \else
5816                 \edef\bbl@tempb{\zap@space#1 \@empty}%
5817                 \bbl@for\bbl@tempa\bbl@tempb{%
5818                     \bbl@fixname\bbl@tempa
5819                     \bbl@iflanguage\bbl@tempa{%
5820                         \bbl@csarg\protected@edef{patterns@\bbl@tempa}{%
5821                             \@ifundefined{\bbl@patterns@\bbl@tempa}%
5822                             \@empty
5823                             {\csname bbl@patterns@\bbl@tempa\endcsname\space}%
5824                             #2}}}%
5825             \fi}}

```

10.6. Southeast Asian scripts

First, some general code for line breaking, used by `\babelposthyphenation`.

Replace regular (i.e., implicit) discretionaries by spaceskips, based on the previous glyph (which I think makes sense, because the hyphen and the previous char go always together). Other discretionaries are not touched. See Unicode UAX 14.

```

5826 \def\bbl@intraspace#1 #2 #3\@{%
5827     \directlua{
5828         Babel.intraspaces = Babel.intraspaces or {}
5829         Babel.intraspaces['\csname bbl@sbc@'\languagename\endcsname'] = %
5830             {b = #1, p = #2, m = #3}
5831         Babel.locale_props[\the\localeid].intraspace = %
5832             {b = #1, p = #2, m = #3}
5833     }}
5834 \def\bbl@intrapenalty#1\@{%
5835     \directlua{
5836         Babel.intrapenalties = Babel.intrapenalties or {}
5837         Babel.intrapenalties['\csname bbl@sbc@'\languagename\endcsname'] = #1
5838         Babel.locale_props[\the\localeid].intrapenalty = #1
5839     }}
5840 \begingroup
5841 \catcode`\%=12

```

```

5842 \catcode`\&=14
5843 \catcode`\'=12
5844 \catcode`\~ =12
5845 \gdef\bbl@seaintraspace{&
5846 \let\bbl@seaintraspace\relax
5847 \directlua{
5848   Babel.sea_enabled = true
5849   Babel.sea_ranges = Babel.sea_ranges or {}
5850   function Babel.set_chranges (script, chrng)
5851     local c = 0
5852     for s, e in string.gmatch(chrng..' ', '(.-%.%.(.-%s)') do
5853       Babel.sea_ranges[script..c]={tonumber(s,16), tonumber(e,16)}
5854       c = c + 1
5855     end
5856   end
5857   function Babel.sea_disc_to_space (head)
5858     local sea_ranges = Babel.sea_ranges
5859     local last_char = nil
5860     local quad = 655360      &% 10 pt = 655360 = 10 * 65536
5861     for item in node.traverse(head) do
5862       local i = item.id
5863       if i == node.id'glyph' then
5864         last_char = item
5865       elseif i == 7 and item.subtype == 3 and last_char
5866         and last_char.char > 0x0C99 then
5867         quad = font.getfont(last_char.font).size
5868         for lg, rg in pairs(sea_ranges) do
5869           if last_char.char > rg[1] and last_char.char < rg[2] then
5870             lg = lg:sub(1, 4) &% Remove trailing number of, e.g., Cyril1
5871             local intraspace = Babel.intraspaces[lg]
5872             local intrapenalty = Babel.intrapenalties[lg]
5873             local n
5874             if intrapenalty ~= 0 then
5875               n = node.new(14, 0)      &% penalty
5876               n.penalty = intrapenalty
5877               node.insert_before(head, item, n)
5878             end
5879             n = node.new(12, 13)      &% (glue, spaceskip)
5880             node.setglue(n, intraspace.b * quad,
5881               intraspace.p * quad,
5882               intraspace.m * quad)
5883             node.insert_before(head, item, n)
5884             node.remove(head, item)
5885           end
5886         end
5887       end
5888     end
5889   end
5890 }&
5891 \bbl@luahyphenate}

```

10.7. CJK line breaking

Minimal line breaking for CJK scripts, mainly intended for simple documents and short texts as a secondary language. Only line breaking, with a little stretching for justification, without any attempt to adjust the spacing. It is based on (but does not strictly follow) the Unicode algorithm.

We first need a little table with the corresponding line breaking properties. A few characters have an additional key for the width (fullwidth vs. halfwidth), not yet used. There is a separate file, defined below.

```

5892 \catcode`\%=14
5893 \gdef\bbl@cjkkintraspace{%
5894 \let\bbl@cjkkintraspace\relax
5895 \directlua{

```

```

5896     require('babel-data-cjk.lua')
5897     Babel.cjk_enabled = true
5898     function Babel.cjk_linebreak(head)
5899         local GLYPH = node.id'glyph'
5900         local last_char = nil
5901         local quad = 655360      % 10 pt = 655360 = 10 * 65536
5902         local last_class = nil
5903         local last_lang = nil
5904         for item in node.traverse(head) do
5905             if item.id == GLYPH then
5906                 local lang = item.lang
5907                 local LOCALE = node.get_attribute(item,
5908                     Babel.attr_locale)
5909                 local props = Babel.locale_props[LOCALE] or {}
5910                 local class = Babel.cjk_class[item.char].c
5911                 if props.cjk_quotes and props.cjk_quotes[item.char] then
5912                     class = props.cjk_quotes[item.char]
5913                 end
5914                 if class == 'cp' then class = 'cl' % )] as CL
5915                 elseif class == 'id' then class = 'I'
5916                 elseif class == 'cj' then class = 'I' % loose
5917                 end
5918                 local br = 0
5919                 if class and last_class and Babel.cjk_breaks[last_class][class] then
5920                     br = Babel.cjk_breaks[last_class][class]
5921                 end
5922                 if br == 1 and props.linebreak == 'c' and
5923                     lang ~= \the\l@nohyphenation\space and
5924                     last_lang ~= \the\l@nohyphenation then
5925                     local intrapenalty = props.intrapenalty
5926                     if intrapenalty ~= 0 then
5927                         local n = node.new(14, 0)      % penalty
5928                         n.penalty = intrapenalty
5929                         node.insert_before(head, item, n)
5930                     end
5931                     local intraspace = props.intraspace
5932                     local n = node.new(12, 13)      % (glue, spaceskip)
5933                     node.setglue(n, intraspace.b * quad,
5934                         intraspace.p * quad,
5935                         intraspace.m * quad)
5936                     node.insert_before(head, item, n)
5937                 end
5938                 if font.getfont(item.font) then
5939                     quad = font.getfont(item.font).size
5940                 end
5941                 last_class = class
5942                 last_lang = lang
5943                 else % if penalty, glue or anything else
5944                     last_class = nil
5945                 end
5946             end
5947             lang.hyphenate(head)
5948         end
5949     }%
5950     \bbl@luahyphenate}
5951 \gdef\bbl@luahyphenate{%
5952     \let\bbl@luahyphenate\relax
5953     \directlua{
5954         luatexbase.add_to_callback('hyphenate',
5955             function (head, tail)
5956                 if Babel.linebreaking.before then
5957                     for k, func in ipairs(Babel.linebreaking.before) do
5958                         func(head)

```

```

5959     end
5960   end
5961   lang.hyphenate(head)
5962   if Babel.cjk_enabled then
5963     Babel.cjk_linebreak(head)
5964   end
5965   if Babel.linebreaking.after then
5966     for k, func in ipairs(Babel.linebreaking.after) do
5967       func(head)
5968     end
5969   end
5970   if Babel.set_hboxed then
5971     Babel.set_hboxed(head)
5972   end
5973   if Babel.sea_enabled then
5974     Babel.sea_disc_to_space(head)
5975   end
5976 end,
5977 'Babel.hyphenate')
5978 }}
5979 \endgroup
5980 %
5981 \def\bbl@provide@intraspace{%
5982   \bbl@ifunset{\bbl@intsp@\languagename}{}%
5983   {\expandafter\ifx\csname bbl@intsp@\languagename\endcsname\@empty\else
5984     \bbl@xin@{/c}{/\bbl@cl{\lnbrk}}}%
5985     \ifin@           % cjk
5986     \bbl@cjkintraspace
5987     \directlua{
5988       Babel.locale_props = Babel.locale_props or {}
5989       Babel.locale_props[\the\localeid].linebreak = 'c'
5990     }%
5991     \bbl@exp{\bbl@intraspace\bbl@cl{intsp}\bbl@cl{}@@}%
5992     \ifx\bbl@KVP@intrapenalty\@nnil
5993       \bbl@intrapenalty0@@
5994     \fi
5995   \else           % sea
5996     \bbl@seaintraspace
5997     \bbl@exp{\bbl@intraspace\bbl@cl{intsp}\bbl@cl{}@@}%
5998     \directlua{
5999       Babel.sea_ranges = Babel.sea_ranges or {}
6000       Babel.set_chranges('\bbl@cl{sbcpr}',
6001         '\bbl@cl{chrng}')
6002     }%
6003     \ifx\bbl@KVP@intrapenalty\@nnil
6004       \bbl@intrapenalty0@@
6005     \fi
6006   \fi
6007 \fi
6008 \ifx\bbl@KVP@intrapenalty\@nnil\else
6009   \expandafter\bbl@intrapenalty\bbl@KVP@intrapenalty@@
6010 \fi}}

```

10.8. Arabic justification

WIP. `\bbl@arabicjust` is executed with both elongated and kashida. This must be fine tuned. The attribute `kashida` is set by transforms with `kashida`.

```

6011 \ifnum\bbl@bidimode>100 \ifnum\bbl@bidimode<200
6012 \def\bblar@chars{%
6013   0628,0629,062A,062B,062C,062D,062E,062F,0630,0631,0632,0633,%
6014   0634,0635,0636,0637,0638,0639,063A,063B,063C,063D,063E,063F,%
6015   0640,0641,0642,0643,0644,0645,0646,0647,0649}
6016 \def\bblar@elongated{%

```

```

6017 0626,0628,062A,062B,0633,0634,0635,0636,063B,%
6018 063C,063D,063E,063F,0641,0642,0643,0644,0646,%
6019 0649,064A}
6020 \begingroup
6021 \catcode`_ =11 \catcode`: =11
6022 \gdef\bblar@nofswarn{\gdef\msg_warning:nx##1##2##3{}}
6023 \endgroup
6024 \gdef\bbl@arabicjust{%
6025 \let\bbl@arabicjust\relax
6026 \newattribute\bblar@kashida
6027 \directlua{ Babel.attr_kashida = luatexbase.registernumber'bblar@kashida' }%
6028 \bblar@kashida=\z@
6029 \bbl@patchfont{\bbl@parsejalt}}%
6030 \directlua{
6031 Babel.arabic.elong_map = Babel.arabic.elong_map or {}
6032 Babel.arabic.elong_map[\the\localeid] = {}
6033 luatexbase.add_to_callback('post_linebreak_filter',
6034 Babel.arabic.justify, 'Babel.arabic.justify')
6035 luatexbase.add_to_callback('hpack_filter',
6036 Babel.arabic.justify_hbox, 'Babel.arabic.justify_hbox')
6037 }%

```

Save both node lists to make replacement.

```

6038 \def\bblar@fetchjalt#1#2#3#4{%
6039 \bbl@exp{\bbl@foreach{#1}}{%
6040 \bbl@ifunset{bblar@JE@##1}%
6041 {\setbox\z@\hbox{\texdir TRT ^^^200d\char"##1#2}}%
6042 {\setbox\z@\hbox{\texdir TRT ^^^200d\char"@nameuse{bblar@JE@##1}#2}}%
6043 \directlua{%
6044 local last = nil
6045 for item in node.traverse(tex.box[0].head) do
6046 if item.id == node.id'glyph' and item.char > 0x600 and
6047 not (item.char == 0x200D) then
6048 last = item
6049 end
6050 end
6051 Babel.arabic.#3['##1#4'] = last.char
6052 }}}

```

Elongated forms. Brute force. No rules at all, yet. The ideal: look at jalt table. And perhaps other tables (falt?, cswb?). What about kaf? And diacritic positioning?

```

6053 \gdef\bbl@parsejalt{%
6054 \ifx\addfontfeature\undefined\else
6055 \bbl@xin@{/e}{/\bbl@cl{\lnbrk}}%
6056 \ifin@
6057 \directlua{%
6058 if Babel.arabic.elong_map[\the\localeid][\fontid\font] == nil then
6059 Babel.arabic.elong_map[\the\localeid][\fontid\font] = {}
6060 tex.print([[string\cswb\space bbl@parsejalti\endcswb]])
6061 end
6062 }%
6063 \fi
6064 \fi}
6065 \gdef\bbl@parsejalti{%
6066 \begingroup
6067 \let\bbl@parsejalt\relax % To avoid infinite loop
6068 \edef\bbl@tempb{\fontid\font}%
6069 \bblar@nofswarn
6070 \@nameuse{tracinglostchars}\z@
6071 \bblar@fetchjalt\bblar@elongated{{from}}}%
6072 \bblar@fetchjalt\bblar@chars{^^^064a}{from}{a}% Alef maksura
6073 \bblar@fetchjalt\bblar@chars{^^^0649}{from}{y}% Yeh
6074 \addfontfeature{RawFeature+=jalt}%
6075 % \@namedef{bblar@JE@0643}{06AA}% todo: catch medial kaf

```



```

6076 \bblar@fetchjalt\bblar@elongated{}{dest}{}%
6077 \bblar@fetchjalt\bblar@chars{^^^064a}{dest}{a}%
6078 \bblar@fetchjalt\bblar@chars{^^^0649}{dest}{y}%
6079 \directlua{%
6080     for k, v in pairs(Babel.arabic.from) do
6081         if Babel.arabic.dest[k] and
6082             not (Babel.arabic.from[k] == Babel.arabic.dest[k]) then
6083             Babel.arabic.elong_map[\the\localeid][\bbl@tempb]
6084             [Babel.arabic.from[k]] = Babel.arabic.dest[k]
6085         end
6086     end
6087 }%
6088 \endgroup}

```

The actual justification (inspired by CHICKENIZE).

```

6089 \beginingroup
6090 \catcode`#=11
6091 \catcode`~=11
6092 \directlua{
6093
6094 Babel.arabic = Babel.arabic or {}
6095 Babel.arabic.from = {}
6096 Babel.arabic.dest = {}
6097 Babel.arabic.justify_factor = 0.95
6098 Babel.arabic.justify_enabled = true
6099 Babel.arabic.kashida_limit = -1
6100
6101 function Babel.arabic.justify(head)
6102     if not Babel.arabic.justify_enabled then return head end
6103     for line in node.traverse_id(node.id'hlist', head) do
6104         Babel.arabic.justify_hlist(head, line)
6105     end
6106     % In case the very first item is a line (eg, in \vbox):
6107     while head.prev do head = head.prev end
6108     return head
6109 end
6110
6111 function Babel.arabic.justify_hbox(head, gc, size, pack)
6112     local has_inf = false
6113     if Babel.arabic.justify_enabled and pack == 'exactly' then
6114         for n in node.traverse_id(12, head) do
6115             if n.stretch_order > 0 then has_inf = true end
6116         end
6117         if not has_inf then
6118             Babel.arabic.justify_hlist(head, nil, gc, size, pack)
6119         end
6120     end
6121     return head
6122 end
6123
6124 function Babel.arabic.justify_hlist(head, line, gc, size, pack)
6125     local d, new
6126     local k_list, k_item, pos_inline
6127     local width, width_new, full, k_curr, wt_pos, goal, shift
6128     local subst_done = false
6129     local elong_map = Babel.arabic.elong_map
6130     local cnt
6131     local last_line
6132     local GLYPH = node.id'glyph'
6133     local KASHIDA = Babel.attr_kashida
6134     local LOCALE = Babel.attr_locale
6135
6136     if line == nil then

```

```

6137     line = {}
6138     line.glue_sign = 1
6139     line.glue_order = 0
6140     line.head = head
6141     line.shift = 0
6142     line.width = size
6143 end
6144
6145 % Exclude last line.
6146 if ((line.next ~= nil or line.glue_sign == 1) and line.glue_order == 0) then
6147     elongs = {}      % Stores elongated candidates of each line
6148     k_list = {}      % And all letters with kashida
6149     pos_inline = 0   % Not yet used
6150
6151     for n in node.traverse_id(GLYPH, line.head) do
6152         pos_inline = pos_inline + 1 % To find where it is. Not used.
6153
6154         % Elongated glyphs
6155         if elong_map then
6156             local locale = node.get_attribute(n, LOCALE)
6157             if elong_map[locale] and elong_map[locale][n.font] and
6158                 elong_map[locale][n.font][n.char] then
6159                 table.insert(elongs, {node = n, locale = locale} )
6160                 node.set_attribute(n.prev, KASHIDA, 0)
6161             end
6162         end
6163
6164         % Tatwil. First create a list of nodes marked with kashida. The
6165         % rest of nodes can be ignored. The list of used weights is build
6166         % when transforms with the key kashida= are declared.
6167         if Babel.kashida_wts then
6168             local k_wt = node.get_attribute(n, KASHIDA)
6169             if k_wt > 0 then % todo. parameter for multi inserts
6170                 table.insert(k_list, {node = n, weight = k_wt, pos = pos_inline})
6171             end
6172         end
6173
6174     end % of node.traverse_id
6175
6176     if #elongs == 0 and #k_list == 0 then goto next_line end
6177     full = line.width
6178     shift = line.shift
6179     goal = full * Babel.arabic.justify_factor % A bit crude
6180     width = node.dimensions(line.head)      % The 'natural' width
6181
6182     % == Elongated ==
6183     % Original idea taken from 'chickenize'
6184     while (#elongs > 0 and width < goal) do
6185         subst_done = true
6186         local x = #elongs
6187         local curr = elongs[x].node
6188         local oldchar = curr.char
6189         curr.char = elong_map[elongs[x].locale][curr.font][curr.char]
6190         width = node.dimensions(line.head) % Check if the line is too wide
6191         % Substitute back if the line would be too wide and break:
6192         if width > goal then
6193             curr.char = oldchar
6194             break
6195         end
6196         % If continue, pop the just substituted node from the list:
6197         table.remove(elongs, x)
6198     end
6199

```

```

6200 % == Tatwil ==
6201 % Traverse the kashida node list so many times as required, until
6202 % the line is filled. The first pass adds a tatweel after each
6203 % node with kashida in the line, the second pass adds another one,
6204 % and so on. In each pass, add first the kashida with the highest
6205 % weight, then with lower weight and so on.
6206 if #k_list == 0 then goto next_line end
6207
6208 width = node.dimensions(line.head) % The 'natural' width
6209 k_curr = #k_list % Traverse backwards, from the end
6210 wt_pos = 1
6211
6212 while width < goal do
6213     subst_done = true
6214     k_item = k_list[k_curr].node
6215     if k_list[k_curr].weight == Babel.kashida_wts[wt_pos] then
6216         d = node.copy(k_item)
6217         d.char = 0x0640
6218         d.yoffset = 0 % TODO. From the prev char. But 0 seems safe.
6219         d.xoffset = 0
6220         line.head, new = node.insert_after(line.head, k_item, d)
6221         width_new = node.dimensions(line.head)
6222         if width > goal or width == width_new then
6223             node.remove(line.head, new) % Better compute before
6224             break
6225         end
6226         if Babel.fix_diacr then
6227             Babel.fix_diacr(k_item.next)
6228         end
6229         width = width_new
6230     end
6231     if k_curr == 1 then
6232         k_curr = #k_list
6233         wt_pos = (wt_pos >= table.getn(Babel.kashida_wts)) and 1 or wt_pos+1
6234     else
6235         k_curr = k_curr - 1
6236     end
6237 end
6238
6239 % Limit the number of tatweel by removing them. Not very efficient,
6240 % but it does the job in a quite predictable way.
6241 if Babel.arabic.kashida_limit > -1 then
6242     cnt = 0
6243     for n in node.traverse_id(GLYPH, line.head) do
6244         if n.char == 0x0640 then
6245             cnt = cnt + 1
6246             if cnt > Babel.arabic.kashida_limit then
6247                 node.remove(line.head, n)
6248             end
6249         else
6250             cnt = 0
6251         end
6252     end
6253 end
6254
6255 ::next_line::
6256
6257 % Must take into account marks and ins, see luatex manual.
6258 % Have to be executed only if there are changes. Investigate
6259 % what's going on exactly.
6260 if subst_done and not gc then
6261     d = node.hpack(line.head, full, 'exactly')
6262     d.shift = shift

```

```

6263      node.insert_before(head, line, d)
6264      node.remove(head, line)
6265    end
6266  end % if process line
6267 end
6268 }
6269 \endgroup
6270 \fi\fi % ends Arabic just block: \ifnum\bbl@bidimode>100...

```

10.9. Common stuff

First, a couple of auxiliary macros to set the renderer according to the script. This is done by patching temporarily the low-level fontspec macro containing the current features set with `\defaultfontfeatures`. Admittedly this is somewhat dangerous, but that way the latter command still works as expected, because the renderer is set just before other settings. In xetex they are set to `\relax`.

```

6271 \def\bbl@scr@node@list{%
6272   ,Armenian,Coptic,Cyrillic,Georgian,,Glagolitic,Gothic,%
6273   ,Greek,Latin,Old Church Slavonic Cyrillic,}
6274 \ifnum\bbl@bidimode=102 % bidi-r
6275   \bbl@add\bbl@scr@node@list{Arabic,Hebrew,Syriac}
6276 \fi
6277 \def\bbl@set@renderer{%
6278   \bbl@xin@{\bbl@cl{sname}}{\bbl@scr@node@list}%
6279   \ifin@
6280     \let\bbl@unset@renderer\relax
6281   \else
6282     \bbl@exp{%
6283       \def\\bbl@unset@renderer{%
6284         \def<g__fontspec_default_fontopts_clist>{%
6285           [g__fontspec_default_fontopts_clist]}%
6286         \def<g__fontspec_default_fontopts_clist>{%
6287           Renderer=Harfbuzz,[g__fontspec_default_fontopts_clist]}%
6288       \fi}
6289 <@Font selection@>

```

10.10. Automatic fonts and ids switching

After defining the blocks for a number of scripts (must be extended and very likely fine tuned), we define a the function `Babel.locale_map`, which just traverse the node list to carry out the replacements. The table `loc_to_scr` stores the script range for each locale (whose id is the key), copied from this table (so that it can be modified on a locale basis); there is an intermediate table named `chr_to_loc` built on the fly for optimization, which maps a char to the locale. This locale is then used to get the `\language` as stored in `locale_props`, as well as the font (as requested). In the latter table a key starting with `/` maps the font from the global one (the key) to the local one (the value). Maths are skipped and discretionaries are handled in a special way.

There are two situations where the replacement is not carried out: either the `letters` option has been set and the character is not a letter (in the $\TeX$ sense), or the current script is the same as the new one.

```

6290 \directlua{% DL6
6291 Babel.script_blocks = {
6292   ['dflt'] = {},
6293   ['Arab'] = {{0x0600, 0x06FF}, {0x08A0, 0x08FF}, {0x0750, 0x077F},
6294     {0xFE70, 0xFEFF}, {0xFB50, 0xFDFF}, {0x1EE00, 0x1EEFF}},
6295   ['Armn'] = {{0x0530, 0x058F}},
6296   ['Beng'] = {{0x0980, 0x09FF}},
6297   ['Cher'] = {{0x13A0, 0x13FF}, {0xAB70, 0xABBF}},
6298   ['Copt'] = {{0x03E2, 0x03EF}, {0x2C80, 0x2CFF}, {0x102E0, 0x102FF}},
6299   ['Cyr'] = {{0x0400, 0x04FF}, {0x0500, 0x052F}, {0x1C80, 0x1C8F},
6300     {0x2DE0, 0x2DFF}, {0xA640, 0xA69F}},
6301   ['Deva'] = {{0x0900, 0x097F}, {0xA8E0, 0xA8FF}},
6302   ['Ethi'] = {{0x1200, 0x137F}, {0x1380, 0x139F}, {0x2D80, 0x2DDF}},

```

```

6303         {0xAB00, 0xAB2F}},
6304 ['Geor'] = {{0x10A0, 0x10FF}, {0x2D00, 0x2D2F}},
6305 % Don't follow strictly Unicode, which places some Coptic letters in
6306 % the 'Greek and Coptic' block
6307 ['Grek'] = {{0x0370, 0x03E1}, {0x03F0, 0x03FF}, {0x1F00, 0x1FFF}},
6308 ['Hans'] = {{0x2E80, 0x2EFF}, {0x3000, 0x303F}, {0x31C0, 0x31EF},
6309             {0x3300, 0x33FF}, {0x3400, 0x4DBF}, {0x4E00, 0x9FFF},
6310             {0xF900, 0xFAFF}, {0xFE30, 0xFE4F}, {0xFF00, 0xFFEF},
6311             {0x20000, 0x2A6DF}, {0x2A700, 0x2B73F},
6312             {0x2B740, 0x2B81F}, {0x2B820, 0x2CEAF},
6313             {0x2CEB0, 0x2EBEF}, {0x2F800, 0x2FA1F}},
6314 ['Hebr'] = {{0x0590, 0x05FF},
6315             {0xFB1F, 0xFB4E}}, % <- Includes some <reserved>
6316 ['Jpan'] = {{0x3000, 0x303F}, {0x3040, 0x309F}, {0x30A0, 0x30FF},
6317             {0x4E00, 0x9FAF}, {0xFF00, 0xFFEF}},
6318 ['Khmr'] = {{0x1780, 0x17FF}, {0x19E0, 0x19FF}},
6319 ['Knda'] = {{0x0C80, 0x0CFF}},
6320 ['Kore'] = {{0x1100, 0x11FF}, {0x3000, 0x303F}, {0x3130, 0x318F},
6321             {0x4E00, 0x9FAF}, {0xA960, 0xA97F}, {0xAC00, 0xD7AF},
6322             {0xD7B0, 0xD7FF}, {0xFF00, 0xFFEF}},
6323 ['Lao'] = {{0x0E80, 0x0EFF}},
6324 ['Latn'] = {{0x0000, 0x007F}, {0x0080, 0x00FF}, {0x0100, 0x017F},
6325             {0x0180, 0x024F}, {0x1E00, 0x1EFF}, {0x2C60, 0x2C7F},
6326             {0xA720, 0xA7FF}, {0xAB30, 0xAB6F}},
6327 ['Mahj'] = {{0x11150, 0x1117F}},
6328 ['Mlym'] = {{0x0D00, 0x0D7F}},
6329 ['Mymr'] = {{0x1000, 0x109F}, {0xAA60, 0xAA7F}, {0xA9E0, 0xA9FF}},
6330 ['Orya'] = {{0x0B00, 0x0B7F}},
6331 ['Sinh'] = {{0x0D80, 0x0DFF}, {0x111E0, 0x111FF}},
6332 ['Syr'] = {{0x0700, 0x074F}, {0x0860, 0x086F}},
6333 ['Taml'] = {{0x0B80, 0x0BFF}},
6334 ['Telu'] = {{0x0C00, 0x0C7F}},
6335 ['Tfng'] = {{0x2D30, 0x2D7F}},
6336 ['Thai'] = {{0x0E00, 0x0E7F}},
6337 ['Tibt'] = {{0x0F00, 0x0FFF}},
6338 ['Vaii'] = {{0xA500, 0xA63F}},
6339 ['Yiii'] = {{0xA000, 0xA48F}, {0xA490, 0xA4CF}}
6340 }
6341
6342 Babel.script_blocks.Cyrs = Babel.script_blocks.Cyrl
6343 Babel.script_blocks.Hant = Babel.script_blocks.Hans
6344 Babel.script_blocks.Kana = Babel.script_blocks.Jpan
6345
6346 function Babel.locale_map(head)
6347   if not Babel.locale_mapped then return head end
6348
6349   local LOCALE = Babel.attr_locale
6350   local GLYPH = node.id('glyph')
6351   local inmath = false
6352   local toloc_save
6353   for item in node.traverse(head) do
6354     local toloc
6355     if not inmath and item.id == GLYPH then
6356       % Optimization: build a table with the chars found
6357       if Babel.chr_to_loc[item.char] then
6358         toloc = Babel.chr_to_loc[item.char]
6359       else
6360         for lc, maps in pairs(Babel.loc_to_scr) do
6361           for _, rg in pairs(maps) do
6362             if item.char >= rg[1] and item.char <= rg[2] then
6363               Babel.chr_to_loc[item.char] = lc
6364               toloc = lc
6365               break

```

```

6366         end
6367     end
6368 end
6369 % Treat composite chars in a different fashion, because they
6370 % 'inherit' the previous locale.
6371 if (item.char >= 0x0300 and item.char <= 0x036F) or
6372    (item.char >= 0x1AB0 and item.char <= 0x1AFF) or
6373    (item.char >= 0x1DC0 and item.char <= 0x1DFF) then
6374     Babel.chr_to_loc[item.char] = -2000
6375     toloc = -2000
6376 end
6377 if not toloc then
6378     Babel.chr_to_loc[item.char] = -1000
6379 end
6380 end
6381 if toloc == -2000 then
6382     toloc = toloc_save
6383 elseif toloc == -1000 then
6384     toloc = nil
6385 end
6386 if toloc and Babel.locale_props[toloc] and
6387    Babel.locale_props[toloc].letters and
6388    tex.getcatcode(item.char) \string~= 11 then
6389     toloc = nil
6390 end
6391 if toloc and Babel.locale_props[toloc].script
6392    and Babel.locale_props[node.get_attribute(item, LOCALE)].script
6393    and Babel.locale_props[toloc].script ==
6394    Babel.locale_props[node.get_attribute(item, LOCALE)].script then
6395     toloc = nil
6396 end
6397 if toloc then
6398     if Babel.locale_props[toloc].lg then
6399         item.lang = Babel.locale_props[toloc].lg
6400         node.set_attribute(item, LOCALE, toloc)
6401     end
6402     if Babel.locale_props[toloc]['/'..item.font] then
6403         item.font = Babel.locale_props[toloc]['/'..item.font]
6404     end
6405 end
6406 toloc_save = toloc
6407 elseif not inmath and item.id == 7 then % Apply recursively
6408     item.replace = item.replace and Babel.locale_map(item.replace)
6409     item.pre      = item.pre and Babel.locale_map(item.pre)
6410     item.post     = item.post and Babel.locale_map(item.post)
6411 elseif item.id == node.id'math' then
6412     inmath = (item.subtype == 0)
6413 end
6414 end
6415 return head
6416 end
6417 }

```

The code for `\babelcharproperty` is straightforward. Just note the modified lua table can be different.

```

6418 \newcommand\babelcharproperty[1]{%
6419   \count@=#1\relax
6420   \ifvmode
6421     \expandafter\bbl@chprop
6422   \else
6423     \bbl@error{charproperty-only-vertical}{}{}{}%
6424   \fi}
6425 \newcommand\bbl@chprop[3][\the\count@]{%

```

```

6426 \@tempcnta=#1\relax
6427 \bbl@ifunset{\bbl@chprop@#2}% {unknown-char-property}
6428   {\bbl@error{unknown-char-property}{\#2}}}%
6429   }%
6430 \loop
6431   \bbl@cs{chprop@#2}{#3}%
6432   \ifnum\count@<\@tempcnta
6433     \advance\count@\@ne
6434   \repeat}
6435 %
6436 \def\bbl@chprop@direction#1{%
6437   \directlua{
6438     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6439     Babel.characters[\the\count@]['d'] = '#1'
6440   }}
6441 \let\bbl@chprop@bc\bbl@chprop@direction
6442 %
6443 \def\bbl@chprop@mirror#1{%
6444   \directlua{
6445     Babel.characters[\the\count@] = Babel.characters[\the\count@] or {}
6446     Babel.characters[\the\count@]['m'] = '\number#1'
6447   }}
6448 \let\bbl@chprop@bmg\bbl@chprop@mirror
6449 %
6450 \def\bbl@chprop@linebreak#1{%
6451   \directlua{
6452     Babel.cjk_characters[\the\count@] = Babel.cjk_characters[\the\count@] or {}
6453     Babel.cjk_characters[\the\count@]['c'] = '#1'
6454   }}
6455 \let\bbl@chprop@lb\bbl@chprop@linebreak
6456 %
6457 \def\bbl@chprop@locale#1{%
6458   \directlua{
6459     Babel.chr_to_loc = Babel.chr_to_loc or {}
6460     Babel.chr_to_loc[\the\count@] =
6461       \bbl@ifblank{#1}{-1000}{\the\bbl@cs{id@#1}}\space
6462   }}

```

Post-handling hyphenation patterns for non-standard rules, like ff to ff-f. There are still some issues with speed (not very slow, but still slow). The Lua code is below.

```

6463 \directlua{% DL7
6464   Babel.nohyphenation = \the\l@nohyphenation
6465 }

```

Now the $\TeX$ high level interface, which requires the function defined above for converting strings to functions returning a string. These functions handle the $\{n\}$ syntax. For example, $\text{pre}=\{1\}\{1\}$ - becomes `function(m) return m[1]..m[1]..'-' end`, where `m` are the matches returned after applying the pattern. With a mapped capture the functions are similar to `function(m) return Babel.capt_map(m[1],1) end`, where the last argument identifies the mapping to be applied to `m[1]`. The way it is carried out is somewhat tricky, but the effect is not dissimilar to `lua load` – save the code as string in a $\TeX$ macro, and expand this macro at the appropriate place. As `\directlua` does not take into account the current catcode of `@`, we just avoid this character in macro names (which explains the internal group, too).

```

6466 \begingroup
6467 \catcode`\~ =12
6468 \catcode`\% =12
6469 \catcode`\& =14
6470 \catcode`\| =12
6471 \gdef\babelprehyphenation{%&
6472   \@ifnextchar[{\bbl@settransform{0}}{\bbl@settransform{0}}{]}
6473 \gdef\babelposthyphenation{%&
6474   \@ifnextchar[{\bbl@settransform{1}}{\bbl@settransform{1}}{]}
6475 %
6476 \gdef\bbl@settransform#1[#2]#3#4#5{%&

```

```

6477 \ifcase#1
6478   \bbl@activateprehyphen
6479 \or
6480   \bbl@activateposthyphen
6481 \fi
6482 \begingroup
6483   \def\babeltempa{\bbl@add@list\babeltempb}&%
6484   \let\babeltempb\@empty
6485   \def\bbl@tempa{#5}&%
6486   \bbl@replace\bbl@tempa{,}{,}&% TODO. Ugly trick to preserve {}
6487   \expandafter\bbl@foreach\expandafter{\bbl@tempa}{&%
6488     \bbl@ifsamestring{##1}{remove}&%
6489     {\bbl@add@list\babeltempb{nil}}&%
6490     {\directlua{
6491       local rep = [=##1]=]
6492       local three_args = '%s*=%s*([%-%d%.%a{}|]|+)%s+([%-%d%.%a{}|]|+)%s+([%-%d%.%a{}|]|+)'
6493       &% Numeric passes directly: kern, penalty...
6494       rep = rep:gsub('^%s*(remove)%s*$', 'remove = true')
6495       rep = rep:gsub('^%s*(insert)%s*', 'insert = true, ')
6496       rep = rep:gsub('^%s*(after)%s*', 'after = true, ')
6497       rep = rep:gsub('(string)%s*=%s*([^\s,]*)', Babel.capture_func)
6498       rep = rep:gsub('node%s*=%s*(%a+)%s*(%a*)', Babel.capture_node)
6499       rep = rep:gsub(' (norule)' .. three_args,
6500         'norule = {' .. '%2, %3, %4' .. '})')
6501       if #1 == 0 or #1 == 2 then
6502         rep = rep:gsub(' (space)' .. three_args,
6503           'space = {' .. '%2, %3, %4' .. '})')
6504         rep = rep:gsub(' (spacefactor)' .. three_args,
6505           'spacefactor = {' .. '%2, %3, %4' .. '})')
6506         rep = rep:gsub('(kashida)%s*=%s*([^\s,]*)', Babel.capture_kashida)
6507         &% Transform values
6508         rep, n = rep:gsub(' {[([%a%-%.]+)|([%a%_%.]+)}',
6509           function(v,d)
6510             return string.format (
6511               '{\the\csname bbl@id@@#3\endcsname,"%s",%s}',
6512               v,
6513               load( 'return Babel.locale_props'..
6514                 '[\the\csname bbl@id@@#3\endcsname].' .. d)() )
6515             end )
6516         rep, n = rep:gsub(' {[([%a%-%.]+)|([%-%d%.]+)}',
6517           '{\the\csname bbl@id@@#3\endcsname,"%1",%2}')
6518       end
6519       if #1 == 1 then
6520         rep = rep:gsub(' (no)%s*=%s*([^\s,]*)', Babel.capture_func)
6521         rep = rep:gsub(' (pre)%s*=%s*([^\s,]*)', Babel.capture_func)
6522         rep = rep:gsub(' (post)%s*=%s*([^\s,]*)', Babel.capture_func)
6523       end
6524       tex.print([[ \string\babeltempa{[]] .. rep .. [[]]])
6525     }]}&%
6526 \bbl@foreach\babeltempb{&%
6527   \bbl@forkv{##1}{&%
6528     \in@{,###1,}{,nil,step,data,remove,insert,string,no,pre,no,&%
6529     post,penalty,kashida,space,spacefactor,kern,node,after,norule,}&%
6530   \ifin@else
6531     \bbl@error{bad-transform-option}{###1}{,}{&%
6532   \fi}}&%
6533 \let\bbl@kv@attribute\relax
6534 \let\bbl@kv@label\relax
6535 \let\bbl@kv@fonts\@empty
6536 \let\bbl@kv@prepend\relax
6537 \bbl@forkv{#2}{\bbl@csarg\edef{kv###1}{##2}}&%
6538 \ifx\bbl@kv@fonts\@empty\else\bbl@settransformfont\fi
6539 \ifx\bbl@kv@attribute\relax

```



```

6540 \ifx\bbk@kv@label\relax\else
6541 \bbk@exp{\bbk@trim@def\bbk@kv@font{\bbk@kv@font}}&%
6542 \bbk@replace\bbk@kv@font{ }{,}&%
6543 \edef\bbk@kv@attribute{\bbk@ATR@bbk@kv@label @#3@bbk@kv@font}&%
6544 \count@z@
6545 \def\bbk@elt##1##2##3{&%
6546 \bbk@ifsamestring{#3,\bbk@kv@label}{##1,##2}&%
6547 {\bbk@ifsamestring{\bbk@kv@font}{##3}&%
6548 {\count@z@ne}&%
6549 {\bbk@error{font-conflict-transforms}{}}}&%
6550 }&%
6551 \bbk@transfont@list
6552 \ifnum\count@z@
6553 \bbk@exp{\global\bbk@add\bbk@transfont@list
6554 {\bbk@elt{#3}{\bbk@kv@label}{\bbk@kv@font}}}&%
6555 \fi
6556 \bbk@ifunset{\bbk@kv@attribute}&%
6557 {\global\bbk@carg\newattribute{\bbk@kv@attribute}}&%
6558 }&%
6559 \global\bbk@carg\setattribute{\bbk@kv@attribute}\@ne
6560 \fi
6561 \else
6562 \edef\bbk@kv@attribute{\expandafter\bbk@stripslash\bbk@kv@attribute}&%
6563 \fi
6564 \directlua{
6565 local lbkr = Babel.linebreaking.replacements[#1]
6566 local u = unicode.utf8
6567 local id, attr, label
6568 if #1 == 0 then
6569 id = \the\csname \bbk@id@#3\endcsname\space
6570 else
6571 id = \the\csname l@#3\endcsname\space
6572 end
6573 \ifx\bbk@kv@attribute\relax
6574 attr = -1
6575 \else
6576 attr = luatexbase.registernumber'\bbk@kv@attribute'
6577 \fi
6578 \ifx\bbk@kv@label\relax\else &% Same refs:
6579 label = [==[\bbk@kv@label]==]
6580 \fi
6581 &% Convert pattern:
6582 local patt = string.gsub([==[#4]==], '%s', '')
6583 if #1 == 0 then
6584 patt = string.gsub(patt, '|', ' ')
6585 end
6586 if not u.find(patt, '()', nil, true) then
6587 patt = '()' .. patt .. '()'
6588 end
6589 patt = string.gsub(patt, '%(%)^', '^()')
6590 patt = string.gsub(patt, '%$(%)', '()$')
6591 patt = u.gsub(patt, '{(.)}',
6592 function (n)
6593 return '%' .. (tonumber(n) and (tonumber(n)+1) or n)
6594 end)
6595 patt = u.gsub(patt, '{(%X%X%X%X+)}',
6596 function (n)
6597 return u.gsub(u.char(tonumber(n, 16)), '(%p)', '%%1')
6598 end)
6599 lbkr[id] = lbkr[id] or {}
6600 table.insert(lbkr[id], \ifx\bbk@kv@prepend\relax\else 1,\fi
6601 { label=label, attr=attr, pattern=patt, replace={\babeltempb} })
6602 }&%

```

```

6603 \endgroup}
6604 \endgroup
6605 %
6606 \let\bbl@transfont@list\@empty
6607 \def\bbl@settransfont{%
6608   \global\let\bbl@settransfont\relax % Execute only once
6609   \gdef\bbl@transfont{%
6610     \def\bbl@elt####1####2####3{%
6611       \bbl@ifblank{####3}%
6612       {\count@tw@}% Do nothing if no fonts
6613       {\count@z@
6614         \bbl@vforeach{####3}{%
6615           \def\bbl@tempd{#####1}%
6616           \edef\bbl@tempe{\bbl@transfam/\f@series/\f@shape}%
6617           \ifx\bbl@tempd\bbl@tempe
6618             \count@@ne
6619           \else\ifx\bbl@tempd\bbl@transfam
6620             \count@@ne
6621           \fi\fi}%
6622         \ifcase\count@
6623           \bbl@csarg\unsetattribute{ATR@###2@###1@###3}%
6624         \or
6625           \bbl@csarg\setattribute{ATR@###2@###1@###3}\@ne
6626         \fi}}%
6627     \bbl@transfont@list}%
6628 \AddToHook{selectfont}{\bbl@transfont}% Hooks are global.
6629 \gdef\bbl@transfam{-unknown-}%
6630 \bbl@foreach\bbl@font@fams{%
6631   \AddToHook{##1family}{\def\bbl@transfam{##1}}%
6632   \bbl@ifsamestring{\@nameuse{##1default}}\familydefault
6633   {\xdef\bbl@transfam{##1}}%
6634   {}}%
6635 %
6636 \DeclareRobustCommand\enablelocaletransform[1]{%
6637   \bbl@ifunset{\bbl@ATR@#1@\language @}%
6638   {\bbl@error{transform-not-available}{#1}{}}}%
6639   {\bbl@csarg\setattribute{ATR@#1@\language @}\@ne}}
6640 \DeclareRobustCommand\disablelocaletransform[1]{%
6641   \bbl@ifunset{\bbl@ATR@#1@\language @}%
6642   {\bbl@error{transform-not-available-b}{#1}{}}}%
6643   {\bbl@csarg\unsetattribute{ATR@#1@\language @}}}

```

The following two macros load the Lua code for transforms, but only once. The only difference is in `add_after` and `add_before`.

```

6644 \def\bbl@activateposthyphen{%
6645   \let\bbl@activateposthyphen\relax
6646   \ifx\bbl@attr@hboxed\@undefined
6647     \newattribute\bbl@attr@hboxed
6648   \fi
6649   \directlua{
6650     require('babel-transforms.lua')
6651     Babel.linebreaking.add_after(Babel.post_hyphenate_replace)
6652   }}
6653 \def\bbl@activateprehyphen{%
6654   \let\bbl@activateprehyphen\relax
6655   \ifx\bbl@attr@hboxed\@undefined
6656     \newattribute\bbl@attr@hboxed
6657   \fi
6658   \directlua{
6659     require('babel-transforms.lua')
6660     Babel.linebreaking.add_before(Babel.pre_hyphenate_replace)
6661   }}
6662 \newcommand\SetTransformValue[3]{%

```

```

6663 \directlua{
6664   Babel.locale_props[\the\csname bbl@id@@#1\endcsname].vars["#2"] = #3
6665 }}

```

The code in `babel-transforms.lua` prints at some points the current string being transformed. This macro first make sure this file is loaded. Then, activates temporarily this feature and typeset inside a box the text in the argument.

```

6666 \newcommand\ShowBabelTransforms[1]{%
6667   \bbl@activateprehyphen
6668   \bbl@activateposthyphen
6669   \begingroup
6670     \directlua{ Babel.show_transforms = true }%
6671     \setbox\z@\vbox{#1}%
6672     \directlua{ Babel.show_transforms = false }%
6673   \endgroup}

```

The following experimental (and unfinished) macro applies the prehyphenation transforms for the current locale to a string (characters and spaces) and processes it in a fully expandable way (among other limitations, the string can't contain `]`). The way it operates is admittedly rather cumbersome: it converts the string to a node list, processes it, and converts it back to a string. The lua code is in the lua file below.

```

6674 \newcommand\localeprehyphenation[1]{%
6675   \directlua{ Babel.string_prehyphenation([==#1]==], \the\localeid) }}

```

10.11.Bidi

Make sure the `\pagedir` is always the same as the `\bodydir` when the document starts. At this point the main language has been selected, and therefore its `\bodydir` has been set.

```

6676 \AtBeginDocument{\pagedir\bodydir}

```

As a first step, add a handler for bidi and digits (and potentially other processes) just before `luaotfload` is applied, which is loaded by default by `TeX`. Just in case, consider the possibility it has not been loaded.

```

6677 \def\bbl@activate@preotf{%
6678   \let\bbl@activate@preotf\relax % only once
6679   \directlua{
6680     function Babel.pre_otfload_v(head)
6681       if Babel.numbers and Babel.digits_mapped then
6682         head = Babel.numbers(head)
6683       end
6684       if Babel.bidi_enabled then
6685         head = Babel.bidi(head, false, dir)
6686       end
6687       return head
6688     end
6689     %
6690     function Babel.pre_otfload_h(head, gc, sz, pt, dir)
6691       if Babel.numbers and Babel.digits_mapped then
6692         head = Babel.numbers(head)
6693       end
6694       if Babel.bidi_enabled then
6695         head = Babel.bidi(head, false, dir)
6696       end
6697       return head
6698     end
6699     %
6700     luatexbase.add_to_callback('pre_linebreak_filter',
6701       Babel.pre_otfload_v,
6702       'Babel.pre_otfload_v',
6703       Babel.priority_in_callback('pre_linebreak_filter',
6704         'luaotfload.node_processor') or nil)
6705     %
6706     luatexbase.add_to_callback('hpack_filter',

```

```

6707     Babel.pre_otfload_h,
6708     'Babel.pre_otfload_h',
6709     Babel.priority_in_callback('hpack_filter',
6710     'luaotfload.node_processor') or nil)
6711 }}

```

The basic setup. The output is modified at a very low level to set the `\bodydir` to the `\pagedir`. Sadly, we have to deal with boxes in math with basic, so the `\bbl@insidemath` hack is activated (set to 1) every math with the package option `bidi=`. The hack for the PUA is no longer necessary with basic (24.8), but it's kept in `basic-r`.

```

6712 \ifnum\bbl@bidimode>\@ne % Any bidi= except default (=1)
6713 \let\bbl@beforeforeign\leavevmode
6714 \AtEndOfPackage{\EnableBabelHook{babel-bidi}}
6715 \RequirePackage{luatexbase}
6716 \bbl@activate@preotf
6717 \directlua{
6718     require('babel-data-bidi.lua')
6719     \ifcase\expandafter\@gobbletwo\the\bbl@bidimode\or
6720         require('babel-bidi-basic.lua')
6721     \or
6722         require('babel-bidi-basic-r.lua')
6723     table.insert(Babel.ranges, {0xE000, 0xF8FF, 'on'})
6724     table.insert(Babel.ranges, {0xF0000, 0xFFFFFD, 'on'})
6725     table.insert(Babel.ranges, {0x100000, 0x10FFFFD, 'on'})
6726 \fi}
6727 \newattribute\bbl@attr@dir
6728 \directlua{ Babel.attr_dir = luatexbase.registernumber'bbl@attr@dir' }
6729 \bbl@exp{\output{\bodydir\pagedir\the\output}}
6730 \fi
6731 %
6732 \chardef\bbl@thetextdir\z@
6733 \chardef\bbl@thepardir\z@
6734 \def\bbl@setluadir#1#2{% l=\text/pardirection 2= 0:l 1:r 2:al
6735     \ifcase#2\relax
6736         \ifcase#1\else#1=\z@\fi
6737     \else
6738         \ifcase#1#1=\@ne\fi
6739     \fi}

```

`\bbl@attr@dir` stores the directions with a mask: `..00PPTT`, with masks `0xC` (`PP` is the `par dir`) and `0x3` (`TT` is the `text dir`). These macro names are shared by the 3 engines, with different definitions.

```

6740 \def\bbl@thedir{0}
6741 \def\bbl@textdir#1{%
6742     \bbl@setluadir\textdirection{#1}%
6743     \chardef\bbl@thetextdir#1\relax
6744     \edef\bbl@thedir{\the\numexpr\bbl@thepardir*4+#1}%
6745     \setattribute\bbl@attr@dir{\numexpr\bbl@thepardir*4+#1}%
6746 \def\bbl@pardir#1{% Used twice
6747     \bbl@setluadir\pardirection{#1}%
6748     \chardef\bbl@thepardir#1\relax}
6749 \def\bbl@bodydir{\bbl@setluadir\bodydirection}% Used once
6750 \def\bbl@dirparastext{\pardirection=\textdirection\relax}% Used once

```

RTL text inside math needs special attention. It affects not only to actual math stuff, but also to ‘tabular’, which is based on a fake math.

```

6751 \ifnum\bbl@bidimode>\z@ % Any bidi=
6752     \def\bbl@insidemath{0}%
6753     \def\bbl@everymath{\def\bbl@insidemath{1}}
6754     \def\bbl@everydisplay{\def\bbl@insidemath{2}}
6755     \frozen@everymath\expandafter{%
6756         \expandafter\bbl@everymath\the\frozen@everymath}
6757     \frozen@everydisplay\expandafter{%
6758         \expandafter\bbl@everydisplay\the\frozen@everydisplay}
6759     \AtBeginDocument{

```

```

6760 \directlua{
6761     function Babel.math_box_dir(head)
6762         if not (token.get_macro('bbl@insidemath') == '0') then
6763             if Babel.hlist_has_bidi(head) then
6764                 local d = node.new(node.id'dir')
6765                 d.dir = '+TRT'
6766                 for item in node.traverse(head) do
6767                     if item.id == 11 or item.id == node.id'glyph' then
6768                         head = node.insert_before(head, item, d)
6769                         break
6770                     end
6771                 end
6772                 local inmath = false
6773                 for item in node.traverse(head) do
6774                     if item.id == 11 then
6775                         inmath = (item.subtype == 0)
6776                     elseif not inmath then
6777                         node.set_attribute(item,
6778                             Babel.attr_dir, token.get_macro('bbl@thedir'))
6779                     end
6780                 end
6781             end
6782         end
6783         return head
6784     end
6785     luatexbase.add_to_callback("hpack_filter", Babel.math_box_dir,
6786         "Babel.math_box_dir", 0)
6787     if Babel.unset_atdir then
6788         luatexbase.add_to_callback("pre_linebreak_filter", Babel.unset_atdir,
6789             "Babel.unset_atdir")
6790         luatexbase.add_to_callback("hpack_filter", Babel.unset_atdir,
6791             "Babel.unset_atdir")
6792     end
6793     luatexbase.add_to_callback("post_mlist_to_hlist_filter", function(head)
6794         local dir = tex.mathdirection
6795         local n = node.new("dir")
6796         n.direction = dir
6797         head = node.insert_before(head, head, n)
6798         n = node.new("dir", 1)
6799         n.direction = dir
6800         head = node.insert_after(head, node.tail(head), n)
6801         return head
6802     end, "Babel.ensure_math_dir")
6803 } }%
6804 \fi

```

Experimental. Tentative name.

```

6805 \DeclareRobustCommand\localebox[1]{%
6806     {\def\bbl@insidemath{0}%
6807         \mbox{\foreignlanguage{\language}{\#1}}}}

```

10.12 Layout

Unlike xetex, luatex requires only minimal changes for right-to-left layouts, particularly in monolingual documents (the engine itself reverses boxes – including column order or headings –, margins, etc.) with `bidi=basic`, without having to patch almost any macro where text direction is relevant.

Still, there are three areas deserving special attention, namely, tabular, math, and graphics, text and intrinsically left-to-right elements are intermingled. I've made some progress in graphics, but they're essentially hacks; I've also made some progress in 'tabular', but when I decided to tackle math (both standard math and 'amsmath') the nightmare began. I'm still not sure how 'amsmath' should be modified, but the main problem is that, boxes are "generic" containers that can hold text,

math, and graphics (even at the same time; remember that inline math is included in the list of text nodes marked with 'math' (11) nodes too).

`\@hangfrom` is useful in many contexts and it is redefined always with the `layout` option.

There are, however, a number of issues when the text direction is not the same as the box direction (as set by `\bodydir`), and when `\parbox` and `\hangindent` are involved. Fortunately, latest releases of `luatex` simplify a lot the solution with `\shapemode`.

With the issue #15 I realized commands are best patched, instead of redefined. With a few lines, a modification could be applied to several classes and packages. Now, `tabular` seems to work (at least in simple cases) with `array`, `tabularx`, `hline`, `colortbl`, `longtable`, `booktabs`, etc. However, `dcolumn` still fails.

```
6808 \bbl@trace{Redefinitions for bidi layout}
6809 %
6810 ({*More package options}) ≡
6811 \chardef\bbl@eqnpos\z@
6812 \DeclareOption{leqno}{\chardef\bbl@eqnpos\@ne}
6813 \DeclareOption{fleqn}{\chardef\bbl@eqnpos\tw@}
6814 ({/More package options})
```

Fixes in `luatex` are sometimes done with flags. We first activate all all them.

```
6815 \ifnum\bbl@bidimode>\z@ % Any bidi=
6816   \matheqdirmode\@ne
6817   \mathemptydisplaymode\@ne
6818   \breakafterdirmode\@ne
6819   \fixupboxesmode\@ne
6820   \let\bbl@eqnodir\relax
6821   \def\bbl@eqdel{()}
6822   \def\bbl@eqnum{%
6823     {\normalfont\normalcolor
6824       \expandafter\@firstoftwo\bbl@eqdel
6825       \theequation
6826       \expandafter\@secondoftwo\bbl@eqdel}}
6827   \def\bbl@puteqno#1{\eqno\hbox{#1}}
6828   \def\bbl@putleqno#1{\leqno\hbox{#1}}
6829   \def\bbl@eqno@flip#1{% So
6830     \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6831       \eqno
6832       \hb@xt@.01pt{%
6833         \hb@xt@\displaywidth{\hss{#1}\glet\bbl@upset\@currentlabel}}\hss}%
6834     \else
6835       \leqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6836     \fi
6837     \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}
6838   \def\bbl@leqno@flip#1{%
6839     \ifdim\predisplaysize=-\maxdimen % For consecutive displays
6840       \leqno
6841       \hb@xt@.01pt{%
6842         \hss\hb@xt@\displaywidth{\[bbl@upset\@currentlabel]\hss}}%
6843     \else
6844       \eqno\hbox{#1}\glet\bbl@upset\@currentlabel}%
6845     \fi
6846     \bbl@exp{\def\\@currentlabel{\[bbl@upset]}}
6847 %
6848 \AtBeginDocument{%
6849   \ifx\bbl@noamsmath\relax\else
6850     \ifx\maketag@@@undefined % Normal equation, eqnarray
6851       \ifnum\bbl@eqnpos=\tw@
6852         \bbl@replace\equation{\hb@xt@\linewidth}%
6853           {\hbox bdir\mathdirection to\linewidth}%
6854         \bbl@carg\bbl@sreplace{[ ]}{\hb@xt@\linewidth}%
6855           {\hbox bdir\mathdirection to\linewidth}%
6856       \fi
6857       \AddToHook{env/equation/begin}{%
6858         \ifnum\bbl@thetextdir>\z@
```

```

6859 \let\@eqnnum\bbledqnum
6860 \edef\bbledqnodir{\noexpand\bbledtextdir{\the\bbledthetextdir}}%
6861 \chardef\bbledthetextdir\z@
6862 \bbledadd\normalfont{\bbledqnodir}%
6863 \ifcase\bbledqnpos
6864 \let\bbledputeqno\bbledeqnoflip
6865 \or
6866 \let\bbledputeqno\bbledleqnoflip
6867 \fi
6868 \fi}%
6869 \ifnum\bbledqnpos=\tw@else
6870 \def\endequation{\bbledputeqno{\@eqnnum}$$\@ignoretrue}%
6871 \fi
6872 \AddToHook{env/eqnarray/begin}{%
6873 \ifnum\bbledthetextdir>\z@
6874 \edef\bbledqnodir{\noexpand\bbledtextdir{\the\bbledthetextdir}}%
6875 \chardef\bbledthetextdir\z@
6876 \bbledadd\normalfont{\bbledqnodir}%
6877 \ifnum\bbledqnpos=\@ne
6878 \def\@eqnnum{%
6879 \setbox\z@\hbox{\bbledeqnum}%
6880 \hbox to0.01pt{\hss\hbox to\displaywidth{\box\z@\hss}}}%
6881 \else
6882 \let\@eqnnum\bbledeqnum
6883 \fi
6884 \fi}
6885 \else % amstex
6886 \bbledexp{% Hack to hide maybe undefined conditionals:
6887 \chardef\bbledqnpos=0<\iftagsleft>1<\else>\<\if@flegn>2<\fi><\fi>\relax}%
6888 \ifnum\bbledqnpos=\@ne
6889 \let\bbledams@lap\hbox
6890 \else
6891 \let\bbledams@lap\llap
6892 \fi
6893 \ExplSyntaxOn % Required by \bbledsreplace with \intertext@
6894 \bbledsreplace\intertext@{\normalbaselines}%
6895 {\normalbaselines
6896 \ifx\bbledqnodir\relax\else\bbledpaddir\@ne\bbledqnodir\fi}%
6897 \ExplSyntaxOff
6898 \def\bbledams@tagbox#1#2{\bbledqnodir#2}% #1=hbox|@lap|flip
6899 \ifx\bbledams@lap\hbox % leqno
6900 \def\bbledams@flip#1{%
6901 \hbox to 0.01pt{\hss\hbox to\displaywidth{{#1}\hss}}}%
6902 \else % eqno
6903 \def\bbledams@flip#1{%
6904 \hbox to 0.01pt{\hbox to\displaywidth{\hss{#1}\hss}}}%
6905 \fi
6906 \def\bbledams@preset#1{%
6907 \ifnum\bbledthetextdir>\z@
6908 \edef\bbledqnodir{\noexpand\bbledtextdir{\the\bbledthetextdir}}%
6909 \bbledsreplace\textdef@{\hbox}{\bbledams@tagbox\hbox}%
6910 \bbledsreplace\maketag@@@{\hbox}{\bbledams@tagbox#1}%
6911 \fi}%
6912 \def\bbledams@equation{%
6913 \ifnum\bbledthetextdir>\z@
6914 \bbledams@preset\hbox
6915 \chardef\bbledthetextdir\z@
6916 \bbledadd\normalfont{\bbledqnodir}%
6917 \ifcase\bbledqnpos
6918 \def\veqno##1##2{\bbledeqnoflip{##1##2}}%
6919 \or
6920 \def\veqno##1##2{\bbledleqnoflip{##1##2}}%
6921 \fi

```

```

6922     \fi}%
6923     \AddToHook{env/equation/begin}{\bbl@ams@equation}%
6924     \AddToHook{env/equation*/begin}{\bbl@ams@equation}%
6925     \AddToHook{env/cases/begin}{\bbl@ams@preset\bbl@ams@lap}%
6926     \AddToHook{env/multline/begin}{\bbl@ams@preset\bbl@ams@lap}%
6927     \AddToHook{env/gather/begin}{\bbl@ams@preset\bbl@ams@lap}%
6928     \AddToHook{env/gather*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6929     \AddToHook{env/align/begin}{\bbl@ams@preset\bbl@ams@lap}%
6930     \AddToHook{env/align*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6931     \AddToHook{env/alignat/begin}{\bbl@ams@preset\bbl@ams@lap}%
6932     \AddToHook{env/alignat*/begin}{\bbl@ams@preset\bbl@ams@lap}%
6933     \AddToHook{env/eqnalign/begin}{\bbl@ams@preset\bbl@ams@lap}%
6934     % Hackish, for proper alignment. Don't ask me why it works!:
6935     \bbl@exp{% Avoid a 'visible' conditional
6936         \\\AddToHook{env/align*/end}{\<iftag>\<else>\\tag*{}\<fi>}%
6937         \\\AddToHook{env/alignat*/end}{\<iftag>\<else>\\tag*{}\<fi>}}%
6938     \AddToHook{env/flalign/begin}{\bbl@ams@preset\bbl@ams@lap}%
6939     \AddToHook{env/split/before}{%
6940         \ifnum\bbl@thetextdir>\z@
6941             \bbl@ifsamestring\@currentenv{equation}%
6942             {\ifx\bbl@ams@lap\hbox % leqno
6943                 \def\bbl@ams@flip#1{%
6944                     \hbox to 0.01pt{\hbox to\displaywidth{{#1}\hss}\hss}}%
6945                 \else
6946                     \def\bbl@ams@flip#1{%
6947                         \hbox to 0.01pt{\hss\hbox to\displaywidth{\hss{#1}}}}%
6948                 \fi}%
6949             {}%
6950         \fi}%
6951     \fi\fi}
6952 \fi

```

Declarations specific to lua, called by \babelprovide.

```

6953 \def\bbl@provide@extra#1{%
6954     % == onchar ==
6955     \ifx\bbl@KVP@onchar\@nnil\else
6956         \bbl@luahyphenate
6957         \bbl@exp{%
6958             \\\AddToHook{env/document/before}{%
6959                 {\let\\bbl@ifrestoring\\@firstoftwo
6960                     \\\select@language{#1}}}%
6961             \directlua{
6962                 if Babel.locale_mapped == nil then
6963                     Babel.locale_mapped = true
6964                     Babel.linebreaking.add_before(Babel.locale_map, 1)
6965                     Babel.loc_to_scr = {}
6966                     Babel.chr_to_loc = Babel.chr_to_loc or {}
6967                 end
6968                 Babel.locale_props[\the\localeid].letters = false
6969             }%
6970             \bbl@xin@{ letters }{ \bbl@KVP@onchar\space}%
6971             \ifin@
6972                 \directlua{
6973                     Babel.locale_props[\the\localeid].letters = true
6974                 }%
6975             \fi
6976             \bbl@xin@{ ids }{ \bbl@KVP@onchar\space}%
6977             \ifin@
6978                 \ifx\bbl@starthyphens\@undefined % Needed if no explicit selection
6979                     \AddBabelHook{babel-onchar}{beforestart}{\bbl@starthyphens}}%
6980             \fi
6981             \bbl@exp{\\bbl@add\\bbl@starthyphens
6982                 {\bbl@patterns@lua{\language\language}}}%

```



```

6983 \directlua{
6984   if Babel.script_blocks['\bbl@cl{sbc}'] then
6985     Babel.loc_to_scr[\the\localeid] = Babel.script_blocks['\bbl@cl{sbc}']
6986     Babel.locale_props[\the\localeid].lg = \the@nameuse{l@{language}}\space
6987   end
6988 }%
6989 \fi
6990 \bbl@xin@{ fonts }{ \bbl@KVP@onchar\space}%
6991 \ifin@
6992   \bbl@ifunset{bbl@lsys@{language}}{ \bbl@provide@lsys@{language}}{}%
6993   \bbl@ifunset{bbl@wdir@{language}}{ \bbl@provide@dirs@{language}}{}%
6994   \directlua{
6995     if Babel.script_blocks['\bbl@cl{sbc}'] then
6996       Babel.loc_to_scr[\the\localeid] =
6997         Babel.script_blocks['\bbl@cl{sbc}']
6998     end}%
6999   \ifx\bbl@mapselect@undefined
7000     \AtBeginDocument{%
7001       \bbl@patchfont{ \bbl@mapselect}%
7002       { \selectfont}%
7003       \def\bbl@mapselect{%
7004         \let\bbl@mapselect\relax
7005         \edef\bbl@prefontid{\fontid\font}}%
7006       \def\bbl@mapdir##1{%
7007         \begingroup
7008           \setbox\z@ \hbox{% Force text mode
7009             \def\language{##1}%
7010             \let\bbl@ifrestoring\@firstoftwo % To avoid font warning
7011             \bbl@switchfont
7012             \ifnum\fontid\font>\z@ % A hack, for the pgf nullfont hack
7013               \directlua{
7014                 Babel.locale_props[\the\csname bbl@id@##1\endcsname]%
7015                   ['/\bbl@prefontid'] = \fontid\font\space}%
7016               \fi}%
7017             \endgroup}%
7018           \fi
7019           \bbl@exp{ \bbl@add \bbl@mapselect{ \bbl@mapdir{ \language} } }%
7020         \fi
7021       \fi
7022       % == mapfont ==
7023       % For bidi texts, to switch the font based on direction. Deprecated
7024       \ifx\bbl@KVP@mapfont\@nnil\else
7025         \bbl@ifsamestring{ \bbl@KVP@mapfont }{ direction }{}%
7026         { \bbl@error{unknown-mapfont} }{}{}%
7027         \bbl@ifunset{ bbl@lsys@{language} }{ \bbl@provide@lsys@{language} }{}%
7028         \bbl@ifunset{ bbl@wdir@{language} }{ \bbl@provide@dirs@{language} }{}%
7029         \ifx\bbl@mapselect@undefined
7030           \AtBeginDocument{%
7031             \bbl@patchfont{ \bbl@mapselect}%
7032             { \selectfont}%
7033             \def\bbl@mapselect{%
7034               \let\bbl@mapselect\relax
7035               \edef\bbl@prefontid{\fontid\font}}%
7036             \def\bbl@mapdir##1{%
7037               \def\language{##1}%
7038               \let\bbl@ifrestoring\@firstoftwo % avoid font warning
7039               \bbl@switchfont
7040               \directlua{Babel.fontmap
7041                 [\the\csname bbl@wdir@##1\endcsname]%
7042                 [\bbl@prefontid]=\fontid\font}}}%
7043             \fi
7044             \bbl@exp{ \bbl@add \bbl@mapselect{ \bbl@mapdir{ \language} } }%
7045           \fi

```

```

7046 % == Line breaking: CJK quotes ==
7047 \ifcase\bbl@engine\or
7048   \bbl@xin@{/c}{/\bbl@cl{l\brk}}%
7049   \ifin@
7050     \bbl@ifunset{\bbl@quote@\languagename}{}%
7051     {\directlua{
7052       Babel.locale_props[\the\localeid].cjk_quotes = {}
7053       local cs = 'op'
7054       for c in string.utfvalues(
7055         [[\csname \bbl@quote@\languagename\endcsname]]) do
7056         if Babel.cjk_characters[c].c == 'qu' then
7057           Babel.locale_props[\the\localeid].cjk_quotes[c] = cs
7058         end
7059         cs = ( cs == 'op') and 'cl' or 'op'
7060       end
7061     }}%
7062   \fi
7063 \fi
7064 % == Counters: mapdigits ==
7065 % Native digits
7066 \ifx\bbl@KVP@mapdigits\@nnil\else
7067   \bbl@ifunset{\bbl@dgnat@\languagename}{}%
7068   {\bbl@activate@preotf
7069     \directlua{
7070       Babel.digits_mapped = true
7071       Babel.digits = Babel.digits or {}
7072       Babel.digits[\the\localeid] =
7073         table.pack(string.utfvalue('\bbl@cl{dgnat}'))
7074       if not Babel.numbers then
7075         function Babel.numbers(head)
7076           local LOCALE = Babel.attr_locale
7077           local GLYPH = node.id'glyph'
7078           local inmath = false
7079           for item in node.traverse(head) do
7080             if not inmath and item.id == GLYPH then
7081               local temp = node.get_attribute(item, LOCALE)
7082               if Babel.digits[temp] then
7083                 local chr = item.char
7084                 if chr > 47 and chr < 58 then
7085                   item.char = Babel.digits[temp][chr-47]
7086                 end
7087               end
7088             elseif item.id == node.id'math' then
7089               inmath = (item.subtype == 0)
7090             end
7091           end
7092           return head
7093         end
7094       end
7095     }}%
7096   \fi
7097 % == transforms ==
7098 \ifx\bbl@KVP@transforms\@nnil\else
7099   \def\bbl@elt##1##2##3{%
7100     \in@{${transforms.}}{##1}%
7101     \ifin@
7102       \def\bbl@tempa{##1}%
7103       \bbl@replace\bbl@tempa{transforms.}{}%
7104       \bbl@carg\bbl@transforms{babel\bbl@tempa}{##2}{##3}%
7105     \fi}%
7106   \bbl@exp{%
7107     \\bbl@ifblank{\bbl@cl{dgnat}}}%
7108     {\let\\bbl@tempa\relax}%

```

```

7109      {\def\\bbl@tempa{%
7110        \\bbl@elt{transforms.prehyphenation}%
7111        {digits.native.1.0}{([0-9])}%
7112        \\bbl@elt{transforms.prehyphenation}%
7113        {digits.native.1.1}{string={1\string|0123456789\string|\bbl@cl{dgnat}}}}}%
7114      \ifx\bbl@tempa\relax\else
7115        \toks@{\expandafter\expandafter\expandafter{%
7116          \csname bbl@inidata@\language\endcsname}%
7117          \bbl@csarg\edef{inidata@\language}{%
7118            \unexpanded\expandafter{\bbl@tempa}%
7119            \the\toks@}%
7120        \fi
7121        \csname bbl@inidata@\language\endcsname
7122        \bbl@release@transforms\relax % \relax closes the last item.
7123      \fi}

```

Start tabular here:

```

7124 \def\localerestoredirs{%
7125   \ifcase\bbl@thetextdir
7126     \ifnum\textdirection=\z@\else\textdirection=\z@\fi
7127   \else
7128     \ifnum\textdirection=\@ne\else\textdirection=\@ne\fi
7129   \fi
7130   \ifcase\bbl@thepardir
7131     \ifnum\pardirection=\z@\else\pardirection=\z@\bodydirection=\z@\fi
7132   \else
7133     \ifnum\pardirection=\@ne\else\pardirection=\@ne\bodydirection=\@ne\fi
7134   \fi}
7135 %
7136 \IfBabelLayout{tabular}%
7137   {\chardef\bbl@tabular@mode\z@}% All RTL
7138   {\IfBabelLayout{lrtabular}%
7139     {\chardef\bbl@tabular@mode\@ne}%
7140     {\chardef\bbl@tabular@mode\z@}}% Mixed, with LTR cols
7141 %
7142 \ifnum\bbl@bidimode>\@ne % Any lua bidi= except default=1
7143   % Redefine: vrules mess up dirs (why?).
7144   \AtBeginDocument{\def\@arstrut{\relax\copy\@arstrutbox}}%
7145   \ifcase\bbl@tabular@mode\else % 1 = Mixed
7146     \let\bbl@parabefore\relax
7147     \AddToHook{para/before}{\bbl@parabefore}
7148     \AtBeginDocument{%
7149       \bbl@replace\@tabular{\hbox\bgroup}{\hbox bdir\z@\bgroup}%
7150       \ifnum\bbl@tabular@mode=\@ne
7151         \bbl@ifunset{\@tabclassz}{\fi}%
7152         \bbl@exp{% Hide conditionals
7153           \\bbl@sreplace\\ \@tabclassz
7154             {\<ifcase>\\ \@chnum}%
7155             {\\ \localerestoredirs\<ifcase>\\ \@chnum}}}%
7156         \@ifpackageloaded{colortbl}%
7157           {\bbl@sreplace\@classz
7158             {\hbox\bgroup\bgroup}{\hbox\bgroup\bgroup\localerestoredirs}}%
7159         {\@ifpackageloaded{array}%
7160           {\bbl@exp{% Hide conditionals
7161             \\bbl@sreplace\\ \@classz
7162               {\<ifcase>\\ \@chnum}%
7163               {\bgroup\\ \localerestoredirs\<ifcase>\\ \@chnum}%
7164               \\bbl@sreplace\\ \@classz
7165               {\do@row@strut\<fi>}{\do@row@strut\<fi>\egroup}}}%
7166           {}}%
7167       \fi}%
7168   \fi

```

Very likely the \output routine must be patched in a quite general way to make sure the \bodydir

is set to \pagedir. Note outside \output they can be different (and often are). For the moment, two *ad hoc* changes.

```

7169 \AtBeginDocument{%
7170   \@ifpackageloaded{multicol}%
7171     {\toks@ \expandafter{\multi@column@out}%
7172      \edef\multi@column@out{\bodydir\pagedir\the\toks@}}%
7173     {}%
7174   \@ifpackageloaded{paracol}%
7175     {\edef\pcol@output{%
7176      \bodydir\pagedir\unexpanded\expandafter{\pcol@output}}}%
7177     {}}%
7178 \fi

```

Finish here if there in no layout.

```

7179 \ifx\bbl@opt@layout\@nnil\endinput\fi

```

\hangindent does not honour direction changes by default, so we need to redefine \@hangfrom.

```

7180 \chardef\bbl@trace@vboxdir\z@
7181 \ifnum\bbl@bidimode>\z@ % Any bidi=
7182   \IfBabelLayout{nopars}{}
7183   {\edef\bbl@opt@layout{\bbl@opt@layout.pars.}}%
7184   \IfBabelLayout{pars}
7185     {\chardef\bbl@trace@vboxdir\@ne
7186      \def\@hangfrom#1{%
7187        \setbox\@tempboxa\hbox{#1}}%
7188        \hangindent\wd\@tempboxa
7189        \ifnum\bbl@vbox@bodydir=\pardirection\else
7190          \shapemode\@ne
7191          \fi
7192          \noindent\box\@tempboxa}}
7193   {}
7194 \fi

```

We need to patch lists in documents with both LTR and RTL paragraphs. See issue #395 in GitHub. There was a partial solution, but a better one has been devised by Udi Fogiel (in 26.4).

```

7195 \IfBabelLayout{lists}
7196   {\chardef\bbl@trace@vboxdir\@ne
7197    \let\bbl@OL@list\list
7198    \bbl@sreplace\list{\parshape}{\bbl@listparshape}%
7199    \let\bbl@NL@list\list
7200    \def\bbl@listparshape#1#2#3{%
7201      \parshape #1 #2 #3 %
7202      \ifnum\bbl@vbox@bodydir=\pardirection\else
7203        \shapemode\tw@
7204        \fi}}
7205   {}
7206 %
7207 \ifcase\bbl@trace@vboxdir\else
7208   \AtBeginDocument{\chardef\bbl@vbox@bodydir\pagedirection}%
7209   \def\bbl@vbox@lists{\chardef\bbl@vbox@bodydir\bodydirection}%
7210   \let\bbl@bidi@vbox\everyvbox
7211   \@nameuse{newtoks}\everyvbox % \outer in Plain
7212   \everyvbox\expandafter{\the\bbl@bidi@vbox}%
7213   \bbl@bidi@vbox{\bbl@vbox@lists\the\everyvbox}%
7214 \fi
7215 %
7216 \IfBabelLayout{graphics}
7217   {\let\bbl@pictresetdir\relax
7218    \def\bbl@pictsetdir#1{%
7219      \ifcase\bbl@thetextdir
7220        \let\bbl@pictresetdir\relax
7221        \else
7222          \ifcase#1\bodydir TLT % Remember this sets the inner boxes

```

```

7223         \or\textdir TLT
7224         \else\bodydir TLT \textdir TLT
7225     \fi
7226     % \(\text|par)dir required in pgf:
7227     \def\bbl@pictresetdir{\bodydir TRT\pardir TRT\textdir TRT\relax}%
7228     \fi}%
7229 \AddToHook{env/picture/begin}{\bbl@pictsetdir\tw@}%
7230 \directlua{
7231     Babel.get_picture_dir = true
7232     Babel.picture_has_bidi = 0
7233     %
7234     function Babel.picture_dir (head)
7235         if not Babel.get_picture_dir then return head end
7236         if Babel.hlist_has_bidi(head) then
7237             Babel.picture_has_bidi = 1
7238         end
7239         return head
7240     end
7241     luatexbase.add_to_callback("hpack_filter", Babel.picture_dir,
7242         "Babel.picture_dir")
7243 }%
7244 \AddToHook{package/graphics/after}{%
7245     \bbl@exp{%
7246         \\bbl@sreplace\\rotatebox{\hbox{{\string#2}}}
7247         {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}}}%
7248 \AddToHook{package/graphicx/after}{%
7249     \bbl@exp{%
7250         \\bbl@sreplace\\Grot@box@std{\hbox{{\string#2}}}
7251         {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#2}}}%
7252         \\bbl@sreplace\\Grot@box@kv{\hbox{\string#3}}
7253         {\hbox bdir\z@{\hbox bdir<ifcase>\bbl@thetextdir\z@<else>\@ne<fi>{\string#3}}}%
7254         \\bbl@sreplace\\Grot@box{\hbox}{\hbox bdir\z@}}}%
7255 \AtBeginDocument{%
7256     \long\def\put(#1,#2)#3{%
7257         \@killglue
7258         % Try:
7259         \ifx\bbl@pictresetdir\relax
7260             \def\bbl@tempc{0}%
7261         \else
7262             \directlua{
7263                 Babel.get_picture_dir = true
7264                 Babel.picture_has_bidi = 0
7265             }%
7266             \setbox\z@\hb@xt\z@{%
7267                 \@defaultunitsset\@tempdimc{#1}\unitlength
7268                 \kern\@tempdimc
7269                 #3\hss}%
7270             \edef\bbl@tempc{\directlua{tex.print(Babel.picture_has_bidi)}}%
7271         \fi
7272         % Do:
7273         \@defaultunitsset\@tempdimc{#2}\unitlength
7274         \raise\@tempdimc\hb@xt\z@{%
7275             \@defaultunitsset\@tempdimc{#1}\unitlength
7276             \kern\@tempdimc
7277             {\ifnum\bbl@tempc>\z@\bbl@pictresetdir\fi#3}\hss}%
7278         \ignorespaces}%
7279     \MakeRobust\put}%
7280 \AtBeginDocument
7281     {\AddToHook{cmd/diagbox@pict/before}{\let\bbl@pictsetdir\@gobble}%
7282     \ifx\pgfpicture\undefined\else
7283         \AddToHook{env/pgfpicture/begin}{\bbl@pictsetdir\@ne}%
7284         \bbl@add\pgfinterruptpicture{\bbl@pictresetdir}%
7285         \bbl@add\pgfsys@beginpicture{\bbl@pictsetdir\z@}%

```

```

7286 \fi
7287 \ifx\tikzpicture\undefined\else
7288 \AddToHook{env/tikzpicture/begin}{\bbl@pictsetdir\tw@}%
7289 \bbl@add\tikz@atbegin@node{\bbl@pictresetdir}%
7290 \bbl@sreplace\tikz{\beginpgroup}{\beginpgroup\bbl@pictsetdir\tw@}%
7291 \bbl@sreplace\tikzpicture{\beginpgroup}{\beginpgroup\bbl@pictsetdir\tw@}%
7292 \fi
7293 }}
7294 {}

```

Implicitly reverses sectioning labels in `bidi=basic-r`, because the full stop is not in contact with L numbers any more. I think there must be a better way. Assumes `bidi=basic`, but there are some additional readjustments for `bidi=default`.

```

7295 \IfBabelLayout{counters*}%
7296 {\bbl@add\bbl@opt@layout{.counters.}%
7297 \directlua{
7298 \luaexecbase.add_to_callback("process_output_buffer",
7299 \Babel.discard_sublr , "Babel.discard_sublr") }%
7300 }{}
7301 \IfBabelLayout{counters}%
7302 {\let\bbl@latinarabic=\@arabic
7303 \let\bbl@0L@arabic=\@arabic
7304 \def\@arabic#1{\babelsublr{\bbl@latinarabic#1}}%
7305 \ifpackagewith{babel}{bidi=default}%
7306 {\let\bbl@asciroman=\@roman
7307 \let\bbl@0L@roman=\@roman
7308 \def\@roman#1{\babelsublr{\ensureascii{\bbl@asciroman#1}}}%
7309 \let\bbl@asciiRoman=\@Roman
7310 \let\bbl@0L@roman=\@Roman
7311 \def\@Roman#1{\babelsublr{\ensureascii{\bbl@asciiRoman#1}}}%
7312 \let\bbl@0L@labelenumii\labelenumii
7313 \def\labelenumii{\theenumii}%
7314 \let\bbl@0L@p@enumiii\p@enumiii
7315 \def\p@enumiii{\p@enumii}\theenumii{}}{}

```

Some $\TeX$ macros use internally the math mode for text formatting. They have very little in common and are grouped here, as a single option.

```

7316 \IfBabelLayout{extras}%
7317 {\bbl@ncarg\let\bbl@0L@underline{underline }%
7318 \bbl@carg\bbl@sreplace{underline }{\hbox}%
7319 {\def\bbl@insidemath{0}\hbox bdir\ifcase\bbl@thetextdir\z@else\@ne\fi}%
7320 \let\bbl@0L@LaTeXe\LaTeXe
7321 \DeclareRobustCommand{\LaTeXe}{\mbox{\m@th
7322 \if b\expandafter\@car\series\@nil\boldmath\fi
7323 \babelsublr{\LaTeX\kern.15em2$\textstyle\varepsilon$}}}
7324 {}
7325  $\text{\textcolor{red}{/luatex}}$ 

```

10.13 Lua: transforms

After declaring the table containing the patterns with their replacements, we define some auxiliary functions: `str_to_nodes` converts the string returned by a function to a node list, taking the node at base as a model (font, language, etc.); `fetch_word` fetches a series of glyphs and discretionary, which pattern is matched against (if there is a match, it is called again before trying other patterns, and this is very likely the main bottleneck).

`post_hyphenate_replace` is the callback applied after `lang.hyphenate`. This means the automatic hyphenation points are known. As empty captures return a byte position (as explained in the `luatex` manual), we must convert it to a utf8 position. With `first`, the last byte can be the leading byte in a utf8 sequence, so we just remove it and add 1 to the resulting length. With `last` we must take into account the capture position points to the next character. Here `word_head` points to the starting node of the text to be matched.

```

7326  $\text{\textcolor{red}{(*transforms)}}$ 
7327 Babel.linebreaking.replacements = {}

```

```

7328 Babel.linebreaking.replacements[0] = {} -- pre
7329 Babel.linebreaking.replacements[1] = {} -- post
7330
7331 function Babel.tovalue(v)
7332   if type(v) == 'table' then
7333     return Babel.locale_props[v[1]].vars[v[2]] or v[3]
7334   else
7335     return v
7336   end
7337 end
7338
7339 Babel.attr_hboxed = luatexbase.registernumber'bbl@attr@hboxed'
7340
7341 function Babel.set_hboxed(head, gc)
7342   for item in node.traverse(head) do
7343     node.set_attribute(item, Babel.attr_hboxed, 1)
7344   end
7345   return head
7346 end
7347
7348 Babel.fetch_subtext = {}
7349
7350 Babel.ignore_pre_char = function(node)
7351   return (node.lang == Babel.nohyphenation)
7352 end
7353
7354 Babel.show_transforms = false
7355
7356 -- Merging both functions doesn't seem feasible, because there are too
7357 -- many differences.
7358 Babel.fetch_subtext[0] = function(head)
7359   local word_string = ''
7360   local word_nodes = {}
7361   local lang
7362   local item = head
7363   local inmath = false
7364
7365   while item do
7366     if item.id == 11 then
7367       inmath = (item.subtype == 0)
7368     end
7369
7370     if inmath then
7371       -- pass
7372     elseif item.id == 29 then
7373       local locale = node.get_attribute(item, Babel.attr_locale)
7374
7375       if lang == locale or lang == nil then
7376         lang = lang or locale
7377         if Babel.ignore_pre_char(item) then
7378           word_string = word_string .. Babel.us_char
7379         else
7380           if node.has_attribute(item, Babel.attr_hboxed) then
7381             word_string = word_string .. Babel.us_char
7382           else
7383             word_string = word_string .. unicode.utf8.char(item.char)
7384           end
7385         end
7386       end
7387       word_nodes[#word_nodes+1] = item
7388     else
7389       break
7390     end

```

```

7391     end
7392
7393     elseif item.id == 12 and item.subtype == 13 then
7394         if node.has_attribute(item, Babel.attr_hboxed) then
7395             word_string = word_string .. Babel.us_char
7396         else
7397             word_string = word_string .. ' '
7398         end
7399         word_nodes[#word_nodes+1] = item
7400
7401         -- Ignore leading unrecognized nodes, too.
7402         elseif word_string ~= '' then
7403             word_string = word_string .. Babel.us_char
7404             word_nodes[#word_nodes+1] = item -- Will be ignored
7405         end
7406
7407         item = item.next
7408     end
7409
7410     -- Here and above we remove some trailing chars but not the
7411     -- corresponding nodes. But they aren't accessed.
7412     if word_string:sub(-1) == ' ' then
7413         word_string = word_string:sub(1,-2)
7414     end
7415     if Babel.show_transforms then texio.write_nl(word_string) end
7416     word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7417     return word_string, word_nodes, item, lang
7418 end
7419
7420 Babel.fetch_subtext[1] = function(head)
7421     local word_string = ''
7422     local word_nodes = {}
7423     local lang
7424     local item = head
7425     local inmath = false
7426
7427     while item do
7428
7429         if item.id == 11 then
7430             inmath = (item.subtype == 0)
7431         end
7432
7433         if inmath then
7434             -- pass
7435         elseif item.id == 29 then
7436             if item.lang == lang or lang == nil then
7437                 lang = lang or item.lang
7438                 if node.has_attribute(item, Babel.attr_hboxed) then
7439                     word_string = word_string .. Babel.us_char
7440                 elseif (item.char == 124) or (item.char == 61) then -- not =, not |
7441                     word_string = word_string .. Babel.us_char
7442                 else
7443                     word_string = word_string .. unicode.utf8.char(item.char)
7444                 end
7445                 word_nodes[#word_nodes+1] = item
7446             else
7447                 break
7448             end
7449         end
7450
7451         elseif item.id == 7 and item.subtype == 2 then
7452             if node.has_attribute(item, Babel.attr_hboxed) then
7453                 word_string = word_string .. Babel.us_char

```



```

7454     else
7455         word_string = word_string .. '='
7456     end
7457     word_nodes[#word_nodes+1] = item
7458
7459 elseif item.id == 7 and item.subtype == 3 then
7460     if node.has_attribute(item, Babel.attr_hboxed) then
7461         word_string = word_string .. Babel.us_char
7462     else
7463         word_string = word_string .. '|'
7464     end
7465     word_nodes[#word_nodes+1] = item
7466
7467     -- (1) Go to next word if nothing was found, and (2) implicitly
7468     -- remove leading USs.
7469     elseif word_string == '' then
7470         -- pass
7471
7472     -- This is the responsible for splitting by words.
7473     elseif (item.id == 12 and item.subtype == 13) then
7474         break
7475
7476     else
7477         word_string = word_string .. Babel.us_char
7478         word_nodes[#word_nodes+1] = item -- Will be ignored
7479     end
7480
7481     item = item.next
7482 end
7483 if Babel.show_transforms then texio.write_nl(word_string) end
7484 word_string = unicode.utf8.gsub(word_string, Babel.us_char .. '+$', '')
7485 return word_string, word_nodes, item, lang
7486 end
7487
7488 function Babel.pre_hyphenate_replace(head)
7489     Babel.hyphenate_replace(head, 0)
7490 end
7491
7492 function Babel.post_hyphenate_replace(head)
7493     Babel.hyphenate_replace(head, 1)
7494 end
7495
7496 Babel.us_char = string.char(31)
7497
7498 function Babel.hyphenate_replace(head, mode)
7499     local u = unicode.utf8
7500     local lbkr = Babel.linebreaking.replacements[mode]
7501     local tovalue = Babel.tovalue
7502
7503     local word_head = head
7504
7505     if Babel.show_transforms then
7506         texio.write_nl('\n==== Showing ' .. (mode == 0 and 'pre' or 'post') .. 'hyphenation ====')
7507     end
7508
7509     while true do -- for each subtext block
7510
7511         local w, w_nodes, nw, lang = Babel.fetch_subtext[mode](word_head)
7512
7513         if Babel.debug then
7514             print()
7515             print((mode == 0) and '@@@<' or '@@@>', w)
7516         end

```

```

7517
7518 if nw == nil and w == '' then break end
7519
7520 if not lang then goto next end
7521 if not lbkr[lang] then goto next end
7522
7523 -- For each saved (pre|post)hyphenation. TODO. Reconsider how
7524 -- loops are nested.
7525 for k=1, #lbkr[lang] do
7526     local p = lbkr[lang][k].pattern
7527     local r = lbkr[lang][k].replace
7528     local attr = lbkr[lang][k].attr or -1
7529
7530     if Babel.debug then
7531         print('*****', p, mode)
7532     end
7533
7534     -- This variable is set in some cases below to the first *byte*
7535     -- after the match, either as found by u.match (faster) or the
7536     -- computed position based on sc if w has changed.
7537     local last_match = 0
7538     local step = 0
7539
7540     -- For every match.
7541     while true do
7542         if Babel.debug then
7543             print('====')
7544         end
7545         local new -- used when inserting and removing nodes
7546         local dummy_node -- used by after
7547
7548         local matches = { u.match(w, p, last_match) }
7549
7550         if #matches < 2 then break end
7551
7552         -- Get and remove empty captures (with ()'s, which return a
7553         -- number with the position), and keep actual captures
7554         -- (from (...)), if any, in matches.
7555         local first = table.remove(matches, 1)
7556         local last = table.remove(matches, #matches)
7557         -- Non re-fetched substrings may contain \31, which separates
7558         -- subsubstrings.
7559         if string.find(w:sub(first, last-1), Babel.us_char) then break end
7560
7561         local save_last = last -- with A()BC()D, points to D
7562
7563         -- Fix offsets, from bytes to unicode. Explained above.
7564         first = u.len(w:sub(1, first-1)) + 1
7565         last = u.len(w:sub(1, last-1)) -- now last points to C
7566
7567         -- This loop stores in a small table the nodes
7568         -- corresponding to the pattern. Used by 'data' to provide a
7569         -- predictable behavior with 'insert' (w_nodes is modified on
7570         -- the fly), and also access to 'remove'd nodes.
7571         local sc = first-1 -- Used below, too
7572         local data_nodes = {}
7573
7574         local enabled = true
7575         for q = 1, last-first+1 do
7576             data_nodes[q] = w_nodes[sc+q]
7577             if enabled
7578                 and attr > -1
7579                 and not node.has_attribute(data_nodes[q], attr)

```

```

7580         then
7581             enabled = false
7582         end
7583     end
7584
7585     -- This loop traverses the matched substring and takes the
7586     -- corresponding action stored in the replacement list.
7587     -- sc = the position in substr nodes / string
7588     -- rc = the replacement table index
7589     local rc = 0
7590
7591     ----- TODO. dummy_node?
7592     while rc < last-first+1 or dummy_node do -- for each replacement
7593         if Babel.debug then
7594             print('.....', rc + 1)
7595         end
7596         sc = sc + 1
7597         rc = rc + 1
7598
7599         if Babel.debug then
7600             Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7601             local ss = ''
7602             for itt in node.traverse(head) do
7603                 if itt.id == 29 then
7604                     ss = ss .. unicode.utf8.char(itt.char)
7605                 else
7606                     ss = ss .. '{' .. itt.id .. '}'
7607                 end
7608             end
7609             print('*****', ss)
7610
7611         end
7612
7613         local crep = r[rc]
7614         local item = w_nodes[sc]
7615         local item_base = item
7616         local placeholder = Babel.us_char
7617         local d
7618
7619         if crep and crep.data then
7620             item_base = data_nodes[crep.data]
7621         end
7622
7623         if crep then
7624             step = crep.step or step
7625         end
7626
7627         if crep and crep.after then
7628             crep.insert = true
7629             if dummy_node then
7630                 item = dummy_node
7631             else -- TODO. if there is a node after?
7632                 d = node.copy(item_base)
7633                 head, item = node.insert_after(head, item, d)
7634                 dummy_node = item
7635             end
7636         end
7637
7638         if crep and not crep.after and dummy_node then
7639             node.remove(head, dummy_node)
7640             dummy_node = nil
7641         end
7642

```

```

7643     if not enabled then
7644         last_match = save_last
7645         goto next
7646
7647     elseif crep and next(crep) == nil then -- = {}
7648         if step == 0 then
7649             last_match = save_last    -- Optimization
7650         else
7651             last_match = utf8.offset(w, sc+step)
7652         end
7653         goto next
7654
7655     elseif crep == nil or crep.remove then
7656         node.remove(head, item)
7657         table.remove(w_nodes, sc)
7658         w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7659         sc = sc - 1 -- Nothing has been inserted.
7660         last_match = utf8.offset(w, sc+1+step)
7661         goto next
7662
7663     elseif crep and crep.kashida then
7664         node.set_attribute(item,
7665             Babel.attr_kashida,
7666             crep.kashida)
7667         last_match = utf8.offset(w, sc+1+step)
7668         goto next
7669
7670     elseif crep and crep.string then
7671         local str = crep.string(matches)
7672         if str == '' then -- Gather with nil
7673             node.remove(head, item)
7674             table.remove(w_nodes, sc)
7675             w = u.sub(w, 1, sc-1) .. u.sub(w, sc+1)
7676             sc = sc - 1 -- Nothing has been inserted.
7677         else
7678             local loop_first = true
7679             for s in string.utfvalues(str) do
7680                 d = node.copy(item_base)
7681                 d.char = s
7682                 if loop_first then
7683                     loop_first = false
7684                     head, new = node.insert_before(head, item, d)
7685                     if sc == 1 then
7686                         word_head = head
7687                     end
7688                     w_nodes[sc] = d
7689                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc+1)
7690                 else
7691                     sc = sc + 1
7692                     head, new = node.insert_before(head, item, d)
7693                     table.insert(w_nodes, sc, new)
7694                     w = u.sub(w, 1, sc-1) .. u.char(s) .. u.sub(w, sc)
7695                 end
7696                 if Babel.debug then
7697                     print('.....', 'str')
7698                     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7699                 end
7700             end -- for
7701             node.remove(head, item)
7702         end -- if ''
7703         last_match = utf8.offset(w, sc+1+step)
7704         goto next
7705

```

```

7706 elseif mode == 1 and crep and (crep.pre or crep.no or crep.post) then
7707     d = node.new(7, 3) -- (disc, regular)
7708     d.pre = Babel.str_to_nodes(crep.pre, matches, item_base)
7709     d.post = Babel.str_to_nodes(crep.post, matches, item_base)
7710     d.replace = Babel.str_to_nodes(crep.no, matches, item_base)
7711     d.attr = item_base.attr
7712     if crep.pre == nil then -- TeXbook p96
7713         d.penalty = tovalue(crep.penalty) or tex.hyphenpenalty
7714     else
7715         d.penalty = tovalue(crep.penalty) or tex.exhyphenpenalty
7716     end
7717     placeholder = '|'
7718     head, new = node.insert_before(head, item, d)
7719
7720 elseif mode == 0 and crep and (crep.pre or crep.no or crep.post) then
7721     -- ERROR
7722
7723 elseif crep and crep.penalty then
7724     d = node.new(14, 0) -- (penalty, userpenalty)
7725     d.attr = item_base.attr
7726     d.penalty = tovalue(crep.penalty)
7727     head, new = node.insert_before(head, item, d)
7728
7729 elseif crep and crep.space then
7730     -- 655360 = 10 pt = 10 * 65536 sp
7731     d = node.new(12, 13) -- (glue, spaceskip)
7732     local quad = font.getfont(item_base.font).size or 655360
7733     node.setglue(d, tovalue(crep.space[1]) * quad,
7734                   tovalue(crep.space[2]) * quad,
7735                   tovalue(crep.space[3]) * quad)
7736     if mode == 0 then
7737         placeholder = ' '
7738     end
7739     head, new = node.insert_before(head, item, d)
7740
7741 elseif crep and crep.norule then
7742     -- 655360 = 10 pt = 10 * 65536 sp
7743     d = node.new(2, 3) -- (rule, empty) = \no*rule
7744     local quad = font.getfont(item_base.font).size or 655360
7745     d.width = tovalue(crep.norule[1]) * quad
7746     d.height = tovalue(crep.norule[2]) * quad
7747     d.depth = tovalue(crep.norule[3]) * quad
7748     head, new = node.insert_before(head, item, d)
7749
7750 elseif crep and crep.spacefactor then
7751     d = node.new(12, 13) -- (glue, spaceskip)
7752     local base_font = font.getfont(item_base.font)
7753     node.setglue(d,
7754                 tovalue(crep.spacefactor[1]) * base_font.parameters['space'],
7755                 tovalue(crep.spacefactor[2]) * base_font.parameters['space_stretch'],
7756                 tovalue(crep.spacefactor[3]) * base_font.parameters['space_shrink'])
7757     if mode == 0 then
7758         placeholder = ' '
7759     end
7760     head, new = node.insert_before(head, item, d)
7761
7762 elseif mode == 0 and crep and crep.space then
7763     -- ERROR
7764
7765 elseif crep and crep.kern then
7766     d = node.new(13, 1) -- (kern, user)
7767     local quad = font.getfont(item_base.font).size or 655360
7768     d.attr = item_base.attr

```

```

7769         d.kern = tovalue(crep.kern) * quad
7770         head, new = node.insert_before(head, item, d)
7771
7772     elseif crep and crep.node then
7773         d = node.new(crep.node[1], crep.node[2])
7774         d.attr = item_base.attr
7775         head, new = node.insert_before(head, item, d)
7776
7777     end -- i.e., replacement cases
7778
7779     -- Shared by disc, space(factor), kern, node and penalty.
7780     if sc == 1 then
7781         word_head = head
7782     end
7783     if crep.insert then
7784         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc)
7785         table.insert(w_nodes, sc, new)
7786         last = last + 1
7787     else
7788         w_nodes[sc] = d
7789         node.remove(head, item)
7790         w = u.sub(w, 1, sc-1) .. placeholder .. u.sub(w, sc+1)
7791     end
7792
7793     last_match = utf8.offset(w, sc+1+step)
7794
7795     ::next::
7796
7797 end -- for each replacement
7798
7799 if Babel.show_transforms then texio.write_nl('> ' .. w) end
7800 if Babel.debug then
7801     print('.....', '/')
7802     Babel.debug_hyph(w, w_nodes, sc, first, last, last_match)
7803 end
7804
7805 if dummy_node then
7806     node.remove(head, dummy_node)
7807     dummy_node = nil
7808 end
7809
7810 end -- for match
7811
7812 end -- for patterns
7813
7814 ::next::
7815 word_head = nw
7816 end -- for substring
7817
7818 if Babel.show_transforms then texio.write_nl(string.rep('-', 32) .. '\n') end
7819 return head
7820 end
7821
7822 -- This table stores capture maps, numbered consecutively
7823 Babel.capture_maps = {}
7824
7825 function Babel.esc_hex_to_char(h)
7826     if tex.getcatcode(tonumber(h, 16)) ~= 11 and
7827         tex.getcatcode(tonumber(h, 16)) ~= 12 then
7828         return string.format("[%\Uchar"%X ]", tonumber(h,16))
7829     else
7830         return unicode.utf8.char(tonumber(h, 16))
7831     end

```

```

7832 end
7833
7834 -- The following functions belong to the next macro
7835 function Babel.capture_func(key, cap)
7836   local ret = "[" .. cap:gsub('{{[0-9]}}', "]]..m[%1]..[" .. "]"
7837   local cnt
7838   local u = unicode.utf8
7839   ret = u.gsub(ret, '{(%x%x%x%x+)}', '\x01%\x04')
7840   ret, cnt = ret:gsub('{{[0-9]}|([^\]|+)|(.-)}', Babel.capture_func_map)
7841   ret = u.gsub(ret, '\x01(%x%x%x%x+)\x04', Babel.esc_hex_to_char)
7842   ret = ret:gsub("%[%]%.%", '')
7843   ret = ret:gsub("%.%.%[%]%", '')
7844   return key .. "[=function(m) return ]] .. ret .. [[ end]]
7845 end
7846
7847 function Babel.capt_map(from, mapno)
7848   return Babel.capture_maps[mapno][from] or from
7849 end
7850
7851 -- Handle the {n|abc|ABC} syntax in captures
7852 function Babel.capture_func_map(capno, from, to)
7853   local u = unicode.utf8
7854   from = u.gsub(from, '\x01(%x%x%x%x+)\x04',
7855     function (n)
7856       return u.char(tonumber(n, 16))
7857     end)
7858   to = u.gsub(to, '\x01(%x%x%x%x+)\x04',
7859     function (n)
7860       return u.char(tonumber(n, 16))
7861     end)
7862   local froms = {}
7863   for s in string.utfcharacters(from) do
7864     table.insert(froms, s)
7865   end
7866   local cnt = 1
7867   table.insert(Babel.capture_maps, {})
7868   local mlen = table.getn(Babel.capture_maps)
7869   for s in string.utfcharacters(to) do
7870     Babel.capture_maps[mlen][froms[cnt]] = s
7871     cnt = cnt + 1
7872   end
7873   return "]]..Babel.capt_map(m[" .. capno .. "], " ..
7874     (mlen) .. " ).." .. "["
7875 end
7876
7877 -- Create/Extend reversed sorted list of kashida weights:
7878 function Babel.capture_kashida(key, wt)
7879   wt = tonumber(wt)
7880   if Babel.kashida_wts then
7881     for p, q in ipairs(Babel.kashida_wts) do
7882       if wt == q then
7883         break
7884       elseif wt > q then
7885         table.insert(Babel.kashida_wts, p, wt)
7886         break
7887       elseif table.getn(Babel.kashida_wts) == p then
7888         table.insert(Babel.kashida_wts, wt)
7889       end
7890     end
7891   else
7892     Babel.kashida_wts = { wt }
7893   end
7894   return 'kashida = ' .. wt

```

```

7895 end
7896
7897 function Babel.capture_node(id, subtype)
7898   local sbt = 0
7899   for k, v in pairs(node.subtypes(id)) do
7900     if v == subtype then sbt = k end
7901   end
7902   return 'node = {' .. node.id(id) .. ', ' .. sbt .. '}'
7903 end
7904
7905 -- Experimental: applies prehyphenation transforms to a string (letters
7906 -- and spaces).
7907 function Babel.string_prehyphenation(str, locale)
7908   local n, head, last, res
7909   head = node.new(8, 0) -- dummy (hack just to start)
7910   last = head
7911   for s in string.utfvalues(str) do
7912     if s == 20 then
7913       n = node.new(12, 0)
7914     else
7915       n = node.new(29, 0)
7916       n.char = s
7917     end
7918     node.set_attribute(n, Babel.attr_locale, locale)
7919     last.next = n
7920     last = n
7921   end
7922   head = Babel.hyphenate_replace(head, 0)
7923   res = ''
7924   for n in node.traverse(head) do
7925     if n.id == 12 then
7926       res = res .. ' '
7927     elseif n.id == 29 then
7928       res = res .. unicode.utf8.char(n.char)
7929     end
7930   end
7931   tex.print(res)
7932 end
7933 (/transforms)

```

10.14 Lua: Auto bidi with basic and basic-r

The file `babel-data-bidi.lua` currently only contains data. It is a large and boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x25]={d='et'},
% [0x26]={d='on'},
% [0x27]={d='on'},
% [0x28]={d='on', m=0x29},
% [0x29]={d='on', m=0x28},
% [0x2A]={d='on'},
% [0x2B]={d='es'},
% [0x2C]={d='cs'},
%

```

For the meaning of these codes, see the Unicode standard.

Now the `basic-r` bidi mode. One of the aims is to implement a fast and simple bidi algorithm, with a single loop. I managed to do it for R texts, with a second smaller loop for a special case. The code is still somewhat chaotic, but its behavior is essentially correct. I cannot resist copying the following text from Emacs `bidi.c` (which also attempts to implement the bidi algorithm with a single loop):

Arrrrgh!! The UAX#9 algorithm is too deeply entrenched in the assumption of batch-style processing [...]. May the fleas of a thousand camels infest the armpits of those who

design supposedly general-purpose algorithms by looking at their own implementations, and fail to consider other possible implementations!

Well, it took me some time to guess what the batch rules in UAX#9 actually mean (in other word, *what* they do and *why*, and not only *how*), but I think (or I hope) I've managed to understand them.

In some sense, there are two bidi modes, one for numbers, and the other for text. Furthermore, setting just the direction in R text is not enough, because there are actually *two* R modes (set explicitly in Unicode with RLM and ALM). In babel the dir is set by a higher protocol based on the language/script, which in turn sets the correct dir (<l>, <r> or <al>).

From UAX#9: “Where available, markup should be used instead of the explicit formatting characters”. So, this simple version just ignores formatting characters. Actually, most of that annex is devoted to how to handle them.

BD14-BD16 are not implemented. Unicode (and the W3C) are making a great effort to deal with some special problematic cases in “streamed” plain text. I don't think this is the way to go – particular issues should be fixed by a high level interface taking into account the needs of the document. And here is where luatex excels, because everything related to bidi writing is under our control.

```
7934 (*basic-r)
7935 Babel.bidi_enabled = true
7936
7937 require('babel-data-bidi.lua')
7938
7939 local characters = Babel.characters
7940 local ranges = Babel.ranges
7941
7942 local DIR = node.id("dir")
7943
7944 local function dir_mark(head, from, to, outer)
7945   dir = (outer == 'r') and 'TLT' or 'TRT' -- i.e., reverse
7946   local d = node.new(DIR)
7947   d.dir = '+' .. dir
7948   node.insert_before(head, from, d)
7949   d = node.new(DIR)
7950   d.dir = '-' .. dir
7951   node.insert_after(head, to, d)
7952 end
7953
7954 function Babel.bidi(head, ispar)
7955   local first_n, last_n      -- first and last char with nums
7956   local last_es             -- an auxiliary 'last' used with nums
7957   local first_d, last_d      -- first and last char in L/R block
7958   local dir, dir_real
```

Next also depends on script/lang (<al>/<r>). To be set by babel. tex.pardir is dangerous, could be (re)set but it should be changed only in vmode. There are two strong's – strong = l/al/r and strong_lr = l/r (there must be a better way):

```
7959   local strong = ('TRT' == tex.pardir) and 'r' or 'l'
7960   local strong_lr = (strong == 'l') and 'l' or 'r'
7961   local outer = strong
7962
7963   local new_dir = false
7964   local first_dir = false
7965   local inmath = false
7966
7967   local last_lr
7968
7969   local type_n = ''
7970
7971   for item in node.traverse(head) do
7972
7973     -- three cases: glyph, dir, otherwise
7974     if item.id == node.id'glyph'
7975       or (item.id == 7 and item.subtype == 2) then
7976
```

```

7977     local itemchar
7978     if item.id == 7 and item.subtype == 2 then
7979         itemchar = item.replace.char
7980     else
7981         itemchar = item.char
7982     end
7983     local chardata = characters[itemchar]
7984     dir = chardata and chardata.d or nil
7985     if not dir then
7986         for nn, et in ipairs(ranges) do
7987             if itemchar < et[1] then
7988                 break
7989             elseif itemchar <= et[2] then
7990                 dir = et[3]
7991                 break
7992             end
7993         end
7994     end
7995     dir = dir or 'l'
7996     if inmath then dir = ('TRT' == tex.mathdir) and 'r' or 'l' end

```

Next is based on the assumption babel sets the language *and* switches the script with its dir. We treat a language block as a separate Unicode sequence. The following piece of code is executed at the first glyph after a 'dir' node. We don't know the current language until then. This is not exactly true, as the math mode may insert explicit dirs in the node list, so, for the moment there is a hack by brute force (just above).

```

7997     if new_dir then
7998         attr_dir = 0
7999         for at in node.traverse(item.attr) do
8000             if at.number == Babel.attr_dir then
8001                 attr_dir = at.value & 0x3
8002             end
8003         end
8004         if attr_dir == 1 then
8005             strong = 'r'
8006         elseif attr_dir == 2 then
8007             strong = 'al'
8008         else
8009             strong = 'l'
8010         end
8011         strong_lr = (strong == 'l') and 'l' or 'r'
8012         outer = strong_lr
8013         new_dir = false
8014     end
8015
8016     if dir == 'nsm' then dir = strong end -- W1

```

Numbers. The dual <al>/<r> system for R is somewhat cumbersome.

```

8017     dir_real = dir -- We need dir_real to set strong below
8018     if dir == 'al' then dir = 'r' end -- W3

```

By W2, there are no <en> <et> <es> if strong == <al>, only <an>. Therefore, there are not <et en> nor <en et>, W5 can be ignored, and W6 applied:

```

8019     if strong == 'al' then
8020         if dir == 'en' then dir = 'an' end -- W2
8021         if dir == 'et' or dir == 'es' then dir = 'on' end -- W6
8022         strong_lr = 'r' -- W3
8023     end

```

Once finished the basic setup for glyphs, consider the two other cases: dir node and the rest.

```

8024     elseif item.id == node.id'dir' and not inmath then
8025         new_dir = true
8026         dir = nil
8027     elseif item.id == node.id'math' then

```

```

8028     inmath = (item.subtype == 0)
8029   else
8030     dir = nil          -- Not a char
8031   end

```

Numbers in R mode. A sequence of <en>, <et>, <an>, <es> and <cs> is typeset (with some rules) in L mode. We store the starting and ending points, and only when anything different is found (including nil, i.e., a non-char), the textdir is set. This means you cannot insert, say, a whatsit, but this is what I would expect (with luacolor you may colorize some digits). Anyway, this behavior could be changed with a switch in the future. Note in the first branch only <an> is relevant if <al>.

```

8032   if dir == 'en' or dir == 'an' or dir == 'et' then
8033     if dir ~= 'et' then
8034       type_n = dir
8035     end
8036     first_n = first_n or item
8037     last_n = last_es or item
8038     last_es = nil
8039   elseif dir == 'es' and last_n then -- W3+W6
8040     last_es = item
8041   elseif dir == 'cs' then          -- it's right - do nothing
8042   elseif first_n then -- & if dir = any but en, et, an, es, cs, inc nil
8043     if strong_lr == 'r' and type_n ~= '' then
8044       dir_mark(head, first_n, last_n, 'r')
8045     elseif strong_lr == 'l' and first_d and type_n == 'an' then
8046       dir_mark(head, first_n, last_n, 'r')
8047       dir_mark(head, first_d, last_d, outer)
8048       first_d, last_d = nil, nil
8049     elseif strong_lr == 'l' and type_n ~= '' then
8050       last_d = last_n
8051     end
8052     type_n = ''
8053     first_n, last_n = nil, nil
8054   end

```

R text in L, or L text in R. Order of dir_ mark's are relevant: d goes outside n, and therefore it's emitted after. See dir_mark to understand why (but is the nesting actually necessary or is a flat dir structure enough?). Only L, R (and AL) chars are taken into account – everything else, including spaces, whatsits, etc., are ignored:

```

8055   if dir == 'l' or dir == 'r' then
8056     if dir ~= outer then
8057       first_d = first_d or item
8058       last_d = item
8059     elseif first_d and dir ~= strong_lr then
8060       dir_mark(head, first_d, last_d, outer)
8061       first_d, last_d = nil, nil
8062     end
8063   end

```

Mirroring. Each chunk of text in a certain language is considered a “closed” sequence. If <r on r> and <l on l>, it's clearly <r> and <l>, resptly, but with other combinations depends on outer. From all these, we select only those resolving <on> → <r>. At the beginning (when last_lr is nil) of an R text, they are mirrored directly. Numbers in R mode are processed. It should not be done, but it doesn't hurt.

```

8064   if dir and not last_lr and dir ~= 'l' and outer == 'r' then
8065     item.char = characters[item.char] and
8066       characters[item.char].m or item.char
8067   elseif (dir or new_dir) and last_lr ~= item then
8068     local mir = outer .. strong_lr .. (dir or outer)
8069     if mir == 'rrr' or mir == 'lrr' or mir == 'rrl' or mir == 'rlr' then
8070       for ch in node.traverse(node.next(last_lr)) do
8071         if ch == item then break end
8072         if ch.id == node.id'glyph' and characters[ch.char] then
8073           ch.char = characters[ch.char].m or ch.char
8074         end

```

```

8075         end
8076     end
8077 end

```

Save some values for the next iteration. If the current node is 'dir', open a new sequence. Since dir could be changed, strong is set with its real value (dir_real).

```

8078     if dir == 'l' or dir == 'r' then
8079         last_lr = item
8080         strong = dir_real          -- Don't search back - best save now
8081         strong_lr = (strong == 'l') and 'l' or 'r'
8082     elseif new_dir then
8083         last_lr = nil
8084     end
8085 end

```

Mirror the last chars if they are no directed. And make sure any open block is closed, too.

```

8086     if last_lr and outer == 'r' then
8087         for ch in node.traverse_id(node.id'glyph', node.next(last_lr)) do
8088             if characters[ch.char] then
8089                 ch.char = characters[ch.char].m or ch.char
8090             end
8091         end
8092     end
8093     if first_n then
8094         dir_mark(head, first_n, last_n, outer)
8095     end
8096     if first_d then
8097         dir_mark(head, first_d, last_d, outer)
8098     end

```

In boxes, the dir node could be added before the original head, so the actual head is the previous node.

```

8099     return node.prev(head) or head
8100 end
8101 (/basic-r)

```

And here the Lua code for bidi=basic:

```

8102 (*basic)
8103 -- e.g., Babel.fontmap[1][<prefontid>]=<dirfontid>
8104
8105 Babel.fontmap = Babel.fontmap or {}
8106 Babel.fontmap[0] = {}          -- l
8107 Babel.fontmap[1] = {}          -- r
8108 Babel.fontmap[2] = {}          -- al/an
8109
8110 -- To cancel mirroring. Also OML, OMS, U?
8111 Babel.symbol_fonts = Babel.symbol_fonts or {}
8112 Babel.symbol_fonts[font.id('tenln')] = true
8113 Babel.symbol_fonts[font.id('tenlnw')] = true
8114 Babel.symbol_fonts[font.id('tencirc')] = true
8115 Babel.symbol_fonts[font.id('tencircw')] = true
8116
8117 Babel.bidi_enabled = true
8118 Babel.mirroring_enabled = true
8119
8120 require('babel-data-bidi.lua')
8121
8122 local characters = Babel.characters
8123 local ranges = Babel.ranges
8124
8125 local DIR = node.id('dir')
8126 local GLYPH = node.id('glyph')
8127
8128 local function insert_implicit(head, state, outer)

```

```

8129 local new_state = state
8130 if state.sim and state.eim and state.sim ~= state.eim then
8131     dir = ((outer == 'r') and 'TLT' or 'TRT') -- i.e., reverse
8132     local d = node.new(DIR)
8133     d.dir = '+' .. dir
8134     node.insert_before(head, state.sim, d)
8135     local d = node.new(DIR)
8136     d.dir = '-' .. dir
8137     node.insert_after(head, state.eim, d)
8138 end
8139 new_state.sim, new_state.eim = nil, nil
8140 return head, new_state
8141 end
8142
8143 local function insert_numeric(head, state)
8144     local new
8145     local new_state = state
8146     if state.san and state.ean and state.san ~= state.ean then
8147         local d = node.new(DIR)
8148         d.dir = '+TLT'
8149         _, new = node.insert_before(head, state.san, d)
8150         if state.san == state.sim then state.sim = new end
8151         local d = node.new(DIR)
8152         d.dir = '-TLT'
8153         _, new = node.insert_after(head, state.ean, d)
8154         if state.ean == state.eim then state.eim = new end
8155     end
8156     new_state.san, new_state.ean = nil, nil
8157     return head, new_state
8158 end
8159
8160 local function glyph_not_symbol_font(node)
8161     if node.id == GLYPH then
8162         return not Babel.symbol_fonts[node.font]
8163     else
8164         return false
8165     end
8166 end
8167
8168 -- TODO - \hbox with an explicit dir can lead to wrong results
8169 -- <R \hbox dir TLT{<R>}> and <L \hbox dir TRT{<L>}>. A small attempt
8170 -- was made to improve the situation, but the problem is the 3-dir
8171 -- model in babel/Unicode and the 2-dir model in LuaTeX don't fit
8172 -- well.
8173
8174 function Babel.bidi(head, ispar, hdir)
8175     local d -- d is used mainly for computations in a loop
8176     local prev_d = ''
8177     local new_d = false
8178
8179     local nodes = {}
8180     local outer_first = nil
8181     local inmath = false
8182
8183     local glue_d = nil
8184     local glue_i = nil
8185
8186     local has_en = false
8187     local first_et = nil
8188
8189     local has_hyperlink = false
8190
8191     local ATDIR = Babel.attr_dir

```

```

8192 local attr_d, temp
8193 local locale_d
8194
8195 local save_outer
8196 local locale_d = node.get_attribute(head, ATDIR)
8197 if locale_d then
8198     locale_d = locale_d & 0x3
8199     save_outer = (locale_d == 0 and 'l') or
8200                 (locale_d == 1 and 'r') or
8201                 (locale_d == 2 and 'al')
8202 elseif ispar then -- Or error? Shouldn't happen
8203     -- when the callback is called, we are just _after_ the box,
8204     -- and the textdir is that of the surrounding text
8205     save_outer = ('TRT' == tex.pardir) and 'r' or 'l'
8206 else -- Empty box
8207     save_outer = ('TRT' == hdir) and 'r' or 'l'
8208 end
8209 local outer = save_outer
8210 local last = outer
8211 -- 'al' is only taken into account in the first, current loop
8212 if save_outer == 'al' then save_outer = 'r' end
8213
8214 local fontmap = Babel.fontmap
8215
8216 for item in node.traverse(head) do
8217
8218     -- Mask: DxxxPPTT (Done, Pardir [0-2], Textdir [0-2])
8219     locale_d = node.get_attribute(item, ATDIR)
8220     node.set_attribute(item, ATDIR, 0x80)
8221
8222     -- In what follows, #node is the last (previous) node, because the
8223     -- current one is not added until we start processing the neutrals.
8224     -- three cases: glyph, dir, otherwise
8225     if glyph_not_symbol_font(item)
8226         or (item.id == 7 and item.subtype == 2) then
8227
8228         if inmath then goto nextnode end
8229
8230         if locale_d == 0x80 then goto nextnode end
8231         locale_d = locale_d or ((save_outer=='l') and 0 or 1)
8232
8233         local d_font = nil
8234         local item_r
8235         if item.id == 7 and item.subtype == 2 then
8236             item_r = item.replace -- automatic discs have just 1 glyph
8237         else
8238             item_r = item
8239         end
8240
8241         local chardata = characters[item_r.char]
8242         d = chardata and chardata.d or nil
8243         if not d or d == 'nsm' then
8244             for nn, et in ipairs(ranges) do
8245                 if item_r.char < et[1] then
8246                     break
8247                 elseif item_r.char <= et[2] then
8248                     if not d then d = et[3]
8249                     elseif d == 'nsm' then d_font = et[3]
8250                     end
8251                     break
8252                 end
8253             end
8254             end

```

```

8255     d = d or 'l'
8256
8257     -- A short 'pause' in bidi for mapfont
8258     -- %%% TODO. move if fontmap here
8259     d_font = d_font or d
8260     d_font = (d_font == 'l' and 0) or
8261               (d_font == 'nsm' and 0) or
8262               (d_font == 'r' and 1) or
8263               (d_font == 'al' and 2) or
8264               (d_font == 'an' and 2) or nil
8265     if d_font and fontmap and fontmap[d_font][item_r.font] then
8266         item_r.font = fontmap[d_font][item_r.font]
8267     end
8268
8269     if new_d then
8270         table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8271         if inmath then
8272             attr_d = 0
8273         else
8274             attr_d = locale_d & 0x3
8275         end
8276         if attr_d == 1 then
8277             outer_first = 'r'
8278             last = 'r'
8279         elseif attr_d == 2 then
8280             outer_first = 'r'
8281             last = 'al'
8282         else
8283             outer_first = 'l'
8284             last = 'l'
8285         end
8286         outer = last
8287         has_en = false
8288         first_et = nil
8289         new_d = false
8290     end
8291
8292     if glue_d then
8293         if (d == 'l' and 'l' or 'r') ~= glue_d then
8294             table.insert(nodes, {glue_i, 'on', nil})
8295         end
8296         glue_d = nil
8297         glue_i = nil
8298     end
8299
8300     elseif item.id == DIR then
8301         if inmath then goto nextnode end
8302         d = nil
8303         new_d = true
8304
8305     elseif item.id == node.id'glue' and item.subtype == 13 then
8306         if inmath then goto nextnode end
8307         glue_d = d
8308         glue_i = item
8309         d = nil
8310
8311     elseif item.id == node.id'math' then
8312         if item.subtype == 0 then -- mathon
8313             inmath = true
8314             d = 'on'
8315             table.insert(nodes, {item, 'on', outer_first})
8316             outer_first = nil
8317             goto nextnode

```

```

8318     else -- mathoff
8319         inmath = false
8320     end
8321
8322 elseif item.id == 8 and item.subtype == 19 then
8323     has_hyperlink = true
8324
8325 else
8326     d = nil
8327 end
8328
8329 -- AL <= EN/ET/ES      -- W2 + W3 + W6
8330 if last == 'al' and d == 'en' then
8331     d = 'an'           -- W3
8332 elseif last == 'al' and (d == 'et' or d == 'es') then
8333     d = 'on'           -- W6
8334 end
8335
8336 -- EN + CS/ES + EN      -- W4
8337 if d == 'en' and #nodes >= 2 then
8338     if (nodes[#nodes][2] == 'es' or nodes[#nodes][2] == 'cs')
8339         and nodes[#nodes-1][2] == 'en' then
8340         nodes[#nodes][2] = 'en'
8341     end
8342 end
8343
8344 -- AN + CS + AN          -- W4 too, because uax9 mixes both cases
8345 if d == 'an' and #nodes >= 2 then
8346     if (nodes[#nodes][2] == 'cs')
8347         and nodes[#nodes-1][2] == 'an' then
8348         nodes[#nodes][2] = 'an'
8349     end
8350 end
8351
8352 -- ET/EN                -- W5 + W7->l / W6->on
8353 if d == 'et' then
8354     first_et = first_et or (#nodes + 1)
8355 elseif d == 'en' then
8356     has_en = true
8357     first_et = first_et or (#nodes + 1)
8358 elseif first_et then    -- d may be nil here !
8359     if has_en then
8360         if last == 'l' then
8361             temp = 'l'    -- W7
8362         else
8363             temp = 'en'   -- W5
8364         end
8365     else
8366         temp = 'on'      -- W6
8367     end
8368     for e = first_et, #nodes do
8369         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8370     end
8371     first_et = nil
8372     has_en = false
8373 end
8374
8375 -- Force mathdir in math if ON (currently works as expected only
8376 -- with 'l')
8377
8378 if inmath and d == 'on' then
8379     d = ('TRT' == tex.mathdir) and 'r' or 'l'
8380 end

```



```

8381
8382     if d then
8383         if d == 'al' then
8384             d = 'r'
8385             last = 'al'
8386         elseif d == 'l' or d == 'r' then
8387             last = d
8388         end
8389         prev_d = d
8390         table.insert(nodes, {item, d, outer_first})
8391     end
8392
8393     outer_first = nil
8394
8395     ::nextnode::
8396
8397 end -- for each node
8398
8399 -- TODO -- repeated here in case EN/ET is the last node. Find a
8400 -- better way of doing things:
8401 if first_et then      -- dir may be nil here !
8402     if has_en then
8403         if last == 'l' then
8404             temp = 'l'    -- W7
8405         else
8406             temp = 'en'   -- W5
8407         end
8408     else
8409         temp = 'on'       -- W6
8410     end
8411     for e = first_et, #nodes do
8412         if glyph_not_symbol_font(nodes[e][1]) then nodes[e][2] = temp end
8413     end
8414 end
8415
8416 -- dummy node, to close things
8417 table.insert(nodes, {nil, (outer == 'l') and 'l' or 'r', nil})
8418
8419 ----- NEUTRAL -----
8420
8421 outer = save_outer
8422 last = outer
8423
8424 local first_on = nil
8425
8426 for q = 1, #nodes do
8427     local item
8428
8429     local outer_first = nodes[q][3]
8430     outer = outer_first or outer
8431     last = outer_first or last
8432
8433     local d = nodes[q][2]
8434     if d == 'an' or d == 'en' then d = 'r' end
8435     if d == 'cs' or d == 'et' or d == 'es' then d = 'on' end --- W6
8436
8437     if d == 'on' then
8438         first_on = first_on or q
8439     elseif first_on then
8440         if last == d then
8441             temp = d
8442         else
8443             temp = outer

```

```

8444     end
8445     for r = first_on, q - 1 do
8446         nodes[r][2] = temp
8447         item = nodes[r][1]    -- MIRRORING
8448         if Babel.mirroring_enabled and glyph_not_symbol_font(item)
8449             and temp == 'r' and characters[item.char] then
8450             local font_mode = ''
8451             if item.font > 0 and font.fonts[item.font].properties then
8452                 font_mode = font.fonts[item.font].properties.mode
8453             end
8454             if font_mode ~= 'harf' and font_mode ~= 'plug' then
8455                 item.char = characters[item.char].m or item.char
8456             end
8457         end
8458     end
8459     first_on = nil
8460 end
8461
8462 if d == 'r' or d == 'l' then last = d end
8463 end
8464
8465 ----- IMPLICIT, REORDER -----
8466
8467 outer = save_outer
8468 last = outer
8469
8470 local state = {}
8471 state.has_r = false
8472
8473 for q = 1, #nodes do
8474
8475     local item = nodes[q][1]
8476
8477     outer = nodes[q][3] or outer
8478
8479     local d = nodes[q][2]
8480
8481     if d == 'nsm' then d = last end          -- W1
8482     if d == 'en' then d = 'an' end
8483     local isdir = (d == 'r' or d == 'l')
8484
8485     if outer == 'l' and d == 'an' then
8486         state.san = state.san or item
8487         state.ean = item
8488     elseif state.san then
8489         head, state = insert_numeric(head, state)
8490     end
8491
8492     if outer == 'l' then
8493         if d == 'an' or d == 'r' then      -- im -> implicit
8494             if d == 'r' then state.has_r = true end
8495             state.sim = state.sim or item
8496             state.eim = item
8497         elseif d == 'l' and state.sim and state.has_r then
8498             head, state = insert_implicit(head, state, outer)
8499         elseif d == 'l' then
8500             state.sim, state.eim, state.has_r = nil, nil, false
8501         end
8502     else
8503         if d == 'an' or d == 'l' then
8504             if nodes[q][3] then -- nil except after an explicit dir
8505                 state.sim = item -- so we move sim 'inside' the group
8506             else

```

```

8507         state.sim = state.sim or item
8508     end
8509     state.eim = item
8510     elseif d == 'r' and state.sim then
8511         head, state = insert_implicit(head, state, outer)
8512     elseif d == 'r' then
8513         state.sim, state.eim = nil, nil
8514     end
8515 end
8516
8517 if isdir then
8518     last = d          -- Don't search back - best save now
8519 elseif d == 'on' and state.san then
8520     state.san = state.san or item
8521     state.ean = item
8522 end
8523
8524 end
8525
8526 head = node.prev(head) or head
8527 % \end{macrocode}
8528 %
8529 % Now direction nodes has been distributed with relation to characters
8530 % and spaces, we need to take into account \TeX-specific elements in
8531 % the node list, to move them at an appropriate place. Firstly, with
8532 % hyperlinks. Secondly, we avoid them between penalties and spaces, so
8533 % that the latter are still discardable.
8534 %
8535 % \begin{macrocode}
8536 --- FIXES ---
8537 if has_hyperlink then
8538     local flag, linking = 0, 0
8539     for item in node.traverse(head) do
8540         if item.id == DIR then
8541             if item.dir == '+TRT' or item.dir == '+TLT' then
8542                 flag = flag + 1
8543             elseif item.dir == '-TRT' or item.dir == '-TLT' then
8544                 flag = flag - 1
8545             end
8546             elseif item.id == 8 and item.subtype == 19 then
8547                 linking = flag
8548             elseif item.id == 8 and item.subtype == 20 then
8549                 if linking > 0 then
8550                     if item.prev.id == DIR and
8551                         (item.prev.dir == '-TRT' or item.prev.dir == '-TLT') then
8552                         d = node.new(DIR)
8553                         d.dir = item.prev.dir
8554                         node.remove(head, item.prev)
8555                         node.insert_after(head, item, d)
8556                     end
8557                 end
8558                 linking = 0
8559             end
8560         end
8561     end
8562
8563     for item in node.traverse_id(10, head) do
8564         local p = item
8565         local flag = false
8566         while p.prev and p.prev.id == 14 do
8567             flag = true
8568             p = p.prev
8569         end

```

```

8570     if flag then
8571         node.insert_before(head, p, node.copy(item))
8572         node.remove(head,item)
8573     end
8574 end
8575
8576 return head
8577 end

8578 function Babel.unset_atdir(head)
8579     local ATDIR = Babel.attr_dir
8580     for item in node.traverse(head) do
8581         node.set_attribute(item, ATDIR, 0x80)
8582     end
8583     return head
8584 end

```

The following code has been contributed by Udi Fogiel. It fixes misplaced glues when combined with penalties at line breaks.

```

8585 local GLUE      = node.id'glue'
8586 local PENALTY   = node.id'penalty'
8587 local HLIST     = node.id'hlist'
8588 local LINE      = table.swapped(node.subtypes("hlist")).line
8589
8590 local characters = Babel.characters
8591 local ranges     = Babel.ranges
8592
8593 -- L1's whitespace set: WS + isolate formats + (per the UBA reference) the
8594 -- explicit embedding/override formats + BN.
8595 -- Babel does not use embedding/override/isolates, but well, I copy pasted it...
8596 local ll_whitespace = {
8597     ws = true, bn = true,
8598     lre = true, rle = true, lro = true, rlo = true, pdf = true,
8599     lri = true, rli = true, fsi = true, pdi = true,
8600 }
8601
8602 local function bidi_class(c)
8603     local cd = characters[c]
8604     local d  = cd and cd.d
8605     if not d then
8606         for _, et in ipairs(ranges) do
8607             if c < et[1] then
8608                 break
8609             elseif c <= et[2] then
8610                 d = et[3]
8611                 break
8612             end
8613         end
8614     end
8615     return d or 'l'
8616 end
8617
8618 -- Nodes we step over as part of a trailing-whitespace run: glue,
8619 -- penalties, and a whitespace glyph.
8620 local function skippable(n)
8621     local id = n.id
8622     if id == GLUE or id == PENALTY then
8623         return true
8624     elseif id == GLYPH then
8625         return ll_whitespace[bidi_class(n.char)] == true
8626     end
8627     return false
8628 end
8629

```

```

8630 local function is_enddir(n)
8631   return n.id == DIR and n.subtype == 1
8632 end
8633
8634 -- Rule L1, clause 4, for one line: pull its trailing whitespace out past the
8635 -- closing dir run so it renders in the box (base) direction.
8636 local function reset_line(head)
8637   local last = node.slide(head)          -- tail, fixes prev links
8638   -- 1. skip the tail that follows the closing dir run (rightskip/parfillskip/...)
8639   local q = last
8640   while q and skippable(q) do q = q.prev end
8641   if not (q and is_enddir(q)) then return head end
8642   local d_last = q
8643   -- 2. step back over the whole closing dir run
8644   while q and q.id == DIR do q = q.prev end
8645   -- 3. what precedes the dirs must be the trailing whitespace run
8646   if not (q and skippable(q)) then return head end
8647   local w_last = q
8648   while q and skippable(q) do q = q.prev end
8649   local before_w = q
8650   if not before_w then return head end    -- whole line is whitespace; leave it
8651   local w_first = before_w.next
8652   local d_first = w_last.next
8653   local after_d = d_last.next
8654   -- detach W = [w_first .. w_last]
8655   before_w.next = d_first
8656   if d_first then d_first.prev = before_w end
8657   -- reinsert W after the dir run (outside every dir => box/base direction)
8658   d_last.next = w_first
8659   w_first.prev = d_last
8660   w_last.next = after_d
8661   if after_d then after_d.prev = w_last end
8662   return head
8663 end
8664
8665 function Babel.reset_lines(head)
8666   for line, subtype in node.traverse_id(HLIST, head) do
8667     if subtype == LINE then
8668       line.list = reset_line(line.list)
8669     end
8670   end
8671   return head
8672 end
8673
8674 luatexbase.add_to_callback('post_linebreak_filter',
8675   function(head, gc)
8676     if Babel.bidi_enabled then head = Babel.reset_lines(head) end
8677     return head
8678   end,
8679   'Babel.reset_lines')
8680 (/basic)

```

11. Data for CJK

It is a boring file and it is not shown here (see the generated file), but here is a sample:

```

% [0x0021]={c='ex'},
% [0x0024]={c='pr'},
% [0x0025]={c='po'},
% [0x0028]={c='op'},
% [0x0029]={c='cp'},

```

```
% [0x002B]={c='pr'},
%
```

For the meaning of these codes, see the Unicode standard.

12. The ‘nil’ language

This ‘language’ does nothing, except setting the hyphenation patterns to nohyphenation. For this language currently no special definitions are needed or available.

The macro `\LdfInit` takes care of preventing that this file is loaded more than once, checking the category code of the `@` sign, etc.

```
8681 (*nil)
8682 \ProvidesLanguage{nil}[<@date@> v<@version@> Nil language]
8683 \LdfInit{nil}{datenil}
```

When this file is read as an option, i.e., by the `\usepackage` command, `nil` could be an ‘unknown’ language in which case we have to make it known.

```
8684 \ifx\l@nil\undefined
8685 \newlanguage\l@nil
8686 \@namedef{bbl@hyphendata@the\l@nil}{}}}% Remove warning
8687 \let\bbl@elt\relax
8688 \edef\bbl@languages{% Add it to the list of languages
8689 \bbl@languages\bbl@elt{nil}{the\l@nil}{}}
8690 \fi
```

This macro is used to store the values of the hyphenation parameters `\lefthyphenmin` and `\righthyphenmin`.

```
8691 \providehyphenmins{\CurrentOption}{\m@ne\m@ne}
```

The next step consists of defining commands to switch to (and from) the ‘nil’ language.

`\captionnil`
`\datenil`

```
8692 \let\captionnil\empty
8693 \let\datenil\empty
```

There is no locale file for this pseudo-language, so the corresponding fields are defined here.

```
8694 \def\bbl@inidata@nil{%
8695 \bbl@elt{identification}{tag.ini}{und}%
8696 \bbl@elt{identification}{load.level}{0}%
8697 \bbl@elt{identification}{charset}{utf8}%
8698 \bbl@elt{identification}{version}{1.0}%
8699 \bbl@elt{identification}{date}{2022-05-16}%
8700 \bbl@elt{identification}{name.local}{nil}%
8701 \bbl@elt{identification}{name.english}{nil}%
8702 \bbl@elt{identification}{name.babel}{nil}%
8703 \bbl@elt{identification}{tag.bcp47}{und}%
8704 \bbl@elt{identification}{language.tag.bcp47}{und}%
8705 \bbl@elt{identification}{tag.opentype}{dflt}%
8706 \bbl@elt{identification}{script.name}{Latin}%
8707 \bbl@elt{identification}{script.tag.bcp47}{Latn}%
8708 \bbl@elt{identification}{script.tag.opentype}{DFLT}%
8709 \bbl@elt{identification}{level}{1}%
8710 \bbl@elt{identification}{encodings}{}%
8711 \bbl@elt{identification}{derivate}{no}}
8712 \@namedef{bbl@tbc@nil}{und}
8713 \@namedef{bbl@lbc@nil}{und}
8714 \@namedef{bbl@casing@nil}{und}
8715 \@namedef{bbl@lotf@nil}{dflt}
8716 \@namedef{bbl@elname@nil}{nil}
8717 \@namedef{bbl@lname@nil}{nil}
8718 \@namedef{bbl@esname@nil}{Latin}
```

```

8719 \@namedef{bbl@sname@nil}{Latin}
8720 \@namedef{bbl@sbc@nil}{Latn}
8721 \@namedef{bbl@sotf@nil}{latn}

```

The macro `\ldf@finish` takes care of looking for a configuration file, setting the main language to be switched on at `\begin{document}` and resetting the category code of `@` to its original value.

```

8722 \ldf@finish{nil}
8723 {/nil}

```

13. Calendars

The code for specific calendars are placed in the specific files, loaded when requested by an ini file in the identification section with `require.calendars`.

Start with function to compute the Julian day. It's based on the little library `calendar.js`, by John Walker, in the public domain.

```

8724 ((*Compute Julian day)) ≡
8725 \def\bbl@fpmmod#1#2{(#1-#2*floor((#1)/(#2)))}
8726 \def\bbl@cs@gregleap#1{%
8727   (\bbl@fpmmod{#1}{4} == 0) &&
8728   (!((\bbl@fpmmod{#1}{100} == 0) && (\bbl@fpmmod{#1}{400} != 0)))}
8729 \def\bbl@cs@jd#1#2#3{% year, month, day
8730   \fpeval{ 1721424.5 + (365 * (#1 - 1)) +
8731     floor((#1 - 1) / 4) + (-floor((#1 - 1) / 100)) +
8732     floor((#1 - 1) / 400) + floor(((367 * #2) - 362) / 12) +
8733     ((#2 <= 2) ? 0 : (\bbl@cs@gregleap{#1} ? -1 : -2)) + #3 } }
8734 (/Compute Julian day)

```

13.1. Islamic

The code for the Civil calendar is based on it, too.

```

8735 (*ca-islamic)
8736 <@Compute Julian day@>
8737 % == islamic (default)
8738 % Not yet implemented
8739 \def\bbl@ca@islamic#1-#2-#3\@#4#5#6{

```

The Civil calendar.

```

8740 \def\bbl@cs@isltojd#1#2#3{ % year, month, day
8741   ((#3 + ceil(29.5 * (#2 - 1)) +
8742     (#1 - 1) * 354 + floor((3 + (11 * #1)) / 30) +
8743     1948439.5) - 1) }
8744 \@namedef{bbl@ca@islamic-civil++}{\bbl@ca@islamicvl{x}{+2}}
8745 \@namedef{bbl@ca@islamic-civil+}{\bbl@ca@islamicvl{x}{+1}}
8746 \@namedef{bbl@ca@islamic-civil}{\bbl@ca@islamicvl{x}{}}
8747 \@namedef{bbl@ca@islamic-civil-}{\bbl@ca@islamicvl{x}{-1}}
8748 \@namedef{bbl@ca@islamic-civil--}{\bbl@ca@islamicvl{x}{-2}}
8749 \def\bbl@ca@islamicvl@x#1#2-#3-#4\@#5#6#7{%
8750   \edef\bbl@tempa{%
8751     \fpeval{ floor(\bbl@cs@jd{#2}{#3}{#4})+0.5 #1 } }%
8752   \edef#5{%
8753     \fpeval{ floor(((30*(\bbl@tempa-1948439.5)) + 10646)/10631) } }%
8754   \edef#6{\fpeval{
8755     min(12,ceil((\bbl@tempa-(29+\bbl@cs@isltojd{#5}{1}{1}))/29.5)+1) } }%
8756   \edef#7{\fpeval{ \bbl@tempa - \bbl@cs@isltojd{#5}{#6}{1} + 1 } }

```

The Umm al-Qura calendar, used mainly in Saudi Arabia, is based on moment-hijri, by Abdullah Alsigar (license MIT).

Since the main aim is to provide a suitable `\today`, and maybe some close dates, data just covers Hijri $\sim 1435/\sim 1460$ (Gregorian $\sim 2014/\sim 2038$).

```

8757 \def\bbl@cs@umalqura@data{56660, 56690,56719,56749,56778,56808,%
8758   56837,56867,56897,56926,56956,56985,57015,57044,57074,57103,%
8759   57133,57162,57192,57221,57251,57280,57310,57340,57369,57399,%

```

```

8760 57429,57458,57487,57517,57546,57576,57605,57634,57664,57694,%
8761 57723,57753,57783,57813,57842,57871,57901,57930,57959,57989,%
8762 58018,58048,58077,58107,58137,58167,58196,58226,58255,58285,%
8763 58314,58343,58373,58402,58432,58461,58491,58521,58551,58580,%
8764 58610,58639,58669,58698,58727,58757,58786,58816,58845,58875,%
8765 58905,58934,58964,58994,59023,59053,59082,59111,59141,59170,%
8766 59200,59229,59259,59288,59318,59348,59377,59407,59436,59466,%
8767 59495,59525,59554,59584,59613,59643,59672,59702,59731,59761,%
8768 59791,59820,59850,59879,59909,59939,59968,59997,60027,60056,%
8769 60086,60115,60145,60174,60204,60234,60264,60293,60323,60352,%
8770 60381,60411,60440,60469,60499,60528,60558,60588,60618,60648,%
8771 60677,60707,60736,60765,60795,60824,60853,60883,60912,60942,%
8772 60972,61002,61031,61061,61090,61120,61149,61179,61208,61237,%
8773 61267,61296,61326,61356,61385,61415,61445,61474,61504,61533,%
8774 61563,61592,61621,61651,61680,61710,61739,61769,61799,61828,%
8775 61858,61888,61917,61947,61976,62006,62035,62064,62094,62123,%
8776 62153,62182,62212,62242,62271,62301,62331,62360,62390,62419,%
8777 62448,62478,62507,62537,62566,62596,62625,62655,62685,62715,%
8778 62744,62774,62803,62832,62862,62891,62921,62950,62980,63009,%
8779 63039,63069,63099,63128,63157,63187,63216,63246,63275,63305,%
8780 63334,63363,63393,63423,63453,63482,63512,63541,63571,63600,%
8781 63630,63659,63689,63718,63747,63777,63807,63836,63866,63895,%
8782 63925,63955,63984,64014,64043,64073,64102,64131,64161,64190,%
8783 64220,64249,64279,64309,64339,64368,64398,64427,64457,64486,%
8784 64515,64545,64574,64603,64633,64663,64692,64722,64752,64782,%
8785 64811,64841,64870,64899,64929,64958,64987,65017,65047,65076,%
8786 65106,65136,65166,65195,65225,65254,65283,65313,65342,65371,%
8787 65401,65431,65460,65490,65520}
8788 \@namedef{bbl@ca@islamic-umalqura+}{\bbl@ca@islamcuqr@x{+1}}
8789 \@namedef{bbl@ca@islamic-umalqura}{\bbl@ca@islamcuqr@x{}}
8790 \@namedef{bbl@ca@islamic-umalqura-}{\bbl@ca@islamcuqr@x{-1}}
8791 \def\bbl@ca@islamcuqr@x#1#2-#3-#4\@#5#6#7{%
8792 \ifnum#2>2014 \ifnum#2<2038
8793 \bbl@afterfi\expandafter\@gobble
8794 \fi\fi
8795 {\bbl@error{year-out-range}{2014-2038}{}}}%
8796 \edef\bbl@tempd{\fpeval{ % (Julian) day
8797 \bbl@cs@jd{#2}{#3}{#4} + 0.5 - 2400000 #1}}%
8798 \count@\@ne
8799 \bbl@foreach\bbl@cs@umalqura@data{%
8800 \advance\count@\@ne
8801 \ifnum##1>\bbl@tempd\else
8802 \edef\bbl@tempe{\the\count@}%
8803 \edef\bbl@tempb{##1}%
8804 \fi}%
8805 \edef\bbl@templ{\fpeval{ \bbl@tempe + 16260 + 949 }}% month~lunar
8806 \edef\bbl@tempa{\fpeval{ floor((\bbl@templ - 1) / 12) }}% annus
8807 \edef#5{\fpeval{ \bbl@tempa + 1 }}%
8808 \edef#6{\fpeval{ \bbl@templ - (12 * \bbl@tempa) }}%
8809 \edef#7{\fpeval{ \bbl@tempd - \bbl@tempb + 1 }}%
8810 \bbl@add\bbl@precalendar{%
8811 \bbl@replace\bbl@ld@calendar{-civil}{}}%
8812 \bbl@replace\bbl@ld@calendar{-umalqura}{}}%
8813 \bbl@replace\bbl@ld@calendar{+}{}}%
8814 \bbl@replace\bbl@ld@calendar{-}{}}%
8815 (/ca-islamic)

```

13.2. Hebrew

This is basically the set of macros written by Michail Rozman in 1991, with corrections and adaptations by Rama Porrat, Misha, Dan Haran and Boris Lavva. This must be eventually replaced by computations with l3fp. An explanation of what's going on can be found in `hebcsl.sty`

```
8816 (*ca-hebrew)
```



```

8817 \newcount\bbl@cntcommon
8818 \def\bbl@remainder#1#2#3{%
8819   #3=#1\relax
8820   \divide #3 by #2\relax
8821   \multiply #3 by -#2\relax
8822   \advance #3 by #1\relax}%
8823 \newif\ifbbl@divisible
8824 \def\bbl@checkifdivisible#1#2{%
8825   {\countdef\tmp=0
8826   \bbl@remainder{#1}{#2}{\tmp}%
8827   \ifnum \tmp=0
8828     \global\bbl@divisibletrue
8829   \else
8830     \global\bbl@divisiblefalse
8831   \fi}}
8832 \newif\ifbbl@gregleap
8833 \def\bbl@ifgregleap#1{%
8834   \bbl@checkifdivisible{#1}{4}%
8835   \ifbbl@divisible
8836     \bbl@checkifdivisible{#1}{100}%
8837     \ifbbl@divisible
8838       \bbl@checkifdivisible{#1}{400}%
8839       \ifbbl@divisible
8840         \bbl@gregleaptrue
8841       \else
8842         \bbl@gregleapfalse
8843       \fi
8844     \else
8845       \bbl@gregleaptrue
8846     \fi
8847   \else
8848     \bbl@gregleapfalse
8849   \fi
8850   \ifbbl@gregleap}
8851 \def\bbl@gregdayspriormonths#1#2#3{%
8852   {#3=\ifcase #1 0 \or 0 \or 31 \or 59 \or 90 \or 120 \or 151 \or
8853     181 \or 212 \or 243 \or 273 \or 304 \or 334 \fi
8854   \bbl@ifgregleap{#2}%
8855   \ifnum #1 > 2
8856     \advance #3 by 1
8857   \fi
8858   \fi
8859   \global\bbl@cntcommon=#3}%
8860   #3=\bbl@cntcommon}
8861 \def\bbl@gregdaysprioryears#1#2{%
8862   {\countdef\tmpc=4
8863   \countdef\tmpb=2
8864   \tmpb=#1\relax
8865   \advance \tmpb by -1
8866   \tmpc=\tmpb
8867   \multiply \tmpc by 365
8868   #2=\tmpc
8869   \tmpc=\tmpb
8870   \divide \tmpc by 4
8871   \advance #2 by \tmpc
8872   \tmpc=\tmpb
8873   \divide \tmpc by 100
8874   \advance #2 by -\tmpc
8875   \tmpc=\tmpb
8876   \divide \tmpc by 400
8877   \advance #2 by \tmpc
8878   \global\bbl@cntcommon=#2\relax}%
8879   #2=\bbl@cntcommon}

```

```

8880 \def\bbl@absfromgreg#1#2#3#4{%
8881   {\countdef\tmpd=0
8882     #4=#1\relax
8883     \bbl@gregdayspriormonths{#2}{#3}{\tmpd}%
8884     \advance #4 by \tmpd
8885     \bbl@gregdaysprioryears{#3}{\tmpd}%
8886     \advance #4 by \tmpd
8887     \global\bbl@cntcommon=#4\relax}%
8888   #4=\bbl@cntcommon}
8889 \newif\ifbbl@hebrleap
8890 \def\bbl@checkleaphebrewyear#1{%
8891   {\countdef\tmpa=0
8892     \countdef\tmpb=1
8893     \tmpa=#1\relax
8894     \multiply \tmpa by 7
8895     \advance \tmpa by 1
8896     \bbl@remainder{\tmpa}{19}{\tmpb}%
8897     \ifnum \tmpb < 7
8898       \global\bbl@hebrleaptrue
8899     \else
8900       \global\bbl@hebrleapfalse
8901     \fi}}
8902 \def\bbl@hebreleapsedmonths#1#2{%
8903   {\countdef\tmpa=0
8904     \countdef\tmpb=1
8905     \countdef\tmpc=2
8906     \tmpa=#1\relax
8907     \advance \tmpa by -1
8908     #2=\tmpa
8909     \divide #2 by 19
8910     \multiply #2 by 235
8911     \bbl@remainder{\tmpa}{19}{\tmpb}% \tmpa=years%19-years this cycle
8912     \tmpc=\tmpb
8913     \multiply \tmpb by 12
8914     \advance #2 by \tmpb
8915     \multiply \tmpc by 7
8916     \advance \tmpc by 1
8917     \divide \tmpc by 19
8918     \advance #2 by \tmpc
8919     \global\bbl@cntcommon=#2}%
8920   #2=\bbl@cntcommon}
8921 \def\bbl@hebreleapseddays#1#2{%
8922   {\countdef\tmpa=0
8923     \countdef\tmpb=1
8924     \countdef\tmpc=2
8925     \bbl@hebreleapsedmonths{#1}{#2}%
8926     \tmpa=#2\relax
8927     \multiply \tmpa by 13753
8928     \advance \tmpa by 5604
8929     \bbl@remainder{\tmpa}{25920}{\tmpc}% \tmpc == ConjunctionParts
8930     \divide \tmpa by 25920
8931     \multiply #2 by 29
8932     \advance #2 by 1
8933     \advance #2 by \tmpa
8934     \bbl@remainder{#2}{7}{\tmpa}%
8935     \ifnum \tmpc < 19440
8936       \ifnum \tmpc < 9924
8937       \else
8938         \ifnum \tmpa=2
8939           \bbl@checkleaphebrewyear{#1}% of a common year
8940           \ifbbl@hebrleap
8941             \else
8942               \advance #2 by 1

```

```

8943         \fi
8944     \fi
8945 \fi
8946 \ifnum \tmpc < 16789
8947 \else
8948     \ifnum \tmpa=1
8949         \advance #1 by -1
8950         \bbl@checkleaphebrewyear{#1}% at the end of leap year
8951         \ifbbl@hebrleap
8952             \advance #2 by 1
8953         \fi
8954     \fi
8955 \fi
8956 \else
8957     \advance #2 by 1
8958 \fi
8959 \bbl@remainder{#2}{7}{\tmpa}%
8960 \ifnum \tmpa=0
8961     \advance #2 by 1
8962 \else
8963     \ifnum \tmpa=3
8964         \advance #2 by 1
8965     \else
8966         \ifnum \tmpa=5
8967             \advance #2 by 1
8968         \fi
8969     \fi
8970 \fi
8971 \global\bbl@cntcommon=#2\relax}%
8972 #2=\bbl@cntcommon}
8973 \def\bbl@daysinhebrewyear#1#2{%
8974 {\countdef\tmpe=12
8975 \bbl@hebreleapseddays{#1}{\tmpe}%
8976 \advance #1 by 1
8977 \bbl@hebreleapseddays{#1}{#2}%
8978 \advance #2 by -\tmpe
8979 \global\bbl@cntcommon=#2}%
8980 #2=\bbl@cntcommon}
8981 \def\bbl@hebrdayspriormonths#1#2#3{%
8982 {\countdef\tmpf= 14
8983 #3=\ifcase #1
8984     0 \or
8985     0 \or
8986     30 \or
8987     59 \or
8988     89 \or
8989     118 \or
8990     148 \or
8991     148 \or
8992     177 \or
8993     207 \or
8994     236 \or
8995     266 \or
8996     295 \or
8997     325 \or
8998     400
8999 \fi
9000 \bbl@checkleaphebrewyear{#2}%
9001 \ifbbl@hebrleap
9002     \ifnum #1 > 6
9003         \advance #3 by 30
9004     \fi
9005 \fi

```

```

9006 \bbl@daysinhebrewyear{#2}{\tmpf}%
9007 \ifnum #1 > 3
9008     \ifnum \tmpf=353
9009         \advance #3 by -1
9010     \fi
9011     \ifnum \tmpf=383
9012         \advance #3 by -1
9013     \fi
9014 \fi
9015 \ifnum #1 > 2
9016     \ifnum \tmpf=355
9017         \advance #3 by 1
9018     \fi
9019     \ifnum \tmpf=385
9020         \advance #3 by 1
9021     \fi
9022 \fi
9023 \global\bbl@cntcommon=#3\relax}%
9024 #3=\bbl@cntcommon}
9025 \def\bbl@absfromhebr#1#2#3#4{%
9026 {#4=#1\relax
9027 \bbl@hebrdayspriormonths{#2}{#3}{#1}%
9028 \advance #4 by #1\relax
9029 \bbl@hebreleapseddays{#3}{#1}%
9030 \advance #4 by #1\relax
9031 \advance #4 by -1373429
9032 \global\bbl@cntcommon=#4\relax}%
9033 #4=\bbl@cntcommon}
9034 \def\bbl@hebrfromgreg#1#2#3#4#5#6{%
9035 {\countdef\tmpx= 17
9036 \countdef\tmpy= 18
9037 \countdef\tmpz= 19
9038 #6=#3\relax
9039 \global\advance #6 by 3761
9040 \bbl@absfromgreg{#1}{#2}{#3}{#4}%
9041 \tmpz=1 \tmpy=1
9042 \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9043 \ifnum \tmpx > #4\relax
9044     \global\advance #6 by -1
9045     \bbl@absfromhebr{\tmpz}{\tmpy}{#6}{\tmpx}%
9046 \fi
9047 \advance #4 by -\tmpx
9048 \advance #4 by 1
9049 #5=#4\relax
9050 \divide #5 by 30
9051 \loop
9052     \bbl@hebrdayspriormonths{#5}{#6}{\tmpx}%
9053     \ifnum \tmpx < #4\relax
9054         \advance #5 by 1
9055         \tmpy=\tmpx
9056 \repeat
9057 \global\advance #5 by -1
9058 \global\advance #4 by -\tmpy}}
9059 \newcount\bbl@hebrday \newcount\bbl@hebrmonth \newcount\bbl@hebrewyear
9060 \newcount\bbl@gregday \newcount\bbl@gregmonth \newcount\bbl@gregyear
9061 \def\bbl@ca@hebrew#1-#2-#3\@#4#5#6{%
9062 \bbl@gregday=#3\relax \bbl@gregmonth=#2\relax \bbl@gregyear=#1\relax
9063 \bbl@hebrfromgreg
9064 {\bbl@gregday}{\bbl@gregmonth}{\bbl@gregyear}%
9065 {\bbl@hebrday}{\bbl@hebrmonth}{\bbl@hebrewyear}%
9066 \edef#4{\the\bbl@hebrewyear}%
9067 \edef#5{\the\bbl@hebrmonth}%
9068 \edef#6{\the\bbl@hebrday}}

```

13.3. Persian

There is an algorithm written in TeX by Jabri, Abolhassani, Pournader and Esfahbod, created for the first versions of the FarsiTeX system (no longer available), but the original license is GPL, so its use with LPPL is problematic. The code here follows loosely that by John Walker, which is free and accurate, but sadly very complex, so the relevant data for the years 2013-2050 have been pre-calculated and stored. Actually, all we need is the first day (either March 20 or March 21).

```

9070 (*ca-persian)
9071 <@Compute Julian day@>
9072 \def\bbl@cs@firstjal@xx{2012,2016,2020,2024,2028,2029,% March 20
9073 2032,2033,2036,2037,2040,2041,2044,2045,2048,2049}
9074 \def\bbl@ca@persian#1-#2-#3\@#4#5#6{%
9075 \edef\bbl@tempa{#1}% 20XX-03-\bbl@tempe = 1 farvardin:
9076 \ifnum\bbl@tempa>2012 \ifnum\bbl@tempa<2051
9077 \bbl@afterfi\expandafter\@gobble
9078 \fi\fi
9079 {\bbl@error{year-out-range}{2013-2050}{}}}%
9080 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9081 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9082 \edef\bbl@tempc{\fpeval{\bbl@cs@jd{\bbl@tempa}{#2}{#3}+.5}}% current
9083 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}% begin
9084 \ifnum\bbl@tempc<\bbl@tempb
9085 \edef\bbl@tempa{\fpeval{\bbl@tempa-1}}% go back 1 year and redo
9086 \bbl@xin@{\bbl@tempa}{\bbl@cs@firstjal@xx}%
9087 \ifin@def\bbl@tempe{20}\else\def\bbl@tempe{21}\fi
9088 \edef\bbl@tempb{\fpeval{\bbl@cs@jd{\bbl@tempa}{03}{\bbl@tempe}+.5}}%
9089 \fi
9090 \edef#4{\fpeval{\bbl@tempa-621}}% set Jalali year
9091 \edef#6{\fpeval{\bbl@tempc-\bbl@tempb+1}}% days from 1 farvardin
9092 \edef#5{\fpeval{% set Jalali month
9093 (#6 <= 186) ? ceil(#6 / 31) : ceil((#6 - 6) / 30)}}
9094 \edef#6{\fpeval{% set Jalali day
9095 (#6 - ((#5 <= 7) ? ((#5 - 1) * 31) : ((#5 - 1) * 30) + 6))}}
9096 (/ca-persian)

```

13.4. Coptic and Ethiopic

Adapted from `jquery.calendars.package-1.1.4`, written by Keith Wood, 2010. Dual license: GPL and MIT. The only difference is the epoch.

```

9097 (*ca-coptic)
9098 <@Compute Julian day@>
9099 \def\bbl@ca@coptic#1-#2-#3\@#4#5#6{%
9100 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9101 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1825029.5}}%
9102 \edef#4{\fpeval{%
9103 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9104 \edef\bbl@tempc{\fpeval{%
9105 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1825029.5}}%
9106 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9107 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}
9108 (/ca-coptic)
9109 (*ca-ethiopic)
9110 <@Compute Julian day@>
9111 \def\bbl@ca@ethiopic#1-#2-#3\@#4#5#6{%
9112 \edef\bbl@tempd{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + 0.5}}%
9113 \edef\bbl@tempc{\fpeval{\bbl@tempd - 1724220.5}}%
9114 \edef#4{\fpeval{%
9115 floor((\bbl@tempc - floor((\bbl@tempc+366) / 1461)) / 365) + 1}}%
9116 \edef\bbl@tempc{\fpeval{%
9117 \bbl@tempd - (#4-1) * 365 - floor(#4/4) - 1724220.5}}%

```

```

9118 \edef#5{\fpeval{floor(\bbl@tempc / 30) + 1}}%
9119 \edef#6{\fpeval{\bbl@tempc - (#5 - 1) * 30 + 1}}%
9120 (/ca-ethiopic)

```

13.5. Julian

Based on [ReinDersh].

```

9121 (*ca-julian)
9122 <@Compute Julian day@>
9123 \def\bbl@ca@julian#1-#2-#3\@@#4#5#6{%
9124 \edef\bbl@tempj{\fpeval{floor(\bbl@cs@jd{#1}{#2}{#3}) + .5}}%
9125 \edef\bbl@tempa{\fpeval{\bbl@tempj + 32082.5}}%
9126 \edef\bbl@tempb{\fpeval{floor((4 * \bbl@tempa + 3) / 1461)}}%
9127 \edef\bbl@tempc{\fpeval{\bbl@tempa - floor(1461*\bbl@tempb/4)}}%
9128 \edef\bbl@tempd{\fpeval{floor((5 * \bbl@tempc + 2) / 153)}}%
9129 \edef#6{\fpeval{\bbl@tempc - floor((153*\bbl@tempd+2) / 5) + 1}}%
9130 \edef#5{\fpeval{\bbl@tempd + 3 - 12 * floor(\bbl@tempd / 10)}}%
9131 \edef#4{\fpeval{\bbl@tempb - 4800 + floor(\bbl@tempd / 10)}}%
9132 (/ca-julian)

```

The Indian National Calendar, Saka era.

```

9133 (*ca-indian)
9134 <@Compute Julian day@>
9135 \def\bbl@sakachaitra#1{%
9136 \bbl@cs@jd{#1}{3}{%
9137 \fpeval{ ((\bbl@fpmmod{#1}{4})==0 && \bbl@fpmmod{#1}{100})!=0)
9138 || \bbl@fpmmod{#1}{400})==0) ? 21 : 22 }}%
9139 \def\bbl@ca@indian#1-#2-#3\@@#4#5#6{%
9140 \edef\bbl@tempa{\fpeval{\bbl@cs@jd{#1}{#2}{#3}}}%
9141 \edef\bbl@tempb{\fpeval{\bbl@sakachaitra{#1}}}%
9142 \edef\bbl@tempc{\fpeval{\bbl@tempa < \bbl@tempb ? #1-1 : #1}}%
9143 \edef\bbl@tempd{\fpeval{\bbl@tempa-\bbl@sakachaitra{\bbl@tempc}}}%
9144 \edef\bbl@tempe{\fpeval{ ((\bbl@fpmmod{\bbl@tempc}{4})==0 &&
9145 \bbl@fpmmod{\bbl@tempc}{100})!=0) || \bbl@fpmmod{\bbl@tempc}{400})==0) ? 31 : 30 }}%
9146 \edef#4{\fpeval{\bbl@tempc-78}}%
9147 \edef#5{\fpeval{\bbl@tempd<\bbl@tempe ? 1 :
9148 (\bbl@tempd-\bbl@tempe < 155 ?
9149 2+trunc((\bbl@tempd-\bbl@tempe)/31,0) :
9150 7+trunc((\bbl@tempd-\bbl@tempe-155)/30,0)}}}%
9151 \edef#6{\fpeval{\bbl@tempd<\bbl@tempe ? \bbl@tempd+1 :
9152 (\bbl@tempd-\bbl@tempe < 155 ?
9153 \bbl@fpmmod{\bbl@tempd-\bbl@tempe}{31}+1 :
9154 \bbl@fpmmod{\bbl@tempd-\bbl@tempe-155}{30}+1)}}%
9155 (/ca-indian)
9156 %

```

13.6. Buddhist

That's very simple.

```

9157 (*ca-buddhist)
9158 \def\bbl@ca@buddhist#1-#2-#3\@@#4#5#6{%
9159 \edef#4{\number\numexpr#1+543\relax}%
9160 \edef#5{#2}%
9161 \edef#6{#3}%
9162 (/ca-buddhist)
9163 %
9164 % \subsection{Chinese}
9165 %
9166 % Brute force, with the Julian day of first day of each month. The
9167 % table has been computed with the help of \textsf{python-lunardate} by
9168 % Ricky Yeung, GPLv2 (but the code itself has not been used). The range
9169 % is 2015-2044.

```

```

9170 %
9171 % \begin{macrocode}
9172 (*ca-chinese)
9173 \ExplSyntaxOn
9174 <@Compute Julian day@>
9175 \def\bbl@ca@chinese#1-#2-#3\@#4#5#6{%
9176 \edef\bbl@tempd{\fpeval{%
9177 \bbl@cs@jd{#1}{#2}{#3} - 2457072.5 }}%
9178 \count@ \z@
9179 \@tempcnta=2015
9180 \bbl@foreach\bbl@cs@chinese@data{%
9181 \ifnum##1>\bbl@tempd\else
9182 \advance\count@\@ne
9183 \ifnum\count@>12
9184 \count@\@ne
9185 \advance\@tempcnta\@ne\fi
9186 \bbl@xin@{,##1,}{, \bbl@cs@chinese@leap,}%
9187 \ifin@
9188 \advance\count@\m@ne
9189 \edef\bbl@tempe{\the\numexpr\count@+12\relax}%
9190 \else
9191 \edef\bbl@tempe{\the\count@}%
9192 \fi
9193 \edef\bbl@tempb{##1}%
9194 \fi}%
9195 \edef#4{\the\@tempcnta}%
9196 \edef#5{\bbl@tempe}%
9197 \edef#6{\the\numexpr\bbl@tempd-\bbl@tempb+1\relax}}
9198 \def\bbl@cs@chinese@leap{%
9199 885,1920,2953,3809,4873,5906,6881,7825,8889,9893,10778}
9200 \def\bbl@cs@chinese@data{0,29,59,88,117,147,176,206,236,266,295,325,
9201 354,384,413,443,472,501,531,560,590,620,649,679,709,738,%
9202 768,797,827,856,885,915,944,974,1003,1033,1063,1093,1122,%
9203 1152,1181,1211,1240,1269,1299,1328,1358,1387,1417,1447,1477,%
9204 1506,1536,1565,1595,1624,1653,1683,1712,1741,1771,1801,1830,%
9205 1860,1890,1920,1949,1979,2008,2037,2067,2096,2126,2155,2185,%
9206 2214,2244,2274,2303,2333,2362,2392,2421,2451,2480,2510,2539,%
9207 2569,2598,2628,2657,2687,2717,2746,2776,2805,2835,2864,2894,%
9208 2923,2953,2982,3011,3041,3071,3100,3130,3160,3189,3219,3248,%
9209 3278,3307,3337,3366,3395,3425,3454,3484,3514,3543,3573,3603,%
9210 3632,3662,3691,3721,3750,3779,3809,3838,3868,3897,3927,3957,%
9211 3987,4016,4046,4075,4105,4134,4163,4193,4222,4251,4281,4311,%
9212 4341,4370,4400,4430,4459,4489,4518,4547,4577,4606,4635,4665,%
9213 4695,4724,4754,4784,4814,4843,4873,4902,4931,4961,4990,5019,%
9214 5049,5079,5108,5138,5168,5197,5227,5256,5286,5315,5345,5374,%
9215 5403,5433,5463,5492,5522,5551,5581,5611,5640,5670,5699,5729,%
9216 5758,5788,5817,5846,5876,5906,5935,5965,5994,6024,6054,6083,%
9217 6113,6142,6172,6201,6231,6260,6289,6319,6348,6378,6408,6437,%
9218 6467,6497,6526,6556,6585,6615,6644,6673,6703,6732,6762,6791,%
9219 6821,6851,6881,6910,6940,6969,6999,7028,7057,7087,7116,7146,%
9220 7175,7205,7235,7264,7294,7324,7353,7383,7412,7441,7471,7500,%
9221 7529,7559,7589,7618,7648,7678,7708,7737,7767,7796,7825,7855,%
9222 7884,7913,7943,7972,8002,8032,8062,8092,8121,8151,8180,8209,%
9223 8239,8268,8297,8327,8356,8386,8416,8446,8475,8505,8534,8564,%
9224 8593,8623,8652,8681,8711,8740,8770,8800,8829,8859,8889,8918,%
9225 8948,8977,9007,9036,9066,9095,9124,9154,9183,9213,9243,9272,%
9226 9302,9331,9361,9391,9420,9450,9479,9508,9538,9567,9597,9626,%
9227 9656,9686,9715,9745,9775,9804,9834,9863,9893,9922,9951,9981,%
9228 10010,10040,10069,10099,10129,10158,10188,10218,10247,10277,%
9229 10306,10335,10365,10394,10423,10453,10483,10512,10542,10572,%
9230 10602,10631,10661,10690,10719,10749,10778,10807,10837,10866,%
9231 10896,10926,10956,10986,11015,11045,11074,11103}
9232 \ExplSyntaxOff

```

14. Support for Plain T_EX (`plain.def`)

14.1. Not renaming `hyphen.tex`

As Don Knuth has declared that the filename `hyphen.tex` may only be used to designate *his* version of the american English hyphenation patterns, a new solution has to be found in order to be able to load hyphenation patterns for other languages in a plain-based T_EX-format. When asked he responded:

That file name is “sacred”, and if anybody changes it they will cause severe upward/downward compatibility headaches.

People can have a file `locallyhyphen.tex` or whatever they like, but they mustn’t diddle with `hyphen.tex` (or `plain.tex` except to preload additional fonts).

The files `bplain.tex` and `lplain.tex` can be used as replacement wrappers around `plain.tex` and `lplain.tex` to achieve the desired effect, based on the `babel` package. If you load each of them with `iniTEX`, you will get a file called either `bplain.fmt` or `lplain.fmt`, which you can use as replacements for `plain.fmt` and `lplain.fmt`.

As these files are going to be read as the first thing `iniTEX` sees, we need to set some category codes just to be able to change the definition of `\input`.

```
9234 (*bplain|bplain)
9235 \catcode`\{=1 % left brace is begin-group character
9236 \catcode`\}=2 % right brace is end-group character
9237 \catcode`\#=6 % hash mark is macro parameter character
```

If a file called `hyphen.cfg` can be found, we make sure that *it* will be read instead of the file `hyphen.tex`. We do this by first saving the original meaning of `\input` (and I use a one letter control sequence for that so as not to waste multi-letter control sequence on this in the format).

```
9238 \openin 0 hyphen.cfg
9239 \ifeof0
9240 \else
9241   \let\input
```

Then `\input` is defined to forget about its argument and load `hyphen.cfg` instead. Once that’s done the original meaning of `\input` can be restored and the definition of `\a` can be forgotten.

```
9242   \def\input #1 {%
9243     \let\input\input
9244     \a hyphen.cfg
9245     \let\input\undefined
9246   }
9247 \fi
9248 (/bplain|bplain)
```

Now that we have made sure that `hyphen.cfg` will be loaded at the right moment it is time to load `plain.tex`.

```
9249 (bplain)\a plain.tex
9250 (bplain)\a lplain.tex
```

Finally we change the contents of `\fmtname` to indicate that this is *not* the plain format, but a format based on plain with the `babel` package preloaded.

```
9251 (bplain)\def\fmtname{babel-plain}
9252 (bplain)\def\fmtname{babel-lplain}
```

When you are using a different format, based on `plain.tex` you can make a copy of `bplain.tex`, rename it and replace `plain.tex` with the name of your format file.

14.2. Emulating some L^AT_EX features

The file `babel.def` expects some definitions made in the L^AT_EX 2_ε style file. So, in Plain we must provide at least some predefined values as well some tools to set them (even if not all options are available). There are no package options, and therefore an alternative mechanism is provided. For the moment, only `\babeloptionstrings` and `\babeloptionmath` are provided, which can be defined before loading `babel`. `\BabelModifiers` can be set too (but not sure it works).


```

9253 <(<*Emulate LaTeX)> ≡
9254 \def\@empty{}
9255 \def\loadlocalcfg#1{%
9256   \openin0#1.cfg
9257   \ifeof0
9258     \closein0
9259   \else
9260     \closein0
9261     {\immediate\writel6{*****}%
9262      \immediate\writel6{* Local config file #1.cfg used}%
9263      \immediate\writel6{*}%
9264     }
9265     \input #1.cfg\relax
9266   \fi
9267   \@endoflfd}

```

14.3. General tools

A number of $\TeX$ macro's that are needed later on.

```

9268 \long\def\@firstofone#1{#1}
9269 \long\def\@firstoftwo#1#2{#1}
9270 \long\def\@secondoftwo#1#2{#2}
9271 \def\@nnil{\@nil}
9272 \def\@gobbletwo#1#2{}
9273 \def\@ifstar#1{\@ifnextchar *{\@firstoftwo{#1}}}
9274 \def\@star@or@long#1{%
9275   \@ifstar
9276   {\let\l@ngrel@x\relax#1}%
9277   {\let\l@ngrel@x\long#1}}
9278 \let\l@ngrel@x\relax
9279 \def\@car#1#2\@nil{#1}
9280 \def\@cdr#1#2\@nil{#2}
9281 \let\@typeset@protect\relax
9282 \let\protected@edef\edef
9283 \long\def\@gobble#1{}
9284 \edef\@backslashchar{\expandafter\@gobble\string\}
9285 \def\strip@prefix#1>{}
9286 \def\g@addto@macro#1#2{{%
9287   \toks@\expandafter{#1#2}%
9288   \xdef#1{\the\toks@}}}
9289 \def\@namedef#1{\expandafter\def\csname #1\endcsname}
9290 \def\@nameuse#1{\csname #1\endcsname}
9291 \def\@ifundefined#1{%
9292   \expandafter\ifx\csname#1\endcsname\relax
9293     \expandafter\@firstoftwo
9294   \else
9295     \expandafter\@secondoftwo
9296   \fi}
9297 \def\@expandtwoargs#1#2#3{%
9298   \edef\reserved@a{\noexpand#1{#2}{#3}}\reserved@a}
9299 \def\zap@space#1 #2{%
9300   #1%
9301   \ifx#2\@empty\else\expandafter\zap@space\fi
9302   #2}
9303 \let\bbl@trace\@gobble
9304 \def\bbl@error#1{% Implicit #2#3#4
9305   \begingroup
9306     \catcode`\=0 \catcode`\==12 \catcode`\'=12
9307     \catcode`\^M=5 \catcode`\%=14
9308     \input errbabel.def
9309   \endgroup
9310   \bbl@error{#1}}
9311 \def\bbl@warning#1{%

```

```

9312 \begingroup
9313   \newlinechar=`^^J
9314   \def\{^J(babel) }%
9315   \message{\#1}%
9316 \endgroup}
9317 \let\bbl@infowarn\bbl@warning
9318 \def\bbl@info#1{%
9319   \begingroup
9320     \newlinechar=`^^J
9321     \def\{^J}%
9322     \wlog{#1}%
9323   \endgroup}

```

$\LaTeX 2_{\epsilon}$ has the command `\@onlypreamble` which adds commands to a list of commands that are no longer needed after `\begin{document}`.

```

9324 \ifx\@preamblecmds\undefined
9325   \def\@preamblecmds{}
9326 \fi
9327 \def\@onlypreamble#1{%
9328   \expandafter\gdef\expandafter\@preamblecmds\expandafter{%
9329     \@preamblecmds\do#1}}
9330 \@onlypreamble\@onlypreamble

```

Mimic $\LaTeX$'s `\AtBeginDocument`; for this to work the user needs to add `\begindocument` to his file.

```

9331 \def\begindocument{%
9332   \@begindocumenthook
9333   \global\let\@begindocumenthook\@undefined
9334   \def\do##1{\global\let##1\@undefined}%
9335   \@preamblecmds
9336   \global\let\do\noexpand}
9337 \ifx\@begindocumenthook\undefined
9338   \def\@begindocumenthook{}
9339 \fi
9340 \@onlypreamble\@begindocumenthook
9341 \def\AtBeginDocument{\g@addto@macro\@begindocumenthook}

```

We also have to mimic $\LaTeX$'s `\AtEndOfPackage`. Our replacement macro is much simpler; it stores its argument in `\@endofldf`.

```

9342 \def\AtEndOfPackage#1{\g@addto@macro\@endofldf{#1}}
9343 \@onlypreamble\AtEndOfPackage
9344 \def\@endofldf{}
9345 \@onlypreamble\@endofldf
9346 \let\bbl@afterlang\@empty
9347 \chardef\bbl@opt@hyphenmap\z@

```

$\LaTeX$ needs to be able to switch off writing to its auxiliary files; plain doesn't have them by default. There is a trick to hide some conditional commands from the outer `\ifx`. The same trick is applied below.

```

9348 \catcode`\&=\z@
9349 \ifx&\if@files\@undefined
9350   \expandafter\let\csname if@files\expandafter\endcsname
9351   \csname iffalse\endcsname
9352 \fi
9353 \catcode`\&=4

```

Mimic $\LaTeX$'s commands to define control sequences.

```

9354 \def\newcommand{\@star@or@long\new@command}
9355 \def\new@command#1{%
9356   \@testopt{\@newcommand#1}0}
9357 \def\@newcommand#1[#2]{%
9358   \@ifnextchar [{\@xargdef#1[#2]}%
9359   {\@argdef#1[#2]}}
9360 \long\def\@argdef#1[#2]#3{%

```

```

9361 \@yargdef#1\@ne{#2}{#3}}
9362 \long\def\@xargdef#1[#2][#3]#4{%
9363 \expandafter\def\expandafter#1\expandafter{%
9364 \expandafter\@protected@testopt\expandafter #1%
9365 \csname\string#1\expandafter\endcsname{#3}}%
9366 \expandafter\@yargdef \csname\string#1\endcsname
9367 \tw@{#2}{#4}}
9368 \long\def\@yargdef#1#2#3{%
9369 \@tempcnta#3\relax
9370 \advance \@tempcnta \@ne
9371 \let\@hash@\relax
9372 \edef\reserved@a{\ifx#2\tw@ [\@hash@1]\fi}%
9373 \@tempcntb #2%
9374 \@whilenum \@tempcntb < \@tempcnta
9375 \do{%
9376 \edef\reserved@a{\reserved@a\@hash@\the\@tempcntb}%
9377 \advance\@tempcntb \@ne}%
9378 \let\@hash@##%
9379 \l@ngrel@x\expandafter\def\expandafter#1\reserved@a}
9380 \def\providecommand{\@star@or@long\provide@command}
9381 \def\provide@command#1{%
9382 \begingroup
9383 \escapechar\m@ne\xdef\@gtempa{\{string#1}}%
9384 \endgroup
9385 \expandafter\@ifundefined\@gtempa
9386 {\def\reserved@a{\new@command#1}}%
9387 {\let\reserved@a\relax
9388 \def\reserved@a{\new@command\reserved@a}}%
9389 \reserved@a}%
9390 \def\DeclareRobustCommand{\@star@or@long\declare@robustcommand}
9391 \def\declare@robustcommand#1{%
9392 \edef\reserved@a{string#1}%
9393 \def\reserved@b{#1}%
9394 \edef\reserved@b{\expandafter\strip@prefix\meaning\reserved@b}%
9395 \edef#1{%
9396 \ifx\reserved@a\reserved@b
9397 \noexpand\x@protect
9398 \noexpand#1%
9399 \fi
9400 \noexpand\protect
9401 \expandafter\noexpand\csname
9402 \expandafter\@gobble\string#1 \endcsname
9403 }%
9404 \expandafter\new@command\csname
9405 \expandafter\@gobble\string#1 \endcsname
9406 }
9407 \def\x@protect#1{%
9408 \ifx\protect\@typeset@protect\else
9409 \@x@protect#1%
9410 \fi
9411 }
9412 \catcode`\&=\z@ % Trick to hide conditionals
9413 \def\@x@protect#1&fi#2#3&fi\protect#1}

```

The following little macro `\in@` is taken from `latex.ltx`; it checks whether its first argument is part of its second argument. It uses the boolean `\in@`; allocating a new boolean inside conditionally executed code is not possible, hence the construct with the temporary definition of `\bbl@tempa`.

```

9414 \def\bbl@tempa{\csname newif\endcsname&ifin@}
9415 \catcode`\&=4
9416 \ifx\in@\@undefined
9417 \def\in@#1#2{%
9418 \def\in@@##1#1##2##3\in@@{%
9419 \ifx\in@@##2\in@false\else\in@true\fi}%

```

```

9420 \in@@#2#1\in@\in@}
9421 \else
9422 \let\bbl@tempa\@empty
9423 \fi
9424 \bbl@tempa

```

$\LaTeX$ has a macro to check whether a certain package was loaded with specific options. The command has two extra arguments which are code to be executed in either the true or false case. This is used to detect whether the document needs one of the accents to be activated (activegrave and activeacute). For plain $\TeX$ we assume that the user wants them to be active by default. Therefore the only thing we do is execute the third argument (the code for the true case).

```

9425 \def\@ifpackagewith#1#2#3#4{#3}

```

The $\LaTeX$ macro `\@ifl@aded` checks whether a file was loaded. This functionality is not needed for plain $\TeX$ but we need the macro to be defined as a no-op.

```

9426 \def\@ifl@aded#1#2#3#4{}

```

For the following code we need to make sure that the commands `\newcommand` and `\providecommand` exist with some sensible definition. They are not fully equivalent to their $\LaTeX 2\epsilon$ versions; just enough to make things work in plain $\TeX$ environments.

```

9427 \ifx\@tempcnta\@undefined
9428 \csname newcount\endcsname\@tempcnta\relax
9429 \fi
9430 \ifx\@tempcntb\@undefined
9431 \csname newcount\endcsname\@tempcntb\relax
9432 \fi

```

To prevent wasting two counters in $\LaTeX$ (because counters with the same name are allocated later by it) we reset the counter that holds the next free counter (`\count10`).

```

9433 \ifx\bye\@undefined
9434 \advance\count10 by -2\relax
9435 \fi
9436 \ifx\@ifnextchar\@undefined
9437 \def\@ifnextchar#1#2#3{%
9438 \let\reserved@d=#1%
9439 \def\reserved@a{#2}\def\reserved@b{#3}%
9440 \futurelet\@let@token\@ifnch}
9441 \def\@ifnch{%
9442 \ifx\@let@token\@sptoken
9443 \let\reserved@c\@xifnch
9444 \else
9445 \ifx\@let@token\reserved@d
9446 \let\reserved@c\reserved@a
9447 \else
9448 \let\reserved@c\reserved@b
9449 \fi
9450 \fi
9451 \reserved@c}
9452 \def\:\let\@sptoken= } \: % this makes \@sptoken a space token
9453 \def\:\@xifnch\expandafter\def\:\{\futurelet\@let@token\@ifnch}
9454 \fi
9455 \def\@testopt#1#2{%
9456 \@ifnextchar[#{1}{#1[#{2}]}
9457 \def\@protected@testopt#1{%
9458 \ifx\protect\@typeset@protect
9459 \expandafter\@testopt
9460 \else
9461 \@x@protect#1%
9462 \fi}
9463 \long\def\@whilenum#1\do #2{\ifnum #1\relax #2\relax\@iwhilenum{#1\relax
9464 #2\relax}\fi}
9465 \long\def\@iwhilenum#1{\ifnum #1\expandafter\@iwhilenum
9466 \else\expandafter\@gobble\fi{#1}}

```

14.4. Encoding related macros

Code from `ltoutenc.dtx`, adapted for use in the plain $\TeX$ environment.

```
9467 \def\DeclareTextCommand{%
9468   \@dec@text@cmd\providecommand
9469 }
9470 \def\ProvideTextCommand{%
9471   \@dec@text@cmd\providecommand
9472 }
9473 \def\DeclareTextSymbol#1#2#3{%
9474   \@dec@text@cmd\chardef#1{#2}#3\relax
9475 }
9476 \def\@dec@text@cmd#1#2#3{%
9477   \expandafter\def\expandafter#2%
9478     \expandafter{%
9479       \csname#3-cmd\expandafter\endcsname
9480       \expandafter#2%
9481       \csname#3\string#2\endcsname
9482     }%
9483 %   \let\@ifdefinable\@rc@ifdefinable
9484   \expandafter#1\csname#3\string#2\endcsname
9485 }
9486 \def\@current@cmd#1{%
9487   \ifx\protect\@typeset@protect\else
9488     \noexpand#1\expandafter\@gobble
9489   \fi
9490 }
9491 \def\@changed@cmd#1#2{%
9492   \ifx\protect\@typeset@protect
9493     \expandafter\ifx\csname\cf@encoding\string#1\endcsname\relax
9494       \expandafter\ifx\csname ?\string#1\endcsname\relax
9495         \expandafter\def\csname ?\string#1\endcsname{%
9496           \@changed@x@err{#1}%
9497         }%
9498       \fi
9499       \global\expandafter\let
9500         \csname\cf@encoding\string#1\expandafter\endcsname
9501         \csname ?\string#1\endcsname
9502     \fi
9503     \csname\cf@encoding\string#1%
9504       \expandafter\endcsname
9505   \else
9506     \noexpand#1%
9507   \fi
9508 }
9509 \def\@changed@x@err#1{%
9510   \errhelp{Your command will be ignored, type <return> to proceed}%
9511   \errmessage{Command \protect#1 undefined in encoding \cf@encoding}}
9512 \def\DeclareTextCommandDefault#1{%
9513   \DeclareTextCommand#1?%
9514 }
9515 \def\ProvideTextCommandDefault#1{%
9516   \ProvideTextCommand#1?%
9517 }
9518 \expandafter\let\csname OT1-cmd\endcsname\@current@cmd
9519 \expandafter\let\csname?-cmd\endcsname\@changed@cmd
9520 \def\DeclareTextAccent#1#2#3{%
9521   \DeclareTextCommand#1{#2}[1]{\accent#3 #1}
9522 }
9523 \def\DeclareTextCompositeCommand#1#2#3#4{%
9524   \expandafter\let\expandafter\reserved@a\csname#2\string#1\endcsname
9525   \edef\reserved@b{\string##1}%
9526   \edef\reserved@c{%
```

```

9527 \expandafter\@strip@args\meaning\reserved@a:-\@strip@args}%
9528 \ifx\reserved@b\reserved@c
9529 \expandafter\expandafter\expandafter\ifx
9530 \expandafter\@car\reserved@a\relax\relax\@nil
9531 \@text@composite
9532 \else
9533 \edef\reserved@b##1{%
9534 \def\expandafter\noexpand
9535 \csname#2\string#1\endcsname###1{%
9536 \noexpand\@text@composite
9537 \expandafter\noexpand\csname#2\string#1\endcsname
9538 ###1\noexpand\@empty\noexpand\@text@composite
9539 {##1}%
9540 }%
9541 }%
9542 \expandafter\reserved@b\expandafter{\reserved@a{##1}}%
9543 \fi
9544 \expandafter\def\csname\expandafter\string\csname
9545 #2\endcsname\string#1-\string#3\endcsname{#4}
9546 \else
9547 \errhelp{Your command will be ignored, type <return> to proceed}%
9548 \errmessage{\string\DeclareTextCompositeCommand\space used on
9549 inappropriate command \protect#1}
9550 \fi
9551 }
9552 \def\@text@composite#1#2#3\@text@composite{%
9553 \expandafter\@text@composite@x
9554 \csname\string#1-\string#2\endcsname
9555 }
9556 \def\@text@composite@x#1#2{%
9557 \ifx#1\relax
9558 #2%
9559 \else
9560 #1%
9561 \fi
9562 }
9563 %
9564 \def\@strip@args#1:#2-#3\@strip@args{#2}
9565 \def\DeclareTextComposite#1#2#3#4{%
9566 \def\reserved@a{\DeclareTextCompositeCommand#1{#2}{#3}}%
9567 \bgroup
9568 \lccode`\@=#4%
9569 \lowercase{%
9570 \egroup
9571 \reserved@a @%
9572 }%
9573 }
9574 %
9575 \def\UseTextSymbol#1#2{#2}
9576 \def\UseTextAccent#1#2#3{}
9577 \def\@use@text@encoding#1{}
9578 \def\DeclareTextSymbolDefault#1#2{%
9579 \DeclareTextCommandDefault#1{\UseTextSymbol{#2}#1}%
9580 }
9581 \def\DeclareTextAccentDefault#1#2{%
9582 \DeclareTextCommandDefault#1{\UseTextAccent{#2}#1}%
9583 }
9584 \def\cf@encoding{OT1}

```

Currently we only use the $\text{\LaTeX 2}_{\epsilon}$ method for accents for those that are known to be made active in *some* language definition file.

```

9585 \DeclareTextAccent{"}{OT1}{127}
9586 \DeclareTextAccent{'}{OT1}{19}

```

```

9587 \DeclareTextAccent{\^}{OT1}{94}
9588 \DeclareTextAccent{\`}{OT1}{18}
9589 \DeclareTextAccent{\~}{OT1}{126}

```

The following control sequences are used in `babel.def` but are not defined for PLAIN $\TeX$.

```

9590 \DeclareTextSymbol{\textquotedblleft}{OT1}{92}
9591 \DeclareTextSymbol{\textquotedblright}{OT1}{`\"}
9592 \DeclareTextSymbol{\textquoteleft}{OT1}{`\'}
9593 \DeclareTextSymbol{\textquoteright}{OT1}{`\'}
9594 \DeclareTextSymbol{\i}{OT1}{16}
9595 \DeclareTextSymbol{\ss}{OT1}{25}

```

For a couple of languages we need the $\LaTeX$ -control sequence `\scriptsize` to be available. Because plain $\TeX$ doesn't have such a sophisticated font mechanism as $\LaTeX$ has, we just `\let` it to `\sevenrm`.

```

9596 \ifx\scriptsize@undefined
9597   \let\scriptsize\sevenrm
9598 \fi

```

And a few more “dummy” definitions.

```

9599 \def\language{english}%
9600 \let\bbl@opt@shorthands@nnil
9601 \def\bbl@ifshorthand#1#2#3{#2}%
9602 \let\bbl@language@opts@empty
9603 \let\bbl@provide@locale\relax
9604 \ifx\babeloptionstrings@undefined
9605   \let\bbl@opt@strings@nnil
9606 \else
9607   \let\bbl@opt@strings\babeloptionstrings
9608 \fi
9609 \def\BabelStringsDefault{generic}
9610 \def\bbl@tempa{normal}
9611 \ifx\babeloptionmath\bbl@tempa
9612   \def\bbl@mathnormal{\noexpand\textormath}
9613 \fi
9614 \def\AfterBabelLanguage#1#2{}
9615 \ifx\BabelModifiers@undefined\let\BabelModifiers\relax\fi
9616 \let\bbl@afterlang\relax
9617 \def\bbl@opt@safe{BR}
9618 \ifx\@uclclist@undefined\let\@uclclist@empty\fi
9619 \ifx\bbl@trace@undefined\def\bbl@trace#1{}\fi
9620 \expandafter\newif\csname ifbbl@single\endcsname
9621 \chardef\bbl@bidimode\z@
9622 ((/Emulate LaTeX))

```

A proxy file:

```

9623 (*plain)
9624 \input babel.def
9625 (/plain)

```

15. Acknowledgements

In the initial stages of the development of `babel`, Bernd Raichle provided many helpful suggestions and Michel Goossens supplied contributions for many languages. Ideas from Nico Poppelier, Piet van Oostrum and many others have been used. Paul Wackers and Werenfried Spit helped find and repair bugs.

More recently, there are significant contributions by Salim Bou, Ulrike Fischer, Loren Davis and Udi Fogiel.

Barbara Beeton has helped in improving the manual.

There are also many contributors for specific languages, which are mentioned in the respective files. Without them, `babel` just wouldn't exist.

References

- [1] Huda Smitshuijzen Abifares, *Arabic Typography*, Saqi, 2001.
- [2] Johannes Braams, Victor Eijkhout and Nico Poppelier, *The development of national $\text{\LaTeX}$ styles*, *TUGboat* 10 (1989) #3, pp. 401–406.
- [3] Yannis Haralambous, *Fonts & Encodings*, O'Reilly, 2007.
- [4] Donald E. Knuth, *The $\text{\TeX}$ book*, Addison-Wesley, 1986.
- [5] Jukka K. Korpela, *Unicode Explained*, O'Reilly, 2006.
- [6] Leslie Lamport, *$\text{\LaTeX}$, A document preparation System*, Addison-Wesley, 1986.
- [7] Leslie Lamport, in: $\text{\TeX}$ hax Digest, Volume 89, #13, 17 February 1989.
- [8] Ken Lunde, *CJKV Information Processing*, O'Reilly, 2nd ed., 2009.
- [9] Edward M. Reingold and Nachum Dershowitz, *Calendrical Calculations: The Ultimate Edition*, Cambridge University Press, 2018.
- [10] Hubert Partl, *German $\text{\TeX}$* , *TUGboat* 9 (1988) #1, pp. 70–72.
- [11] Joachim Schrod, *International $\text{\LaTeX}$ is ready to use*, *TUGboat* 11 (1990) #1, pp. 87–90.
- [12] Apostolos Syropoulos, Antonis Tsolomitis and Nick Sofroniu, *Digital typography using $\text{\LaTeX}$* , Springer, 2002, pp. 301–373.
- [13] K.F. Treebus. *Tekstwijzer; een gids voor het grafisch verwerken van tekst*, SDU Uitgeverij ('s-Gravenhage, 1988).